Mathematics in Collaboration

Learning to ask, understand, prove, and investigate with AI

From first mathematical sentences to complete proofs, with a look at current research

Worked proofs • Exercises with complete solutions
Small computational experiments • An introduction to Lean

To the reader

This book starts from ordinary arithmetic: if you can add, subtract, multiply, and divide fractions and whole numbers, including negative numbers and zero, you can begin. Mathematicians call the whole numbers $\ldots, -2, -1, 0, 1, 2, \ldots$ the *integers*. You do not need calculus, linear algebra, experience with proofs, or programming. Chapter 1 reviews the algebra and notation we need and explains what definitions, theorems, and proofs are.

We use the real numbers—all the points of the number line, including numbers such as $\sqrt{2}$ that are not fractions—as a familiar tool. Besides the ordinary rules of arithmetic, such as $a + b = b + a$, we accept four *starting facts* without proof; each is stated again where it is first used:

- every nonnegative real number has exactly one nonnegative square root (Chapter 1);
- for every real number, however large, there is a larger integer (the *Archimedean property*, Chapter 2);
- every nonempty set of positive integers has a smallest member (the *well-ordering property*, Chapter 4);
- the real numbers are *complete*: roughly, the number line has no holes. If, for every distance however small, all the terms of a sequence from some point on lie within that distance of one another, then the terms approach some point of the line. Cutting off $\sqrt{2} = 1.41421\ldots$ after more and more digits gives such a sequence, 1, 1.4, 1.41, 1.414, ..., which approaches $\sqrt{2}$; a line containing only fractions would have a hole there (Chapter 11).

Many students begin a mathematics course by asking how much they must memorize before they can start. This book is organized around a different question: *How can I enter a mathematical conversation, notice what is missing from my understanding, and learn enough to take part responsibly?* An AI assistant can be a useful partner in that conversation, suggesting examples, explanations, arguments, and code. It can also make convincing-sounding mistakes, and it cannot understand the mathematics for you. So throughout the book you will practice three skills: asking for exactly the step you are missing, noticing when an answer has quietly changed an assumption, and explaining what an argument really establishes.

Three problems run through the book. Can each person in a group be given a different task that they find acceptable? Can a process that repeatedly improves an approximate answer settle down to an exact solution? Can a sequence of plus and minus signs be arranged so that it has almost no resemblance to shifted copies of itself? Each starts with small examples you can work out by hand. Followed far enough, they lead to Hall's matching theorem (Chapters 8 and 9), the contraction theorem (Chapters 11 and 12), and a question about special matrices that has been open since the 1960s and for which a proof was claimed in 2026 (Chapter 14). A fourth strand, finite probability (Chapter 13), shows how computing an average can prove that an object with a desired property exists, even when the argument does not say how to build it.

Most definitions are introduced as answers to problems, so that you can see why they are worded

as they are. Major proofs appear first as plans, then as complete arguments, and finally as arguments you can modify and test, for instance by removing a hypothesis (one of the assumptions a theorem makes) and finding the step that breaks. Along the way the book collects eight *habits*, short pieces of advice about making progress, each in a shaded box headed by its number and name. Habit 1, for example, is *choose a representation that exposes the goal*: write your objects in a form in which what you want to prove can be seen, as when you write a number as $2k$, with k an integer, to show that it is even.

How to use a chapter

Each chapter opens with a shaded box headed "Overview" that says what the chapter is for. When the text raises a question or points out a pattern, try a small example of your own before looking for the answer. Many sections stop at a question marked **Pause and decide**, followed by a *Check*: cover the check, commit to an answer, and only then read on. When you read a proof, ask two questions about every line: *Why is this line true?* and *Why is this line here?*—that is, which later step needs it? You understand the proof when you can answer both.

Near the end of each chapter there are six exercises, numbered by chapter: Exercise 7.3 is the third exercise of Chapter 7. The first four are *core* exercises, which practice the main ideas directly. The last two are *transfer* exercises, which use the same ideas in a new setting or with a changed hypothesis. The one exception is Chapter 9, whose sixth exercise is a review portfolio drawing on Chapters 8 and 9. Complete solutions are in Appendix A, grouped by chapter: Section A.7 holds the solutions for Chapter 7. Look at a solution only after a genuine attempt; the introduction to Appendix A suggests how to use the solutions.

Two further appendices are for reference. Appendix B explains just enough of the programming language Python to run the book's computer experiments. It also says where to find the official installation instructions for Lean, the *proof assistant* (a program that checks every step of a proof) used in Chapters 15 and 16, and how to have Lean check the book's Lean examples on your own computer. Appendix C begins with a table of the book's main symbols, each with the place where it is explained, and returns to several of the habits.

Keeping track of evidence

For each important statement you meet, keep two separate records in your notebook. The first says how well you understand it: *understood* (you can explain it in your own words and use it in a new situation), *can follow* (you can check each step but could not yet explain it yourself), or *open* (you cannot yet follow it). Chapter 17, which studies a long proof, adds *borrowed*, for a result that you use without having studied its proof. The second record says what supports the statement: *proved* (in this book or by you), *cited from another source*, *checked by computer for finitely many cases*, or *not established*. For example, if a computer confirms that $1 + 3 + 5 + \cdots + (2n - 1) = n^2$ for every n from 1 to 1000, the second record says "checked by computer for finitely many cases"; only after the induction proof of Theorem 5.1 in Chapter 5 does it say "proved." Record the four starting facts as *assumptions*: we use them without proof, although more advanced books prove them from

a careful construction of the numbers.

A proof that a computer has checked step by step can still be poorly understood, and a pattern you understand well from experiments can still be unproved. Keeping both records stops these two kinds of confidence from blurring into one.

Where the book ends

The book ends with an independent investigation of your own, the *capstone*. It is open-ended, so it has no answer key; instead, Chapter 18 works through a complete model investigation, and Section C.6 gives a *rubric*: a table of the qualities on which your investigation can be assessed.

By the end of the book you should be able to rebuild several complete proofs from their main ideas, without having memorized their wording; run and check small computer experiments; carry out a small, carefully limited investigation with an AI assistant; and read short proofs written in Lean. Most research papers will still be out of reach, but you will have a method for an unfamiliar proof: find out exactly which background it needs, learn that background, and connect it back to the proof.

Whether an argument is correct and whether it is new are separate questions. The theorems in this book are well-established results. A result you discover for yourself may be correct and still be well known; it becomes a contribution to research only after it has been compared carefully with what is already known and its evidence has been checked.

Contents

To the reader **i**

1 A first mathematical conversation **1**

1.1 What does a pattern promise? . . . 1
1.2 Letters, brackets, and the equals sign . . . 1
1.3 The small algebra moves we will use . . . 2
1.4 A definition that makes a proof possible . . . 3
1.5 Definition, example, claim, theorem, proof . . . 4
1.6 Reading and writing a first proof . . . 5
1.7 Keeping track of what depends on what . . . 6
1.8 Sums, number systems, inequalities, and square roots . . . 7

2 Statements, choices, and assumptions **10**

2.1 An implication is a conditional promise . . . 10
2.2 And, or, and not . . . 12
2.3 The order of choices changes the meaning . . . 13
2.4 Negation: describing how a statement fails . . . 15
2.5 Contraposition and vacuous truth . . . 17
2.6 Absolute value, squares, and intervals . . . 18

3 Objects, maps, and information **22**

3.1 Sets express the boundary of a question . . . 22
3.2 A function includes its input and output sets . . . 24
3.3 Injective, surjective, and reversible . . . 26
3.4 Images and preimages answer different questions . . . 28

4 How a proof finds its next step **31**

4.1 Working backward and working forward . . . 31
4.2 The contrapositive and proof by contradiction . . . 32
4.3 The square root of two is irrational . . . 34
4.4 Existence is not uniqueness . . . 36
4.5 Reading a long proof as a sequence of smaller promises . . . 37

5 Induction, invariants, and progress **39**

5.1 From a few sums to every sum 39

5.2 Why induction is not circular, and when strong induction helps 41

5.3 A sequence defined by a recurrence 42

5.4 An invariant shows what cannot change 44

5.5 Euclid's algorithm: why it is right and why it stops 45

6 Sameness, symmetry, and structure **49**

6.1 Which differences matter? 49

6.2 Congruence and residue classes 50

6.3 Well-definedness: does the answer depend on the representative? 53

6.4 Structure-preserving maps 54

6.5 What symmetry permits us to assume 57

6.6 Order is a different structure 58

7 Graphs: counting, paths, and extremal choices **61**

7.1 A graph is a pattern of connections 61

7.2 Double counting and the handshake theorem 62

7.3 The pigeonhole principle and the average 64

7.4 Walks, paths, and shortest objects 65

7.5 Directed edges and a guaranteed sink 66

7.6 Tournaments and a vertex of maximum outdegree 68

7.7 Second out-neighborhoods and a research problem 69

8 Assignments, repairs, and obstructions **72**

8.1 Why one acceptable task per person is not enough 72

8.2 Maximal does not mean maximum 73

8.3 An alternating repair 74

8.4 Searching for an augmenting path 76

8.5 A failed search exposes a shortage 79

8.6 The whole procedure on a small example 80

9 Hall's theorem: a complete proof apprenticeship **83**

9.1 A condition that checks every group of students 83

9.2 First prove the necessary direction 84

9.3 A constructive proof from the search procedure 85

9.4 A second proof: equality tells us where to split 86

9.5 Why every part of the case split is needed . . . 89
9.6 Mathematical taste from equality cases . . . 90

10 Linearity: changing the language of a problem 92
10.1 Vectors are ordered measurements . . . 92
10.2 A linear map is determined by its building blocks . . . 94
10.3 Kernel: the information lost at zero . . . 96
10.4 The geometry of an inner product . . . 98
10.5 Deriving a bound by removing a component . . . 99
10.6 Matrix multiplication records composition . . . 102

11 Distance, error, and the meaning of a limit 105
11.1 Distance begins with absolute value . . . 105
11.2 A metric keeps only the properties we use . . . 106
11.3 Convergence promises eventual control . . . 108
11.4 Limits are unique . . . 110
11.5 Cauchy sequences do not need a known destination . . . 110
11.6 Completeness excludes missing limits . . . 112
11.7 A geometric bound controls an entire tail . . . 114
11.8 The edge of a set: bounds, suprema, and infima . . . 116

12 The contraction theorem: an unfamiliar proof made readable 120
12.1 From solving an equation to repeating a map . . . 120
12.2 The theorem and its dependency map . . . 122
12.3 The proof, one step at a time . . . 123
12.4 What the error estimates are for . . . 125
12.5 A quadratic example, worked by hand . . . 126
12.6 Remove the hypotheses one at a time . . . 128

13 Averaging, randomness, and existence 131
13.1 A finite probability model . . . 131
13.2 Expectation is a weighted average . . . 133
13.3 Indicators convert events into quantities . . . 136
13.4 Every graph has a large cut . . . 137
13.5 Turning the average argument into a construction . . . 138
13.6 A bound on rare large values . . . 140

14 Sign sequences: from experiments to a research boundary **143**
14.1 A sequence with small echoes . . . 143
14.2 Periodic correlation is a different quantity . . . 145
14.3 Circulant matrices turn correlations into dot products . . . 146
14.4 The sum of all correlations reveals a square . . . 148
14.5 Orthogonality also forces divisibility by four . . . 149
14.6 An exact finite experiment . . . 151
14.7 What current research adds . . . 153

15 Lean I: reading the state of a proof **155**
15.1 A checker is not the same as an explanation . . . 155
15.2 Reading Lean's basic notation . . . 156
15.3 An implication as a change of goal . . . 158
15.4 Conjunction means two obligations . . . 159
15.5 Composition of injective functions in Lean . . . 160
15.6 Existence statements, and the type of a number . . . 162
15.7 Three ways to use an equation . . . 163

16 Lean II: definitions, dependencies, and trust **165**
16.1 The statement must be right before the proof can help . . . 165
16.2 A complete existence proof in Lean . . . 166
16.3 Sets as membership predicates . . . 169
16.4 Encoding the exclusions in a second out-neighborhood . . . 170
16.5 A successful proof can still be irrelevant . . . 172
16.6 Placeholders and axiom dependencies . . . 172
16.7 A debugging sequence that does not lose the mathematics . . . 174

17 Reading an unfamiliar proof one piece at a time **178**
17.1 A new theorem on a familiar foundation . . . 178
17.2 Capsule one: small regions that cover the space . . . 179
17.3 Capsule two: keeping infinitely many indices . . . 181
17.4 Capsule three: subsequences, and a plan for the proof . . . 181
17.5 The complete proof . . . 182
17.6 Why neither hypothesis can be dropped . . . 184
17.7 The prerequisite map after the work is done . . . 185
17.8 Reading a formalized research proof with the same method . . . 187

18 A complete AI-assisted investigation **191**

18.1 The question that starts the project . . . 191
18.2 Testing the first guess . . . 192
18.3 The one-step estimate . . . 193
18.4 Can the constant be improved? . . . 196
18.5 A numerical experiment with exact arithmetic . . . 197
18.6 The finished investigation as a short paper . . . 200
18.7 Your independent capstone . . . 200

A Solutions **203**

A.1 Chapter 1 . . . 203
A.2 Chapter 2 . . . 205
A.3 Chapter 3 . . . 207
A.4 Chapter 4 . . . 209
A.5 Chapter 5 . . . 211
A.6 Chapter 6 . . . 213
A.7 Chapter 7 . . . 215
A.8 Chapter 8 . . . 217
A.9 Chapter 9 . . . 219
A.10 Chapter 10 . . . 222
A.11 Chapter 11 . . . 224
A.12 Chapter 12 . . . 226
A.13 Chapter 13 . . . 229
A.14 Chapter 14 . . . 231
A.15 Chapter 15 . . . 234
A.16 Chapter 16 . . . 235
A.17 Chapter 17 . . . 237
A.18 Chapter 18 . . . 240

B Running the examples on a computer **245**

B.1 The smallest amount of Python needed . . . 245
B.2 An exact fraction is not a rounded decimal . . . 248
B.3 Three experiments . . . 249
B.4 Checking the Lean examples . . . 254

C A compact guide to notation and mathematical judgment **257**

C.1 A notation map, with places to return to . . . 257

C.2 Structures, what proofs use, and changes of description 261
C.3 Choosing the next move 262
C.4 Turning a reading difficulty into a precise question 264
C.5 What a finished reading record contains 266
C.6 A suggested capstone rubric 266
C.7 What you can do at the end of the book 267

References and further reading **268**

CHAPTER 1

A first mathematical conversation

Overview

Turn a familiar numerical pattern into a precise statement, and prove it. Review the algebra and notation needed to state such a pattern exactly, and learn how definitions, examples, claims, theorems, and proofs differ. Begin reading an AI assistant's arguments critically: which numbers does an argument really cover, and has it skipped a step?

1.1 What does a pattern promise?

Suppose two people each have an odd number of tokens. If they put their tokens together, must the total be even? Try a few cases: $3 + 5 = 8$, $7 + 9 = 16$, and $1 + 1 = 2$. Every total is even. But there are infinitely many pairs of odd numbers, and no list of examples can include them all. To know what must happen for every pair, including those nobody has tried, we need a *proof*: an argument that explains why the total is always even.

First we must say exactly which numbers the pattern is about. An *integer* is a whole number that may be positive, negative, or zero: $\ldots, -2, -1, 0, 1, 2, \ldots$, where the dots mean that the list continues forever in both directions. The collection of all integers is written $\mathbb{Z}$. The pattern becomes a precise statement:

For every pair of odd integers, their sum is even.

This covers more than the token story: nobody holds -3 tokens, but the statement includes sums such as $-3 + 5 = 2$. Once negative numbers are allowed, "odd" and "even" need exact meanings too; Section 1.4 supplies them.

So when you bring a claim to a classmate or to an AI assistant, the first question is not "Can you prove this?" but "Which numbers does the claim talk about, and what exactly does it say about them?" An assistant that quietly replaces "integers" by "positive integers" and then argues correctly has answered a different question: its proof says nothing about $-3 + 5$.

1.2 Letters, brackets, and the equals sign

A letter such as n that stands for an integer whose value we have not fixed is called a *variable*. A number written directly in front of a letter means multiplication: $2n$ means $2 \times n$. Two letters written side by side are multiplied too, so ab means $a \times b$. A raised dot also means multiplication: $2 \cdot 4$ is 2×4. Between two numerals the dot cannot be left out, because 24 already means twenty-four.

The quantities multiplied in a product are its *factors*, and the quantities added in a sum are its *terms*: the product $2n$ has the factors 2 and n, and the sum $2n + 1$ has the terms $2n$ and 1. The square n^2 means $n \times n$. Parentheses, also called brackets, group an expression: $2(n + 1)$ is twice the whole quantity $n + 1$. It equals $2n + 2$, not $2n + 1$, because the 1 inside the brackets is doubled too. At $n = 5$, for instance, $2(5 + 1) = 12$, while $2 \cdot 5 + 1 = 11$.

An equals sign says that the expressions on either side of it have the same value; it does not mean "and now calculate the next thing." The chain $2(3 + 1) = 2 \cdot 4 = 8$ is correct, because all three expressions have the value 8. The chain $2 + 3 = 5 \cdot 2 = 10$ is wrong, because $2 + 3$ is not equal to $5 \cdot 2$. The writer meant two steps, and should have written two equations.

We will use the familiar rules of arithmetic freely. Addition and multiplication can each be done in either order and grouped in either way, and multiplication distributes over addition:

$$a + b = b + a, \qquad (a + b) + c = a + (b + c),$$
$$ab = ba, \qquad (ab)c = a(bc), \qquad a(b + c) = ab + ac.$$

The last rule is called the *distributive rule*. Because multiplication can be done in either order, it also works with the sum written first: $(a + b)c = ac + bc$. These rules hold for all integers, and more generally for all the real numbers described in Section 1.8.

A fraction a/b means a divided by b, and it is defined only when $b \neq 0$ (the symbol $\neq$ means "is not equal to"). If two expressions are equal, then adding the same number to both, subtracting it from both, or multiplying both by it keeps them equal, and so does dividing both by the same nonzero number.

Pause and decide. An assistant starts from the equation $x^2 = x$, divides both sides by x, and concludes that $x = 1$. Has it found every solution?

Check. No. Dividing by x is allowed only when $x \neq 0$, so the argument says nothing about $x = 0$; forgetting such a case is a common way to lose solutions. A complete argument treats two cases. If $x = 0$, the equation holds, since $0^2 = 0$. If $x \neq 0$, dividing both sides by x gives $x = 1$, and indeed $1^2 = 1$. So the solutions are exactly 0 and 1.

1.3 The small algebra moves we will use

To evaluate an expression such as $2k + 1$ at $k = 3$, replace every occurrence of k by 3, getting $2 \cdot 3 + 1 = 7$. This is called *substitution*.

Writing $f(n) = n + 1$ names a rule f that adds one to its input, so $f(3) = 4$. Read $f(n)$ as "the output of f at the input n," not as f times n. Such a rule is called a *function*; Chapter 3 studies functions.

To rewrite an expression without changing its value, we may read any arithmetic rule from Section 1.2 in either direction. Reading the distributive rule from left to right, as in $2(k+1) = 2k+2$, is called *expanding* or *multiplying out*. Reading it from right to left, as in $2k + 2 = 2(k + 1)$, is called *factoring*; in a proof, factoring is often useful because it brings a common factor, here 2, into the open.

An equality that holds for every allowed value of its letters, such as $2(k+1) = 2k+2$, is called an *identity*. An equation to be solved is different: $2k+1 = 7$ asks which values of k make it true, and subtracting one and then dividing by two gives $k = 3$.

For a longer example of multiplying out, take the product $(2k+1)(2\ell+1)$. The letter ℓ is a curly lowercase L, read "ell," so that it is not mistaken for the digit 1. Two different letters can stand for different numbers; Section 1.7 explains why that matters. We multiply out one bracket at a time:

$$\begin{aligned}(2k+1)(2\ell+1) &= 2k(2\ell+1) + 1\cdot(2\ell+1)\\ &= (4k\ell + 2k) + (2\ell+1)\\ &= 4k\ell + 2k + 2\ell + 1.\end{aligned}$$

The first line uses $(a+b)c = ac+bc$ with $a = 2k$, $b = 1$, and $c = 2\ell+1$. The second multiplies out each copy of $2\ell+1$ with $a(b+c) = ab+ac$, using $2k\cdot 2\ell = (2\cdot 2)(k\ell) = 4k\ell$, since factors may be reordered and regrouped. The third drops the brackets, since addition may be regrouped.

A *power* records repeated multiplication: $2^3 = 2\cdot 2\cdot 2 = 8$. The *exponent* 3 counts equal factors, not additions: $2+2+2 = 6$ is $2\cdot 3$, not 2^3. In general, for a positive integer n, a^n is the product of n factors each equal to a, and a is called the *base*; in particular, $a^1 = a$. Counting factors gives two rules. Two factors of 2 followed by three more make five, so $2^2\cdot 2^3 = 2^5$, and in general $a^n a^m = a^{n+m}$ for positive integers n and m. Division cancels factors instead:

$$\frac{2^5}{2^2} = \frac{2\cdot 2\cdot 2\cdot 2\cdot 2}{2\cdot 2} = 2^3.$$

In general, if $n > m$, the m factors below the line cancel m of the n factors above it, so $a^n/a^m = a^{n-m}$. Cancelling a factor a means dividing by a, so this rule requires $a \neq 0$.

We extend the notation to zero and negative exponents in the only way that keeps these rules working. Dividing a^n by itself gives 1, while the subtraction rule would give a^0; so we agree that $a^0 = 1$ whenever $a \neq 0$. Similarly, the subtraction rule would turn $1/a^n = a^0/a^n$ into $a^{0-n} = a^{-n}$; so we agree that $a^{-n} = 1/a^n$ for each positive integer n and $a \neq 0$. For example, $2^{-3} = 1/8$. With these agreements, both rules hold for all integers n and m, as long as $a \neq 0$; for instance, $2^3\cdot 2^{-1} = 8\cdot\frac{1}{2} = 4 = 2^2$. We skip the routine check, which sorts the signs of n and m into cases, and use both rules freely.

Pause and decide. Why is $(a+b)^2$ not in general equal to a^2+b^2?

Check. Multiplying out $(a+b)(a+b)$ one bracket at a time gives $a^2+ab+ba+b^2 = a^2+2ab+b^2$. The extra term $2ab$ is not zero in general: for $a = 1$ and $b = 2$, $(1+2)^2 = 9$ but $1^2+2^2 = 5$, and the difference 4 is exactly $2ab$.

1.4 A definition that makes a proof possible

To prove anything about odd and even numbers, we need to know exactly what those words mean. A *definition* gives an exact meaning to a word or symbol; ours must let us decide, for any integer at all, whether that integer qualifies. Definitions are set apart under numbered headings; in this book,

definitions, examples, and proved results share one numbering within each chapter, while exercises are numbered separately.

Definition 1.1 (Even and odd integers)**.** An integer a is *even* if $a = 2k$ for some integer k. It is *odd* if $a = 2k + 1$ for some integer k.

For example, 8 is even because $8 = 2 \cdot 4$, and 7 is odd because $7 = 2 \cdot 3 + 1$. Negative integers are included: -3 is odd because $-3 = 2 \cdot (-2) + 1$. Zero is even because $0 = 2 \cdot 0$. Whether an integer is even or odd is called its *parity*. Every integer is in fact either even or odd, but not both; that separate fact needs its own argument, given in Lemma 4.1 of Chapter 4, and this chapter does not need it.

The words "for some integer k" mean that at least one integer k makes the equation true, and the requirement that k be an *integer* is essential. If k could be any number, every integer a would have the form $2k$, simply by choosing $k = a/2$; for instance, $7 = 2 \cdot 3.5$. The definition asks not whether a number can be divided by two, but whether the result is an integer, and 3.5 is not.

When a definition asks for *some* integer k with a certain property, a particular integer with that property is called a *witness*. For the odd integer 7, the witness is $k = 3$, because $7 = 2 \cdot 3 + 1$. The witness is not 7 itself, but the extra integer that makes $7 = 2k + 1$ true. For -3, the witness is $k = -2$.

A definition can be used in two directions. If we are told that a is odd, we may introduce an integer k with $a = 2k + 1$ and reason with that equation; this *unpacks* the information. To show that a given integer a is odd, it is enough to name an integer k and check that $a = 2k + 1$; this *recognizes* that the definition is satisfied. Our first proof does both: it unpacks the information that two numbers are odd, and it recognizes that their sum is even.

1.5 Definition, example, claim, theorem, proof

Mathematical sentences do different jobs, and a reader who mixes up the jobs can easily misjudge what has been shown.

A definition sets a meaning. Definition 1.1 is an agreement about how the words *even* and *odd* will be used, not a discovery about numbers, so there is nothing to prove before we adopt it.

An example shows the meaning in one instance, as $7 = 2 \cdot 3 + 1$ does for the word *odd*. A *nonexample* is an object that does not meet the definition. For instance, 4 is not odd: if $4 = 2k + 1$, then $2k = 3$ and $k = 3/2$, which is not an integer, so no integer witness exists. Examples and nonexamples together show where the boundary of a definition lies.

A claim asserts something that needs justification. "Adding two odd integers always gives an even integer" is not part of any definition. To establish it, we must show that whenever two integers are each one more than twice an integer, their sum is twice an integer.

A theorem is a claim that has been proved. The heading *theorem* is usually kept for the most important results. A *proposition* is a proved result of more moderate importance, a *lemma* is proved mainly to help prove something else, and a *corollary* follows quickly from an earlier result. All are held to the same standard of proof. A *conjecture*, by contrast, has been proposed but not yet proved. In logic and in the proof assistant Lean (Chapter 15), the word *proposition* has a second

meaning: any statement that is either true or false, whether or not anyone has proved it, such as the false proposition "2 + 2 = 5." Context makes clear which meaning is intended.

A proof supplies the justification. It starts from what we are given and from facts already accepted, makes deductions that can each be justified, and arrives at exactly the promised conclusion, so that the reader can see for themselves that no allowed case has been left out. The square □ marks the end of a proof.

A *hypothesis*, also called an assumption or premise, is a condition under which a result is asserted. In Proposition 1.2 in the next section, the hypotheses are that the two numbers are integers and that both are odd; the *conclusion* is that their sum is even. A proof may use the hypotheses at any point, but not the conclusion, because that is exactly what it must establish.

Facts accepted without proof as the starting rules of a theory are called *axioms*. We accept the familiar rules of arithmetic, such as $a + b = b + a$, and four further *starting facts* about numbers, listed in *To the reader*; the first, about square roots, appears in Section 1.8. A claim does not become a starting fact just because it is hard to prove: an argument that relies on a claim that is neither proved nor accepted as a starting fact has a gap.

Pause and decide. Classify these three sentences. (1) "Even means twice an integer." (2) "Six is even because $6 = 2 \cdot 3$." (3) "The sum of two odd integers is even."

Check. (1) is a definition. (2) is an example: it checks the definition for one number and names the witness 3. (3) is a general claim about every pair of odd integers; it becomes an established result once it is proved, as Proposition 1.2 in the next section.

1.6 Reading and writing a first proof

Proofs use a few small words in a precise way. "Let a and b be odd integers" gives the names a and b to two odd integers that could be *any* odd integers at all, not your two favorite examples. Since the proof may use only the fact that they are odd integers, the finished argument works whichever odd integers are supplied. Words such as "hence" and "therefore" announce a deduction, and the reader should always be able to see its reason.

Read the next result in three passes: first what is assumed and what is promised; then the plan, which shows how one might discover the proof but proves nothing itself; and then the proof, line by line.

Proposition 1.2. *The sum of any two odd integers is even.*

Plan. Write each odd integer in the form given by the definition of odd, so that its oddness is visible. Then rearrange the sum until it has the form required by the definition of even: twice an integer.

Proof. Let a and b be odd integers. By the definition of odd (Definition 1.1), there are integers k and ℓ with $a = 2k + 1$ and $b = 2\ell + 1$. The integers k and ℓ may be equal or different; the argument

does not depend on which. Substituting these expressions gives

$$\begin{aligned} a + b &= (2k + 1) + (2\ell + 1) && \text{substitute for } a \text{ and } b \\ &= 2k + 2\ell + 2 && \text{reorder and regroup the terms, add } 1 + 1 \\ &= 2(k + \ell + 1) && \text{factor out 2.} \end{aligned}$$

The number $k + \ell + 1$ is an integer, because a sum of integers is an integer. Hence $a + b$ is twice an integer. By the definition of even, $a + b$ is even. □

The final sentence connects the expression we computed to the property we promised: a proof ends not when the algebra looks tidy, but when the required conclusion has been reached.

To see what each symbol does, take $a = 7$ and $b = -3$, with witnesses $k = 3$ and $\ell = -2$. The proof builds the witness $k + \ell + 1 = 2$ for the sum, and indeed $7 + (-3) = 4 = 2 \cdot 2$. This check is not a second proof.

The proposition promises an even sum only when *both* numbers are odd; about $2 + 3 = 5$ it says nothing. Keep the hypotheses attached to a result when you remember it.

> **Habit 1: choose a representation that exposes the goal**
>
> Before calculating, ask what form the answer must take. Then choose a way of writing the objects, a *representation*, in which that form can appear. To show that a number is even, try to display a factor of 2; to show that an approximation is close to the true value, write down the difference between them. When an assistant proposes a representation, ask why it helps before letting it continue.

1.7 Keeping track of what depends on what

In the proof of Proposition 1.2, a and b can be any odd integers, but the witnesses are not chosen freely. Solving $a = 2k + 1$ gives $k = (a - 1)/2$, and likewise $\ell = (b - 1)/2$, so each witness is determined by its odd integer. If a changes from 7 to 9 while b stays the same, then k changes from 3 to 4, but ℓ does not. The witness $k + \ell + 1$ for the sum is built in turn from k and ℓ.

Suppose an assistant writes both odd integers as $2k + 1$, using the same k for each, and computes $(2k + 1) + (2k + 1) = 2(2k + 1)$, which is certainly even. The algebra is correct, but the argument covers only the case in which the two odd integers are equal, as in $5 + 5$. It says nothing about $3 + 5$, because no single integer k makes both $3 = 2k + 1$ and $5 = 2k + 1$ true. So ask of any argument: "Which quantities can be anything, and which are determined by the others?" Chapter 2 makes this question precise with the phrases "for every" and "there exists."

Separate witnesses matter just as much when we show that the sum of three odd integers is odd. Let a, b, and c be odd integers. By the definition of odd, there are integers k, ℓ, and m, possibly different, with $a = 2k + 1$, $b = 2\ell + 1$, and $c = 2m + 1$. Adding gives

$$a + b + c = (2k + 1) + (2\ell + 1) + (2m + 1) = 2k + 2\ell + 2m + 3.$$

This may "look odd," but the definition asks for the form *twice an integer, plus one*. So split 3 as $2 + 1$ and factor 2 out of the terms that contain it:

$$2k + 2\ell + 2m + 3 = 2k + 2\ell + 2m + 2 + 1 = 2(k + \ell + m + 1) + 1.$$

The number $k + \ell + m + 1$ is an integer, so $a + b + c$ is odd by Definition 1.1.

1.8 Sums, number systems, inequalities, and square roots

To name the numbers in a list, we often use one letter with a small label below and to the right, called a *subscript*: a_1, a_2, a_3, a_4 are the first four numbers of a list. A subscript is a label, not a multiplication: a_2 is the second number, whereas $2a$ is twice a. The notation

$$\sum_{j=1}^{4} a_j = a_1 + a_2 + a_3 + a_4$$

means: add a_1, a_2, a_3, and a_4. The symbol $\sum$ is the Greek capital letter sigma, an S for "sum." The letter j is an *index*, a counter that runs from the value below the sigma to the value above it.

If $a_1 = 1$, $a_2 = 3$, $a_3 = 5$, and $a_4 = 7$, the sum is $1 + 3 + 5 + 7 = 16$. Renaming the index does not change the sum, because the index is only a counter: $\sum_{r=1}^{4} a_r$ is also 16. Changing the value above the sigma from 4 to 3 does: $\sum_{j=1}^{3} a_j = 1 + 3 + 5 = 9$.

These totals are *perfect squares*, squares of integers: $1 + 3 + 5 = 3^2$ and $1 + 3 + 5 + 7 = 4^2$, and likewise $1 = 1^2$ and $1 + 3 = 2^2$. Is the sum of the first n odd positive integers always n^2? As in Section 1.1, examples suggest a pattern, but only a proof can settle it; Theorem 5.1 in Chapter 5 supplies one.

The quantity added can be any expression involving the index; substitute each value of the index in turn and add. With the same numbers,

$$\sum_{j=1}^{3} (a_j + 5) = (a_1 + 5) + (a_2 + 5) + (a_3 + 5) = 6 + 8 + 10 = 24,$$

and the index itself may appear in the expression:

$$\sum_{j=1}^{3} j^2 = 1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14.$$

A *set* is a collection of objects, called its *members*; curly braces list the members, and Chapter 3 studies sets carefully. Several sets of numbers have standard names.

- $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$ is the set of *natural numbers*. In this book zero counts as a natural number; when we want only the positive integers $1, 2, 3, \ldots$, we say so.
- $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ is the set of integers.
- $\mathbb{Q}$ is the set of *rational numbers*: numbers that can be written as a fraction a/b in which a and b are integers and $b \neq 0$, such as $3/4$, $-5/2$, and $7 = 7/1$.

- $\mathbb{R}$ is the set of *real numbers*: all the numbers on the number line, including some that are not fractions of integers. By Pythagoras' theorem, the diagonal of a square with sides of length 1 has a length whose square is $1^2 + 1^2 = 2$; that length is $\sqrt{2}$, and Theorem 4.3 in Chapter 4 proves that $\sqrt{2}$ is not such a fraction.

Each of these sets contains the one before it.

Four signs compare real numbers: $x < y$ means that x is strictly smaller than y, and $x \le y$ means that x is smaller than or equal to y, that is, at most y; likewise $x > y$ means strictly larger, and $x \ge y$ means at least y. So $3 \le 3$ is true, but $3 < 3$ is false. The signs $<$ and $>$ give *strict* inequalities, and $\le$ and $\ge$ *non-strict* ones. The two kinds disagree only when the two sides are equal, and many mistakes in proofs happen at that boundary case. A chain such as $0 \le x \le 1$ states two conditions at once: $0 \le x$ and $x \le 1$.

A number x is *positive* if $x > 0$, *negative* if $x < 0$, and *nonnegative* if $x \ge 0$. So zero is nonnegative, but neither positive nor negative.

A number whose square is t is called a *square root* of t. We accept without proof that every nonnegative real number t has exactly one nonnegative square root, written $\sqrt{t}$; this is the first of the starting facts listed in *To the reader*. For example, $\sqrt{9} = 3$: both 3 and -3 have square 9, and the square-root sign picks the nonnegative one. A negative number has no real square root, because the square of a real number is never negative.

Because the square-root sign gives a nonnegative answer, $\sqrt{x^2}$ is not always x: $\sqrt{(-3)^2} = \sqrt{9} = 3$. In general, $\sqrt{x^2} = x$ when $x \ge 0$, and $\sqrt{x^2} = -x$ when $x < 0$, because then $-x$ is nonnegative and $(-x)^2 = x^2$. In both cases $\sqrt{x^2}$ is the distance from x to 0 on the number line, which Section 2.6 calls the *absolute value* $|x|$.

Exercises

Exercises 1.1–1.4 are core exercises, and Exercises 1.5 and 1.6 are transfer exercises. Complete solutions are in Appendix A, Section A.1.

Exercise 1.1. Evaluate $2(x + 3)$ and $2x + 3$ at $x = 4$. Explain in words why the two answers differ.

Exercise 1.2. Prove that the sum of an even integer and an odd integer is odd. Name the integer that serves as the witness for the oddness of the sum.

Exercise 1.3. An assistant argues that the product of two odd integers is always odd, because $(2k + 1)^2$ is odd for every integer k. Explain which products of odd integers this argument fails to cover, and then give a complete proof that the product of any two odd integers is odd.

Exercise 1.4. Let $a_1 = 2$, $a_2 = -1$, and $a_3 = 4$. Write out the sum $\sum_{j=1}^{3}(2a_j + 1)$ term by term, and then evaluate it.

Exercise 1.5. Find every real solution of the equation $x(x - 2) = 0$. Then explain why dividing both sides by x does not, on its own, give a complete answer. You may use the fact that a product of two real numbers is zero only when at least one of the two factors is zero.

Exercise 1.6. Someone checks that a formula $f(n)$ is zero for $n = 1, 2, \ldots, 10$ and concludes that $f(n) = 0$ for every positive integer n. Show that this reasoning is unsafe: construct a formula $f(n)$ with $f(n) = 0$ for $n = 1, 2, \ldots, 10$ but $f(11) \neq 0$, and compute $f(11)$. Hint: use a product with one factor for each checked value of n.

Before moving on

Close the book and explain, in your own words, why the sum of two odd integers is even: what the witnesses k and ℓ are, why they get different letters, and why the last step returns to the definition of even. Then say how a definition, a checked example, and a claim about every allowed input differ.

CHAPTER 2

Statements, choices, and assumptions

> **Overview**
> Read exactly what a mathematical statement promises and what it leaves open. Keep track of which choices may depend on which inputs. Negate statements correctly, use counterexamples to test claims, and work with absolute values and squares without losing cases.

2.1 An implication is a conditional promise

In mathematics, a *statement* is a sentence that is either true or false, such as "seven is odd" (true) or "$2 + 2 = 5$" (false). A sentence such as "n is even" becomes a statement only once a value of n is fixed. A question such as "Is 15 odd?" or an instruction such as "Add 3 to both sides" is not a statement at all.

Our first example of an "if . . . , then . . . " statement needs one new word. An integer n is *divisible by* an integer d when $n = dk$ for some integer k. For example, 20 is divisible by four, because $20 = 4 \cdot 5$. But 6 is not: $6 = 4k$ forces $k = 3/2$, which is not an integer. As with even and odd numbers in Section 1.4, the number k is called a witness, and it must be an integer.

Now consider the statement

If an integer is divisible by four, then it is even.

As in Section 1.5, the part after "if" is the *hypothesis*, the condition that is assumed, and the part after "then" is the *conclusion*, what is promised once the hypothesis holds. For an integer n, let P stand for "n is divisible by four" and Q for "n is even." A statement of the form "if P, then Q" is called an *implication*. We write it $P \Rightarrow Q$ and read it "P implies Q." Like any statement, an implication can be true or false. Our sentence makes its promise for every integer n at once; Section 2.3 introduces a symbol for "for every."

The promise is easy to prove. Let n be an integer that is divisible by four. Then $n = 4k$ for some integer k, so $n = 2(2k)$, and $2k$ is an integer. By the definition of even (Definition 1.1), n is even.

The implication makes no promise about an integer that is not divisible by four, which may be even, like 6, or odd, like 7. A common error is to run it backward. The backward statement $Q \Rightarrow P$ is called the *converse* of $P \Rightarrow Q$. Here the converse says "if an integer is even, then it is divisible by four," and it is false, since 6 is even but not divisible by four.

A particular input that shows a statement about every input to be false is called a *counterexample*. For an implication, it is an input for which the hypothesis is true and the conclusion is false. So 6 is a counterexample to the converse above, and one counterexample is enough, because the converse claims to hold for *every* integer. An input such as 7, which does not satisfy the hypothesis of

"divisible by four implies even," can never be a counterexample to it. When you ask an assistant to test a claim, ask for inputs that satisfy the hypothesis; inputs chosen at random often miss it and so test nothing.

Sometimes an implication and its converse are both true. The statement $P \Leftrightarrow Q$, read "P if and only if Q," promises both $P \Rightarrow Q$ and $Q \Rightarrow P$. Here "P if Q" means $Q \Rightarrow P$, while "P only if Q" says that P can hold only when Q holds as well, which is $P \Rightarrow Q$. For example, "an integer is divisible by four only if it is even" means "divisible by four implies even." When $P \Leftrightarrow Q$ holds, we say that P and Q are *equivalent*. To prove an "if and only if" statement, we prove the two implications separately. Here is one such statement: for every integer n,

$$n \text{ is even if and only if } n^2 \text{ is even.}$$

One direction is a short calculation: if $n = 2k$ for an integer k, then $n^2 = 2(2k^2)$, which is twice an integer. The other direction is harder, because knowing that $n^2 = 2w$ for an integer w gives no direct way to write n itself as twice an integer. Section 2.5 describes the idea that gets around this.

A result applies only after its hypothesis is checked

To use "divisible by four implies even" for 20, we must first check the hypothesis, $20 = 4 \cdot 5$; quoting the implication does not supply this evidence. Suppose an assistant writes: "30 is even, because every integer divisible by four is even." The conclusion is true, but the reason is not: 30 is not divisible by four ($30 = 4k$ would force $k = 15/2$), so the result says nothing about 30. A correct reason is $30 = 2 \cdot 15$.

Necessary and sufficient, stronger and weaker

In a true implication $P \Rightarrow Q$, we say that P is a *sufficient* condition for Q: knowing P is enough to guarantee Q. We also say that Q is a *necessary* condition for P: P cannot hold unless Q holds too. So divisibility by four is sufficient for evenness, and evenness is necessary for divisibility by four, but not sufficient for it, as 6 shows. When $P \Leftrightarrow Q$ holds, P is both necessary and sufficient for Q.

When every object satisfying P also satisfies Q, but not the other way around, we call P the *stronger* condition and Q the *weaker* one; the stronger condition demands more. Divisibility by four is stronger than evenness. Whole statements are compared in the same way: a stronger statement promises more. For instance, a statement about every integer is stronger than the same statement about positive integers only.

Pause and decide. Consider the claim "If an integer is divisible by six, then it is divisible by three." For each of 12, 9, and 7, decide whether it could be a counterexample. Is the claim true? Is its converse true?

Check. A counterexample would have to be divisible by six but not by three. The number 12 is divisible by six and also by three, so it agrees with the claim. Neither 9 nor 7 is divisible by six, so neither can be a counterexample. The claim is true: if $n = 6k$ for an integer k, then $n = 3(2k)$. Its converse, "divisible by three implies divisible by six," is false, with counterexample 9: $9 = 3 \cdot 3$, but $9 = 6k$ would force $k = 3/2$.

2.2 And, or, and not

Statements can be combined into longer ones. Throughout this section, n is an integer, P is the statement "n is even," and Q is the statement "n is divisible by three." The letters P and Q stand for whole statements, not for numbers.

The *conjunction* $P \wedge Q$, read "P and Q," is true when both P and Q are true. The *disjunction* $P \vee Q$, read "P or Q," is true when at least one of them is true, including when both are: for $n = 6$, which is even and divisible by three, $P \vee Q$ is true. In everyday speech, "tea or coffee" often means one but not both; in mathematics, "or" allows both unless the author says otherwise. Finally, the *negation* $\neg P$, read "not P," is true exactly when P is false.

The table works these out for four integers, one for each way in which P and Q can be true (T) or false (F).

n	P	Q	$P \wedge Q$	$P \vee Q$	$\neg(P \vee Q)$	$(\neg P) \wedge (\neg Q)$
6	T	T	T	T	F	F
4	T	F	F	T	F	F
3	F	T	F	T	F	F
5	F	F	F	F	T	T

The last two columns apply the same rules to the earlier columns: $\neg(P \vee Q)$ is the opposite of the $P \vee Q$ column, and $(\neg P) \wedge (\neg Q)$ is true only in a row where P and Q are both false. These two columns agree in every row:

$$\neg(P \vee Q) \quad \Longleftrightarrow \quad (\neg P) \wedge (\neg Q).$$

In words: an integer fails to be "even or divisible by three" exactly when it is not even *and* not divisible by three. To show that 5 lacks this property, it is not enough to show that 5 is not even.

By contrast, an integer fails to be "even and divisible by three" as soon as one of the two requirements fails: 4 fails because it is not divisible by three, and 3 because it is not even. In symbols,

$$\neg(P \wedge Q) \quad \Longleftrightarrow \quad (\neg P) \vee (\neg Q).$$

These two rules are called *De Morgan's laws*. The long arrow $\Longleftrightarrow$ is the "if and only if" of Section 2.1: the statements on its two sides are true in exactly the same situations.

The table proves the first law for *any* two statements P and Q, not only for these four integers: its last two columns were filled in using only the truth values of P and Q in the same row, and there are only four combinations of these, one per row. The second law can be checked in the same way: each side is false only in the row where P and Q are both true, since only there are $\neg P$ and $\neg Q$ both false.

The same bookkeeping describes how an implication fails, for any two statements P and Q. The implication $P \Rightarrow Q$ fails exactly when its hypothesis holds and its conclusion fails:

$$\neg(P \Rightarrow Q) \quad \Longleftrightarrow \quad P \wedge (\neg Q).$$

This is the description of a counterexample from Section 2.1. In the other three situations the implication counts as true:

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

The last two rows surprise many people. Think of the promise "if it rains, I will bring an umbrella." On a dry day the promise is not broken, whether I bring an umbrella or not. Likewise, "if n is divisible by four, then n is even" is not broken by $n = 6$ or by $n = 7$: neither is divisible by four, so at both values the implication is true. Section 2.5 looks at what this means when the hypothesis can never be true.

Parentheses and grouping

Parentheses show which statements belong together, and moving them can change the meaning. With P and Q as at the start of this section, take $n = 5$: the statement $\neg(P \wedge Q)$ is true, because 5 is not both even and divisible by three, but $(\neg P) \wedge Q$ is false, because 5 is not divisible by three. By convention, $\neg$ applies only to the statement immediately after it, so $\neg P \wedge Q$ means $(\neg P) \wedge Q$; we usually write such parentheses anyway.

2.3 The order of choices changes the meaning

A *universal* statement, such as "every integer divisible by four is even," claims that every allowed input has a property. A proof cannot check infinitely many integers one at a time; instead, as in the proof of Proposition 1.2, it works with an *arbitrary* input, one about which we know nothing except that it is allowed, so the same argument applies to each input. An *existence* statement, such as "some integer is both even and divisible by three," claims that at least one allowed input has the property. A single example, such as 6, proves it; there may be others, such as 12.

The symbol $\forall$ means "for every," and the symbol $\exists$ means "there exists" or "there is at least one." They are called *quantifiers*, because they say for how many values of a letter a condition must hold. We write $x \in \mathbb{R}$, read "x in $\mathbb{R}$," to say that x belongs to the set $\mathbb{R}$ of real numbers. The set that a quantified letter ranges over is called its *domain*; in "$\forall x \in \mathbb{R}$" the domain of x is $\mathbb{R}$. When several quantifiers appear in one statement, their order matters. Compare

$$\forall x \in \mathbb{R}\ \exists y \in \mathbb{R}: \quad y > x, \tag{A}$$

$$\exists y \in \mathbb{R}\ \forall x \in \mathbb{R}: \quad y > x. \tag{B}$$

Read aloud, (A) says "for every real number x, there is a real number y such that $y > x$," and (B) says "there is a real number y such that, for every real number x, $y > x$." The colon separates the quantifiers from the condition that the chosen values must satisfy.

It helps to read a quantified statement as a game between a skeptic, who doubts the statement, and a prover, who defends it. The quantifiers are read from left to right, and each one is a move: at a $\forall$, the skeptic picks any member of the domain, and at an $\exists$, the prover picks a value. The prover wins if the condition after the colon holds for the values picked, and the statement is true exactly when the prover has a way to win however the skeptic plays.

Statement (A) is true. The skeptic picks x first, and the prover may then respond with a y that suits it: $y = x + 1$ works. The response depends on the input: for $x = 3$ the prover answers 4, and for $x = -10$ it answers -9. Statement (B) is false. The prover must commit to a single y *first*, and whatever y is chosen, the skeptic's reply $x = y + 1$ defeats it, because $y > y + 1$ is false.

In general, a value chosen at an $\exists$ may depend on every value chosen before it, but not on anything chosen after it; this is the dependence of Section 1.7, where the witnesses k and ℓ were determined by a and b. Swapping two neighboring quantifiers of the same kind, as in $\forall x \in \mathbb{R}\ \forall y \in \mathbb{R}$, does not change the meaning, as long as the domain of neither letter refers to the other. (In "for every integer N and every integer $n \geq N$," the domain of n depends on N, so N must come first.) When a proof that chooses several quantities becomes confusing, ask: *at what stage is this quantity chosen, and which other quantities may it depend on?*

Tolerances and the inequality rules they need

In later chapters we often need one number to be within some allowed error of another. A *tolerance* is a positive amount of error that we allow, traditionally written ε, the Greek letter epsilon. A statement that begins "for every positive ε," written $\forall \varepsilon > 0$, must hold for $\varepsilon = 1/10$, for $\varepsilon = 1/1000$, and for every other positive real number.

Tolerances lead to inequalities, where the sign of a number matters. Three rules cover almost everything we need.

- Adding the same number to both sides keeps the direction of an inequality: from $2 < 5$ we get $2 + 7 < 5 + 7$.
- Multiplying or dividing both sides by the same *positive* number keeps the direction: multiplying $2 < 5$ by 3 gives $6 < 15$.
- Multiplying or dividing both sides by the same *negative* number reverses the direction: multiplying $2 < 5$ by -1 gives $-2 > -5$.

The same three rules hold with $\leq$ in place of $<$.

The *reciprocal* of a nonzero number a is $1/a$. Suppose $0 < a < b$. The product ab is positive, so by the second rule we may divide both sides of $a < b$ by ab:

$$\frac{a}{ab} < \frac{b}{ab}, \qquad \text{that is,} \qquad \frac{1}{b} < \frac{1}{a}.$$

So for positive numbers, taking reciprocals reverses the order: from $2 < 5$ we get $1/5 < 1/2$. The same argument shows that $0 < a \leq b$ implies $1/b \leq 1/a$. The rule can fail when the numbers are not both positive: $-1 < 2$, but $1/(-1) = -1$ is smaller than $1/2$.

Given a tolerance ε, we will often want a positive integer N with $1/N < \varepsilon$, and for this it is

enough that $N > 1/\varepsilon$. Such an integer always exists, by the following property of the real numbers, which we accept without proof; it is the second of the starting facts listed in *To the reader*.

The Archimedean property: for every real number r, there is an integer N with $N > r$.

Example 2.1. We prove that for every positive real number ε, there is a positive integer N with $1/N < \varepsilon$. Let ε be a positive real number. By the Archimedean property, there is an integer N with $N > 1/\varepsilon$. This N is positive, since $N > 1/\varepsilon > 0$ (divide both sides of $1 > 0$ by ε to get $1/\varepsilon > 0$). As $1/\varepsilon$ and N are both positive, taking reciprocals reverses the inequality $1/\varepsilon < N$:

$$\frac{1}{N} < \frac{1}{1/\varepsilon} = \varepsilon.$$

The order of the quantifiers allows N to depend on ε, and it has to: a smaller tolerance may need a larger N. For instance, if $\varepsilon = 0.003$, then $1/\varepsilon = 333.33\ldots$, so $N = 334$ works, with $1/334 = 0.002994\ldots$, while $N = 333$ fails, because $1/333 = 0.003003\ldots$ is larger than 0.003.

Pause and decide. Can a single positive integer N satisfy $1/N < \varepsilon$ for every positive ε?

Check. No. Once N is fixed, the skeptic may choose the positive number $\varepsilon = 1/(2N)$. Then $1/N = 2\varepsilon > \varepsilon$, so $1/N < \varepsilon$ fails. This claim has the shape of (B): one N chosen before ε. The true statement in Example 2.1 has the shape of (A): N is chosen after ε is known.

2.4 Negation: describing how a statement fails

To negate statements with quantifiers, we write $P(x)$ for a statement about x that becomes true or false once a value of x is chosen. For example, if $P(x)$ is "$x^2 > 4$," then $P(3)$ is the true statement $9 > 4$, and $P(1)$ is the false statement $1 > 4$. The rules below work whatever the domain of x is, so in stating them we leave out the domain and the colon.

A universal statement $\forall x\ P(x)$ is false exactly when at least one x fails to satisfy $P(x)$, and such an x is a counterexample. An existence statement $\exists x\ P(x)$ is false exactly when every x fails. In symbols,

$$\neg(\forall x\ P(x)) \iff \exists x\ \neg P(x), \qquad \neg(\exists x\ P(x)) \iff \forall x\ \neg P(x).$$

For example, over the real numbers, $\forall x\ (x^2 > 4)$ is false, and $x = 1$ is both a counterexample to it and a witness for its negation $\exists x\ \neg(x^2 > 4)$. On the other hand, $\exists x\ (x^2 > 4)$ is true, with witness $x = 3$, so its negation $\forall x\ \neg(x^2 > 4)$ is false. In both negations the domain stays the same: the negation of a statement about every real number is a statement about some real number, not about things that are not real numbers.

To negate a statement with several quantifiers, apply these rules one quantifier at a time, from left to right. Here is the negation of statement (A) from Section 2.3:

$$\begin{aligned}\neg\big(\forall x \in \mathbb{R}\ \exists y \in \mathbb{R} :\ y > x\big) &\iff \exists x \in \mathbb{R}\ \neg\big(\exists y \in \mathbb{R} :\ y > x\big)\\ &\iff \exists x \in \mathbb{R}\ \forall y \in \mathbb{R} :\ \neg(y > x)\\ &\iff \exists x \in \mathbb{R}\ \forall y \in \mathbb{R} :\ y \le x.\end{aligned}$$

In the first step, the whole statement "$\exists y \in \mathbb{R} :\ y > x$" plays the role of $P(x)$, since it becomes true or false once x is chosen. In the second, x is fixed and "$y > x$" is the statement about y. The third negates the final condition. So each $\forall$ became an $\exists$ and each $\exists$ a $\forall$, the quantifiers kept their order and their domains, and the condition after the colon was negated. The result claims that some real number x is at least as large as every real number; such an x would be an *upper bound* for all the real numbers. This negation is false, as it must be, since (A) is true.

Notice that $y > x$ became $y \leq x$, not $y < x$. When $y > x$ is false, two possibilities remain, $y < x$ and $y = x$, and $y \leq x$ covers both: the failure of a strict inequality includes the case of equality.

Pause and decide. Negate the statement "Every real number x satisfies $x^2 \geq x$." Which is true, the statement or its negation?

Check. The negation is "There is a real number x with $x^2 < x$": "every" became "there is," the domain is still the real numbers, and $\geq$ became $<$. The negation is true, with witness $x = 1/2$, since $(1/2)^2 = 1/4 < 1/2$. So the original statement is false.

One acceptable task each, or different tasks for all?

The order of choices matters outside arithmetic too. Suppose a class has some students and a list of tasks, and each student either finds each task acceptable or does not. Compare "for every student, there is an acceptable task" with the stronger statement "all the students can be given different tasks, each acceptable to the student who receives it." In the first statement, each student's task is chosen by looking at that student alone, so two students may be offered the same task. The second asks for a single assignment, chosen once, that works for all the students at the same time. This is the contrast between (A) and (B) again, and the first statement can be true while the second is false; Exercise 2.5 asks you to build an example. Chapters 8 and 9 work out exactly when such an assignment exists; the answer, Hall's theorem, is one of the main results of this book.

A negation worked out slowly

We negate the statement

For every real number x, there is an integer m that is divisible by three and satisfies $m > x$.

Fix a real number x. A proposed response m succeeds only if it meets *both* requirements: m is divisible by three, and $m > x$. By De Morgan's law, m fails when it is not divisible by three or $m \leq x$. The whole statement fails if, for at least one x, *every* integer m fails. So the negation is

There is a real number x such that every integer m fails at least one requirement: m is not divisible by three, or $m \leq x$.

In symbols, the original statement is

$$\forall x \in \mathbb{R}\ \exists m \in \mathbb{Z} : \quad (m \text{ is divisible by } 3) \wedge (m > x),$$

and its negation is

$$\exists x \in \mathbb{R}\ \forall m \in \mathbb{Z}: \quad \neg(m \text{ is divisible by } 3) \vee (m \leq x).$$

Negating a statement describes what its failure would look like; it does not tell us whether failure happens. Here the original statement is true, so its negation is false: given a real number x, the Archimedean property gives an integer $k > x/3$, and multiplying by 3 shows that $m = 3k$, which is divisible by three, satisfies $m > x$.

Describing the failure in plain words first, as here, is also the way to check a negation that you or an assistant produced by flipping symbols: a sentence can look correct and still describe the wrong kind of failure.

2.5 Contraposition and vacuous truth

The *contrapositive* of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$: if the conclusion fails, then the hypothesis must have failed too. An implication and its contrapositive are *logically equivalent*: each is true exactly when the other is, whatever the statements P and Q are. "Logically" signals that this follows from the form of the statements alone, as the following argument shows.

- Suppose $P \Rightarrow Q$ is true, and suppose Q fails. If P held, the implication would force Q to hold, which it does not. So P fails. This proves $\neg Q \Rightarrow \neg P$.
- Suppose $\neg Q \Rightarrow \neg P$ is true, and suppose P holds. If Q failed, the contrapositive would force P to fail, which it does not. So Q holds. This proves $P \Rightarrow Q$.

The second argument concludes that Q holds because the possibility that Q fails has been ruled out. That step uses the principle that every statement is either true or false, so ruling out one possibility leaves the other. Ordinary mathematics accepts this principle, and so do we.

Pause and decide. Use the rule $\neg(P \Rightarrow Q) \Leftrightarrow P \wedge (\neg Q)$ from Section 2.2 to describe exactly when the contrapositive $\neg Q \Rightarrow \neg P$ is false. Compare this with the situation in which $P \Rightarrow Q$ is false.

Check. The contrapositive is false exactly when its hypothesis $\neg Q$ is true and its conclusion $\neg P$ is false, that is, when Q is false and P is true. This is the same situation in which $P \Rightarrow Q$ is false. Two statements that fail in exactly the same situations are equivalent.

Do not confuse the contrapositive with the converse. "Divisible by four implies even" is equivalent to its contrapositive, "not even implies not divisible by four": for example, 7 is not even, so 7 is not divisible by four. It is *not* equivalent to its converse, "even implies divisible by four," which fails at $n = 6$.

The contrapositive is often the easier version to prove. Section 4.2 proves the harder direction of "n is even if and only if n^2 is even" (Lemma 4.2), namely "if n^2 is even, then n is even," through its contrapositive, "if n is not even, then n^2 is not even." The new hypothesis is about n itself rather than about n^2, and since an integer that is not even is odd (Lemma 4.1), it lets us write $n = 2k + 1$, a form we can calculate with.

Vacuous truth

What happens when the hypothesis of an implication can never hold? Consider the statement

Every integer that is both even and odd is larger than 100.

As an implication: for every integer n, if n is both even and odd, then $n > 100$. A counterexample would have to be an integer that is both even and odd and at most 100. But no integer is both even and odd: if $n = 2k$ and $n = 2\ell + 1$ for integers k and ℓ, then $2(k - \ell) = 1$, so $k - \ell = 1/2$, which is impossible because $k - \ell$ is an integer and $1/2$ is not. So there is no counterexample, and the statement counts as true, in agreement with Section 2.2: an implication fails only when its hypothesis holds and its conclusion fails.

A statement of this kind is called *vacuously true*: it is true because there is nothing for it to be about. It does not tell us that an integer which is both even and odd exists, nor anything about integers larger than 100; "every integer that is both even and odd is smaller than 0" is vacuously true as well.

A theorem whose hypotheses can never all hold at once is vacuously true in the same way: its proof may be correct and still tell us nothing. Suppose an assistant proves a result about "every real number x with $x^2 < 0$." No real number has a negative square, so the result is vacuously true however impressive its conclusion looks. Before relying on a theorem, check that at least one object satisfies all of its hypotheses. Vacuous truth returns in Chapter 3, where it explains why the empty set, the set with no members, is contained in every set.

2.6 Absolute value, squares, and intervals

The *absolute value* $|x|$ of a real number x is its distance from zero on the number line. Thus $|3| = |-3| = 3$ and $|0| = 0$. As a formula,

$$|x| = \begin{cases} x & \text{if } x \geq 0, \\ -x & \text{if } x < 0. \end{cases}$$

In the second case the result is positive, since the negative of a negative number is positive: $|-3| = -(-3) = 3$. So $|x| \geq 0$ for every x. Section 1.8 found that $\sqrt{x^2}$ is x when $x \geq 0$ and $-x$ when $x < 0$, so $\sqrt{x^2} = |x|$.

For all real numbers c and x,

$$|cx| = |c|\,|x|.$$

We check this case by case, according to the signs of c and x.

- If $c = 0$ or $x = 0$, then $cx = 0$, and both sides are zero.
- If $c > 0$ and $x > 0$, then $cx > 0$, and both sides equal cx.
- If $c < 0$ and $x < 0$, then $cx > 0$, so $|cx| = cx$, while $|c|\,|x| = (-c)(-x) = cx$.
- If one of c and x is positive and the other negative, then $cx < 0$, so $|cx| = -cx$, while $|c|\,|x|$ is $c(-x)$ if $c > 0$ and $(-c)x$ if $c < 0$, and both equal $-cx$.

Taking $c = -1$ gives $|-x| = |x|$: changing the sign of a number does not change its distance from zero.

For a number $b \geq 0$, the statement $|x| \leq b$ says that x is at distance at most b from zero:

$$|x| \leq b \quad \Longleftrightarrow \quad -b \leq x \leq b.$$

To check this, use the two cases in the formula for $|x|$. If $x \geq 0$, then $|x| \leq b$ says $x \leq b$, and $-b \leq x$ holds automatically, since $-b \leq 0 \leq x$. If $x < 0$, then $|x| \leq b$ says $-x \leq b$, which becomes $x \geq -b$ after multiplying by -1, and $x \leq b$ holds automatically, since $x < 0 \leq b$. The same argument with strict inequalities shows that, for $b > 0$,

$$|x| < b \quad \Longleftrightarrow \quad -b < x < b.$$

Both rules apply to a difference $x - c$ in place of x. The number $|x - c|$ is the distance between x and c on the number line; for instance, $|7 - 5| = |5 - 7| = 2$. For $b > 0$, the second rule turns $|x - c| < b$ into $-b < x - c < b$, and adding c to each part gives $c - b < x < c + b$: the number x lies less than b away from c. For example, $|x - 5| < 1$ means $4 < x < 6$.

Now take $b = 2$: $|x| \leq 2$ is the conjunction "$-2 \leq x$ and $x \leq 2$." By De Morgan's law, its negation $|x| > 2$ is the disjunction of the negations of the two parts, "$x < -2$ or $x > 2$": a condition with two separate pieces, one on each side of zero. The same two-sided form appears when an inequality involves a square.

We need two facts about squares. First, squaring preserves order for nonnegative numbers. Suppose $0 \leq a < b$. Multiplying out gives

$$(b - a)(b + a) = b^2 + ba - ab - a^2 = b^2 - a^2.$$

Both factors on the left are positive: $b - a > 0$ because $a < b$, and $b + a > 0$ because $b > a \geq 0$. So $b^2 - a^2 > 0$, that is, $a^2 < b^2$. In the same way, if $0 \leq a \leq b$, then both factors are nonnegative, so $a^2 \leq b^2$. The condition that the numbers are nonnegative cannot be dropped: $-3 < 2$, but $(-3)^2 = 9$ is larger than $2^2 = 4$.

Second, $x^2 = |x|^2$ for every real number x. If $x \geq 0$, this says $x^2 = x^2$. If $x < 0$, it says $x^2 = (-x)^2$, which is true because $(-x)(-x) = x \cdot x$.

Now we can solve the inequality $x^2 > 4$ completely, by comparing $|x|$ with 2.

- If $|x| > 2$, the squaring rule for $<$ with $a = 2$ and $b = |x|$ gives $4 < |x|^2 = x^2$.
- If $|x| \leq 2$, the squaring rule for $\leq$ with $a = |x|$ and $b = 2$, allowed because $|x| \geq 0$, gives $x^2 = |x|^2 \leq 4$.

The first case shows that $|x| > 2$ implies $x^2 > 4$. The second shows that $|x| \leq 2$ implies $x^2 \leq 4$, which is the contrapositive of "$x^2 > 4$ implies $|x| > 2$." Together they prove

$$x^2 > 4 \quad \Longleftrightarrow \quad |x| > 2 \quad \Longleftrightarrow \quad x < -2 \text{ or } x > 2.$$

For instance, $x = -3$ satisfies $x^2 = 9 > 4$ even though x is not greater than 2. The tempting shortcut "take square roots to get $x > 2$" loses the whole negative half of the answer.

Ranges of real numbers like these have a standard notation. For real numbers $a < b$, each of the following sets is called an *interval*:

- $[a, b]$ is the set of all real x with $a \le x \le b$;
- (a, b) is the set of all real x with $a < x < b$;
- $[a, b)$ is the set of all real x with $a \le x < b$, and $(a, b]$ is the set of all real x with $a < x \le b$.

A square bracket includes its endpoint, and a round bracket excludes it: 1 and 3 belong to $[1, 3]$ but not to $(1, 3)$. An interval may also have no endpoint on one side: $[0, \infty)$ is the set of real x with $x \ge 0$, and $(-\infty, -2)$ the set of real x with $x < -2$. The symbol ∞, read "infinity," is not a real number; it only says that there is no endpoint on that side, so it never goes next to a square bracket. In this notation, $|x| \le 2$ says that x belongs to $[-2, 2]$, and $x^2 > 4$ says that x belongs to $(-\infty, -2)$ or to $(2, \infty)$. Round brackets are also used for an *ordered pair*, such as $(1, 3)$, which lists two entries in a definite order (Chapter 3); the surrounding sentence tells you which is meant.

> **Habit 2: keep a statement contract**
>
> Before working on a statement, write down its *contract*: the domain of each variable, the hypotheses, the conclusion, and the order in which the quantities are chosen. Whenever the statement changes, for example when you or an assistant repair or generalize it, compare the new version with the contract: did a hypothesis or the conclusion become weaker or stronger? A changed statement can be true and still fail to answer the original question. In Section 1.1, an assistant that replaced "integers" by "positive integers" gave a correct proof of a weaker statement, one that says nothing about a sum such as $-3 + 5$.

Exercises

Exercises 2.1–2.4 are core exercises, and Exercises 2.5 and 2.6 are transfer exercises. Complete solutions are in Appendix A, Section A.2.

Exercise 2.1. For a real number x, consider the implication "If $x = 3$, then $x^2 = 9$." State its converse and its contrapositive. Decide which of the three statements are true, and justify each answer.

Exercise 2.2. Negate the statement "For every integer n, there is an integer m with $m > n$ and m even." Keep the domain of each variable unchanged, and take care when you negate the "and." Give the negation both in words and in symbols.

Exercise 2.3. Decide whether each of the statements

$$\forall x \in \mathbb{R}\ \exists y \in \mathbb{R} :\ x + y = 0 \qquad \text{and} \qquad \exists y \in \mathbb{R}\ \forall x \in \mathbb{R} :\ x + y = 0$$

is true. For a true statement, explain how to choose y. For a false statement, show that no choice of y works.

Exercise 2.4. Find a counterexample to the claim "If a real number x satisfies $x^2 > 4$, then $x > 2$." Then repair the claim by changing only its conclusion, so that the repaired claim is true for every real number x. Choose a conclusion that describes exactly which real numbers satisfy $x^2 > 4$.

Exercise 2.5. Give an example of two students and a list of tasks in which each student accepts at least one task, but the two students cannot be given different acceptable tasks. Then find a condition on *groups* of students that is stronger than "each student accepts at least one task" and can be checked by counting. It should fail in your example, and it should hold whenever all the students can be given different acceptable tasks. Explain why it must hold in that case. (Hint: for a group of students, count the tasks that at least one member of the group accepts, and compare with the number of students in the group.)

Exercise 2.6. For each positive real number ε, show how to choose a positive integer N such that every integer $n \geq N$ satisfies $1/n < \varepsilon$. Explain why N may depend on ε, and why it must not depend on the integer n that is tested afterwards.

Before moving on

Close the book and check that you can do the following in your own words. State the converse and the contrapositive of "if an integer is divisible by four, then it is even," and say which of the three are true and why. Explain, using choices made in order, why statement (A) of Section 2.3 is true and statement (B) is false; then write the negation of (A) and say what it claims. Explain why the negation of an "and" statement is an "or" statement, and why "every integer that is both even and odd is larger than 100" is true but tells us nothing about any integer. Show that for every positive ε some positive integer N satisfies $1/N < \varepsilon$, and say why N must be allowed to depend on ε. Finally, find all real numbers x with $x^2 > 4$ without losing the negative ones.

These skills return in the definition of a limit (Chapter 11), whose meaning depends on the order of three quantifiers, in Hall's theorem (Chapters 8 and 9), and in the proof assistant Lean (Chapters 15 and 16), which usually proves an existence statement by supplying an actual witness.

CHAPTER 3

Objects, maps, and information

Overview

Use sets to say exactly which objects a question is about, and functions to describe how inputs become outputs. Prove facts about sets and functions by unpacking their definitions, one membership condition at a time. Decide whether a function loses information by sending different inputs to the same output, and notice when a proof needs a property only at the points it actually uses.

3.1 Sets express the boundary of a question

Section 1.1 suggested that the first question to ask about any claim is: which objects does it talk about? A *set* records the answer. It is a collection of objects, called its *members* or *elements*, and it is completely determined by which objects belong to it. So listing a member twice changes nothing, $\{1, 1, 3\} = \{1, 3\}$, and neither does the order of the list: $\{3, 1\} = \{1, 3\}$.

We write $x \in A$, read "x is in A," when x is a member of A, and $x \notin A$ when it is not; for example, $3 \in \{1, 3, 5\}$ and $2 \notin \{1, 3, 5\}$. The *empty set* $\varnothing$ has no members at all. We write $A \subseteq B$, read "A is a *subset* of B" or "A is contained in B," when every member of A is also a member of B. For example, $\{1, 3\} \subseteq \{1, 2, 3\}$, but $\{1, 4\}$ is not a subset of $\{1, 2, 3\}$, because 4 is missing from the second set. Every set is a subset of itself.

To prove $A \subseteq B$, begin with "Let $x \in A$," where x is *arbitrary*: as in Chapter 1, it stands for any member of A, so the argument may use only the fact that $x \in A$. Then show that $x \in B$, without assuming it along the way, since deriving it is the whole point. Because x was arbitrary, this covers every member of A.

Three operations build new sets from old ones. The *union* $A \cup B$ consists of the objects in at least one of A and B; this is the inclusive "or" of Section 2.2, so an object in both sets is in the union. The *intersection* $A \cap B$ consists of the objects in both A and B. The *difference* $A \setminus B$ consists of the objects in A but not in B. For $A = \{1, 2, 3\}$ and $B = \{3, 4\}$,

$$A \cup B = \{1, 2, 3, 4\}, \qquad A \cap B = \{3\}, \qquad A \setminus B = \{1, 2\}.$$

Order matters for a difference, since $B \setminus A = \{4\}$, but not for a union or an intersection.

To describe a set by a property instead of a list, we write $\{x \in A : P(x)\}$ for the set of members of A that have the property P; read the colon as "such that." For example, $\{n \in \mathbb{Z} : n \text{ is even}\}$ is the set of even integers, and $\{x \in \mathbb{R} : x^2 \leq 4\}$ is the interval $[-2, 2]$, because $x^2 \leq 4$ exactly when $-2 \leq x \leq 2$ (Section 2.6).

Membership, containment, and equality

The symbols $\in$ and $\subseteq$ are easy to confuse, but they connect different kinds of objects: $\in$ has an object on its left and a set on its right, while $\subseteq$ has a set on both sides. Thus $1 \in \{1, 2\}$ says that the number 1 is a member, and $\{1\} \subseteq \{1, 2\}$ says that every member of the set $\{1\}$ is a member. Both are true; in general, $x \in B$ exactly when $\{x\} \subseteq B$, since the only member of $\{x\}$ is x. But the symbols cannot be exchanged. The statement $\{1\} \in \{1, 2\}$ is false, because neither member of $\{1, 2\}$ is the set $\{1\}$, and $1 \subseteq \{1, 2\}$ has no meaning, because 1 is not a set.

Two sets are *equal* when they have exactly the same members. So to prove $D = E$, we prove the two containments $D \subseteq E$ and $E \subseteq D$.

The empty set is a subset of every set E. A counterexample to $\varnothing \subseteq E$ would be a member of $\varnothing$ that is not in E, and since $\varnothing$ has no members, there is none. This is the set version of vacuous truth from Section 2.5. Likewise, a proof beginning "Let $x \in D$" remains valid when D is empty: it says what happens whenever a member is supplied, and never claims that one exists.

Try a first set equality on small sets: for $A = \{1, 2, 3\}$, $B = \{2, 4\}$, and $C = \{3, 4\}$, we get $A \cap (B \cup C) = A \cap \{2, 3, 4\} = \{2, 3\}$, and $(A \cap B) \cup (A \cap C) = \{2\} \cup \{3\}$ is again $\{2, 3\}$. The proof explains why the two sides agree for all sets.

Proposition 3.1. *For any sets A, B, C, we have $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.*

Proof. We prove the two containments separately.

First let $x \in A \cap (B \cup C)$. Then $x \in A$ and $x \in B \cup C$, so $x \in B$ or $x \in C$. If $x \in B$, then $x \in A \cap B$; if $x \in C$, then in the same way $x \in A \cap C$. In either case $x \in (A \cap B) \cup (A \cap C)$. Since x was an arbitrary member of $A \cap (B \cup C)$, this proves $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$.

Conversely, let $x \in (A \cap B) \cup (A \cap C)$. Then $x \in A \cap B$ or $x \in A \cap C$. In the first case $x \in A$ and $x \in B$; in the second, $x \in A$ and $x \in C$. In both cases $x \in A$, and x lies in B or in C, that is, $x \in B \cup C$. Thus $x \in A \cap (B \cup C)$, which proves the reverse containment, and with it the equality. □

Pause and decide. For $A = \{1, 2, 3\}$, $B = \{2, 4\}$, and $C = \{3, 4\}$, compare $A \setminus (B \cup C)$ with $(A \setminus B) \cap (A \setminus C)$. Do you expect the two sides to agree for all sets?

Check. Here $A \setminus (B \cup C) = A \setminus \{2, 3, 4\} = \{1\}$, and $(A \setminus B) \cap (A \setminus C) = \{1, 3\} \cap \{1, 2\} = \{1\}$. The sides agree for all sets. An object x belongs to $A \setminus (B \cup C)$ when $x \in A$ and "$x \in B$ or $x \in C$" is false. By De Morgan's law $\neg(P \vee Q) \Leftrightarrow (\neg P) \wedge (\neg Q)$ (Section 2.2), this says that $x \in A$, $x \notin B$, and $x \notin C$, which is exactly the condition for membership in $(A \setminus B) \cap (A \setminus C)$.

Counting members and forming pairs

A set is *finite* if its members can be written down in a list that ends, and *infinite* otherwise, like $\mathbb{N}$ and $\mathbb{Z}$. For a finite set S, the *size* or *cardinality* $|S|$ is the number of its members. For example, $|\{2, 4, 6\}| = 3$ (we count the members, we do not add them up), and $|\varnothing| = 0$. Bars around a real number mean its absolute value (Section 2.6); the kind of object inside the bars tells you which meaning is intended.

Two sets A and B are *disjoint* if they have no member in common, that is, if $A \cap B = \varnothing$. For disjoint finite sets, $|A \cup B| = |A| + |B|$, because each member of the union lies in exactly one of the two sets. For overlapping sets the sum counts too much: $|\{1, 2\}| + |\{2, 3\}| = 4$, but $|\{1, 2, 3\}| = 3$, because the common member 2 is counted twice. In general, $|A| + |B|$ counts every member of $A \cap B$ twice and every other member of $A \cup B$ once, so

$$|A \cup B| = |A| + |B| - |A \cap B| \qquad \text{for all finite sets } A, B.$$

When order matters, we use an *ordered pair* (a, b), with first entry a and second entry b. Two ordered pairs are equal exactly when their first entries are equal and their second entries are equal. So $(1, 3) \neq (3, 1)$, although $\{1, 3\} = \{3, 1\}$. For example, the winner and the runner-up of a race form an ordered pair, because swapping them changes the result.

The *Cartesian product* $A \times B$ is the set of all ordered pairs (a, b) with $a \in A$ and $b \in B$. If $A = \{1, 2\}$ and $B = \{p, q\}$, where p and q are two different objects, then

$$A \times B = \{(1, p), (1, q), (2, p), (2, q)\}.$$

Each of the $|A|$ choices of first entry combines with each of the $|B|$ choices of second entry, so $|A \times B| = |A| \cdot |B|$ for finite sets. Despite the multiplication sign, $A \times B$ is a set of pairs; only its *size* is a product of numbers. Products describe inputs made of two pieces: adding two real numbers takes a pair (x, y) from $\mathbb{R} \times \mathbb{R}$ and returns $x + y$.

3.2 A function includes its input and output sets

A *function* $f : A \to B$, read "f from A to B," assigns to each input $x \in A$ exactly one output $f(x) \in B$. The set A of allowed inputs is the *domain* of f. The set B, in which the outputs are required to lie, is the *codomain*, and its elements are called *targets*. The words *map* and *mapping* mean the same as function.

Not every target needs to occur as an output. The outputs that do occur form the *image* of f,

$$f(A) = \{f(x) : x \in A\},$$

the set of all values $f(x)$ obtained as x runs through A. For example, let $A = \{p, q, r\}$, where p, q, r are three different objects, let $B = \{1, 2, 3, 4\}$, and define $f(p) = 1$ and $f(q) = f(r) = 3$. Each input has exactly one output in B, so this is a function; two inputs may share an output. The image is $f(A) = \{1, 3\}$, and the targets 2 and 4 are never reached.

For $f : \mathbb{R} \to \mathbb{R}$ given by $f(x) = x^2$, the image is $[0, \infty)$: every output x^2 is nonnegative, and every $t \geq 0$ is an output, because $f(\sqrt{t}) = t$ (Section 1.8). The codomain, however, is all of $\mathbb{R}$. A function is specified by three pieces of information, its domain, its codomain, and its rule; the image is something we work out from these.

To name a rule without giving it a letter, we write $x \mapsto x^2$, read "x maps to x^2." Two functions $f, g : A \to B$ are *equal* when $f(x) = g(x)$ for every $x \in A$. Functions with different domains or codomains are different, even with the same rule: $x \mapsto x^2$ from $\mathbb{R}$ to $\mathbb{R}$ differs from the same rule

from $\mathbb{R}$ to $[0, \infty)$: every target of the second is an output, but the negative targets of the first are never reached. Section 3.3 gives this difference a name.

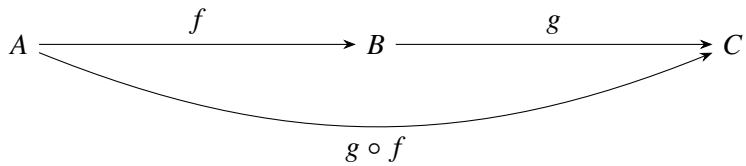
Pause and decide. Does the rule "send x to a number y with $y^2 = x$" define a function from $\mathbb{R}$ to $\mathbb{R}$?

Check. No, for two reasons. The input -1 has no output, since no real number has a negative square, and the input 4 has two candidate outputs, 2 and -2. A function must give *exactly one* output for *every* input. The rule $t \mapsto \sqrt{t}$ from $[0, \infty)$ to $\mathbb{R}$ repairs both problems: it shrinks the domain to the numbers that have a square root, and it always picks the nonnegative root.

The *identity map* $\mathrm{id}_A : A \to A$ sends each element to itself: $\mathrm{id}_A(x) = x$ for every $x \in A$. If $f : A \to B$ and $g : B \to C$, their *composition* $g \circ f : A \to C$ applies f first and g second:

$$(g \circ f)(x) = g(f(x)).$$

This makes sense because every output of f lies in B, the set of allowed inputs of g. Watch the order: in $g \circ f$, the function written on the right acts first, so read $g \circ f$ as "g after f."



Composing two concrete maps

Let $f, g : \mathbb{R} \to \mathbb{R}$ be given by $f(x) = x + 1$ and $g(y) = 2y$. Then $(g \circ f)(3) = g(4) = 8$, while $(f \circ g)(3) = f(6) = 7$, so $g \circ f$ and $f \circ g$ are different functions. In general,

$$(g \circ f)(x) = g(x + 1) = 2(x + 1) = 2x + 2 \qquad \text{and} \qquad (f \circ g)(x) = f(2x) = 2x + 1,$$

the two expressions compared in Section 1.2.

When the codomain of f is not the domain of g, the composition can fail to make sense. Let $f : \mathbb{R} \to \mathbb{R}$ be $f(x) = x - 1$, and let $g : [0, \infty) \to \mathbb{R}$ be $g(t) = \sqrt{t}$. Then $g(f(5)) = g(4) = 2$, but $g(f(0))$ makes no sense, because $f(0) = -1$ is not an allowed input of g; so $g \circ f$ is not defined. Using the same rule $x \mapsto x - 1$ as a function from $[1, \infty)$ to $[0, \infty)$ fixes this: the composition $x \mapsto \sqrt{x - 1}$ is defined on $[1, \infty)$.

Undoing a function

A function $f : A \to B$ is *reversible*, or *invertible*, if some function $h : B \to A$ undoes it from both sides:

$$h(f(a)) = a \text{ for every } a \in A \qquad \text{and} \qquad f(h(b)) = b \text{ for every } b \in B.$$

Such an h is called an *inverse* of f. Because two functions are equal when they agree at every input, the two conditions say that $h \circ f = \mathrm{id}_A$ and $f \circ h = \mathrm{id}_B$. For $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x + 1$, an inverse is $h(y) = y - 1$: $h(f(x)) = (x + 1) - 1 = x$ and $f(h(y)) = (y - 1) + 1 = y$.

Both conditions must hold at *every* input. The square root seems to undo squaring, since $\sqrt{3^2} = 3$, but for the square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, we get $\sqrt{f(-3)} = 3$, not -3. In fact this f has no inverse at all: an inverse h would need $h(1) = h(f(1)) = 1$ and also $h(1) = h(f(-1)) = -1$, which is impossible.

A function has at most one inverse. Suppose that h and k are both inverses of f, and let $b \in B$. Since $f(k(b)) = b$, applying h to both sides gives $h(f(k(b))) = h(b)$. Since $h(f(a)) = a$ for every $a \in A$, taking $a = k(b)$ gives $h(f(k(b))) = k(b)$. Hence $h(b) = k(b)$ for every $b \in B$, so $h = k$. We may therefore speak of *the* inverse of f, usually written f^{-1}. This symbol does not mean the reciprocal $1/f$: for $f(x) = x + 1$, we have $f^{-1}(1) = 0$, while $1/f(1) = 1/2$.

3.3 Injective, surjective, and reversible

Which functions can be undone? The answer combines two properties that are useful on their own.

A function $f : A \to B$ is *injective* if different inputs always give different outputs: whenever $x \neq y$, also $f(x) \neq f(y)$. An injective function loses no information, because each output tells us which input produced it. In proofs we usually use the contrapositive form (Section 2.5):

$$\forall x, y \in A : \qquad f(x) = f(y) \Rightarrow x = y,$$

where "$\forall x, y \in A$" is short for "$\forall x \in A\ \forall y \in A$." This form is easier to use, because an equation gives us something to calculate with (from $x + 1 = y + 1$ we get $x = y$ by subtracting 1), while an assumption such as $x \neq y$ gives very little to go on. So to prove that f is injective, take arbitrary $x, y \in A$, assume $f(x) = f(y)$, and deduce $x = y$.

A function $f : A \to B$ is *surjective* if every target is an output:

$$\forall b \in B\ \exists a \in A : \qquad f(a) = b.$$

Equivalently, the image $f(A)$ is the whole codomain B. In the game of Section 2.3, the skeptic hands us a target b, and we respond with an input a, which may depend on b. An input a with $f(a) = b$ is a *witness* that b is reached, as 3 witnesses that $7 = 2 \cdot 3 + 1$ is odd (Section 1.4). So to prove that f is surjective, take an arbitrary $b \in B$ and produce a witness.

A function that is both injective and surjective is *bijective*, or a *bijection*. Other books often say *one-to-one* for injective and *onto* for surjective. A function can have either property without the other. The map $f : \{p, q\} \to \{1, 2, 3\}$ with $f(p) = 1$ and $f(q) = 2$ is injective, but not surjective, because no input reaches 3. The map $g : \{p, q, r\} \to \{1, 2\}$ with $g(p) = g(q) = 1$ and $g(r) = 2$ is surjective, but not injective: told only that the output is 1, we cannot tell whether the input was p or q.

Now take $f : \{p, q, r\} \to \{1, 2, 3\}$ with $f(p) = 2$, $f(q) = 3$, and $f(r) = 1$. It reaches every target exactly once, so it is bijective. To undo it, read each assignment backward: the inverse is $h(1) = r$, $h(2) = p$, $h(3) = q$; for instance, $h(f(q)) = h(3) = q$.

Why does this work for every bijection? Call an input a a *preimage* of a target b when $f(a) = b$, that is, when a is a witness that b is reached. To build an inverse h, we must decide, for every target b, which input $h(b)$ should be.

- Surjectivity says that every target b has *at least one* preimage, so there is something to choose.
- Injectivity says that every target has *at most one* preimage: if $f(a_1) = b$ and $f(a_2) = b$, then $f(a_1) = f(a_2)$, and so $a_1 = a_2$. So there is no choice to make.

We may therefore define $h(b)$ to be *the* unique preimage of b. Then $f(h(b)) = b$ by the definition of h. And for any input a, the input a itself is a preimage of $f(a)$, and the only one, so $h(f(a)) = a$.

Conversely, a function with an inverse must be bijective; Exercise 3.5 proves this in a sharper form, using each condition on an inverse separately. So a function is reversible exactly when it is bijective.

The square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, is neither injective nor surjective: $f(1) = f(-1)$, and the target -1 is never reached. That is why it has no inverse. Now keep the rule but change both sets: $s : [0, \infty) \to [0, \infty)$, $s(x) = x^2$, is a bijection. For injectivity, if $s(x) = s(y) = t$, then x and y are nonnegative square roots of t, and t has only one (Section 1.8), so $x = y$. For surjectivity, a target $t \geq 0$ is reached by the nonnegative input $\sqrt{t}$. The inverse of s sends each t to its unique preimage $\sqrt{t}$: it is the square-root function from $[0, \infty)$ to $[0, \infty)$. Shrinking the domain removed the collision between 1 and -1, and shrinking the codomain removed the targets that could never be reached.

These properties also compare the sizes of finite sets. If A and B are finite and $f : A \to B$ is injective, its $|A|$ different inputs have $|A|$ different outputs, all in B, so $|A| \leq |B|$. If f is also surjective, those outputs make up all of B, so $|A| = |B|$: a bijection shows that two finite sets have the same size, without counting either one. And if $|A| = |B|$, then the $|A|$ different outputs of an injective f are $|B|$ different members of B, so they make up all of B: f is surjective as well. Read as a contrapositive, the first fact says that if $|A| > |B|$, then every map from A to B sends two different inputs to the same output. This is the simplest case of the pigeonhole principle, stated in general in Chapter 7 as Proposition 7.4.

Both properties survive composition. Here is the statement for injectivity; Exercise 3.2 asks you to prove the one for surjectivity.

Theorem 3.2. *If $f : A \to B$ and $g : B \to C$ are injective, then $g \circ f$ is injective.*

Plan. An equality of final outputs should let us recover an equality at the intermediate stage, and then an equality of inputs.

Proof. Take any $x, y \in A$ and assume $g(f(x)) = g(f(y))$. Since $f(x)$ and $f(y)$ belong to B, injectivity of g gives $f(x) = f(y)$. Injectivity of f then gives $x = y$. This is the implication required for $g \circ f$ to be injective. □

An assistant may summarize this proof as "cancel g, then cancel f." That shorthand is fine once it is clear that each cancellation is an application of injectivity. Functions cannot be cancelled in general: for $g : \mathbb{R} \to \mathbb{R}$, $g(y) = y^2$, we have $g(1) = g(-1)$, yet $1 \neq -1$.

Proposition 3.3. *Let $f : A \to B$ and $g : B \to C$. If $g \circ f$ is injective, then f is injective. However, g need not be injective.*

Proof. Let $x, y \in A$ with $f(x) = f(y)$. Applying the same function g to equal objects gives equal results, so $g(f(x)) = g(f(y))$. Injectivity of $g \circ f$ then gives $x = y$. Hence f is injective.

For the second assertion, one example is enough. Take $A = \{0, 1\}$, $B = \{0, 1, 2\}$, $C = \{0, 1\}$, and define

$$f(0) = 0, \quad f(1) = 1, \qquad g(0) = 0, \quad g(1) = g(2) = 1.$$

The composition sends 0 to 0 and 1 to 1, so it is injective. Yet g is not injective, because the different inputs 1 and 2 have the same output. □

Why does the example work? The composition applies g only to outputs of f, the points of the image $f(A) = \{0, 1\}$. The collision $g(1) = g(2)$ involves the point 2, which f never reaches, so the composition cannot detect it. Studying why a counterexample works often shows what is true instead: here, that g must be injective on the image of f, though not necessarily elsewhere.

To say this precisely: if S is a subset of the domain of g, the *restriction* of g to S is the function from S to C that uses the same rule as g but accepts only inputs from S. Whenever $g \circ f$ is injective, the restriction of g to $f(A)$ is injective. To check this, let $b_1, b_2 \in f(A)$ with $g(b_1) = g(b_2)$. Since both lie in the image, $b_1 = f(x)$ and $b_2 = f(y)$ for some $x, y \in A$. Then $g(f(x)) = g(f(y))$, so injectivity of $g \circ f$ gives $x = y$, and therefore $b_1 = f(x) = f(y) = b_2$. In the example, the restriction of g to $f(A) = \{0, 1\}$ sends 0 to 0 and 1 to 1, so it is injective.

If, in addition, f is surjective, then $f(A)$ is all of B, so no input of g goes untested, and g itself must be injective.

Proposition 3.4. *Let $f : A \to B$ and $g : B \to C$. If $g \circ f$ is injective and f is surjective, then g is injective.*

Proof. Since f is surjective, $f(A) = B$, so the restriction of g to $f(A)$ is g itself. We showed above that this restriction is injective whenever $g \circ f$ is injective. Hence g is injective. □

3.4 Images and preimages answer different questions

Let $f : A \to B$. For $S \subseteq A$, the *image* of S is

$$f(S) = \{f(x) : x \in S\},$$

the set of outputs of inputs from S. It answers "where do these inputs go?" When $S = A$, this is the image $f(A)$ from Section 3.2. Going the other way, for $T \subseteq B$, the *preimage* of T is

$$f^{-1}(T) = \{x \in A : f(x) \in T\},$$

the set of inputs whose outputs land in T. It answers "which inputs end up here?" Despite the notation, $f^{-1}(T)$ makes sense for every function f, whether or not f has an inverse. When $T = \{b\}$, the preimage $f^{-1}(\{b\})$ is the set of all preimages of b in the sense of Section 3.3. So f is surjective

exactly when every such set has at least one member, and injective exactly when every such set has at most one member.

For the square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, here are some images and preimages:

$$f(\{-1, 1, 2\}) = \{1, 4\}, \qquad f^{-1}(\{1\}) = \{-1, 1\},$$
$$f^{-1}(\{4, 9\}) = \{-3, -2, 2, 3\}, \qquad f^{-1}(\{-1\}) = \varnothing.$$

Notice the braces: the value $f(3) = 9$ is a number, while the image $f(\{3\}) = \{9\}$ is a set. Images and preimages are always sets. The set T may also be an interval: $f^{-1}([0, 4]) = [-2, 2]$, because $0 \le x^2$ holds for every real x, and $x^2 \le 4$ holds exactly when $-2 \le x \le 2$ (Section 3.1).

The square map has no inverse, yet all of its preimages make sense. When f does have an inverse $h : B \to A$, the two readings of $f^{-1}(T)$ agree: the preimage of T under f is the image $h(T)$ of T under the inverse. Indeed, take $x \in A$. If x lies in the preimage, then $t = f(x)$ belongs to T, and $x = h(f(x)) = h(t)$, so $x \in h(T)$. Conversely, if $x = h(t)$ for some $t \in T$, then $f(x) = f(h(t)) = t \in T$, so x lies in the preimage.

Pause and decide. For the square map $f : \mathbb{R} \to \mathbb{R}$, compute $f^{-1}(f(\{2\}))$. Do you get back $\{2\}$?

Check. No: $f(\{2\}) = \{4\}$, and $f^{-1}(\{4\}) = \{-2, 2\}$. Going forward and then backward returns every input that shares an output with 2, and -2 is such an input. In general $S \subseteq f^{-1}(f(S))$, because each $x \in S$ has its output $f(x)$ in $f(S)$. If f is injective, nothing extra comes back: if $f(x) \in f(S)$, then $f(x) = f(s)$ for some $s \in S$, and injectivity gives $x = s \in S$. So $f^{-1}(f(S)) = S$ whenever f is injective.

Preimages fit well with the set operations of Section 3.1. For any $S, T \subseteq B$,

$$f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T).$$

We can justify this one membership condition at a time. For any $x \in A$,

$$\begin{aligned}
x \in f^{-1}(S \cap T) &\iff f(x) \in S \cap T && \text{definition of preimage} \\
&\iff f(x) \in S \text{ and } f(x) \in T && \text{definition of intersection} \\
&\iff x \in f^{-1}(S) \text{ and } x \in f^{-1}(T) && \text{definition of preimage} \\
&\iff x \in f^{-1}(S) \cap f^{-1}(T) && \text{definition of intersection.}
\end{aligned}$$

Each step is an "if and only if," valid in both directions. Reading the chain downward shows that every member of the left side lies in the right side, and reading it upward shows the reverse containment. Both sides are subsets of A, and $x \in A$ was arbitrary, so the two sets are equal. No inverse function was used. (Exercise 3.6 asks whether images behave as well.)

Habit 3: identify the information actually used

When you read or write a proof, ask which facts it uses and where. A statement about every target can be much stronger than one about the outputs that are reached. In Proposition 3.3, the composition $g \circ f$ tests g only on the image $f(A)$, so a conclusion about g on all of B asks for more than the argument can give, and here more than is true. The same question can show that a result reaches further than it seems: the proof of Proposition 3.1 used only the meanings of "and" and "or" applied to membership, never what the elements were, so it holds for sets of numbers, people, tasks, or any other objects. Before you propose a generalization, or accept one from an assistant, find the part of each object that the proof actually visits.

Exercises

Exercises 3.1–3.4 are core exercises, and Exercises 3.5 and 3.6 are transfer exercises. Complete solutions are in Appendix A, Section A.3.

Exercise 3.1. For $A = \{1, 2, 4\}$ and $B = \{2, 3, 4\}$, find $A \cup B$, $A \cap B$, and $A \setminus B$.

Exercise 3.2. Let $f : A \to B$ and $g : B \to C$ be surjective. Prove that $g \circ f : A \to C$ is surjective. Think about which witness you must find first.

Exercise 3.3. Let $f : \mathbb{Z} \to \mathbb{Z}$ be given by $f(n) = 2n$. Decide whether f is injective and whether it is surjective, and prove your answers. Then keep the domain and the rule, change only the codomain, and obtain a bijection.

Exercise 3.4. Let $f : A \to B$ and $S, T \subseteq B$. Prove that $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$ by following the membership conditions on both sides.

Exercise 3.5. Let $f : A \to B$ and $h : B \to A$. Suppose that $h \circ f = \mathrm{id}_A$; such an h is called a *left inverse* of f. Prove that f is injective. Suppose instead that $f \circ h = \mathrm{id}_B$; such an h is called a *right inverse* of f. Prove that f is surjective.

Exercise 3.6. Let $f : A \to B$ and $S, T \subseteq A$. Is $f(S \cap T) = f(S) \cap f(T)$ always true? Prove the containment that always holds, and give a counterexample to equality.

CHAPTER 4

How a proof finds its next step

Overview

Find the next step of a proof by working backward from what you must show and forward from what you know. Use direct proof, proof of the contrapositive, proof by contradiction, and separate proofs of existence and uniqueness, and learn when each helps. Prove that every integer is even or odd, and that $\sqrt{2}$ is irrational. Read a long proof by checking that each earlier result it uses really applies.

4.1 Working backward and working forward

Every proof has a *goal*, the statement it must establish, and a supply of facts it may use: the hypotheses, the definitions, and earlier results. When you do not know what to write next, look in two directions. *Working backward* starts from the goal and asks: what would be enough to establish this? *Working forward* starts from the facts and asks: what follows from what I already have? A proof is found when the two meet: something derived from the facts is exactly what working backward said would be enough.

Unpacking the definitions in the goal often tells you exactly what to produce. To show that an integer is even, you must find an integer w such that the number equals $2w$ (Definition 1.1). To show that a function f is injective, you take inputs x and y with $f(x) = f(y)$ and deduce that $x = y$ (Section 3.3). To show that an object with some property exists, you usually name one and check that it has the property; such an object is called a *witness* (Section 1.4). Occasionally existence is proved without a witness, by showing that if no such object existed, something impossible would follow (a proof by contradiction, Section 4.2); Chapter 13 contains proofs of this kind.

A proof state on paper

Consider the claim

$$\textit{If } n \textit{ is an even integer, then } n^2 \textit{ is even.}$$

Section 2.1 proved it in one line; here we slow down to watch how such a proof is found. At each moment of the search we keep two lists: the facts we already have, and what we still have to show. Together they are called the *proof state*. Proof assistants such as Lean display exactly this pair after every step (Chapter 15).

At the start, the only fact is the hypothesis, and the goal is "n^2 is even." Working backward, the definition of even turns the goal into *find an integer w with $n^2 = 2w$*. Working forward, the same

definition turns the hypothesis into $n = 2k$ for some integer k. The two letters play different roles: k is handed to us by the hypothesis, while w is an integer we still have to find. Squaring gives $n^2 = 4k^2 = 2(2k^2)$, exactly the shape the goal asks for, so we choose $w = 2k^2$, an integer because k is. Only now does the goal move to the "known" list. The table records the search.

Step	Known	Still to show
Start	n is an even integer	n^2 is even
Unpack the goal	n is an even integer	an integer w with $n^2 = 2w$
Unpack the hypothesis	$n = 2k$, where k is an integer	an integer w with $n^2 = 2w$
Square and factor	$n = 2k$ and $n^2 = 2(2k^2)$	an integer w with $n^2 = 2w$
Choose $w = 2k^2$	$n^2 = 2w$, and w is an integer	nothing: the proof is finished

The search moved in both directions, but the written proof runs forward only:

> Let n be an even integer. Then $n = 2k$ for some integer k, so $n^2 = 4k^2 = 2(2k^2)$. Since $2k^2$ is an integer, n^2 is even.

A proof like this one, which starts from the hypotheses and reaches the conclusion through a chain of deductions, is called a *direct proof*.

Keeping the goal on the "still to show" list until it has been earned guards against a common mistake: writing "since n^2 is even, $n^2 = 2w$ for some integer w" in the middle of the proof. That uses the conclusion as though it were already known, so the argument goes in a circle. Working backward is still the right way to *search*, but in the finished proof the goal may appear only at the end, as the conclusion reached.

Pause and decide. Let $f : \mathbb{Z} \to \mathbb{Z}$ be given by $f(n) = n + 3$. To prove that f is surjective, what does the goal ask you to produce? Find it by working backward, and then check it by working forward.

Check. By the definition of surjective (Section 3.3), for each integer b you must find an integer a with $a + 3 = b$; working backward, this suggests $a = b - 3$. Working forward, $b - 3$ is an integer and $f(b - 3) = (b - 3) + 3 = b$. So $b - 3$ is a witness for each b, and f is surjective.

4.2 The contrapositive and proof by contradiction

Now turn the claim of Section 4.1 around. Its converse (Section 2.1) asks: if n^2 is even, must the integer n be even? Working forward gives $n^2 = 2w$ for some integer w, which describes n^2, not n; taking square roots gives only $n = \sqrt{2w}$ or $n = -\sqrt{2w}$, neither visibly twice an integer. The forward direction stalls.

So try the contrapositive (Section 2.5):

If n is not even, then n^2 is not even.

Since an implication and its contrapositive are equivalent, proving this proves the converse. But the new hypothesis, "n is not even," says only what n is *not*. If we knew that such an n must be odd, we could write $n = 2k + 1$ and square it. Section 1.4 stated without proof that every integer is either even or odd, but not both. We now prove this, using a method called *proof by contradiction*.

Proof by contradiction

To prove a claim by contradiction, temporarily suppose that the claim is false, keep all the original hypotheses, and reason until you reach a *contradiction*: some statement and its negation both follow. If the supposition were true, everything correctly deduced from it and the hypotheses would be true as well; but a statement and its negation cannot both be true. So the supposition is false. This last step uses the principle that every statement is either true or false (Section 2.5): ruling out "the claim is false" leaves only "the claim is true."

Section 2.5 already used this method, in its discussion of vacuous truth, to show that *no integer is both even and odd*. If some integer n were both, then $n = 2k$ and $n = 2\ell + 1$ for integers k and ℓ, so $2(k - \ell) = 1$, and the integer $k - \ell$ would equal $1/2$, which is not an integer. A statement ("$k - \ell$ is an integer") and its negation would both hold, so the supposition is impossible.

The end point must be a genuine impossibility, not something that only "looks unlikely." Mark the supposition clearly, for example with "suppose, for contradiction," because it is temporary: inside the argument we may use it freely, but once the contradiction is reached it is discarded, and nothing deduced from it counts as established afterward.

Every integer is even or odd

The other half, that every integer is even or odd, needs one more fact, which we accept without proof; it is the third of the starting facts listed in *To the reader*.

Well-ordering property: every nonempty set of positive integers has a smallest member.

For example, the positive integers whose square is larger than 50 form a nonempty set, since it contains 10. Its smallest member is 8: the squares of $1, 2, \ldots, 7$ are at most 49, while $8^2 = 64$. Each part of the property matters. The empty set has no smallest member, since it has no members. The negative integers have none either, since each negative integer n has a smaller negative integer, $n - 1$; nor do the positive rational numbers, since each positive rational r has a smaller positive rational, $r/2$.

Lemma 4.1. *Every integer is either even or odd, and no integer is both.*

Proof. We showed above that no integer is both even and odd. It remains to show that every integer n is even or odd. If $n = 0$, then n is even, since $0 = 2 \cdot 0$.

Next, suppose, for contradiction, that some positive integer is neither even nor odd. Then the set of such positive integers is nonempty, so by the well-ordering property it has a smallest member; call it m. Now $m \neq 1$, because $1 = 2 \cdot 0 + 1$ is odd. So $m \geq 2$, and $m - 1$ is a positive integer smaller than m. Since m is the *smallest* positive integer that is neither even nor odd, $m - 1$ is

even or odd. If $m - 1 = 2k$ for an integer k, then $m = 2k + 1$, so m is odd. If $m - 1 = 2k + 1$ for an integer k, then $m = 2k + 2 = 2(k + 1)$, so m is even. Either way m is even or odd, which contradicts the choice of m. Hence every positive integer is even or odd.

Finally, let n be negative. Then $-n$ is positive, so by the previous paragraph $-n = 2k$ or $-n = 2k+1$ for some integer k. In the first case $n = 2(-k)$ is even. In the second, $n = -2k - 1 = 2(-k - 1) + 1$ is odd; for instance, $7 = 2 \cdot 3 + 1$ and $-7 = 2 \cdot (-4) + 1$. □

Here the contradiction is that m has a property it was chosen *not* to have. Proofs that begin "choose the smallest . . . " often end this way: the next section chooses a smallest denominator, and Chapter 5 uses the same pattern to explain why induction works.

Back to the contrapositive

Lemma 4.1 lets us finish the argument. Since the next section uses the result twice, we record it as a lemma.

Lemma 4.2. *For every integer n, the integer n is even if and only if n^2 is even.*

Proof. Let n be an integer. If n is even, then n^2 is even, by the direct proof in Section 4.1.

For the other direction we prove the contrapositive: if n is not even, then n^2 is not even. So suppose n is not even. By Lemma 4.1, n is odd, so $n = 2k + 1$ for some integer k. Then

$$\begin{aligned} n^2 &= (2k + 1)(2k + 1) \\ &= 4k^2 + 4k + 1 \\ &= 2(2k^2 + 2k) + 1. \end{aligned}$$

Since $2k^2 + 2k$ is an integer, n^2 is odd, so by Lemma 4.1 again, n^2 is not even. This proves the contrapositive, and hence that if n^2 is even, then n is even. □

In general, try the contrapositive when the negation of the conclusion is easier to unpack than the hypothesis; when the hypothesis is the easier one, as in Section 4.1, prove the statement directly.

Pause and decide. The proof of Lemma 4.2 used Lemma 4.1 twice. Which half of Lemma 4.1 was used each time?

Check. The first use needs "every integer is even or odd": it turns "n is not even" into "n is odd," which gives $n = 2k + 1$. The second needs "no integer is both": it turns "n^2 is odd" into "n^2 is not even."

4.3 The square root of two is irrational

Recall from Section 1.8 that a *rational number* can be written as a fraction a/b of integers with $b \neq 0$; a real number that is not rational is called *irrational*. The symbol $\sqrt{2}$ denotes the nonnegative real number whose square is 2.

Theorem 4.3. *The number $\sqrt{2}$ is irrational.*

Why argue by contradiction? The theorem says what $\sqrt{2}$ is *not*, and a direct proof would have to rule out every fraction at once. Supposing the opposite hands us one fraction of integers equal to $\sqrt{2}$, with a numerator and denominator to calculate with. Statements that no object of some kind exists are often proved by contradiction for this reason.

The proof chooses a fraction for $\sqrt{2}$ with the smallest possible positive denominator. For an ordinary rational number such as $0.75 = 3/4 = 6/8 = 9/12 = \cdots$, the possible positive denominators are $4, 8, 12, \ldots$, and choosing the smallest picks out $3/4$, which cannot be reduced further.

Plan. Suppose $\sqrt{2} = a/b$ with b as small as possible. Squaring will force both a and b to be even. Then cancelling a factor of two gives a fraction for $\sqrt{2}$ with a smaller positive denominator, which contradicts the choice of b.

Proof. Suppose, for contradiction, that $\sqrt{2}$ is rational: $\sqrt{2} = p/q$ for some integers p and q with $q \neq 0$. If q is negative, we may rewrite p/q as $(-p)/(-q)$, whose denominator is positive. So the set of positive integers j for which $\sqrt{2} = a/j$ with some integer a is nonempty. By the well-ordering property (Section 4.2), it has a smallest member; call it b, and let a be an integer with $\sqrt{2} = a/b$.

Squaring gives $2 = a^2/b^2$, and multiplying by b^2 gives $a^2 = 2b^2$. Since b^2 is an integer, a^2 is even, so a is even by Lemma 4.2; write $a = 2c$ with c an integer. Substituting into $a^2 = 2b^2$ gives

$$(2c)^2 = 2b^2, \qquad \text{that is,} \qquad 4c^2 = 2b^2, \qquad \text{so} \qquad b^2 = 2c^2.$$

Thus b^2 is even, and Lemma 4.2 shows that b is even; write $b = 2d$ with d an integer. Since b is positive, so is $d = b/2$, and $d < b$ because $b = d + d$. Cancelling the common factor 2 gives

$$\sqrt{2} = \frac{a}{b} = \frac{2c}{2d} = \frac{c}{d}.$$

So $\sqrt{2} = c/d$ with c an integer and d a positive integer smaller than b, contradicting the choice of b. Therefore $\sqrt{2}$ is irrational. □

Parity alone gives no contradiction: fractions with even numerator and denominator, such as $6/8$, certainly exist. The contradiction comes from parity together with the choice of b.

No computation could replace this proof. Computers have produced billions of digits of $\sqrt{2} \approx 1.41421356$, but any finite list of digits is a fraction, such as $141421356/100000000$. Only a proof can show that *no* fraction of integers, however large, equals $\sqrt{2}$.

Pause and decide. Where did the proof use the fact that the denominator b is positive?

Check. In two places. First, the well-ordering property applies only to sets of *positive* integers, and negative denominators have no smallest one: from $\sqrt{2} = a/b$ with $b > 0$ we also get $\sqrt{2} = (-a)/(-b) = (-2a)/(-2b) = \cdots$, with denominators $-b, -2b, \ldots$ decreasing without end. Second, we needed $d = b/2$ to be smaller than b, which fails for negative b: if $b = -4$, then $b/2 = -2 > b$.

4.4 Existence is not uniqueness

The equation $x^2 = 1$ has real solutions, but not a *unique* one, that is, exactly one. It has exactly two, 1 and -1: both square to 1, and there are no others, because a solution makes $x^2 - 1 = (x-1)(x+1)$ equal to zero, and a product of real numbers is zero only when at least one factor is zero. In general, "there is a unique solution" makes two separate claims:

- *existence*: at least one number satisfies the condition;
- *uniqueness*: any two numbers that satisfy the condition are equal, so there is at most one.

This is the split of Section 3.3, where surjectivity gave every element of the codomain at least one preimage and injectivity at most one. Each claim needs its own argument.

Consider the equation

$$x = \frac{x+1}{2},$$

where x is a real number. *Existence* is settled by a witness: 1 is a solution, because $(1+1)/2 = 1$.

For *uniqueness*, suppose that real numbers x and y both satisfy the equation:

$$x = \frac{x+1}{2} \qquad \text{and} \qquad y = \frac{y+1}{2}.$$

Subtract the second equation from the first. The constant terms 1 cancel, leaving

$$x - y = \frac{x-y}{2}.$$

Multiplying by two gives $2(x-y) = x-y$, and subtracting $x-y$ gives $x-y = 0$, so $x = y$. Together, the two parts show that the equation has exactly one real solution, namely 1.

The uniqueness argument never solves the equation: it takes two hypothetical solutions and shows that they cannot differ. This can be simpler than solving, because whatever the two equations share, such as the constant terms here, cancels on subtraction. Chapter 12 turns this comparison into the uniqueness part of the contraction theorem, starting from the same equation $x = (x+1)/2$.

A uniqueness argument on its own proves only that there is *at most one* solution, which is also true of an equation with no solutions. Take the equation $2x = 1$, where x must be an integer. If integers x and y satisfy $2x = 1$ and $2y = 1$, then $2(x-y) = 0$, so $x = y$. The argument is correct, yet there is no integer solution: the only real solution, $1/2$, is not an integer. That is why the existence check cannot be skipped.

Be careful, too, with the phrase "the solution": the word "the" claims that there is exactly one. A proof that begins "let x be the solution of $x^2 = 1$" has already gone wrong, since a conclusion such as "so x is positive" may hold for one solution and fail for the other. Until existence and uniqueness are both established, say what you are really assuming, for example "suppose x is a solution" or "if a solution exists, then . . . ."

Pause and decide. Try the comparison argument on the equation $x = x$, where x is a real number. What happens, and why is that the right outcome?

Check. Subtracting $y = y$ from $x = x$ gives $x - y = x - y$, which is true for all real x and y and says nothing about whether $x = y$. That is as it should be: every real number solves $x = x$, so uniqueness is false, and no correct argument could prove it.

4.5 Reading a long proof as a sequence of smaller promises

Long proofs are usually assembled from shorter results: lemmas, propositions and theorems (Section 1.5). Each works like a small promise: supply its hypotheses, and it delivers its conclusion. So whenever a proof uses an earlier result, ask three questions. What does the result need? Why does the proof have it at this point? What does the result give back?

Here is that check for the proof of Theorem 4.3, which uses one starting fact once and one lemma twice.

Result used	What it needs	Why it holds here	What it gives
Well-ordering property	a nonempty set of positive integers	the assumed fraction for $\sqrt{2}$, with both signs changed if necessary, supplies a positive denominator	a smallest positive denominator b
Lemma 4.2	an integer whose square is even	a is an integer, and $a^2 = 2b^2$ is twice the integer b^2	a is even
Lemma 4.2 again	an integer whose square is even	b is an integer, and $b^2 = 2c^2$ is twice the integer c^2	b is even

The third column must be checked, not assumed. A sentence such as "by a standard result, a is even" makes this impossible: the reader cannot tell which result is meant, so cannot see what it needs. A careful proof names the result and shows that its hypotheses hold.

> **Habit 4: ask for a local contract, not just more words**
>
> Habit 2 (Section 2.6) asked for the contract of a whole statement. When a single step of a proof (your own, a book's, or an assistant's) is hard to follow, ask for the contract of that step. What is known at this point? What is the next goal? Which exact result is used, and why do its hypotheses hold here? Which quantity is constructed, and from what? These questions locate the missing mathematics better than a request for a "simpler" explanation.

For example, take this step from the proof of Lemma 4.1: "If $m - 1 = 2k + 1$ for an integer k, then $m = 2k + 2 = 2(k + 1)$, so m is even." Known: $m - 1 = 2k + 1$, where k is an integer. Next goal: m is even or odd. Result used: none; Definition 1.1 alone says that m is even once it is twice an integer. Quantity constructed: the witness $k + 1$, an integer because k is. With these four answers written down, each part of the step can be checked on its own.

Exercises

Exercises 4.1–4.4 are core exercises, and Exercises 4.5 and 4.6 are transfer exercises. Complete solutions are in Appendix A, Section A.4.

Exercise 4.1. Give a direct proof that if a and b are even integers, then ab is divisible by four.

Exercise 4.2. Let n be an integer. Prove, by proving the contrapositive, that if $3n + 1$ is even, then n is odd.

Exercise 4.3. Let r be a rational number and u an irrational number. Prove that $r + u$ is irrational. Your proof will need the fact that the difference of two rational numbers is rational; explain why this fact holds.

Exercise 4.4. Prove that the equation $x = 3x - 4$ has exactly one real solution. Give the existence part and the uniqueness part separately.

Exercise 4.5. An assistant is asked to solve the equation $x^2 = 2$ for a real number x, and answers "so $x = \sqrt{2}$." What has the assistant missed? Find all real solutions of $x^2 = 2$, and prove that your list is complete.

Exercise 4.6. Prove that $\sqrt{8}$ is irrational in two ways. First, deduce it from Theorem 4.3. Second, without using Theorem 4.3, adapt its proof so that it works for $\sqrt{8}$, and explain how the parity steps have to change.

CHAPTER 5

Induction, invariants, and progress

> **Overview**
> Prove a statement about every natural number from a first case and one general step, and use strong induction, whose step may use all the earlier cases. Use an invariant to show what a process can never change, and a decreasing nonnegative integer to show that a process must stop; Euclid's algorithm for the greatest common divisor uses both. See how a more general statement can make an induction proof simpler.

5.1 From a few sums to every sum

Add up the odd positive integers, one at a time:

$$1 = 1, \qquad 1 + 3 = 4, \qquad 1 + 3 + 5 = 9, \qquad 1 + 3 + 5 + 7 = 16.$$

The totals are the squares $1^2, 2^2, 3^2, 4^2$. The nth odd positive integer is $2n - 1$, so the pattern suggests that

$$1 + 3 + \cdots + (2n - 1) = n^2 \qquad \text{for every } n \geq 1.$$

Section 1.8 asked whether this always holds, and as Section 1.1 explained, examples cannot settle it: even a computer check of the first thousand values of n tells us only about those values. We need a reason why the pattern can never break. The idea is to show that whenever the formula holds for one value of n, it also holds for the next; since it holds for $n = 1$, it then holds for $n = 2$, hence for $n = 3$, and so on forever. This way of reasoning is called *induction*.

The principle of induction

Let $P(n)$ be a statement about a natural number n that becomes true or false once a value of n is chosen, as in Section 2.4; for instance, "$1 + 3 + \cdots + (2n - 1) = n^2$," whose meaning at $n = 0$ is explained below. To prove that $P(n)$ is true for every natural number n, it is enough to prove two things:

1. the *base case*: $P(0)$ is true;
2. the *induction step*: for every natural number k, if $P(k)$ is true, then $P(k + 1)$ is true.

In the induction step, k stands for a natural number, but we do not say which one, and we assume that $P(k)$ is true. This temporary assumption is the *induction hypothesis*, and from it we must deduce $P(k + 1)$. Because the argument never uses a particular value of k, it works for every k at once.

These two facts give $P(n)$ for every n: the base case gives $P(0)$, and n uses of the induction step, with $k = 0, 1, \ldots, n-1$, carry it on to $P(1), P(2), \ldots, P(n)$. Section 5.2 makes this argument more careful.

Induction does not have to start at 0. To prove $P(n)$ for every integer $n \geq 1$, prove $P(1)$ as the base case and the induction step for every $k \geq 1$; the chain then reaches $1, 2, 3, \ldots$. Any other starting point works the same way.

Starting at zero

Our formula also makes sense at $n = 0$, which is a natural number in this book. In sigma notation its left side is $\sum_{j=1}^{n}(2j-1)$, and for $n = 0$ the index j would run from 1 up to 0: there are no such j, so the sum has no terms. A sum with no terms is called an *empty sum*, and its value is defined to be 0, since adding no numbers to a running total leaves it unchanged. The book uses this agreement throughout. It keeps the rule "the sum of the first $k+1$ terms is the sum of the first k terms plus the last term" true when $k = 0$: the one-term sum $\sum_{j=1}^{1}(2j-1)$ equals $0 + 1 = 1$.

Watching one induction step

Before writing the general step, watch it happen at $k = 3$. The formula at 3 says $1 + 3 + 5 = 3^2 = 9$. The next sum is the old sum plus the next odd number, 7:

$$1 + 3 + 5 + 7 = (1 + 3 + 5) + 7 = 9 + 7 = 16 = 4^2.$$

The induction step below does exactly the same thing with k in place of 3. When an induction argument puzzles you, rewrite its step like this with a specific number.

Theorem 5.1. *For every $n \in \mathbb{N}$,*

$$\sum_{j=1}^{n}(2j-1) = n^2.$$

For $n = 0$ the left side is the empty sum, which is zero.

Proof. We use induction on n.

Base case. For $n = 0$ the left side is the empty sum, which is 0, and the right side is $0^2 = 0$.

Induction step. Let k be a natural number, and assume the induction hypothesis

$$\sum_{j=1}^{k}(2j-1) = k^2.$$

We must show that $\sum_{j=1}^{k+1}(2j-1) = (k+1)^2$. Split the sum for $k+1$ into its first k terms and its

last term, the term with $j = k + 1$:

$$\begin{aligned}\sum_{j=1}^{k+1}(2j-1) &= \sum_{j=1}^{k}(2j-1) + \big(2(k+1)-1\big) \\ &= k^2 + \big(2(k+1)-1\big) && \text{by the induction hypothesis} \\ &= k^2 + 2k + 1 \\ &= (k+1)^2.\end{aligned}$$

This is the formula with $k + 1$ in place of n. By induction, the formula holds for every natural number n. □

5.2 Why induction is not circular, and when strong induction helps

A first reaction to induction is often suspicion: the induction step *assumes* the formula at k, so are we assuming what we want to prove? No: the induction step proves only the conditional statement "*if* the formula holds at k, *then* it holds at $k + 1$," for every k, and never claims that the formula holds at any particular k. The base case supplies the first actual truth, and the conditional statement carries it forward one step at a time, like a row of dominoes: each knocks over the next, and the base case pushes the first.

Here is a more careful way to see that no natural number can slip through. Suppose we have proved the base case $P(0)$ and the induction step, but P fails for some natural number. Then the set of natural numbers at which P fails is nonempty, and every nonempty set of natural numbers has a smallest member. This is the well-ordering property of Section 4.2, stated there for positive integers; allowing 0 changes nothing, because if 0 belongs to the set it is the smallest member, and otherwise the set consists of positive integers. Call the smallest member m. Then:

- $m \neq 0$, because $P(0)$ was proved directly. So $m \geq 1$, and $m - 1$ is a natural number.
- $P(m - 1)$ is true, because $m - 1$ is smaller than m, and m is the *smallest* number where P fails.
- The induction step, applied with $k = m - 1$, turns $P(m - 1)$ into $P(m)$. So $P(m)$ is true.

So $P(m)$ is both false, by the choice of m, and true. This contradiction shows that $P(n)$ is true for every natural number n. The pattern "choose a smallest counterexample and derive a contradiction" is called the *minimal-counterexample* method; the proof of Lemma 4.1 used it to show that every positive integer is even or odd.

Strong induction

Sometimes the step to $k + 1$ needs more than the case immediately before it. *Strong induction* lets us assume all of $P(0), P(1), \ldots, P(k)$ when proving $P(k + 1)$. To prove $P(n)$ for every natural number n by strong induction, prove

1. the base case: $P(0)$ is true; and

2. the strong induction step: for every natural number k, if $P(0), P(1), \ldots, P(k)$ are all true, then $P(k+1)$ is true.

The same minimal-counterexample argument shows that this works: at the smallest failure m, which is not 0, all of $P(0), \ldots, P(m-1)$ are true, and the strong induction step with $k = m-1$ gives $P(m)$, a contradiction. Strong induction can also start at a number other than 0. To prove $P(n)$ for every integer $n \geq 2$, say, prove $P(2)$, and prove for every $k \geq 2$ that if $P(2), P(3), \ldots, P(k)$ are all true, then $P(k+1)$ is true.

A classic example shows why the extra strength helps. Recall from Section 2.1 that an integer n is divisible by an integer d when $n = dk$ for some integer k; a positive integer d with this property is called a *divisor* of n. An integer $p \geq 2$ is *prime* if its only positive divisors are 1 and p. For example, 2, 3, 5 and 7 are prime, but 12 is not, since 3 is a divisor of 12. By a *product of primes* we mean a product of two or more primes, which need not be different, such as $12 = 2 \cdot 2 \cdot 3$.

Proposition 5.2. *Every integer $n \geq 2$ is a prime or a product of primes.*

Proof. We use strong induction on n, starting at 2.

Base case. The number 2 is prime, because its only positive divisors are 1 and 2.

Induction step. Let $n \geq 3$, and assume that every integer m with $2 \leq m < n$ is a prime or a product of primes. This is the induction hypothesis; here n plays the role of $k+1$ in the strong induction step.

If n is prime, there is nothing to prove. Otherwise n has a positive divisor a other than 1 and n, so $n = ab$ for some integer b. We check that a and b both lie between 2 and $n-1$, the range covered by the induction hypothesis:

- Since a and n are positive, $b = n/a$ is positive; as an integer, $b \geq 1$. Hence $n = ab \geq a$. Together with $a \neq 1$ and $a \neq n$, this gives $2 \leq a \leq n-1$.
- Since $a \geq 2$, we have $b = n/a \leq n/2 < n$. Since $a < n$, we have $b = n/a > 1$, so $b \geq 2$. Hence $2 \leq b \leq n-1$.

By the induction hypothesis, a is a prime or a product of primes, and so is b. Multiplying these together writes $n = ab$ as a product of primes. □

Ordinary induction would let us assume the result only for $n-1$, which is useless here: to handle $12 = 3 \cdot 4$ we need the cases 3 and 4, not 11. We cannot know in advance which smaller numbers a and b will turn up, so we need the result for all of them. Notice also that the proof checked that a and b lie between 2 and $n-1$ before applying the induction hypothesis to them; every use of an induction hypothesis needs a check of this kind.

5.3 A sequence defined by a recurrence

A *sequence* is an endless list of numbers $x_0, x_1, x_2, \ldots$, one for each natural number. Each x_n is a *term* of the sequence, and the subscript n is its position, counting from 0. A *recurrence* defines

a sequence step by step: it gives the first term and a rule that computes each later term from the earlier ones. Let

$$x_0 = 0 \qquad \text{and} \qquad x_{n+1} = \frac{x_n + 1}{2} \quad \text{for every } n \in \mathbb{N}.$$

Read the notation carefully: x_{n+1} is the term after x_n, whereas $x_n + 1$ is the current term plus one. So the rule says: add one to the current term and halve the result. Starting from $x_0 = 0$,

$$x_1 = \frac{0+1}{2} = \frac{1}{2}, \qquad x_2 = \frac{\frac{1}{2}+1}{2} = \frac{3}{4}, \qquad x_3 = \frac{\frac{3}{4}+1}{2} = \frac{7}{8}.$$

The terms fall short of 1 by 1, $\frac{1}{2}$, $\frac{1}{4}$ and $\frac{1}{8}$, each gap half the one before. This suggests the formula

$$x_n = 1 - 2^{-n},$$

where $2^{-n} = 1/2^n$ as in Section 1.3. We prove it by induction on n.

For the base case, $n = 0$, the formula gives $1 - 2^0 = 1 - 1 = 0$, which is x_0. For the induction step, let n be a natural number and assume that $x_n = 1 - 2^{-n}$. (The letter n now plays the role of k; the name does not matter.) Substitute the induction hypothesis into the rule:

$$\begin{aligned} x_{n+1} &= \frac{x_n + 1}{2} && \text{the rule for the next term} \\ &= \frac{(1 - 2^{-n}) + 1}{2} && \text{induction hypothesis} \\ &= \frac{2 - 2^{-n}}{2} \\ &= 1 - \frac{2^{-n}}{2} \\ &= 1 - 2^{-(n+1)}. \end{aligned}$$

The last equality uses $2^{-n}/2 = 1/(2^n \cdot 2) = 2^{-(n+1)}$. This is the formula at $n + 1$, so by induction it holds for every natural number n.

The gap between x_n and 1 is exactly $1 - x_n = 2^{-n}$, which halves at every step, so the terms come as close to 1 as we like; for instance, $x_{10} = 1 - 1/1024$ is within 0.001 of 1. Yet the gap is never zero, so no term ever equals 1.

Tracking the error instead

Why is 1 the right number to compare with? If some term were exactly 1, the next term would be $(1 + 1)/2 = 1$ again. A number that an update rule leaves unchanged is called a *fixed point* of the rule, and by Section 4.4, 1 is the only real number x with $x = (x + 1)/2$, so it is the only fixed point of this rule.

So instead of tracking x_n itself, track its *error* $x_n - 1$, the difference between the current term and the fixed point; its absolute value $|x_n - 1|$ is the distance from x_n to 1. Subtract 1 from both sides of the rule:

$$x_{n+1} - 1 = \frac{x_n + 1}{2} - 1 = \frac{x_n + 1 - 2}{2} = \frac{x_n - 1}{2}.$$

Each update exactly halves the error, and this calculation never used the starting value $x_0 = 0$. So for the same rule with *any* real starting value x_0, induction gives

$$x_n - 1 = 2^{-n}(x_0 - 1) \qquad \text{for every } n \in \mathbb{N}.$$

At $n = 0$ both sides equal $x_0 - 1$, and if the formula holds at n, halving both sides gives it at $n + 1$, by the rule just found; Exercise 5.5 asks you to write out this induction in full. With $x_0 = 0$ the formula becomes $x_n - 1 = -2^{-n}$, that is, $x_n = 1 - 2^{-n}$ again.

This more general statement covers every starting value at once, yet its induction step is shorter than the one above, because it records only what each step does, namely halve the error, and leaves out the starting value 0. So when an induction proof gets tangled, try proving a more general statement; its induction step can be simpler. The contraction theorem of Chapter 12 uses the same idea.

5.4 An invariant shows what cannot change

Many problems describe a process: there is a collection of possible *states*, and there are *legal moves*, each of which turns one state into another. An *invariant* of such a process is a quantity or property that every legal move leaves unchanged.

Here is an example. The states are rows of n signs, each a plus or a minus, where $n \geq 2$. A legal move picks two different positions in the row and flips the sign in each: a plus becomes a minus, and a minus becomes a plus. Can we start from a row of all plus signs and reach a row with exactly one minus sign?

To find an invariant, replace each plus sign by the number 1 and each minus sign by -1, and multiply all the numbers in the row together. Call the result the *product of the row*. Flipping one sign multiplies the product by -1, so a legal move, which flips two signs, multiplies it by $(-1)(-1) = 1$: the product never changes. For instance, the row $+\ -\ +\ +$ has product $1 \cdot (-1) \cdot 1 \cdot 1 = -1$, and flipping the first two signs gives $-\ +\ +\ +$, whose product is still -1.

A row of all plus signs has product 1. Call a row *reachable* if some sequence of legal moves turns the all-plus row into it. Induction on the number of moves shows that every reachable row also has product 1. Let $P(k)$ be the statement "every row obtained from the all-plus row by k legal moves has product 1." The base case $P(0)$ holds, because after 0 moves the row is still all plus signs. For the induction step, assume $P(k)$. A row obtained by $k + 1$ moves comes from a row obtained by k moves, whose product is 1 by the induction hypothesis, followed by one more move, which does not change the product; so $P(k + 1)$ holds. A row with exactly one minus sign has product -1, so it can never be reached, however many moves we make. Notice that the proof never examined sequences of moves one by one, which would be hopeless, since moves can be repeated and there are infinitely many sequences.

A row with m minus signs has product $(-1)^m$, so the invariant says that the number of minus signs stays even. Here the invariant is the only obstacle: every row with an even number of minus signs *can* be reached from the all-plus row, by pairing up the positions that should hold minus signs and flipping one pair at a time. In general, though, an invariant can only show that a state is unreachable; to show that a state can be reached, we must exhibit moves that get there.

An invariant also does not show that a process comes to an end. Nothing in the sign-flipping rules forces us to stop: we can flip the first two signs, flip them back, and repeat forever, and every row along the way has product 1. For a process that is meant to finish, we need a separate argument that it makes *progress*. Euclid's algorithm, in the next section, needs both kinds of argument.

5.5 Euclid's algorithm: why it is right and why it stops

Recall from Section 5.2 that a positive integer d is a divisor of an integer a when $a = dk$ for some integer k. We also say that d *divides* a, and we write $d \mid a$. Do not confuse the statement $d \mid a$, which is true or false, with the number d/a: for example, $6 \mid 84$ is true, because $84 = 6 \cdot 14$, while $6/84 = 1/14$. Every positive integer d divides 0, because $0 = d \cdot 0$.

Let a and b be nonnegative integers that are not both zero. A *common divisor* of a and b is a positive integer that divides both of them. Their *greatest common divisor*, written $\gcd(a, b)$, is the largest common divisor. For example, the positive divisors of 12 are $1, 2, 3, 4, 6, 12$ and those of 18 are $1, 2, 3, 6, 9, 18$, so the common divisors of 12 and 18 are $1, 2, 3, 6$, and $\gcd(12, 18) = 6$. The definition has to exclude $a = b = 0$: every positive integer divides 0, so 0 and 0 have no largest common divisor. Every other pair has one. Its common divisors include 1, and they all lie among $1, 2, \ldots, c$, where c is the larger of a and b, which is positive: as in the proof of Proposition 5.2, a positive divisor d of c satisfies $c = dk$ with $k = c/d$ a positive integer, so $k \geq 1$ and $d \leq c$. A nonempty finite set of integers has a largest member.

Division with remainder

Let a be any integer and b a positive integer. Then there are unique integers q and r with

$$a = qb + r, \qquad 0 \leq r < b.$$

The integer q is the *quotient*; when $a \geq 0$, it counts how many whole copies of b fit into a. The integer r is the *remainder*, what is left over, also written $a \bmod b$.

For example, the largest multiple of 5 that is at most 17 is $15 = 3 \cdot 5$, so $17 = 3 \cdot 5 + 2$ and $17 \bmod 5 = 2$. The equation $17 = 2 \cdot 5 + 7$ is also true, but it is not division with remainder, because 7 is not less than 5. For a negative number, the largest multiple of 5 that is at most -7 is $-10 = (-2) \cdot 5$, so $-7 = (-2) \cdot 5 + 3$ and $(-7) \bmod 5 = 3$. The remainder is never negative.

The well-ordering property shows where q and r come from. Let S be the set of all numbers of the form $a - qb$, with q an integer, that are at least 0; for $a = 17$ and $b = 5$, for example, $S = \{2, 7, 12, 17, 22, \ldots\}$. The set S is never empty: if $a \geq 0$, then $q = 0$ puts a itself in S, and if $a < 0$, then $q = a$ puts $a - ab = (-a)(b - 1)$ in S, because $-a$ is positive and $b - 1 \geq 0$. So S is a nonempty set of natural numbers, and by the well-ordering property (in the form of Section 5.2, which allows 0) it has a smallest member. Call it r, and let q be an integer with $r = a - qb$. Then $a = qb + r$ and $r \geq 0$. Also $r < b$: if $r \geq b$, then $r - b = a - (q + 1)b$ would also belong to S and be smaller than r, which is impossible.

No other pair of integers works. Suppose also that $a = q'b + r'$ with $0 \leq r' < b$. Subtracting $a = qb + r$ gives $0 = (q' - q)b + (r' - r)$, so $r' - r = (q - q')b$. Since r and r' both lie between

0 and $b - 1$, their difference satisfies $-b < r' - r < b$, so $-b < (q - q')b < b$. Dividing by the positive number b shows that the integer $q - q'$ lies strictly between -1 and 1, so $q - q' = 0$. Hence $q' = q$, and then $r' = a - qb = r$.

One step of Euclid's algorithm

An *algorithm* is a list of precise instructions that can be followed mechanically, one step at a time, like the school method for long division. Euclid's algorithm, from Euclid's *Elements* (around 300 BC), computes greatest common divisors by repeating a single step: replace the pair (a, b) by (b, r), where $r = a \bmod b$. For example, $(84, 30)$ is replaced by $(30, 24)$, because $84 = 2 \cdot 30 + 24$. The common divisors of 84 and 30 are $1, 2, 3, 6$, and so are those of 30 and 24. This is no coincidence.

Lemma 5.3. *Let $a \geq 0$ and $b > 0$ be integers, and write $a = qb + r$ with integers q, r and $0 \leq r < b$. Then the common divisors of a and b are exactly the common divisors of b and r. Consequently* $\gcd(a, b) = \gcd(b, r)$.

Proof. Let d be a positive integer. First suppose that d divides both a and b, say $a = ds$ and $b = dt$ with integers s and t. Then

$$r = a - qb = ds - qdt = d(s - qt).$$

Since $s - qt$ is an integer, d divides r. So every common divisor of a and b is a common divisor of b and r.

Conversely, suppose that d divides both b and r, say $b = dt$ and $r = du$ with integers t and u. Then

$$a = qb + r = qdt + du = d(qt + u),$$

so d divides a as well. So every common divisor of b and r is a common divisor of a and b.

The two directions together show that the pairs (a, b) and (b, r) have exactly the same set of common divisors. Both greatest common divisors are defined, because $b > 0$. Each is the largest member of the same set, so they are equal. □

Both directions are needed. With only the first, the pair (b, r) might have an extra common divisor, larger than every common divisor of a and b, and then $\gcd(b, r)$ would be bigger than $\gcd(a, b)$.

The algorithm, and why its answer is right

Euclid's algorithm starts with a pair (a, b) of nonnegative integers, not both zero. To keep the starting pair in view, we call the current pair (u, v).

1. Start with $(u, v) = (a, b)$.
2. If $v = 0$, stop and output u.
3. Otherwise replace (u, v) by $(v,\ u \bmod v)$ and go back to step 2.

For $(a, b) = (84, 30)$ the divisions are

$$84 = 2 \cdot 30 + 24, \qquad 30 = 1 \cdot 24 + 6, \qquad 24 = 4 \cdot 6 + 0.$$

The successive pairs are $(84, 30)$, $(30, 24)$, $(24, 6)$, $(6, 0)$, and the algorithm outputs 6. If the first number is smaller than the second, the first step simply swaps them: for $(30, 84)$, we have $30 = 0 \cdot 84 + 30$, so the next pair is $(84, 30)$.

Why is the output the greatest common divisor? Every number in the pairs is nonnegative, because a and b are and a remainder is never negative. A step is taken only when v is positive, so Lemma 5.3, with u and v in place of a and b, applies to every step: no step changes the greatest common divisor of the current pair. By induction on the number of steps, $\gcd(u, v) = \gcd(a, b)$ for every pair that the algorithm produces. This is the invariant of the algorithm.

When the algorithm stops, the current pair has the form $(g, 0)$ with $g > 0$. Indeed, if no step was taken, then $b = 0$ and $g = a$, which is positive because a and b are not both zero; otherwise g was the second number of the previous pair, which was positive. Since every positive integer divides 0, the common divisors of g and 0 are exactly the positive divisors of g, the largest of which is g. So $\gcd(g, 0) = g$, and by the invariant, $g = \gcd(a, b)$.

Why the algorithm stops

The invariant shows that *if* the algorithm stops, its answer is correct; to show that it stops, we need a measure of progress. Watch the second number v of the current pair. Each step replaces it by $u \bmod v$, which is less than v. So the second number is a nonnegative integer that strictly decreases at every step.

An integer that decreases goes down by at least 1: if w and v are integers with $w < v$, then $w \le v - 1$. The second number starts at b, so by induction on s, after s steps it is at most $b - s$. It is never negative, so $0 \le b - s$, that is, $s \le b$. Hence at most b steps can occur, so the algorithm stops within b steps. For $(84, 30)$ the second numbers were $30, 24, 6, 0$: the bound allows at most 30 steps, and in fact 3 were enough. The same reasoning shows that a strictly decreasing list of nonnegative integers cannot go on forever: if it starts at c, it has at most $c + 1$ entries.

Pause and decide. Does a strictly decreasing nonnegative real quantity prove that a process stops?

Check. No. The gaps $1 - x_n = 2^{-n}$ of Section 5.3, namely $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$, decrease forever and remain positive. A nonnegative *integer* must drop by at least 1 each time it decreases, but a real number can decrease by smaller and smaller amounts and never run out of room.

> **Habit 5: separate preservation from progress**
>
> For a process that repeats one step, ask two separate questions. What stays true after every step? And what quantity measures progress: a nonnegative integer that must decrease, or an error that must shrink? A preserved property can prove that an answer is correct without proving that the process ever stops. An error that keeps shrinking shows that the process approaches the answer, not that it stops.

Exercises

Exercises 5.1–5.4 are core exercises, and Exercises 5.5 and 5.6 are transfer exercises. Complete solutions are in Appendix A, Section A.5.

Exercise 5.1. Prove by induction that $\sum_{j=1}^{n} j = n(n+1)/2$ for every $n \in \mathbb{N}$. Write out every line of algebra in the induction step.

Exercise 5.2. Let $y_0 = 3$ and $y_{n+1} = 2y_n$ for every $n \in \mathbb{N}$. Compute y_1, y_2 and y_3, guess a formula for y_n, and prove your formula by induction.

Exercise 5.3. Run Euclid's algorithm on the pair $(1071, 462)$. Record each quotient and remainder, the successive pairs, and the greatest common divisor that the algorithm outputs.

Exercise 5.4. A row of five signs starts with all five signs plus. As in Section 5.4, a legal move flips the signs in two different positions. Can some sequence of legal moves produce a row with exactly three minus signs? Justify your answer without listing sequences of moves.

Exercise 5.5. For the recurrence $x_{n+1} = (x_n + 1)/2$ with an arbitrary real starting value x_0, prove the error formula $x_n - 1 = 2^{-n}(x_0 - 1)$ for every $n \in \mathbb{N}$ by induction. Write out both the base case and the induction step in full. Then explain why the error $x_n - 1$ is never exactly zero when $x_0 \neq 1$.

Exercise 5.6. A proof establishes $P(0)$, and it shows that for every natural number k, if $P(k)$ is true, then $P(k+2)$ is true. For which natural numbers n does it follow that $P(n)$ is true? Which additional base case would give $P(n)$ for every natural number n? Explain why.

CHAPTER 6

Sameness, symmetry, and structure

> **Overview**
> Recognize when two different descriptions name the same object, and say exactly which differences are being ignored. Check that a rule defined on equivalence classes does not depend on which representatives are chosen. Identify maps that preserve a structure, and see why the symmetries of an object form a group. Use symmetry to simplify a problem without silently discarding cases. Compare objects with a partial order, and tell a maximal element from a maximum.

6.1 Which differences matter?

The fractions $1/2$ and $2/4$ look different, but they name the same rational number. Similarly, suppose Alice knows Ben and Ben knows Chen, but Alice and Chen have never met. Replacing Alice, Ben, and Chen by 1, 2, 3 gives "1 is linked to 2, and 2 is linked to 3." The labels have changed, but the pattern of connections has not. (Chapter 7 studies such patterns under the name *graphs*.)

In both examples we have chosen to ignore certain differences: how a fraction is written, or what the people are called. Such a choice should be stated precisely, because whether two things count as "the same" depends on the question: if the question is "what is the numerator?", then $1/2$ and $2/4$ give different answers.

Relations and equivalence relations

To describe sameness precisely, we need the idea of a relation. A *relation* on a set A is a rule that tells us, for every ordered pair (x, y) of elements of A, whether x is related to y; when it is, we write $x \sim y$. Two familiar examples are the order $x < y$ on the integers and "x has the same birth month as y" on a set of people. The pair is ordered because the answer can depend on the order: $1 < 2$ is true, but $2 < 1$ is false. A relation is recorded completely by the set of pairs (x, y) with $x \sim y$, a subset of the Cartesian product $A \times A$ of Section 3.1.

Definition 6.1 (Equivalence relation)**.** A relation $\sim$ on a set A is an *equivalence relation* if it has the following three properties.

- *Reflexivity:* $x \sim x$ for every $x \in A$.
- *Symmetry:* for all $x, y \in A$, if $x \sim y$, then $y \sim x$.
- *Transitivity:* for all $x, y, z \in A$, if $x \sim y$ and $y \sim z$, then $x \sim z$.

To show that a relation is an equivalence relation, we must check all three properties for arbitrary

elements of A, not only for a few convenient ones. For instance, on the students of a course, let $x \sim y$ mean that x and y were born in the same month. For any students x, y, z: certainly x shares a birth month with x (reflexivity); if x shares a birth month with y, then y shares one with x (symmetry); and if x and y share a birth month, and so do y and z, then all three were born in y's month, so x and z share it (transitivity). So this is an equivalence relation.

An equivalence relation sorts the elements of its set into separate classes. The *equivalence class* of an element $x \in A$ is the set of all elements of A that are related to x:

$$[x] = \{y \in A : y \sim x\}.$$

For birth months, the class of a student born in March is the set of all students in the course born in March, and every student belongs to exactly one class. Proposition 6.2 in Section 6.2 proves that every equivalence relation sorts its set in this clean way.

Many natural relations are not equivalence relations; relations of rough similarity usually fail transitivity. For example, on the real numbers, let $x \approx y$ mean $|x - y| \leq 1$. This relation is reflexive, because $|x - x| = 0$, and symmetric, because $|x - y| = |y - x|$. It is not transitive: $0 \approx 0.8$ and $0.8 \approx 1.6$, but $|0 - 1.6| = 1.6 > 1$. So "numbers close to each other" cannot be sorted into separate classes: the numbers close to 0 form the interval $[-1, 1]$, and those close to 0.8 form $[-0.2, 1.8]$, two sets that overlap (both contain 0.5) without being equal (only the first contains -1). Proposition 6.2 will show that the classes of an equivalence relation never overlap in this partial way; transitivity, which $\approx$ lacks, is what rules it out.

Pause and decide. Is the relation $x \leq y$ on the integers an equivalence relation? Check each of the three properties.

Check. No. Reflexivity holds, since $x \leq x$, and so does transitivity: if $x \leq y$ and $y \leq z$, then $x \leq z$. Symmetry fails, because $1 \leq 2$ is true but $2 \leq 1$ is false, and one failure is enough. The relation $\leq$ describes comparison rather than sameness; Section 6.6 studies relations of that kind.

6.2 Congruence and residue classes

Fix a positive integer m. For integers a and b, we write

$$a \equiv b \pmod{m},$$

and say that a is *congruent to b modulo m*, when m divides $a - b$ (Section 5.5), that is, when $a - b = mk$ for some integer k. A clock gives an everyday example with $m = 12$: the times 15:00 and 3:00 put the hour hand in the same place, because $15 - 3 = 12$. For $m = 4$, we have $1 \equiv 5$ and $1 \equiv -3 \pmod 4$, since $1 - 5 = 4 \cdot (-1)$ and $1 - (-3) = 4 \cdot 1$, but $1 \not\equiv 2 \pmod 4$, because $1 - 2 = -1$ is not a multiple of four.

Congruence modulo m is an equivalence relation on $\mathbb{Z}$. We check the three properties of Definition 6.1 for arbitrary integers a, b, c.

- *Reflexivity.* $a - a = 0 = m \cdot 0$, so $a \equiv a \pmod m$.

- *Symmetry.* If $a \equiv b \pmod{m}$, then $a - b = mk$ for an integer k, so $b - a = m(-k)$ with $-k$ an integer, and $b \equiv a \pmod{m}$.
- *Transitivity.* If $a \equiv b$ and $b \equiv c \pmod{m}$, then $a - b = mk$ and $b - c = m\ell$ for integers k and ℓ. So
$$a - c = (a - b) + (b - c) = mk + m\ell = m(k + \ell),$$
with $k + \ell$ an integer, so $a \equiv c \pmod{m}$.

By symmetry, it does not matter whether we test $a - b$ or $b - a$: one is a multiple of m exactly when the other is.

Residue classes

The equivalence class of an integer a under this relation is called its *residue class modulo m* and is written $[a]_m$. (*Residue* is an old word for remainder.) An integer x satisfies $x \equiv a \pmod{m}$ exactly when $x = a + mk$ for some integer k, so

$$[a]_m = \{x \in \mathbb{Z} : x \equiv a \pmod{m}\} = \{a + mk : k \in \mathbb{Z}\}.$$

For modulus four, the class of 1 is

$$[1]_4 = \{\ldots, -7, -3, 1, 5, 9, \ldots\} :$$

start at 1 and move up or down in steps of four. Any member of a class can name it, and is then called a *representative* of the class. Starting at 5 and moving in steps of four reaches exactly the same integers, so $[5]_4 = [1]_4$, even though $5 \neq 1$. Keep three levels apart here: the integer 1; the set $[1]_4$ of all integers congruent to it; and the collection of all residue classes modulo four, described at the end of this section.

By contrast, $[2]_4 = \{\ldots, -6, -2, 2, 6, 10, \ldots\}$ has no member in common with $[1]_4$. In general, two classes of an equivalence relation are either equal or disjoint.

Proposition 6.2. *Let $\sim$ be an equivalence relation on a set A, and let $x, y \in A$.*

(a) $x \in [x]$.

(b) $[x] = [y]$ *if and only if* $x \sim y$.

(c) The classes $[x]$ *and* $[y]$ *are either equal or disjoint.*

Proof. (a) By reflexivity, $x \sim x$, and this says that $x \in [x]$.

(b) First suppose $[x] = [y]$. By part (a), $x \in [x]$, so $x \in [y]$, which means $x \sim y$.

Conversely, suppose $x \sim y$. Let $z \in [x]$, so that $z \sim x$. Together with $x \sim y$, transitivity gives $z \sim y$, so $z \in [y]$. This proves $[x] \subseteq [y]$. For the reverse containment, symmetry turns $x \sim y$ into $y \sim x$, and the same argument with x and y exchanged gives $[y] \subseteq [x]$. So $[x] = [y]$.

(c) Suppose $[x]$ and $[y]$ are not disjoint. Then some element z belongs to both, so $z \sim x$ and $z \sim y$. Symmetry turns $z \sim x$ into $x \sim z$, and transitivity applied to $x \sim z$ and $z \sim y$ gives $x \sim y$. By part (b), $[x] = [y]$. □

So reflexivity puts every element into its own class, and symmetry and transitivity prevent classes from overlapping partly.

A *partition* of a set A is a collection of nonempty subsets of A, called the *pieces*, such that every element of A lies in exactly one piece. By Proposition 6.2, the equivalence classes of any equivalence relation form a partition. Each class $[x]$ is nonempty, and every element x lies in at least one class, namely $[x]$, both by part (a). No element lies in two different classes, because by part (c) two classes that share an element are equal. The birth-month classes of Section 6.1 are an example.

For congruence, part (b) gives a test that we will use again and again:

$$[a]_m = [b]_m \quad \text{exactly when} \quad a \equiv b \pmod{m}, \text{ that is, when } m \text{ divides } a - b.$$

A standard name for each class

Every residue class modulo m contains *exactly one* of the integers $0, 1, \ldots, m-1$.

At least one. Let a be any integer. Division with remainder (Section 5.5) gives integers q and r with $a = qm + r$ and $0 \le r < m$. Then $r - a = m \cdot (-q)$, so $r \equiv a \pmod{m}$, and r belongs to $[a]_m$.

At most one. Suppose, for a contradiction, that two different integers $r < s$ from the list $0, 1, \ldots, m-1$ both belong to $[a]_m$. Then $s \equiv a$ and, by symmetry, $a \equiv r$, so transitivity gives $s \equiv r \pmod{m}$, and $s - r = mk$ for some integer k. Since $mk = s - r$ and m are positive, so is k (if $k \le 0$, then $mk \le 0$); hence $k \ge 1$ and $s - r = mk \ge m$. This contradicts $s - r \le m - 1$, which holds because $s \le m - 1$ and $r \ge 0$.

It follows that there are exactly m residue classes modulo m, namely

$$[0]_m,\ [1]_m,\ \ldots,\ [m-1]_m.$$

Every class is one of these: if r is the member of $[a]_m$ in the list, then $r \equiv a$, so $[a]_m = [r]_m$ by the test above. And no two of them are equal: if $[r]_m = [s]_m$ for different r and s in the list, this one class would contain both r and s by part (a) of Proposition 6.2, which we have just ruled out.

The "at least one" argument also tells us which integer of the list lies in $[a]_m$: it is the remainder of a on division by m, which Section 5.5 writes as $a \bmod m$. This gives a second form of the test: *two integers are congruent modulo m exactly when they have the same remainder on division by m.* Indeed, if $a \equiv b \pmod{m}$, then $[a]_m = [b]_m$, and the one member of this class in the list is both $a \bmod m$ and $b \bmod m$. Conversely, if both remainders equal r, then $a \equiv r$ and $b \equiv r$, so $a \equiv b$ by symmetry and transitivity. For example, $17 \equiv -7 \pmod{4}$, since $17 - (-7) = 24 = 4 \cdot 6$, and indeed $17 \bmod 4 = 1 = (-7) \bmod 4$.

The word "mod" is thus used in two ways. The statement $17 \equiv -7 \pmod{4}$ relates two integers and is true or false; the expression $17 \bmod 4$ is a single integer, the remainder 1. So $17 \equiv 1 \pmod{4}$ is true, while $17 = 1$ is false.

Pause and decide. Which of the integers $0, 1, \ldots, 6$ belongs to the class $[-20]_7$? Name two other representatives of the same class.

Check. Division with remainder gives $-20 = (-3) \cdot 7 + 1$ with $0 \le 1 < 7$, so the answer is 1, and $[-20]_7 = [1]_7$. Any integer $1 + 7k$ is another representative, for instance 8 and -6. A common slip is to write $-20 = (-2) \cdot 7 - 6$ and report -6. That equation is true, but a remainder always lies between 0 and $m - 1$, even for a negative number.

The set of all residue classes modulo m is written $\mathbb{Z}/m\mathbb{Z}$, where $m\mathbb{Z}$ stands for the set of all multiples of m. Read it as "the integers, with differences that are multiples of m ignored." For example,

$$\mathbb{Z}/4\mathbb{Z} = \{[0]_4, [1]_4, [2]_4, [3]_4\}$$

is a set with four elements, each of which is itself an infinite set of integers. More generally, the set of all equivalence classes of an equivalence relation on a set A is called the *quotient* of A by the relation, and $\mathbb{Z}/m\mathbb{Z}$ is the quotient of $\mathbb{Z}$ by congruence modulo m.

6.3 Well-definedness: does the answer depend on the representative?

We would like to add residue classes by the natural rule: choose a representative of each class, add the two integers, and take the class of the sum,

$$[a]_m + [b]_m = [a + b]_m.$$

But a class has many representatives, and the rule does not say which ones to choose. If two people choose different representatives of the same two classes, do they always get the same answer? If not, the rule does not define an addition of classes at all. Try this with $[1]_4 + [2]_4$. Using the representatives 1 and 2 gives $[3]_4$. Choosing 5 and 6 instead gives $[11]_4$, which is the same class, because $11 - 3 = 8$ is a multiple of four. The integer sum changes, but its class does not. Here is why this always happens.

Suppose the first person uses a and b, and the second uses a' and b', where $[a']_m = [a]_m$ and $[b']_m = [b]_m$. By Proposition 6.2(b), $a' \equiv a$ and $b' \equiv b \pmod m$, so $a' = a + mk$ and $b' = b + m\ell$ for some integers k and ℓ. Then

$$a' + b' = a + b + m(k + \ell).$$

Since $k + \ell$ is an integer, m divides $(a' + b') - (a + b)$, and so $[a' + b']_m = [a + b]_m$.

A rule whose inputs are classes, but which is computed from chosen representatives, is called *well-defined* when its output depends only on the classes and not on which representatives were chosen. We have just shown that addition of residue classes is well-defined. Notice that the calculation did not prove $a' + b' = a + b$, which is usually false: $5 + 6 = 11$, not 3. It proved that the two sums differ by a multiple of m, which is exactly what is needed for the two output classes to be equal. Before asking whether two outputs agree, ask what kind of object the output is.

Rules that fail, and how to spot them

Try to define an integer-valued function F on $\mathbb{Z}/4\mathbb{Z}$ by $F([a]_4) = a$. Since $[1]_4 = [5]_4$, the rule would give both $F([1]_4) = 1$ and $F([5]_4) = 5$ for the same input. A function must give exactly

one output for each input (Section 3.2), so this rule does not define a function. We can repair it by saying which representative to use: let $F([a]_4)$ be the unique member of $[a]_4$ in $\{0, 1, 2, 3\}$, that is, a mod 4 (Section 6.2). Now the output is determined by the class itself; for instance, $F([5]_4) = F([1]_4) = F([-7]_4) = 1$.

The same check catches proposals that sound reasonable, such as "the parity of a class modulo three is the parity of any of its representatives." Here 0 and 3 represent the same class modulo three, and 0 is even while 3 is odd, so the rule does not define anything. A single pair of representatives that gives different answers is enough to show that a rule is not well-defined. To show that a rule *is* well-defined, we need an argument that covers all representatives, like the calculation for addition above. Does the parity rule fare better for classes modulo four?

Pause and decide. Does the rule $G([a]_4) = [a]_2$ depend on the representative? What does the answer tell you about the parity of the members of a class modulo four?

Check. It does not. If $[a']_4 = [a]_4$, then $a' = a + 4k$ for an integer k, so $a' - a = 2(2k)$ and $[a']_2 = [a]_2$. So G is a well-defined function from $\mathbb{Z}/4\mathbb{Z}$ to $\mathbb{Z}/2\mathbb{Z}$. Since $[0]_2$ is the set of even integers and $[1]_2$ the set of odd integers, $G([a]_4)$ records the parity of a, and all members of a class modulo four have the same parity. The step that made this work is that 2 divides 4; Exercise 6.5 explores this further.

The map G loses information: it sends both $[0]_4$ and $[2]_4$ to $[0]_2$, so it is not injective. Yet it respects addition. Addition on $\mathbb{Z}/2\mathbb{Z}$ is well-defined by the calculation above with $m = 2$, and for all integers a and b,

$$G([a]_4 + [b]_4) = G([a + b]_4) = [a + b]_2 = [a]_2 + [b]_2 = G([a]_4) + G([b]_4).$$

(The first and third equalities are the rules for adding classes, and the second and fourth are the definition of G.) So adding first and then applying G gives the same result as applying G first and then adding: a map can respect addition without keeping every distinction between its inputs. Section 6.4 gives this property a name.

6.4 Structure-preserving maps

A *structure* on a set is extra information that the set carries, such as an operation or a relation. An *operation* on a set S takes two elements of S and returns an element of S; in the language of Section 3.1, it is a function from $S \times S$ to S. Addition of integers and addition of residue classes are operations. A *graph* (Chapter 7) is a set of points, called *vertices*, together with the information of which pairs of vertices are joined; joined vertices are called *adjacent*.

For sets that have an addition, a map T *preserves addition* when

$$T(x + y) = T(x) + T(y) \qquad \text{for all } x \text{ and } y \text{ in its domain.}$$

The sum on the left is computed in the domain and the sum on the right in the codomain, and these may be different kinds of addition, as for the map G at the end of Section 6.3. For a simpler example, $V : \mathbb{Z} \to \mathbb{Z}$ with $V(x) = 3x$ preserves addition, because $3(x + y) = 3x + 3y$. The map

$U : \mathbb{Z} \to \mathbb{Z}$ with $U(x) = x + 1$ does not: $U(x + y) = x + y + 1$, while $U(x) + U(y) = x + y + 2$. Preserving addition is part of the definition of a linear map in Chapter 10.

For graphs, the structure is adjacency. An *isomorphism* from one graph to another is a bijection f from the vertices of the first graph to the vertices of the second that preserves adjacency in both directions: for all vertices u and v of the first graph, if u and v are adjacent, then $f(u)$ and $f(v)$ are adjacent, and if $f(u)$ and $f(v)$ are adjacent, then u and v are adjacent. In the acquaintance example of Section 6.1, the map with $f(\text{Alice}) = 1$, $f(\text{Ben}) = 2$, $f(\text{Chen}) = 3$ is an isomorphism onto the pattern in which 1 is linked to 2 and 2 to 3. The bijection with $g(\text{Alice}) = 2$, $g(\text{Ben}) = 1$, $g(\text{Chen}) = 3$ is not: Ben and Chen know each other, but 1 and 3 are not linked. The second direction matters too: if 1, 2, and 3 were all linked to one another, f would still send linked people to linked numbers, but it would send Alice and Chen, who have never met, to the linked pair 1 and 3. So the second direction fails, as it should: a chain of three and a triangle are different patterns.

That second condition says that the inverse map f^{-1} also preserves adjacency. In general, an *isomorphism* is a bijection that preserves the structure and whose inverse also preserves it. What must be preserved depends on the subject: adjacency for graphs, addition for sets with an addition, distance for the sets with a distance of Chapter 11. Two objects joined by an isomorphism are called *isomorphic*; they differ only in the names of their elements.

Symmetries

A *symmetry* of an object is an isomorphism from the object to itself. (This meaning of the word is unrelated to the symmetry property in Definition 6.1.) Picture four seats around a round table, labeled 0, 1, 2, 3 in order, and take as the structure the information of which seats are next to each other, so that the seats are the vertices of a graph. Moving everyone one seat along is a symmetry. As a map on seat labels, it is

$$r(i) = (i + 1) \bmod 4,$$

so $r(0) = 1$, $r(1) = 2$, $r(2) = 3$, and $r(3) = 0$. Each seat is reached exactly once, so r is a bijection. The neighboring pairs $0, 1$ and $1, 2$ and $2, 3$ and $3, 0$ go to $1, 2$ and $2, 3$ and $3, 0$ and $0, 1$, again neighbors, and the opposite pairs $0, 2$ and $1, 3$ go to $1, 3$ and $2, 0$, again opposite. So two seats are neighbors exactly when their images are. The inverse of r is rotation by three seats, because $1 + 3 = 4 \equiv 0 \pmod 4$.

The symmetries of an object have four useful features. Let f, g, h be symmetries.

- *Closure.* Doing g and then f is again a symmetry. The composition $f \circ g$ is a bijection, because compositions of injective maps are injective (Theorem 3.2) and compositions of surjective maps are surjective (Exercise 3.2). It preserves the structure: for a graph, if u and v are adjacent, then so are $g(u)$ and $g(v)$, because g is a symmetry, and then so are $f(g(u))$ and $f(g(v))$, because f is one. Its inverse is $g^{-1} \circ f^{-1}$, since $(f \circ g)(g^{-1}(f^{-1}(y))) = f(f^{-1}(y)) = y$ and $g^{-1}(f^{-1}(f(g(x)))) = g^{-1}(g(x)) = x$ for all x and y, and this inverse preserves the structure for the same reason.
- *Associativity.* $(f \circ g) \circ h = f \circ (g \circ h)$, because for every input x both sides give $f(g(h(x)))$: apply h, then g, then f.

- *Identity.* The identity map of Section 3.2, which we write simply id rather than id_A, is a bijection, is its own inverse, and changes nothing, so it is a symmetry. Moreover $f \circ \mathrm{id} = \mathrm{id} \circ f = f$.
- *Inverses.* A symmetry f is a bijection, so it has an inverse f^{-1} (Section 3.3), and $f \circ f^{-1} = f^{-1} \circ f = \mathrm{id}$. These equations also say that f is an inverse of f^{-1}, so f^{-1} is a bijection, because a map with an inverse is bijective (Exercise 3.5). It preserves the structure, because an isomorphism preserves the structure in both directions. So f^{-1} is again a symmetry.

These four features occur so often that they have a name.

Definition 6.3 (Group)**.** A *group* is a set S together with an operation $*$ on S, which combines any two elements $x, y \in S$ into an element $x * y \in S$, such that the following hold.

- *Associativity:* $(x * y) * z = x * (y * z)$ for all $x, y, z \in S$.
- *Identity:* there is an element $e \in S$ with $e * x = x * e = x$ for every $x \in S$.
- *Inverses:* for every $x \in S$ there is an element $y \in S$ with $x * y = y * x = e$, where e is the identity element from the previous condition.

Closure is built into the definition, because an operation on S must return an element of S. So the four features say that the symmetries of an object form a group under composition. The integers form a group under addition: addition is associative, 0 is an identity, and $-a$ is an inverse of a. So does $\mathbb{Z}/m\mathbb{Z}$ under the addition of Section 6.3, with identity $[0]_m$ and $[-a]_m$ as an inverse of $[a]_m$, since $[a]_m + [-a]_m = [0]_m$. Associativity is inherited from the integers: both $([a]_m + [b]_m) + [c]_m$ and $[a]_m + ([b]_m + [c]_m)$ equal $[a + b + c]_m$. Not every operation gives a group. The integers under multiplication have the identity 1, but 2 has no inverse, because no integer y satisfies $2y = 1$.

The definition does not require that $x * y = y * x$. A group in which this holds for all x and y is called *commutative*. The integers under addition form a commutative group, but many groups of symmetries do not. Place three letters in a row as ABC. Let s swap the letters in the first two positions, and let t swap the letters in the last two positions. Doing s and then t gives

$$ABC \;\to\; BAC \;\to\; BCA,$$

while doing t and then s gives

$$ABC \;\to\; ACB \;\to\; CAB.$$

So $t \circ s \neq s \circ t$. The rearrangements of three positions, that is, the bijections from a three-element set to itself, are the symmetries of a set with no extra structure, and they form a group that s and t show is not commutative.

Back at the table, let r_j move everyone j seats along, so that $r_j(i) = (i + j) \bmod 4$ for $j = 0, 1, 2, 3$; thus $r_1 = r$. Applying r_3 and then r_2 moves everyone five seats, which is the same as moving one seat, so $r_2 \circ r_3 = r_1$. In general, composing two rotations adds their amounts modulo four, so the result is again a rotation. The identity is r_0, the rotations r_1 and r_3 are inverses of each other, and r_2 is its own inverse (Exercise 6.3). Associativity holds for all compositions of maps. So the four rotations form a group on their own, and a commutative one, because addition of their amounts is commutative.

The rotation r_j thus behaves exactly like the residue class $[j]_4$. To say this precisely, let φ send $[j]_4$ to r_j, for $j = 0, 1, 2, 3$. Since every class modulo four is exactly one of $[0]_4$, $[1]_4$, $[2]_4$, $[3]_4$

(Section 6.2), φ is a well-defined function from $\mathbb{Z}/4\mathbb{Z}$ to the set of rotations, and a bijection, since the four classes go to the four rotations, one each. It preserves the group operation:

$$\varphi(x + y) = \varphi(x) \circ \varphi(y) \qquad \text{for all classes } x \text{ and } y \text{ modulo four.}$$

Indeed, take $x = [j]_4$ and $y = [k]_4$ with $j, k \in \{0, 1, 2, 3\}$, and let $c = (j + k) \bmod 4$. Then $x + y = [c]_4$, so $\varphi(x + y) = r_c$, while $r_j \circ r_k$ moves everyone k seats and then j more, which is r_c as well. Read backward, the same calculation shows that φ^{-1} preserves the structure: $\varphi^{-1}(r_j \circ r_k) = [c]_4 = [j]_4 + [k]_4 = \varphi^{-1}(r_j) + \varphi^{-1}(r_k)$. So φ is an isomorphism from the group $\mathbb{Z}/4\mathbb{Z}$ to the group of rotations: the two differ only in the names of their elements.

6.5 What symmetry permits us to assume

Consider a sequence of signs $a = (a_0, a_1, \ldots, a_{n-1})$ with $n \geq 1$, where each entry a_i is 1 or -1; we number the positions from 0, so the last entry is a_{n-1}. Rows of signs like this appeared in Section 5.4, and Chapter 14 studies them in depth. A *pair product* is a product $a_i a_j$ of two entries of the sequence. Negating every sign turns a into $-a = (-a_0, -a_1, \ldots, -a_{n-1})$ and does not change any pair product, because

$$(-a_i)(-a_j) = a_i a_j.$$

For example, $(-1, 1, 1, -1)$ and its negation $(1, -1, -1, 1)$ have the same products of neighboring entries: -1, 1, -1. So any quantity built only from pair products, such as the sum of neighboring products $a_0a_1 + a_1a_2 + \cdots + a_{n-2}a_{n-1}$, takes the same value on a and on $-a$.

Say that two sign sequences a and b of the same length *have the same pair products* if $a_i a_j = b_i b_j$ for all positions i and j. A property of sign sequences *depends only on the pair products* if any two sequences with the same pair products either both have the property or both lack it; "the sum of neighboring products is -1" is an example. When we study such a property, we may assume that the first sign is 1: if a sequence starts with -1, negate it.

Replacing every object by a convenient standard form in this way is called a *normalization*. Two facts make this one legitimate.

- The change preserves the property under study: since a and $-a$ have the same pair products, a sequence has the property exactly when its negation has it.
- Every case can be brought into the standard form: a sequence either begins with 1 already, or it begins with -1 and its negation begins with 1.

If either fact fails, the reduction fails too. For a failure of the first fact, consider the claim that the two signs of a sequence of length two add up to at least 0. It holds for the sequences $(1, 1)$ and $(1, -1)$ that begin with 1, with sums 2 and 0, but it is false for $(-1, -1)$, with sum -2. Negation turns the sum into its negative, so it does not preserve the property, and checking the sequences that begin with 1 told us nothing about the others.

The second fact can fail as well. Negation can always make the first sign equal to 1, but not always the first and the last: $(1, -1)$ and its negation $(-1, 1)$ each have one sign of each kind. So the normalization "the first and last signs are both 1" leaves such sequences out, even for a property

preserved by negation. For example, the property $a_0 a_{n-1} = 1$ depends only on a pair product and holds whenever the first and last signs are both 1, yet it fails for $(1, -1)$. Checking only the normalized sequences would wrongly suggest that it always holds.

What "without loss of generality" must justify

A reduction like this is often announced with the phrase *without loss of generality*: "Without loss of generality, $a_0 = 1$." Read the phrase as a promise that no case has been lost, not as permission to add a convenient extra condition. It must be backed by the two facts above.

Here is the argument behind the promise for sign sequences. Let P be a property of sign sequences that depends only on the pair products, and suppose we have proved that P holds for every sign sequence of length n whose first entry is 1. Let a be any sign sequence of length n. If $a_0 = 1$, we are done. If $a_0 = -1$, put $b = -a$. Then $b_0 = 1$, so P holds for b. Since a and b have the same pair products and P depends only on these, P holds for a as well. So P holds for every sign sequence of length n. Both facts were used: the second to produce b with $b_0 = 1$, and the first to carry P from b back to a.

Pause and decide. You want to prove that $a_0 a_1 + a_0 a_2 + a_1 a_2 \geq -1$ for every sign sequence (a_0, a_1, a_2). Is it justified to assume $a_0 = 1$? If so, how many sequences are left to check? Check them.

Check. It is justified: the expression is built only from pair products, so negating all three signs does not change its value. With $a_0 = 1$, four sequences remain: $(1, 1, 1)$, $(1, 1, -1)$, $(1, -1, 1)$, and $(1, -1, -1)$. The expression equals 3, -1, -1, and -1 on them, all at least -1, so by the normalization the inequality holds for all eight sequences.

6.6 Order is a different structure

Equivalence relations describe sameness. Other relations describe *comparison*: one object is smaller than another, or lies below it, or is contained in it. Write $x \preceq y$ for such a relation on a set A, and read it as "x is below y."

Definition 6.4 (Partial order)**.** A relation $\preceq$ on a set A is a *partial order* if it has the following three properties.

- *Reflexivity:* $x \preceq x$ for every $x \in A$.
- *Antisymmetry:* for all $x, y \in A$, if $x \preceq y$ and $y \preceq x$, then $x = y$.
- *Transitivity:* for all $x, y, z \in A$, if $x \preceq y$ and $y \preceq z$, then $x \preceq z$.

Reflexivity and transitivity are as for an equivalence relation, but symmetry has been replaced by antisymmetry. Symmetry says that whenever the relation holds in one direction, it also holds in the other; antisymmetry says that it holds in both directions only between an object and itself. The usual order $\leq$ on the real numbers is a partial order. The strict order $<$ is not, because it is not reflexive: $1 < 1$ is false.

Set inclusion $\subseteq$ is also a partial order on any collection of sets. Every set is a subset of itself; if

$X \subseteq Y$ and $Y \subseteq X$, then $X = Y$ (Section 3.1); and if $X \subseteq Y$ and $Y \subseteq Z$, then every member of X lies in Y and hence in Z, so $X \subseteq Z$.

Two elements x and y are *comparable* if $x \preceq y$ or $y \preceq x$. Any two real numbers are comparable under $\leq$, but under inclusion neither of $\{1\}$ and $\{2\}$ contains the other. The word *partial* signals that a partial order need not compare every pair.

Maximal and maximum

Because some pairs may be incomparable, there are two different notions of a "largest" element. Let C be a subset of A, called a *collection* because in the main examples its members are themselves sets, and let x belong to C.

- The element x is *maximal* in C if nothing in C lies strictly above it: there is no y in C with $x \preceq y$ and $y \neq x$.
- The element x is a *maximum* of C if everything in C lies below it: $y \preceq x$ for every y in C.

A maximum is always maximal. Indeed, let x be a maximum, and suppose y is in C with $x \preceq y$. Since x is a maximum, also $y \preceq x$, and antisymmetry gives $y = x$. The same use of antisymmetry shows that a collection has at most one maximum: if x and x' are both maxima, then $x' \preceq x$ and $x \preceq x'$, so $x = x'$.

The converse fails. In the collection $\{\{1\}, \{2\}\}$, ordered by inclusion, both members are maximal, because neither lies inside the other, and for the same reason neither is a maximum. So a collection can have several maximal elements and no maximum. It can also have no maximal element at all: in the integers under $\leq$, every integer n has $n + 1$ strictly above it.

With numbers ordered by $\leq$ we never see this difference, because when every two elements are comparable, a maximal element x is automatically a maximum. Indeed, take any y in the collection. Either $y \preceq x$, which is what we want, or $x \preceq y$, and then maximality forces $y = x$, so $y \preceq x$ by reflexivity. The distinction matters because inclusion, and many other natural orders, leave some pairs incomparable.

Chapter 8 meets a similar contrast for matchings: there too, being impossible to improve by one obvious step is weaker than being best overall.

> **Habit 6: ask what a change of description preserves**
>
> Before you treat different objects as the same, or replace an object by a standard form, say exactly which relation you are using and which property the change preserves. Before you apply a formula to equivalence classes, check that the result does not depend on the representatives. When an argument, yours or an assistant's, says "without loss of generality," ask which symmetry justifies it and whether every case can be brought into the assumed form.

Exercises

Exercises 6.1–6.4 are core exercises, and Exercises 6.5 and 6.6 are transfer exercises. Complete solutions are in Appendix A, Section A.6.

Exercise 6.1. List the members of $[-2]_5$ that lie between -12 and 12 inclusive. Which integer in $\{0, 1, 2, 3, 4\}$ represents the same class?

Exercise 6.2. Prove that multiplication of residue classes, defined by $[a]_m \cdot [b]_m = [ab]_m$, is well-defined. That is, show that if $[a']_m = [a]_m$ and $[b']_m = [b]_m$, then $[a'b']_m = [ab]_m$.

Exercise 6.3. For the four seats around the table in Section 6.4, show that the rotation r_2 by two seats is its own inverse. Explain why this does not make r_2 the identity.

Exercise 6.4. Is "has the same number of elements as" an equivalence relation on a collection of finite sets? Check each of the three properties.

Exercise 6.5. Let m and d be positive integers. Prove that the rule $[a]_m \mapsto [a]_d$ is a well-defined function from $\mathbb{Z}/m\mathbb{Z}$ to $\mathbb{Z}/d\mathbb{Z}$ exactly when d divides m.

Exercise 6.6. Give a family of three sets, ordered by inclusion, that has exactly two maximal members and no maximum. Then add one set to the family so that the new family has a maximum.

CHAPTER 7

Graphs: counting, paths, and extremal choices

> **Overview**
>
> Describe a pattern of connections as a graph. Prove an identity by counting one collection of objects in two ways. Use the pigeonhole principle and averages to show that something must exist. Choose an object that is as short or as long as possible, and show that this choice has the property we want. Learn the language of graphs used in the chapters on matchings, and the language of directed graphs needed to state a research problem exactly.

7.1 A graph is a pattern of connections

Many situations consist of objects, some pairs of which are connected: people who have shaken hands, towns joined by roads. A graph keeps only which objects there are and which pairs of them are connected, and sets aside everything else, such as how long a road is.

Definition 7.1 (Graph)**.** A *graph* consists of a finite set V, whose members are called *vertices*, and a set E, whose members are called *edges*. Each edge is a set $\{u, v\}$ of two different vertices u and v. These two vertices are the *endpoints* of the edge, and we say that the edge *joins* them.

The full name of this kind of graph is a *finite simple undirected graph*, and each word records a choice.

- *Finite*: the set V is finite (Section 3.1), and therefore so is E, because a finite set has only finitely many pairs of members.
- *Undirected*: since $\{u, v\} = \{v, u\}$, an edge does not run from one endpoint to the other; it simply joins them.
- *Simple*: there are no *loops*, edges joining a vertex to itself, because the two endpoints of an edge are different. And two vertices are joined by at most one edge, because E is a set: listing $\{u, v\}$ twice does not give it a second member, just as $\{1, 1, 3\} = \{1, 3\}$.

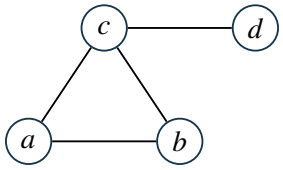
In this book, the word *graph* on its own always means a graph in the sense of Definition 7.1; graphs whose edges have a direction are called *directed* graphs and appear in Section 7.5.

We usually write the edge $\{u, v\}$ as uv, or equally as vu. (In Section 1.2 two letters side by side meant a product; here they mean an edge, and the context always makes clear which is intended.)

Here is an example that we will use throughout the chapter. Its vertex set is $V = \{a, b, c, d\}$, and its edge set is

$$E = \{ab,\ bc,\ ca,\ cd\}.$$

We call it the *example graph* G. In a drawing, each vertex is a small circle, and each edge is a line between its two endpoints:



A drawing is only a picture of a graph: the positions of the circles, the shapes of the lines, and any points where two lines cross carry no information. In particular, a crossing point is not a vertex, and moving the circle for d to the far left would give another picture of the same graph.

Two vertices u and v are *neighbors*, or *adjacent*, when uv is an edge. The *degree* of a vertex v, written $\deg(v)$, is the number of its neighbors. In the example graph, the neighbors of c are a, b, and d, so $\deg(c) = 3$. The four degrees are

$$\deg(a) = 2, \qquad \deg(b) = 2, \qquad \deg(c) = 3, \qquad \deg(d) = 1.$$

A vertex with no neighbors has degree 0; it still belongs to the graph.

The degree of v is also the number of edges at v, that is, edges with v as an endpoint: each neighbor w gives one such edge, vw, different neighbors give different edges, and every edge at v arises in this way from its other endpoint. For example, the edges at c are ca, cb, and cd. This uses simplicity: if two edges could join the same pair of vertices, one neighbor could account for two edges.

Pause and decide. In a graph with n vertices, can a vertex have degree larger than $n - 1$?

Check. No. The neighbors of v are different from v, because the two endpoints of an edge are different, so they lie among the other $n - 1$ vertices, and $\deg(v) \le n - 1$. Equality holds exactly when v is joined to every other vertex, as c is in the example graph.

7.2 Double counting and the handshake theorem

Add up the degrees in the example graph G, which has four edges: $2 + 2 + 3 + 1 = 8 = 2 \cdot 4$. This is no accident.

Consider every pair (v, e) in which v is a vertex, e is an edge, and v is an endpoint of e, that is, $v \in e$. The table below has one row for each vertex of G and one column for each edge; an entry 1 marks a pair (v, e) with $v \in e$.

	ab	bc	ca	cd	row total
a	1	0	1	0	2
b	1	1	0	0	2
c	0	1	1	1	3
d	0	0	0	1	1
column total	2	2	2	2	8

Count the entries equal to 1 in two ways. The row of a vertex has a 1 for each edge at that vertex, so the row totals are the degrees, which add up to 8. Every edge has two endpoints, so every column total is 2, and the columns also add up to 8: both totals count the same entries.

Counting one collection of objects in two ways and setting the two answers equal is called *double counting*. Its essential step is to describe the collection precisely, so that we can check that each grouping counts every member exactly once.

To state the general result, we extend the sum notation of Section 1.8: $\sum_{v \in V} \deg(v)$ means that we add $\deg(v)$ once for each vertex v in V; for the example graph it is $\deg(a) + \deg(b) + \deg(c) + \deg(d) = 8$. The order of the terms does not matter, because the rules $a + b = b + a$ and $(a + b) + c = a + (b + c)$ of Section 1.2 let us reorder and regroup the terms of any finite sum. In the same way, $\sum_{e \in E} 2$ adds 2 once for each edge, so it equals $2|E|$.

Theorem 7.2 (Handshake theorem)**.** *In every graph with vertex set V and edge set E,*

$$\sum_{v \in V} \deg(v) = 2|E|.$$

The name comes from a party: if the guests are vertices and the handshakes are edges, then $\deg(v)$ is the number of hands that v shook, and each handshake is counted twice in the total, once by each of the two people involved.

Proof. Let P be the set of all pairs (v, e) with $v \in V$, $e \in E$, and $v \in e$. We count P in two ways.

First group the pairs by their vertex. For a fixed vertex v, the pairs in P with first entry v are the pairs (v, e) in which e is an edge at v, and by Section 7.1 there are exactly $\deg(v)$ such edges. Each pair in P has just one first entry, so it lies in exactly one of these groups, and $|P| = \sum_{v \in V} \deg(v)$.

Next group the pairs by their edge. For a fixed edge $e = \{u, w\}$, the pairs in P with second entry e are (u, e) and (w, e), two different pairs because $u \neq w$. Again every pair lies in exactly one group, so $|P| = \sum_{e \in E} 2 = 2|E|$.

Both expressions equal $|P|$, so they are equal to each other. □

In the example graph, two vertices, c and d, have odd degree. That this number is even is no coincidence either.

Proposition 7.3. *In every graph, the number of vertices of odd degree is even.*

Proof. Let k be the number of vertices of odd degree. By Lemma 4.1, every degree is either even or odd, and not both. So each vertex contributes its degree to exactly one of two sums: S_{even}, over the vertices of even degree, and S_{odd}, over the vertices of odd degree. Together they make up the degree sum, so $S_{\text{even}} + S_{\text{odd}} = 2|E|$ by the handshake theorem.

- S_{even} is even, because a factor 2 can be taken out of every term. (If no vertex has even degree, S_{even} is the empty sum 0, which is even too.)
- Therefore $S_{\text{odd}} = 2|E| - S_{\text{even}}$ is even, since $2x - 2y = 2(x - y)$.

Now write the degrees of the k vertices of odd degree, using Definition 1.1, as $2\ell_1 + 1, \ldots, 2\ell_k + 1$

with integers $\ell_1, \dots, \ell_k$. Adding them gives

$$S_{\text{odd}} = (2\ell_1 + 1) + \cdots + (2\ell_k + 1) = 2(\ell_1 + \cdots + \ell_k) + k,$$

because each of the k terms contributes one 1. Hence $k = S_{\text{odd}} - 2(\ell_1 + \cdots + \ell_k)$ is the difference of two even numbers, so k is even. (If $k = 0$, the conclusion holds at once, since 0 is even.) □

For instance, the proposition shows that seven people at a party cannot each shake hands with exactly three of the others: all seven vertices would have the odd degree 3, and 7 is odd. (The handshake theorem shows this directly: the degree sum $7 \cdot 3 = 21$ would be odd, but it equals $2|E|$.)

7.3 The pigeonhole principle and the average

If many objects are placed in a few boxes, some box must be crowded. Here is the precise statement.
Proposition 7.4 (Pigeonhole principle)**.** *Let n, r, and k be natural numbers. If n objects are placed in r boxes and $n > rk$, then some box contains more than k objects.*

Proof. We prove the contrapositive (Section 2.5): suppose that every box contains at most k objects, and show that $n \le rk$. Each object lies in exactly one box, so n is the sum of the numbers of objects in the r boxes. Each of these r numbers is at most k, so their sum is at most rk. □

The name pictures pigeons in a pigeon house: if there are more pigeons than holes, some hole holds at least two. This is the case $k = 1$, the one used most often. For example, among any 13 people, two have their birthdays in the same month: the people are the objects, the 12 months are the boxes, and $13 > 12 \cdot 1$. The principle tells us that such a month *exists*, but not which month it is. Since placing objects in boxes defines a function from objects to boxes, the case $k = 1$ is also the fact from Section 3.3 that a function from a finite set A to a finite set B with $|A| > |B|$ cannot be injective.

Pause and decide. A class has 30 students. Must some month contain the birthdays of at least three of them? Of at least four?

Check. Take the 12 months as boxes. Since $30 > 12 \cdot 2$, the principle with $k = 2$ gives a month with more than two birthdays, that is, at least three. For four we would need $30 > 12 \cdot 3 = 36$, which is false. Indeed no guarantee is possible: six months with three birthdays and six with two account for all $6 \cdot 3 + 6 \cdot 2 = 30$ students, and no month holds four.

A closely related fact concerns averages. The *average* of real numbers $a_1, \dots, a_n$, where $n \ge 1$, is

$$\bar{a} = \frac{a_1 + \cdots + a_n}{n},$$

read "a bar."
Proposition 7.5 (Averaging principle)**.** *Let $a_1, \dots, a_n$ be real numbers, where $n \ge 1$, and let $\bar{a}$ be their average. Then at least one of the numbers is at least $\bar{a}$, and at least one of them is at most $\bar{a}$.*

Proof. Suppose, for contradiction, that every a_i is strictly less than $\bar{a}$. Adding these n strict inequalities gives

$$a_1 + \cdots + a_n < n\bar{a}.$$

(From $x < y$ and $x' < y'$ we get $x + x' < y + x' < y + y'$, by adding the same number to both sides as in Section 2.3; repeating this adds all n inequalities. Since $n \geq 1$, there is at least one inequality to add, so the sum is strict.) But $n\bar{a} = a_1 + \cdots + a_n$ by the definition of the average, so this number would be strictly less than itself, which is impossible. Hence some a_i satisfies $a_i \geq \bar{a}$. Reversing every inequality proves the second claim. □

For instance, the numbers $1, 3, 8$ have average $12/3 = 4$; the number 8 is at least 4, and 1 and 3 are at most 4. The condition $n \geq 1$ is needed for the average to make sense at all: with $n = 0$ the formula would divide by zero.

The pigeonhole principle is a special case of the averaging principle: if n objects lie in $r \geq 1$ boxes, the box counts add up to n and have average n/r, which is larger than k when $n > rk$; so some box holds at least $n/r > k$ objects.

Apply the averaging principle to the degrees of a graph with $n \geq 1$ vertices and m edges. By the handshake theorem the degrees add up to $2m$, so their average is $2m/n$, and some vertex has degree at least $2m/n$. In the example graph G, the average degree is $8/4 = 2$, and c has degree 3. The words "at least one" are essential: nothing is promised about the other degrees, and Exercise 7.5 asks how far from the average they can be. Chapter 13 turns this into a method for proving existence: compute an average over random choices, and conclude that some choice is at least as good as the average.

7.4 Walks, paths, and shortest objects

A *walk* from a vertex u to a vertex w is a finite sequence of vertices $v_0, v_1, \ldots, v_k$ that starts at $v_0 = u$, ends at $v_k = w$, and in which every two consecutive vertices are joined by an edge: $v_{i-1}v_i$ is an edge for every $i = 1, \ldots, k$. Its *length* is k, the number of steps, one less than the number of entries. A walk may visit a vertex or use an edge more than once; each use of an edge counts as a separate step. A *path* is a walk in which no vertex appears twice. A single vertex v_0 counts as a walk and a path of length 0 from v_0 to itself.

In the example graph G, the sequence a, c, b, c, d is a walk of length 4 from a to d, since ac, cb, bc, and cd are edges. It visits c twice, so it is not a path. The sequence a, c, d is a path of length 2 from a to d, and a, d is not a walk at all, because ad is not an edge. Since edges have no direction, reading a walk backward gives a walk in the opposite direction: d, c, a is a walk from d to a. We call two vertices *connected* when there is a walk from one to the other.

A walk that repeats a vertex contains a detour that can be cut out. Between its two visits to c, the walk a, c, b, c, d makes the round trip c, b, c; deleting it, but keeping one copy of c, leaves the shorter walk a, c, d. The step to d now follows the first visit to c instead of the second, which is legitimate because both visits are at the same vertex. The next lemma uses this cut to show that a shortest walk never repeats a vertex.

Lemma 7.6. *Let u and w be vertices of a graph, and suppose that there is a walk from u to w. Then there is a walk from u to w of smallest possible length, and every such shortest walk is a path. In particular, there is a path from u to w.*

Proof. The lengths of walks from u to w form a set of natural numbers, which is nonempty because at least one such walk exists. By the well-ordering property (Section 4.2, extended to natural numbers in Section 5.2), this set has a smallest member L. Let $v_0, v_1, \ldots, v_L$ be any walk from u to w of this length.

Suppose, for contradiction, that $v_i = v_j$ for some positions $i < j$. Delete the entries $v_{i+1}, \ldots, v_j$, leaving the sequence

$$v_0,\ \ldots,\ v_i,\ v_{j+1},\ \ldots,\ v_L$$

(if $j = L$, the sequence simply ends at v_i). This is a walk from u to w:

- Each pair of consecutive entries is joined by an edge. Every such pair was already consecutive in the original walk, except for the new pair v_i, v_{j+1}, which occurs only when $j < L$. Since $v_i = v_j$, the edge $v_i v_{j+1}$ is the same as $v_j v_{j+1}$, which is an edge of the original walk.
- It starts at $v_0 = u$.
- It ends at w: if $j < L$, its last entry is $v_L = w$, and if $j = L$, its last entry is $v_i = v_j = v_L = w$.

Its length is $L - (j - i)$, because $j - i$ entries were deleted. Since $j > i$, this is less than L, which contradicts the choice of L. Hence no vertex appears twice. Since the shortest walk was arbitrary, every shortest walk from u to w is a path, and in particular such a path exists. □

When u and w are connected, the *distance* from u to w is the length of a shortest walk from u to w. By Lemma 7.6, it is also the length of a shortest path, because a shortest walk is a path and every path is a walk. In the example graph, the distance from a to d is 2: the path a, c, d has length 2, and no shorter walk exists, because a walk of length 0 would need $a = d$ and a walk of length 1 would need ad to be an edge. Establishing a distance takes both facts: a walk of that length, and a reason why no shorter walk exists.

The proof of Lemma 7.6 never built a path step by step. It chose a walk that was as short as possible and showed that this choice already has the property we wanted. An object chosen to be as small or as large as possible is called *extremal*, and an argument that starts from such a choice is called an *extremal argument*. We have met this idea before: Section 4.3 chose a fraction for $\sqrt{2}$ with the smallest possible positive denominator, and Section 5.2 chose a smallest counterexample.

7.5 Directed edges and a guaranteed sink

Some connections have a direction: a one-way street, a link from one web page to another, a game won by one player against another. We record such a connection as an arrow.

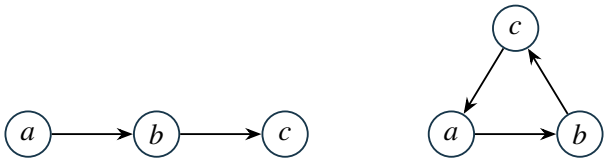
Definition 7.7 (Directed and oriented graphs)**.** A *directed graph* consists of a set V of vertices and a set of *directed edges*. Each directed edge is an ordered pair (u, v) of two different vertices; we write it $u \to v$ and draw it as an arrow from u to v. An *oriented graph* is a directed graph in which no two vertices are joined in both directions: $u \to v$ and $v \to u$ never both occur.

Because the entries of an ordered pair come in a definite order (Section 3.1), $u \to v$ and $v \to u$ are different directed edges. As in a graph, there are no loops $u \to u$. Some pairs of vertices may have no edge in either direction. Unlike a graph, a directed graph may have infinitely many vertices, so we say "finite" explicitly whenever a result needs it.

We also use $u \to w$ as a statement, saying that (u, w) is a directed edge; $u \to w$ *fails* when it is not. (This arrow joins two vertices; it is not the arrow of a function $f : A \to B$.) If $u \to w$, we call w an *out-neighbor* of u. The *outdegree* of u is the number of its out-neighbors, that is, the number of arrows leaving u. A *sink* is a vertex with outdegree 0: no arrow leaves it.

Walks and paths now have to follow the arrows. A *directed walk* from v_0 to v_k is a sequence of vertices $v_0, v_1, \ldots, v_k$ with $v_{i-1} \to v_i$ for every $i = 1, \ldots, k$; its length is k. A *directed path* is a directed walk with no repeated vertex. A *directed cycle* is a directed walk $v_0, v_1, \ldots, v_k$ with at least one step, $k \geq 1$, that returns to its start, $v_k = v_0$, and has no other repetition: the vertices $v_0, v_1, \ldots, v_{k-1}$ are all different. A directed graph is *acyclic* when it has no directed cycle.

In an oriented graph, a directed cycle passes through at least three vertices, because a cycle with one step would be a loop and a cycle with two steps would need arrows in both directions. Here are two oriented graphs, each with vertices a, b, c:



On the left is the *chain* $a \to b \to c$. It is acyclic, since every arrow leads further along the chain, and its only sink is c. On the right is the directed cycle $a \to b \to c \to a$. Replacing its arrow $c \to a$ by $a \to c$ gives a triangle with no directed cycle, since then every arrow goes to a later letter.

Does every directed graph have a sink? A directed graph with no vertices has none. An infinite directed graph can fail too: take the natural numbers as vertices, with an edge $n \to n + 1$ for every n. There is no sink, yet the graph is acyclic, because the numbers strictly increase along any directed walk. Exercise 7.3 asks what can happen when the graph is finite but not acyclic.

Theorem 7.8. *Every nonempty finite acyclic oriented graph has a sink.*

Here *nonempty* means that there is at least one vertex. The theorem promises *one* sink, not that every vertex is a sink; in the chain, only c is. The word "oriented" excludes no acyclic directed graph: a directed graph with both $u \to v$ and $v \to u$ has the directed cycle u, v, u. We keep the word because the conjecture in Section 7.7 is about oriented graphs.

Plan. Take a directed path that is as long as possible, and look at its last vertex. An arrow leaving that vertex would either reach a new vertex, which would give a longer path, or return to the path, which would close a directed cycle.

Proof. A directed path never repeats a vertex, so it has at most $|V|$ vertices and therefore length at most $|V| - 1$. (By the pigeonhole principle, a list of more than $|V|$ vertices contains some vertex twice.) Since the graph is nonempty, it has a vertex, and a single vertex is a directed path of length

0. So the lengths of the directed paths form a nonempty subset of $\{0, 1, \ldots, |V| - 1\}$, which has a largest member. Choose a directed path of this greatest length, and write it $v_0 \to v_1 \to \cdots \to v_k$.

Suppose, for contradiction, that its last vertex v_k has an out-neighbor w.

- If w is not on the path, then $v_0 \to v_1 \to \cdots \to v_k \to w$ is a directed path of length $k + 1$, which contradicts the choice of a path of greatest length.
- If w is on the path, say $w = v_j$, then $j \neq k$, because $v_k \to v_k$ would be a loop. So $j < k$. Then $v_j \to v_{j+1} \to \cdots \to v_k \to v_j$ is a directed walk with $k - j + 1$ steps that returns to its start, and its vertices $v_j, \ldots, v_k$ before the return are all different. It is a directed cycle, which contradicts acyclicity.

Both cases are impossible. Hence v_k has no out-neighbor, so it is a sink. □

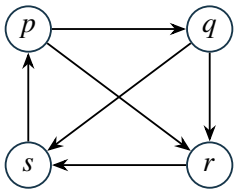
Each hypothesis has a visible job in this proof. Nonemptiness supplies a path to choose from. Finiteness guarantees a directed path of greatest length; in the directed graph on the natural numbers, every directed path can be extended. Acyclicity rules out an arrow from the last vertex back into the path.

7.6 Tournaments and a vertex of maximum outdegree

In a round-robin competition, every pair of players meets exactly once, and every game has a winner. We can record the results as an oriented graph: the players are the vertices, and $u \to v$ means that u beat v.

Definition 7.9 (Tournament)**.** A *tournament* is an oriented graph in which every pair of different vertices is joined by exactly one directed edge: for all vertices $u \neq v$, exactly one of $u \to v$ and $v \to u$ holds.

Here is a tournament with four players p, q, r, s and the six results $p \to q$, $p \to r$, $q \to r$, $q \to s$, $r \to s$, and $s \to p$, one for each pair:



The outdegrees, the numbers of games won, are 2 for p and q, and 1 for r and s.

Call a vertex v a *king* if it can reach every other vertex by a directed path of length at most two: for every other player u, either v beat u, or v beat someone who beat u. In the example, p is a king: it beat q and r, and it reaches s through $p \to q \to s$. So is q, which beat r and s and reaches p through $q \to s \to p$. The player r is not a king: its only win was against s, and s beat only p, so within two steps r does not reach q. Must every nonempty finite tournament have a king? An extremal choice answers this question.

Proposition 7.10. *In a nonempty finite tournament, every vertex of maximum outdegree is a king: it can reach every other vertex by a directed path of length at most two.*

A vertex of maximum outdegree always exists, because the outdegrees form a nonempty finite set of natural numbers, which has a largest member. So the proposition shows that every nonempty finite tournament has a king.

Proof. Let v be any vertex of maximum outdegree, and let A be the set of its out-neighbors, so that the outdegree of v is $|A|$. Let u be any vertex other than v. There are two ways in which u might be reached quickly.

- If $v \to u$, then u is reached in one step.
- If some $a \in A$ satisfies $a \to u$, then u is reached in two steps by $v \to a \to u$. This is a directed path, because its three vertices are different: $a \neq v$ because $v \to a$ and there are no loops, $u \neq v$ by the choice of u, and $a \neq u$ because $a \to u$.

Suppose, for contradiction, that neither happens. Since $v \to u$ fails and $u \neq v$, the tournament condition gives $u \to v$. Also $u \notin A$, because $v \to u$ fails, so u is different from every $a \in A$. For each $a \in A$, the tournament condition gives $a \to u$ or $u \to a$, and $a \to u$ has been excluded, so $u \to a$. Thus u beat every member of A, and it also beat v, which is not in A because a vertex is never its own out-neighbor. So u has at least $|A| + 1$ out-neighbors, while v has exactly $|A|$. This contradicts the choice of v as a vertex of maximum outdegree. Therefore one of the two ways always occurs, and v is a king. □

Pause and decide. In the example tournament, s won only one game, so it does not have maximum outdegree. Is s a king? What does your answer say about the converse of Proposition 7.10?

Check. Yes: $s \to p$, $s \to p \to q$, and $s \to p \to r$ reach all three other players within two steps. So the converse, "every king has maximum outdegree," is false, with s as a counterexample: the proposition gives a sufficient condition for being a king, not a necessary one. The kings of the example are p, q, and s.

7.7 Second out-neighborhoods and a research problem

This section states a problem about directed graphs that resisted every attempt at a proof for more than thirty years. First we measure distance along the arrows.

The deletion argument of Lemma 7.6 works for directed walks too, since after the cut, the arrow that left the second visit to a vertex leaves the first visit, which is the same vertex. So whenever there is a directed walk from v to w, a shortest one exists, and it is a directed path; its length is the *directed distance* from v to w. Unlike distance in a graph, it depends on the order of the two vertices: in the chain $a \to b \to c$, the directed distance from a to c is 2, but no directed walk goes from c to a.

Now let v be a vertex of a directed graph with vertex set V. Write $N_1^+(v)$ for the set of out-neighbors of v, the vertices at directed distance 1 from v, and call it the *first out-neighborhood* of v. The *second out-neighborhood* of v is the set

$$N_2^+(v) = \{w \in V : \ w \neq v, \ w \notin N_1^+(v), \text{ and } v \to u \to w \text{ for some vertex } u\}.$$

(The + means that we follow the arrows outward from v; the subscript counts the steps.) Thus $N_2^+(v)$ is the set of vertices at directed distance exactly two from v: a two-step walk $v \to u \to w$ reaches w, but w is neither v itself, at distance 0, nor a first out-neighbor, at distance 1. It is a set of vertices, not of routes: if several vertices u lead to the same w, then w is still a single member of $N_2^+(v)$ and is counted once in $|N_2^+(v)|$.

In an oriented graph the condition $w \neq v$ holds automatically, because a route $v \to u \to v$ would need edges in both directions between v and u. In other directed graphs it keeps v from being its own second out-neighbor.

For example, in the oriented graph with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$, the two-step routes from a are $a \to b \to c$ and $a \to c \to d$. The vertex c ends the first route, but the edge $a \to c$ makes it a first out-neighbor, so it is not in $N_2^+(a)$. Thus

$$N_1^+(a) = \{b, c\}, \qquad N_2^+(a) = \{d\}.$$

In 1990 Paul Seymour proposed the following statement, now known as *Seymour's second-neighborhood conjecture*:

> Every nonempty finite oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.

In words: some vertex reaches at least as many new vertices in its second step as in its first. In the four-vertex example above, a fails the inequality, since $|N_2^+(a)| = 1 < 2 = |N_1^+(a)|$. The vertex b satisfies it, since $N_1^+(b) = \{c\}$ and $N_2^+(b) = \{d\}$. So does d, for a simpler reason: it is a sink, and a sink has no out-neighbors and no two-step routes, so both of its out-neighborhoods are empty and $0 \geq 0$.

The word "oriented" cannot be dropped. In the directed graph with two vertices u and v and both edges $u \to v$ and $v \to u$, the only two-step route from u returns to u, so $N_1^+(u) = \{v\}$ and $N_2^+(u) = \varnothing$. The same holds for v, so neither vertex satisfies the inequality.

For decades only special cases were settled; for example, David Fisher proved the case of tournaments in 1996. In September 2026, a *preprint*, that is, a research paper made public before a journal has reviewed it, claimed a proof of the full conjecture. It was released in OpenAI's public mathematics repository together with a formal proof in the Lean proof assistant [3]. Its formalized statement uses the conventions of this section: second out-neighbors are the vertices at directed distance exactly two, and graphs with sinks are allowed. The proof draws on ideas well beyond this book. Section 16.4 shows how the definition of $N_2^+(v)$ is written in Lean, and Section 17.8 describes how to begin reading a formalized proof of this kind.

If a news report or an AI assistant tells you that "Seymour's conjecture has been proved," first check *which statement* was proved. Counting two-step routes instead of vertices, or letting first out-neighbors count as second out-neighbors, can only make the second count larger and the inequality easier to satisfy, so a proof of such a variant would prove a weaker statement.

We can already prove one special case completely.

Proposition 7.11. *Every nonempty finite acyclic oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.*

Proof. By Theorem 7.8, the graph has a sink v. As we saw above, both out-neighborhoods of a sink are empty. Hence $|N_2^+(v)| = 0 \geq 0 = |N_1^+(v)|$. □

This proof says nothing about graphs with directed cycles, and that is where all the difficulty lies: a counterexample to the conjecture would have no sink, so by Theorem 7.8 it would contain a directed cycle. Keeping track of which graphs a proof covers is part of the statement contract of Habit 2 (Section 2.6): removing the hypothesis "acyclic" from Proposition 7.11 turns it into the conjecture itself.

Exercises

Exercises 7.1–7.4 are core exercises, and Exercises 7.5 and 7.6 are transfer exercises. Solutions are in Appendix A, Section A.7.

Exercise 7.1. Let G be the example graph of Section 7.1. (a) Add the edge bd to G. List the degrees of the new graph, and find their sum and the number of vertices of odd degree. Check that both answers agree with Theorem 7.2 and Proposition 7.3. (b) Do the same for the graph obtained from G by deleting the edge cd instead.

Exercise 7.2. Let x_1, x_2, x_3, x_4, x_5 be five integers, not necessarily different. Prove that there are two indices $i \neq j$ for which x_i and x_j leave the same remainder on division by four. Then give four integers that leave four different remainders on division by four, to show that five cannot be replaced by four.

Exercise 7.3. Show that Theorem 7.8 becomes false if the word "acyclic" is removed: find a nonempty finite oriented graph with no sink. Use as few vertices as possible, and prove that every nonempty oriented graph with fewer vertices has a sink. Which step of the proof of Theorem 7.8 breaks down for your graph?

Exercise 7.4. Consider the oriented graph with vertices a, b, c, d and edges $a \to b$, $a \to c$, $b \to d$, and $c \to d$. Find $N_1^+(v)$ and $N_2^+(v)$ for every vertex v, and decide which vertices satisfy $|N_2^+(v)| \geq |N_1^+(v)|$. Be careful with d, which can be reached from a along two different routes.

Exercise 7.5. Consider a graph with $n \geq 1$ vertices and m edges. Prove that some vertex has degree at most $2m/n$. Then explain, using an example, why this gives no guarantee that all the degrees are close to the average $2m/n$.

Exercise 7.6. Let v be a king in a tournament, so that every other vertex can be reached from v by a directed path of length at most two. Does it follow that $|N_2^+(v)| \geq |N_1^+(v)|$? A tournament on three vertices is enough to settle the question.

CHAPTER 8

Assignments, repairs, and obstructions

> **Overview**
> Describe an assignment problem precisely as a graph. See why choices that look reasonable one at a time can lead to a dead end, and learn to repair an earlier choice without creating a new conflict. Build a search that either enlarges the current assignment or finds a group of students who together accept too few tasks, proving that no full assignment exists.

8.1 Why one acceptable task per person is not enough

Section 2.4 looked at the statement that all the students of a class can be given different tasks, each acceptable to the student who receives it. The *assignment problem* asks when this can be done, and how to find such an assignment. The simplest failure is the kind Exercise 2.5 asked you to build: Ada and Ben each accept only task 1. Each of them has an acceptable task, but they cannot be given different ones.

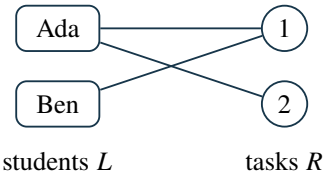
Now let Ada accept tasks 1 and 2, while Ben still accepts only task 1. A solution exists: Ada receives task 2 and Ben task 1. But suppose we hand out tasks *greedily*: one student at a time, each receives some acceptable task that is still free, and no task is ever taken back. If Ada comes first and happens to get task 1, nothing is left for Ben. Getting stuck does not prove that there is no solution; it shows only that the choices made so far cannot be extended without changing an earlier one.

The graph of acceptable pairs

We describe such problems by a graph in the sense of Definition 7.1. The vertices are the students and the tasks, and an edge joins a student to a task exactly when the student accepts that task. No edge joins two students, and no edge joins two tasks.

Definition 8.1 (Bipartite graph)**.** A graph is *bipartite*, with *parts* L and R, if its vertex set is the union of two disjoint sets L and R and every edge has one endpoint in L and the other endpoint in R.

Throughout this chapter, L (drawn on the left) is the set of students and R (on the right) is the set of tasks. We write the edge joining a student s to a task t as s–t, with the student first; the notation st of Chapter 7 is hard to read when the vertices are called Ada or 1. Here is the graph of the second example:



Its three edges are Ada–1, Ada–2 and Ben–1, and we call it the *Ada–Ben graph*; the point where two lines cross is not a vertex. The neighbors of a student s are exactly the tasks that s accepts, and we write $N(s)$ for this set. Here $N(\text{Ada}) = \{1, 2\}$ and $N(\text{Ben}) = \{1\}$.

Matchings

An assignment selects some of the edges. Since no task goes to two students and no student receives two tasks, the selected edges never share an endpoint.

Definition 8.2 (Matching)**.** A *matching* in a graph is a set M of edges of the graph, no two of which share an endpoint. A vertex is *matched* by M if it is an endpoint of some edge of M, and *unmatched* otherwise. Two vertices joined by an edge of M are *partners*. In a bipartite graph with parts L and R, a matching *saturates* L if every vertex of L is matched.

A matched vertex lies on exactly one edge of M: on at least one because it is matched, and on at most one because no two edges of M share an endpoint. So a matched student holds exactly one task, its partner, and a matched task is held by exactly one student, its *owner*. For tasks we also say *used* and *unused* instead of matched and unmatched. The empty set of edges is a matching too, since it has no two edges that could share an endpoint; it leaves every vertex unmatched.

In the Ada–Ben graph, {Ada–2, Ben–1} is a matching that saturates L, while {Ada–1, Ben–1} is not a matching, because both of its edges have the endpoint 1. A matching that saturates L is exactly a solution of the assignment problem, and we call it a *full assignment*. It need not use every task.

8.2 Maximal does not mean maximum

In the Ada–Ben graph, the greedy procedure that gives Ada task 1 gets stuck at the matching {Ada–1}, although {Ada–2, Ben–1} solves the problem. Two words describe the difference.

Definition 8.3 (Maximal and maximum matchings)**.** A matching M is *maximal* if no edge can be added to it: for every edge e of the graph that is not in M, the set $M \cup \{e\}$ is not a matching. A matching M is *maximum* if no matching in the graph has more edges than M.

The matching {Ada–1} is maximal, because each of the other two edges shares an endpoint with Ada–1: Ada–2 shares Ada, and Ben–1 shares task 1. But it is not maximum, because {Ada–2, Ben–1} has two edges. To get from the first matching to the second we must remove Ada–1, and a greedy procedure never removes anything, so it can stop at a maximal matching that is not maximum.

Every maximum matching is maximal: if some edge e could be added to M, then $M \cup \{e\}$ would be a matching with one more edge than M. The example shows that the converse is false.

Pause and decide. Is the empty matching maximal?

Check. Only when the graph has no edges. A single edge is always a matching, since there is no second edge for it to share an endpoint with, so any edge of the graph can be added to the empty matching. If there are no edges, there is nothing to add, and the empty matching is both maximal and maximum.

Comparison with Chapter 6

Section 6.6 defined maximal and maximum elements of a partial order. For the matchings of a graph ordered by inclusion, being maximal, that is, contained in no other matching, agrees with Definition 8.3. If an edge e can be added to M, then $M \cup \{e\}$ is a larger matching containing M. Conversely, if a matching $M' \neq M$ contains M, then any edge e of M' not in M can be added to M: the set $M \cup \{e\}$ lies inside M', and removing edges from a matching cannot create a shared endpoint.

Maximum is different. A maximum in the sense of Section 6.6 would be a matching containing every other matching, but the Ada–Ben graph has no matching containing both Ada–1 and Ben–1. For matchings, *maximum* always means *largest number of edges*. If students a and b both accept tasks 1 and 2, then $\{a\text{–}1,\ b\text{–}2\}$ and $\{a\text{–}2,\ b\text{–}1\}$ are both maximum, and neither contains the other. So an extremal argument (Section 7.4) that begins "take a largest matching" must say which kind it means: one contained in no other matching, or one with as many edges as possible.

A matching as a function

A matching M that saturates L describes a function $f : L \to R$ (Chapter 3): $f(s)$ is the partner of s. Each $s\text{–}f(s)$ is an edge, and f is injective (Section 3.3), since two students with the same task t would give two edges of M sharing the endpoint t. Conversely, if $f : L \to R$ is injective and each $s\text{–}f(s)$ is an edge, these edges form a matching that saturates L. Each student lies on one of them, and two of them, for students $s \neq s'$, have different tasks $f(s) \neq f(s')$ by injectivity; as L and R are disjoint, no student is a task, so the two edges share no endpoint. Both conditions are needed. In the Ada–Ben graph, $f(\text{Ada}) = f(\text{Ben}) = 1$ satisfies the edge condition but is not injective, while $f(\text{Ada}) = 1$, $f(\text{Ben}) = 2$ is injective, but Ben does not accept task 2.

8.3 An alternating repair

In the Ada–Ben graph with the stuck matching $M = \{\text{Ada–1}\}$, Ben's only acceptable task is held by Ada. The repair is to move Ada to task 2 and give task 1 to Ben. This follows the route

$$\text{Ben} - 1 - \text{Ada} - 2,$$

the path Ben, 1, Ada, 2 in the notation of Section 7.4. Of its three edges, the first and third, Ben–1 and Ada–2, are not in M, while the second, Ada–1, is. The repair removes the edge that was in M and adds the two that were not, giving $\{\text{Ben–1},\ \text{Ada–2}\}$.

Vertex	Before the repair	After the repair
Ben	no task	task 1
task 1	held by Ada	held by Ben
Ada	task 1	task 2
task 2	unused	held by Ada

The middle vertices, task 1 and Ada, still have one partner each, and the two ends, Ben and task 2, have become matched.

Alternating and augmenting paths

To describe this repair in general, fix a matching M. We call the edges of M *matched edges*, and the other edges of the graph, the allowed pairs not currently chosen, *unmatched edges*. Recall from Section 7.4 that a walk $v_0, v_1, \ldots, v_k$ takes k steps, the ith along the edge $v_{i-1}v_i$, and that a path is a walk with no repeated vertex.

Definition 8.4 (Alternating walk, augmenting path)**.** Let M be a matching. A walk is *alternating* if its steps use matched and unmatched edges alternately: of any two consecutive steps, one uses an edge of M and the other uses an edge not in M. An *augmenting path* for M is an alternating path that has at least one step and whose first and last vertices are both unmatched.

Let $v_0, v_1, \ldots, v_k$ be an augmenting path. Its first edge v_0v_1 is unmatched, because v_0 is unmatched and so lies on no edge of M; for the same reason its last edge is unmatched. Since the edges alternate, the ith edge is unmatched when i is odd and matched when i is even. The last edge is unmatched, so k is odd. Writing $k = 2r + 1$, the path has $r + 1$ unmatched edges, in the odd positions, and r matched edges, in the even positions.

In a bipartite graph every edge joins L to R, so the vertices of a walk alternate between students and tasks, and a walk with an odd number of steps ends in the other part from where it started. So one end of an augmenting path is an unmatched student and the other an unused task. We always list the path starting from the student. The simplest augmenting path is a single edge from an unmatched student to an unused task, with $k = 1$ and $r = 0$; in the Ada–Ben repair, $k = 3$ and $r = 1$.

Pause and decide. With $M = \{\text{Ada–1}\}$, is the single edge Ben–1 an augmenting path?

Check. No. Ben is unmatched, but task 1 is held by Ada, so adding Ben–1 to M would give task 1 two owners. An augmenting path must end at an unused task, so here it must continue through Ada to task 2.

Flipping along an augmenting path

On a longer augmenting path the same exchange happens all along the path: each student inside it trades the task just before them for the task just after them. To *flip* M along a walk P means to remove from M the matched edges that P uses and to add the unmatched edges that P uses; edges not on P stay as they were.

Lemma 8.5. *Let M be a matching and let P be an augmenting path for M. Flipping M along P gives a matching M' with exactly one more edge than M.*

Proof. Write P as $v_0, v_1, \ldots, v_k$. Every edge of M' is in M or on P, so it is an edge of the graph. To show that M' is a matching, we check that every vertex lies on at most one edge of M'.

The vertices of a path are all different. Therefore v_i lies on the edge $v_{i-1}v_i$ (if $i \geq 1$) and on the edge v_iv_{i+1} (if $i \leq k-1$), and on no other edge of P: an edge $v_{j-1}v_j$ contains v_i only if $v_i = v_{j-1}$ or $v_i = v_j$, that is, only if $j = i+1$ or $j = i$. There are three kinds of vertices to check.

- *A vertex v not on P.* The flip changes no edge at v, so v lies on at most one edge of M', as it did for M.
- *An end of P.* Take v_0; the argument for v_k is the same. The only edge of P at v_0 is v_0v_1, which is unmatched, so the flip adds it. Since v_0 is unmatched, no edge of M contains it, and the edges at v_0 not on P stay outside M'. So v_0 lies on exactly one edge of M'.
- *A vertex v_i inside P, with $0 < i < k$.* Its edges $v_{i-1}v_i$ and v_iv_{i+1} are consecutive edges of P, so one is in M and the other is not; the flip removes the one in M and adds the other. Because M is a matching and v_i already lies on a matched edge of P, no edge of M off P contains v_i, and the flip adds no edge off P. So the only edge of M' at v_i is the path edge that was added.

Hence no vertex lies on two edges of M', and M' is a matching.

Now count. The edges of P are all different, because they join different pairs of vertices. As shown before the lemma, P has $r+1$ unmatched edges and r matched edges for some natural number r. The flip removes the r matched edges and adds the $r+1$ unmatched ones, none of which was in M. Therefore $|M'| = |M| - r + (r+1) = |M| + 1$. □

The proof needs P to be a path: on an alternating walk that visits a vertex twice, the flip could give that vertex two partners (Exercise 8.4).

What is preserved and what makes progress

Habit 5 (Section 5.5) asks of a repeated step what stays true and what measures progress. For repeated flips along augmenting paths, Lemma 8.5 answers both: the selected edges always form a matching, and their number $|M|$ grows by one at every flip. It cannot grow forever: each edge of a matching contains exactly one student, and no two edges contain the same one, so sending each edge of M to its student is an injective map from M to L, and therefore $|M| \leq |L|$ (Section 3.3). So the nonnegative integer $|L| - |M|$ decreases by one at every flip, and a procedure that starts from the empty matching makes at most $|L|$ flips.

8.4 Searching for an augmenting path

Let M be a matching and let x be an unmatched student. How can we find an augmenting path that starts at x, or find out that there is none? Such a path leaves x along an unmatched edge to some task. If that task is unused, the path can stop there. If it is used, the path must go on, and since the

steps alternate, its next step uses a matched edge; the only one at a used task leads to its owner. From there the path continues along an unmatched edge, and so on. So we explore the graph with two kinds of moves:

- from a student s, move to any task t for which s–t is an unmatched edge;
- from a used task t, move to its owner.

The search

The search keeps track of which vertices it has *reached*, and records for each newly reached vertex the vertex it came from, its *predecessor*. To *inspect* a reached student s, go through the unmatched edges s–t at s one at a time, and for each of them do the following.

- If the task t has already been reached, go on to the next edge.
- Otherwise, mark t as reached, with predecessor s. If t is unused, stop: the search has *succeeded*. If t is used, also mark its owner as reached, with predecessor t, and go on to the next edge.

The search itself runs as follows.

1. Mark x as reached. We call x the *root* of the search; it has no predecessor.
2. Choose a reached student that has not been inspected yet, and inspect it.
3. Repeat step 2 until the search succeeds, or until every reached student has been inspected. In the second case the search has *failed*.

No vertex is marked twice. For a task this is built into the rule. The root x owns no task, since it is unmatched, so it is never marked again. Any other student is marked only as the owner of a task being reached for the first time; since a student owns only one task, an earlier marking of that student would have been as the owner of the same task, which would then already have been reached. So each student is inspected at most once, each inspection looks at finitely many edges, and there are finitely many students, so the search always ends.

Suppose the search succeeds at an unused task t. Follow the predecessors backward from t: the predecessor of a task is a student, and the predecessor of a student other than x is the task it holds. Every vertex was reached later than its predecessor, so no vertex is met twice; as there are finitely many vertices, the backward walk stops, and it can stop only at x, the one reached vertex without a predecessor. Read forward, it is a path from x to t. Every step into a task used an unmatched edge and every step into a student a matched edge, so the path is alternating. Its ends are unmatched, and it has at least one step, because x is a student and t a task. So it is an augmenting path, and Lemma 8.5 lets us flip along it.

The rules do not say which uninspected student to choose next. Inspecting students in the order in which they were reached is called *breadth-first search*, but nothing in this chapter depends on the order.

The whole procedure

Repeating the search gives a procedure for the assignment problem:

```
Start with the empty matching M.
While some student is unmatched:
    choose an unmatched student x and search from x;
    if the search succeeds:
        flip M along the augmenting path it found;
    if the search fails:
        stop, and report the set A of students it reached.
If the loop ends because every student is matched, report M.
```

As shown at the end of Section 8.3, the procedure makes at most $|L|$ flips. When a search fails, the procedure reports a set of students, and Section 8.5 shows that this set proves that no matching saturates L.

What a failed search guarantees

When the search fails, the reached vertices have two properties.

(C1) If s is a reached student and s–t is an unmatched edge, then t has been reached.

(C2) Every reached task is used, and its owner has been reached.

Property (C1) holds because a failed search has inspected every reached student, and inspecting s follows every unmatched edge at s. Property (C2) holds because reaching an unused task would have ended the search with success, and because the owner of a used task is marked as reached at the moment the task is.

We call a search *complete* if it stops only when it has found an unused task or has inspected every reached student; the search of this section is complete. A search that gives up earlier, say when its first route is blocked, can stop while (C1) is still false, and then its failure tells us nothing: an augmenting path might use an edge it never tried.

When a complete search fails, no augmenting path starts at x. Suppose there were one, $x = v_0, v_1, \ldots, v_k$. Its vertices alternate between students and tasks, starting with x, and its edges alternate, starting with an unmatched one (Section 8.3), so the edge $v_i v_{i+1}$ is unmatched when i is even, that is, when v_i is a student, and matched when v_i is a task. By induction on i, every v_i has been reached. Certainly $v_0 = x$ has. If a reached v_i is a student, the edge $v_i v_{i+1}$ is unmatched, so v_{i+1} has been reached by (C1). If a reached v_i is a task, then by (C2) it is used and its owner has been reached; the matched edge $v_i v_{i+1}$ can only join it to its owner, so v_{i+1} is that owner. Hence the last vertex v_k, a task, has been reached, and by (C2) it is used, whereas the last vertex of an augmenting path is unused. This contradiction proves the claim.

8.5 A failed search exposes a shortage

To say that a group of students is short of tasks, we need all the tasks the group accepts. For a set S of students, let $N(S)$ be the set of tasks that at least one student of S accepts, that is, the union of the sets $N(s)$ for s in S. A task accepted by several students of S is still only one member of $N(S)$: in the Ada–Ben graph, $N(\{\text{Ada}, \text{Ben}\}) = \{1, 2\} \cup \{1\} = \{1, 2\}$. For a single student, $N(\{s\}) = N(s)$, and $N(\varnothing) = \varnothing$.

An example

Let three students a, b, c each accept exactly the tasks 1 and 2, let $M = \{a\text{–}1,\ b\text{–}2\}$, and run the complete search of Section 8.4 from the root $x = c$. Write A for the set of students it reaches, including x, and B for the set of tasks it reaches. Inspecting c follows both of its edges, which are unmatched, and reaches task 1 with its owner a and task 2 with its owner b. Inspecting a and b finds only the unmatched edges a–2 and b–1, whose tasks have already been reached. So the search fails, with $A = \{c, a, b\}$ and $B = \{1, 2\}$. Here $|B| = |A| - 1$, since each task of B is held by a different student of A and the root c holds none, and $N(A) = B$, since the students of A accept only tasks 1 and 2; three students who together accept two tasks cannot all be given different ones. The next proposition shows that this happens whenever a complete search fails.

Proposition 8.6. *Let M be a matching in a bipartite graph and let x be an unmatched student. Suppose that the complete search from x fails, and let A and B be the sets of students and tasks that it reached. Then*

$$N(A) = B \qquad \text{and} \qquad |N(A)| = |A| - 1.$$

Proof. We use properties (C1) and (C2) of a failed search from Section 8.4.

Step 1: $|B| = |A| - 1$. For each task t in B, let $f(t)$ be the owner of t. By (C2), t is used, so it has exactly one owner, and this owner is a reached student. It is not x, because x is unmatched. So f is a function from B to $A \setminus \{x\}$. It is injective: if two different tasks t and t' had the same owner s, then s would lie on two different edges s–t and s–t' of M. It is surjective: a student s in A other than x can only have been reached by a move from the task it holds, so that task lies in B, and f sends it to s. Hence f is a bijection, and by Section 3.3, $|B| = |A \setminus \{x\}| = |A| - 1$.

Step 2: $N(A) = B$. Every task t in B was reached by a move from some reached student s along an edge s–t, so $t \in N(s) \subseteq N(A)$. Thus $B \subseteq N(A)$. Conversely, let t be any task in $N(A)$, and choose a student $s \in A$ that accepts t.

- If the edge s–t is unmatched, then t has been reached, by (C1).
- If the edge s–t is matched, then s is matched, so $s \neq x$, and t is the task that s holds. As in Step 1, s was reached by a move from that task, so t was reached before s.

In both cases $t \in B$. Thus $N(A) \subseteq B$, and so $N(A) = B$.

Combining the two steps gives $|N(A)| = |B| = |A| - 1$. □

In the smallest case the root x accepts no task. The search fails at once, with $A = \{x\}$ and $B = \varnothing$, and indeed $N(A) = \varnothing = B$ and $|N(A)| = 0 = |A| - 1$.

Step 1 compares A only with the tasks the search happened to reach; Step 2, which needs the completeness of the search, counts *all* the tasks the students of A accept. Suppose the search in the example had stopped right after reaching task 1 and its owner a, before following the edge c–2. Then $A = \{c, a\}$ and $B = \{1\}$, and still $|B| = |A| - 1$. But $N(A) = \{1, 2\}$ has as many members as A, so these two students are not short of tasks at all. The complete search cannot stop in this way, because (C1) forces it to follow c–2 as well.

Shortage sets are certificates

A set S of students with $|N(S)| < |S|$ is called a *shortage set*. If a shortage set exists, then no matching saturates L. Indeed, if some matching saturated L, each student s of S would have a partner, a task that s accepts and hence a member of $N(S)$, and different students of S would have different partners, because two edges of a matching never share an endpoint. So $N(S)$ would contain $|S|$ different tasks, which is impossible when it has fewer than $|S|$ members.

By Proposition 8.6, the set A reported by a failed search satisfies $|N(A)| = |A| - 1 < |A|$, so it is a shortage set, and no full assignment exists. A shortage set is an *obstruction*: it blocks every possible way of assigning tasks, not just the choices one search happened to make. It is also easy to check, without trusting or rerunning the search that found it: list the tasks its students accept and count them. Evidence that can be checked directly in this way is called a *certificate*, and a shortage set is also called a *shortage certificate*. A full assignment is a certificate in the other direction: check that every student appears in exactly one pair, that each pair is an edge, and that no task appears twice. So the procedure of Section 8.4 always ends with a certificate, and Chapter 9 turns this observation into a theorem.

> **Habit 7: make failure informative**
>
> A failed attempt to build something is not, by itself, a proof that it cannot be built. Ask whether the failure can be turned into a concrete object that violates a necessary condition. Here the condition is that every group of students accepts at least as many tasks as it has members, and the object is a group that accepts too few. Elsewhere, a failed attempt to invert a function may turn up two inputs with the same output, which shows that no inverse exists. When a search, yours or an assistant's, reports that nothing was found, ask whether it produced such an object, and whether it could have missed anything.

8.6 The whole procedure on a small example

Let three students a, b, c accept tasks from 1, 2, 3 as follows:

$$N(a) = \{1, 2\}, \qquad N(b) = \{1\}, \qquad N(c) = \{2, 3\}.$$

We run the procedure of Section 8.4, taking the unmatched students as roots in the order a, c, b, and inspecting the edges at each student in increasing order of task number.

1. Starting from the empty matching, the search from a finds the unused task 1 at once. The augmenting path is the single edge a–1, and flipping gives $M = \{a\text{–}1\}$.
2. Likewise, the search from c finds the unused task 2 at once, and flipping gives $M = \{a\text{–}1,\ c\text{–}2\}$.
3. Now b is unmatched, and its only task is held by a. The first two rounds made exactly the choices of a greedy procedure, which would now be stuck. The search from b reaches the following vertices, in this order.

Vertex reached	Predecessor	Reason
b	—	the root of the search
task 1	b	inspecting b: the edge b–1 is unmatched
a	task 1	task 1 is held by a
task 2	a	inspecting a: the edge a–2 is unmatched (a–1 is matched)
c	task 2	task 2 is held by c
task 3	c	inspecting c: the edge c–3 is unmatched (c–2 is matched)

Task 3 is unused, so the search succeeds. Following the predecessors backward from task 3 gives 3, c, 2, a, 1, b, and read forward this is the augmenting path

$$b - 1 - a - 2 - c - 3.$$

The picture shows the flip.



Solid: matched edges, removed by the flip.
Dashed: unmatched edges, added by the flip.

After the flip, the matching $\{b\text{–}1,\ a\text{–}2,\ c\text{–}3\}$ has three edges. Every student is matched, so the procedure stops and reports this assignment. Anyone can check it: $1 \in N(b)$, $2 \in N(a)$, $3 \in N(c)$, and the three tasks are different.

Now remove task 3 from c's options, so that $N(c) = \{2\}$, and run the procedure again in the same order. The first two rounds again give $M = \{a\text{–}1,\ c\text{–}2\}$, and the search from b again reaches task 1, a, task 2, and c. But now the only edge at c is the matched edge c–2, so inspecting c reaches nothing new, and the search fails with

$$A = \{b, a, c\}, \qquad B = \{1, 2\}.$$

As Proposition 8.6 predicts, $|B| = |A| - 1$ and

$$N(A) = N(a) \cup N(b) \cup N(c) = \{1, 2\} \cup \{1\} \cup \{2\} = \{1, 2\} = B.$$

So $|N(A)| = 2 < 3 = |A|$, and A is a shortage set: no order of choices could have succeeded.

Exercises

Exercises 8.1–8.4 are core exercises, and Exercises 8.5 and 8.6 are transfer exercises. Complete solutions are in Appendix A, Section A.8.

Exercise 8.1. List all the matchings of the Ada–Ben graph of Section 8.1, where Ada accepts tasks 1 and 2 and Ben only task 1. Decide which are maximal and which are maximum, and conclude that {Ada–1} is the only matching that is maximal but not maximum.

Exercise 8.2. Students a, b, c accept the tasks $N(a) = \{1, 2\}$, $N(b) = \{1, 3\}$, $N(c) = \{2\}$. Start from the matching $\{a\text{–}2,\ b\text{–}1\}$. Use the search of Section 8.4 to find an augmenting path, and write down the matching obtained by flipping along it.

Exercise 8.3. Three students each accept exactly the tasks 1 and 2, as in the example of Section 8.5. Give a shortage certificate, and explain why the fact that every student has two acceptable tasks is not enough for a full assignment. Then show that no smaller group of these students is a shortage set.

Exercise 8.4. The proof of Lemma 8.5 uses the fact that an augmenting path has no repeated vertex. (a) Explain where the proof uses this fact. Give an example of a matching and an alternating walk from an unmatched student to an unused task that visits some vertex twice, such that flipping along the walk does not give a matching. (b) Suppose that some alternating walk leads from an unmatched student to an unused task. Prove that a shortest such walk has no repeated vertex, so that it is an augmenting path.

Exercise 8.5. A procedure works with matchings in a bipartite graph whose set of students is L, and each of its steps enlarges the current matching by exactly one edge, for example by flipping along an augmenting path. Explain why growth at every step does not, on its own, prove that the procedure stops, and why the finiteness of L supplies the missing ingredient. State an upper bound on the number of steps when the procedure starts from a matching with k edges.

Exercise 8.6. Suppose that the complete search of Section 8.4, started at an unmatched student x, fails, and let A and B be the sets of students and tasks it reached. (a) Without looking back at the proof of Proposition 8.6, prove carefully that $N(A) = B$, treating matched and unmatched edges separately, and point out exactly where the completeness of the search is used. (b) Now suppose instead that the search is stopped early: it has reached no unused task, but some reached student has not been completely inspected. Let A and B be the sets of students and tasks reached so far. Which of the two containments $B \subseteq N(A)$ and $N(A) \subseteq B$ must still hold? Give an example, different from the one in Section 8.5, in which the other one fails. Then show that whenever it fails, A is not a shortage set.

CHAPTER 9

Hall's theorem: a complete proof apprenticeship

> **Overview**
>
> Prove Hall's theorem, which says exactly when every student can be given a different acceptable task: the easy direction directly, and the harder direction twice, with the search procedure of Chapter 8 and by strong induction. Read a proof with several stages and rebuild it yourself. See why a group of students who between them accept too few tasks is the only possible obstruction to a full assignment, and how the case of equality in an inequality can show where to split a problem.

9.1 A condition that checks every group of students

We use the model of Chapter 8: a finite bipartite graph (Definition 8.1) with the set L of students and the set R of tasks as its parts, and an edge s–t when student s accepts task t. A matching (Definition 8.2) is a set of edges no two of which share an endpoint, and it saturates L when every student lies on one of its edges; such a matching is exactly a full assignment. For a set $S \subseteq L$, $N(S)$ is the set of tasks acceptable to at least one student in S (Section 8.5; N stands for *neighbors*), and $N(x)$ means $N(\{x\})$.

Section 8.5 showed that a *shortage set*, a group S of students with $|N(S)| < |S|$, makes a full assignment impossible. Hall's theorem says that this is the only thing that can go wrong. It concerns the condition

$$|N(S)| \geq |S| \qquad \text{for every subset } S \subseteq L. \tag{H}$$

In words: every group of students accepts, between them, at least as many tasks as there are students in the group. We call (H) *Hall's condition*; it is the group condition in the solution to Exercise 2.5.

Why must we check every group? Checking each student on their own misses students who compete for the same few tasks, and checking only the whole class misses a smaller group that is short of tasks. For example, let

$$N(a) = N(b) = \{1\}, \qquad N(c) = \{2, 3\}.$$

Every student accepts at least one task, and the whole class $L = \{a, b, c\}$ accepts three tasks, as many as there are students. Yet the group $\{a, b\}$ accepts only task 1, and two students cannot both receive it, so no full assignment exists.

So read (H) as a list of tests, one for each group S: collect into $N(S)$ every task that some member of S accepts (a task accepted by several members counts once, since it can be given to only one of them), and compare $|N(S)|$ with $|S|$. With n students, a group is formed by deciding for each student separately whether they belong to it, so there are $2 \cdot 2 \cdots 2 = 2^n$ groups, counting the empty group and the whole class; for three students there are eight tests. The empty group has $N(\varnothing) = \varnothing$, so its test $0 \geq 0$ always holds, and a single student x tests $|N(x)| \geq 1$.

Theorem 9.1 (Hall's matching theorem)**.** *Let G be a finite bipartite graph with parts L and R. Then G has a matching that saturates L if and only if $|N(S)| \geq |S|$ for every subset $S \subseteq L$.*

The theorem makes two claims. If a full assignment exists, then (H) holds: in the language of Section 2.1, Hall's condition is *necessary*. If (H) holds, then a full assignment exists: the condition is also *sufficient*.

We prove the sufficient direction twice: once with the search procedure of Chapter 8, which actually constructs an assignment, and once by strong induction (Chapter 5). Philip Hall published the theorem in 1935, and it has become one of the basic tools of combinatorics, the study of finite arrangements and how to count them. For further reading on graphs and matchings, see [1].

9.2 First prove the necessary direction

We begin with the easier half of Theorem 9.1: if a full assignment exists, then every group passes its test. This is the argument that Section 8.5 used for shortage sets, written as a direct proof.

Proof that Hall's condition is necessary. Suppose that a matching M saturates L, and let S be any subset of L. Each student $s \in S$ lies on exactly one edge of M: on at least one because M saturates L, and on at most one because no two edges of M share an endpoint. Call the task at the other end of that edge $f(s)$. Since s–$f(s)$ is an edge, s accepts $f(s)$, so $f(s) \in N(S)$. This defines a function $f : S \to N(S)$.

The function f is injective: if two different students $s \neq s'$ had $f(s) = f(s')$, that task would be an endpoint of two different edges of M, which a matching does not allow. By Section 3.3, an injective function from a finite set S to a finite set $N(S)$ produces $|S|$ different members of $N(S)$, so $|S| \leq |N(S)|$. (When $S = \varnothing$, f has no inputs and this reads $0 \leq 0$.) Since S was an arbitrary subset of L, condition (H) holds. □

For example, in the graph of Section 8.6, with

$$N(a) = \{1, 2\}, \qquad N(b) = \{1\}, \qquad N(c) = \{2, 3\}$$

and the matching b–1, a–2, c–3, the group $S = \{a, b\}$ gives $f(a) = 2$ and $f(b) = 1$: two different tasks in $N(S) = \{1, 2\}$, as Hall's condition requires.

The other direction is harder. The inequalities in (H) say nothing directly about which student should receive which task, yet from them alone we must produce an assignment in which the choices for all the students fit together at once.

9.3 A constructive proof from the search procedure

Chapter 8 already contains almost everything we need for the sufficient direction. Recall the complete search of Section 8.4 for a matching M, started at an unmatched student x. For each vertex it reaches, it records the predecessor, the vertex from which it was reached. It succeeds when it reaches an unused task, and fails when it has inspected every reached student without finding one. Chapter 8 proved a fact about each outcome.

- If the search succeeds, then following the predecessors back to x gives an augmenting path (Section 8.4). Flipping M along this path gives a matching with one more edge (Lemma 8.5).
- If the search fails, let A be the set of students it reached, x included, and let B be the set of tasks it reached. Then $N(A) = B$ and $|B| = |A| - 1$ (Proposition 8.6). Hence $|N(A)| = |A| - 1 < |A|$.

In the second case A is a shortage set, so under (H) the second case can never happen. That is the whole idea of the proof.

Constructive proof that Hall's condition is sufficient. Assume that the graph satisfies (H). Start with the empty matching. As long as some student is unmatched, carry out the following round. Choose an unmatched student x and run the complete search from x. If the search failed, the set A of students it reached would satisfy $|N(A)| = |A| - 1 < |A|$ by Proposition 8.6, contradicting (H). So the search reaches an unused task, and flipping the matching along the resulting augmenting path increases its number of edges by one.

Each edge of a matching contains exactly one student, and different edges contain different students, so a matching has at most $|L|$ edges. The matching starts with 0 edges and gains one in every round, so there are at most $|L|$ rounds. The rounds stop only when no student is unmatched, so the final matching saturates L. (If $L = \varnothing$, there are no rounds, and the empty matching saturates L.) □

Section 8.6 shows both outcomes. The graph with $N(a) = \{1, 2\}$, $N(b) = \{1\}$ and $N(c) = \{2, 3\}$ satisfies Hall's condition, and an unlucky start there, a–1 and c–2, in which a takes the only task that b accepts, is repaired along the augmenting path b–1–a–2–c–3, as the proof promises. When task 3 is removed from c's options, the search from b fails and reports the group $\{a, b, c\}$, which accepts only tasks 1 and 2.

The procedure always ends with a certificate, a matching that saturates L or a shortage set (Section 8.5), and Hall's theorem says that for each graph exactly one of the two exists: the necessary direction rules out a shortage set when a matching saturates L, and the sufficient direction, read as a contrapositive, says that when no matching saturates L, some group fails its test, and that group is a shortage set. This is Habit 7 (Section 8.5) carried to its end: every failure of the procedure produces a concrete obstruction, and a shortage set is the only kind of obstruction there is.

9.4 A second proof: equality tells us where to split

A second proof, by strong induction, needs no search at all. It brings out an idea that is useful far beyond matchings: pay special attention to the cases in which an inequality holds with equality. Throughout this section, G is a finite bipartite graph with parts L and R.

Definition 9.2 (Tight group)**.** A set $S \subseteq L$ of students is *tight* if $|N(S)| = |S|$.

Under Hall's condition, a tight group is one with no spare tasks. The empty group is always tight, because $|N(\varnothing)| = 0 = |\varnothing|$. In the graph of Section 8.6, where $N(a) = \{1, 2\}$, $N(b) = \{1\}$ and $N(c) = \{2, 3\}$, the groups $\{b\}$, $\{a, b\}$ and $\{a, b, c\}$ are tight, since they accept exactly the tasks $\{1\}$, $\{1, 2\}$ and $\{1, 2, 3\}$, as many as they have members. The group $\{c\}$ is not tight, because c accepts two tasks.

Why do tight groups matter? Suppose that S is tight and that a full assignment exists. The students of S receive $|S|$ different tasks, all in $N(S)$, which has only $|S|$ members. So they use up every task in $N(S)$, and all the other students must be served by tasks outside $N(S)$. A tight group is therefore a self-contained block, and this suggests splitting the problem in two: the students of S with the tasks in $N(S)$, and everyone else with the remaining tasks.

For an induction, both pieces must be smaller than the original problem. A subset S of L is called *proper* if $S \neq L$, and we then write $S \subsetneq L$. If S is nonempty and proper, then $|S| < |L|$, because some student of L lies outside S, and $|L \setminus S| = |L| - |S| < |L|$, because $|S| > 0$.

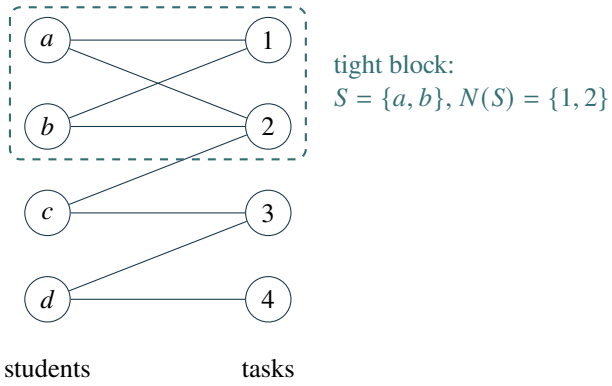
Proof plan. Use strong induction on the number of students. If some nonempty proper group S is tight, split the problem as just described. If no nonempty proper group is tight, every such group has at least one spare task. This spare capacity makes it safe to give any one student any task that student accepts, and we show that the smaller problem that remains still satisfies Hall's condition.

The two cases in small examples

Here is the first case of the plan at work. Four students have the options

$$N(a) = N(b) = \{1, 2\}, \qquad N(c) = \{2, 3\}, \qquad N(d) = \{3, 4\}.$$

In the drawing, the dashed box surrounds a tight block.



This graph satisfies Hall's condition, as you can confirm by checking all $2^4 = 16$ inequalities of (H). The group $S = \{a, b\}$ is tight, so any full assignment must give a and b the tasks 1 and 2 in some order. So we set a and b aside, together with tasks 1 and 2. Of the remaining tasks, c accepts only task 3, and d accepts tasks 3 and 4, so c–3 and d–4, together with a–1 and b–2, give a full assignment.

For a group T of remaining students, write

$$N'(T) = N(T) \setminus N(S)$$

for the set of tasks that someone in T still accepts once the tasks of $N(S)$ are removed (the mark $'$, read "prime", shows that $N'(T)$ belongs to the smaller problem). For $T = \{c\}$, the set $N(T) = \{2, 3\}$ loses task 2, and $N'(T) = \{3\}$. Why must some task of c survive? Not because of Hall's condition for T alone: $|N(c)| \geq 1$ would still hold if c accepted only task 2, leaving c nothing. What helps is Hall's condition for the combined group $S \cup T = \{a, b, c\}$: it accepts at least three tasks, and only two of them can lie in $N(S) = \{1, 2\}$, so it accepts some task outside $N(S)$. No student of S accepts that task, so c does. The general proof shows $|N'(T)| \geq |T|$ for every group T of remaining students in the same way.

Here is the second case of the plan. Let $N(a) = \{1, 2\}$ and $N(b) = \{2, 3\}$. The nonempty proper groups $\{a\}$ and $\{b\}$ each accept one task more than they need, so neither is tight. The plan lets us give a any task it accepts, even task 2, which b also wants: then b still has task 3, and $\{a$–2, b–3$\}$ is a full assignment. The spare task of the group $\{b\}$ absorbs the loss of task 2.

The inductive proof

We prove the sufficient direction of Theorem 9.1 by strong induction on the number of students (Section 5.2). For each natural number n, let $P(n)$ be the statement:

> *Every finite bipartite graph whose left part has exactly n vertices, and which satisfies Hall's condition, has a matching that saturates its left part.*

The statement places no restriction on the right part, because the smaller graphs to which we apply the induction hypothesis lose tasks as well as students. In any graph, Hall's condition is computed with that graph's own edges: every set U of vertices in its left part must be joined to at least $|U|$ vertices of its right part. In a smaller graph some edges are gone, so the tasks joined to U there may be fewer than those in $N(U)$.

Inductive proof that Hall's condition is sufficient. Base case, $n = 0$. If L is empty, the empty matching saturates L, because there is no student to cover. So $P(0)$ holds.

Induction step. Let $n \geq 1$, and assume the induction hypothesis: $P(m)$ holds for every natural number $m < n$. (Here n plays the role of $k + 1$ in the strong induction step of Section 5.2.) Let G be a finite bipartite graph with parts L and R, where $|L| = n$, and suppose that G satisfies (H). We must find a matching of G that saturates L. There are two cases.

Case 1: some nonempty proper subset of L is tight. Choose a set S with $\varnothing \neq S \subsetneq L$ and $|N(S)| = |S|$.

The students in S. Let G_1 be the graph with left part S, right part $N(S)$, and as edges all the edges of G that join a student of S to a task of $N(S)$. Every task that a student of S accepts lies in $N(S)$, so G_1 keeps every edge of G at the students of S. Hence, for every $U \subseteq S$, the set of tasks joined to U in G_1 is exactly $N(U)$, and since $U \subseteq L$, (H) for G gives $|N(U)| \geq |U|$. So G_1 satisfies Hall's condition. Since $0 < |S| < n$, the induction hypothesis $P(|S|)$ gives a matching M_1 of G_1 that saturates S.

The other students. Let G_2 be the graph with left part $L \setminus S$, right part $R \setminus N(S)$, and as edges all the edges of G that join a student of $L \setminus S$ to a task of $R \setminus N(S)$. Let T be any subset of $L \setminus S$. A task is joined to T in G_2 exactly when someone in T accepts it and it does not lie in $N(S)$. So the set of tasks joined to T in G_2 is $N'(T) = N(T) \setminus N(S)$.

We show that $|N'(T)| \geq |T|$. The sets S and T are disjoint, because $T \subseteq L \setminus S$, so $|S \cup T| = |S| + |T|$. A task is acceptable to someone in $S \cup T$ exactly when it is acceptable to someone in S or to someone in T, so $N(S \cup T) = N(S) \cup N(T)$. Applying (H) to $S \cup T$ therefore gives

$$|N(S) \cup N(T)| \geq |S| + |T|.$$

Every member of $N(S) \cup N(T)$ lies in exactly one of the two disjoint sets $N(S)$ and $N'(T) = N(T) \setminus N(S)$, so

$$|N(S) \cup N(T)| = |N(S)| + |N'(T)|$$

(Exercise 9.3 asks you to check this carefully). Combining the last two displays,

$$|N(S)| + |N'(T)| \geq |S| + |T|.$$

Now we use tightness. Substitute $|N(S)| = |S|$ and subtract $|S|$ from both sides to obtain $|N'(T)| \geq |T|$. This holds for every $T \subseteq L \setminus S$, including $T = \varnothing$. So G_2 satisfies Hall's condition. Since S is nonempty, G_2 has $n - |S| < n$ students, and the induction hypothesis $P(n - |S|)$ gives a matching M_2 of G_2 that saturates $L \setminus S$.

Combining the two matchings. The edges of G_1 and of G_2 are edges of G. Every edge of M_1 joins a student of S to a task of $N(S)$, and every edge of M_2 joins a student of $L \setminus S$ to a task of $R \setminus N(S)$, so no vertex lies both on an edge of M_1 and on an edge of M_2. Within each of the two matchings, no two edges share an endpoint either. Hence $M_1 \cup M_2$ is a matching of G. Every student of L lies in S or in $L \setminus S$, so it is matched by M_1 or by M_2. Thus $M_1 \cup M_2$ saturates L.

Case 2: no nonempty proper subset of L is tight. Let S be a nonempty proper subset of L. Condition (H) gives $|N(S)| \geq |S|$, and $|N(S)| \neq |S|$ because S is not tight, so $|N(S)| > |S|$. Both sides are integers, and an integer larger than the integer $|S|$ is at least $|S| + 1$ (Exercise 9.4). Therefore

$$|N(S)| \geq |S| + 1 \qquad \text{for every set } S \text{ with } \varnothing \neq S \subsetneq L.$$

Since $n \geq 1$, we can choose a student $x \in L$; any student will do. Condition (H) for the group $\{x\}$ gives $|N(x)| \geq 1$, so x accepts at least one task y. Let G' be the graph obtained from G by deleting the student x, the task y, and every edge at x or at y. Its left part $L \setminus \{x\}$ has $n - 1$ students, and its right part is $R \setminus \{y\}$.

We check Hall's condition for G'. Let $T \subseteq L \setminus \{x\}$. If $T = \varnothing$, the condition $0 \geq 0$ holds. Otherwise, note that G' has lost only the edges of G at x and at y, and the edges at x do not touch

T, because $x \notin T$. So a task is joined to T in G' exactly when it lies in $N(T)$ and is not y; the set of these tasks is $N(T) \setminus \{y\}$. The set T is nonempty, and it is a proper subset of L because it does not contain x. So $|N(T)| \geq |T| + 1$. Removing the single task y removes at most one member of $N(T)$, so

$$|N(T) \setminus \{y\}| \geq |N(T)| - 1 \geq |T|.$$

Thus G' satisfies Hall's condition, and the induction hypothesis $P(n-1)$ gives a matching M' of G' that saturates $L \setminus \{x\}$.

The edges of M' are edges of G, and so is x–y. This edge shares no endpoint with any edge of M', because x and y are not vertices of G'. So $M' \cup \{x\text{–}y\}$ is a matching of G, and it matches x and every student of $L \setminus \{x\}$, so it saturates L.

When $n = 1$, the set $L = \{x\}$ has no nonempty proper subset, so Case 2 applies, M' is the empty matching given by $P(0)$, and the single edge x–y saturates L.

In both cases G has a matching that saturates L, so $P(n)$ holds. By strong induction, $P(n)$ holds for every natural number n, and this proves the sufficient direction of Hall's theorem. □

9.5 Why every part of the case split is needed

Each condition in the case split of the inductive proof has a job, and the way to see it is to drop the condition and watch the proof break.

Why must S be nonempty? The empty group is always tight. If Case 1 allowed $S = \varnothing$, the first smaller problem would have no students, and the second would be the original problem again, because $L \setminus \varnothing = L$ and $R \setminus N(\varnothing) = R$. The induction hypothesis says nothing about a graph with n students, so the proof would make no progress.

Why must S be proper? The whole class can be tight too: in the four-student example of Section 9.4, it accepts exactly the four tasks $1, 2, 3, 4$. If Case 1 allowed $S = L$, the first smaller problem would again contain all n students, and the induction hypothesis could not be applied to it.

Why must S be tight? Case 1 removes every task in $N(S)$ from the second problem. If S had spare tasks, this could throw away tasks that other students need. Take $N(a) = \{1, 2\}$ and $N(b) = \{2\}$, and let $S = \{a\}$, which is nonempty and proper but not tight, since $|N(S)| = 2 > 1$. Removing $N(S) = \{1, 2\}$ leaves b with no acceptable task, although a–1, b–2 is a full assignment.

Pause and decide. In Case 1, why do we remove every task in $N(S)$, rather than only the tasks that the matching M_1 actually gives to the students in S?

Check. Because S is tight, these are the same tasks: M_1 gives the $|S|$ students of S different tasks inside $N(S)$, which has only $|S|$ members, so it uses them all. Describing them as $N(S)$ lets us name them before M_1 has been found, so the two smaller problems can be set up and solved independently.

Why does Case 2 need the extra unit? Giving x the task y can cost each remaining group one of its acceptable tasks, and the inequality $|N(T)| \geq |T| + 1$ is exactly enough to pay for that loss. Without it, an arbitrary first choice can be fatal. In the Ada–Ben example of Section 8.1, Ada accepts tasks 1 and 2 and Ben accepts only task 1. Giving Ada task 1 leaves Ben with nothing,

although Ada–2, Ben–1 is a full assignment. The proof never makes this mistake: the group {Ben} is nonempty, proper and tight, so this graph belongs to Case 1, which first serves the tight block {Ben} with task 1 and then gives Ada task 2.

Applying a theorem means checking all of its hypotheses

Suppose an assistant proposes the proof "split the students into a group and the rest, and apply induction to each part." That is not yet a proof. The induction hypothesis does not say that smaller graphs have matchings that saturate their left parts, which is false: the graph with $N(a) = N(b) = \{1\}$ has only two students and no such matching. It says this only for graphs *that satisfy Hall's condition*. So a proof must show that both smaller problems satisfy Hall's condition, and the example under "Why must S be tight?" shows that this can fail for a badly chosen group.

Each time the proof applies the induction hypothesis, it must supply three things:

1. a finite bipartite graph, with its left part and right part named;
2. a check that its left part has fewer than n students;
3. a check that it satisfies Hall's condition, that is, an inequality for every subset of its left part.

The proof supplies all three for G_1 and G_2 in Case 1 and for G' in Case 2. The third item is where the work lies, because deleting tasks can destroy Hall's condition: for each group of students in a smaller graph, the proof first works out exactly which tasks are joined to it there ($N(U)$, $N'(T)$ or $N(T) \setminus \{y\}$), and then counts them. In general, when a proof says "by the induction hypothesis" or "by the theorem," write out the hypotheses and check each one for the object at hand (compare Section 2.1).

9.6 Mathematical taste from equality cases

The inductive proof uses the inequality $|N(S)| \geq |S|$ in two ways. When it holds with equality, the group S is rigid: any full assignment must use up all of $N(S)$ on the students of S, and that told us where to split the problem. When it is strict, the sizes, being integers, differ by at least one, enough to absorb the loss of one task.

This rigidity is a general fact from Section 3.3: an injective function between finite sets of the same size is also surjective. Here it is applied to the function from S to $N(S)$ that sends each student of S to the task the matching gives that student.

Whenever you meet an inequality, ask more than whether it can be improved: ask what happens when it holds with equality. If equality forces a recognizable structure, that structure may let you break the problem into smaller pieces. When the inequality is strict, ask how large the margin is and what it can pay for. You will meet a margin of this kind in Chapter 12: the contraction theorem (Theorem 12.2) requires a number q with $0 \leq q < 1$, and its error bounds divide by the gap $1 - q$, so they make sense only because the gap is positive, and they are strong only when it is not too small.

Exercises and review portfolio

Exercises 9.1–9.4 are core exercises, and Exercise 9.5 is a transfer exercise. Exercise 9.6 is a review portfolio, which draws on this chapter and Chapter 8. Solutions are in Appendix A, Section A.9; the solution to Exercise 9.6 is a model portfolio.

Exercise 9.1. Give a graph with four students and four tasks in which every student accepts at least two tasks and $|N(L)| = |L|$, but no matching saturates L. Find a group S of students with $|N(S)| < |S|$, and explain why the tests for single students and for the whole class do not detect the problem.

Exercise 9.2. Let $N(a) = \{1\}$, $N(b) = \{1, 2\}$ and $N(c) = \{2, 3\}$. Check Hall's condition for every subset of $L = \{a, b, c\}$, find every nonempty proper tight subset, and find a matching that saturates L.

Exercise 9.3. In Case 1 of the inductive proof in Section 9.4, prove that $|N(S) \cup N(T)| = |N(S)| + |N(T) \setminus N(S)|$. Do not assume that $N(S)$ and $N(T)$ are disjoint; in the example of that section they are not.

Exercise 9.4. In Case 2 of the inductive proof, a nonempty proper subset S of L satisfies $|N(S)| > |S|$. Explain why it follows that $|N(S)| \geq |S| + 1$. Would the same step be valid if $|N(S)|$ and $|S|$ were replaced by arbitrary real numbers?

Exercise 9.5. Let d be a positive integer. Suppose that every student accepts exactly d tasks and that every task is acceptable to at most d students. Use double counting (Section 7.2) and Hall's theorem to prove that a full assignment exists. Where does your proof use $d > 0$?

Exercise 9.6. Review portfolio. Assemble a short portfolio with four parts. (a) For the graph of Exercise 9.2, give a matching that saturates L, and explain how a reader can check it. (b) For a graph with no such matching, such as the one you found in Exercise 9.1, give a group S of students with $|N(S)| < |S|$, explain how a reader can check it, and explain why it proves that no full assignment exists. (c) Write out, in your own words, a complete proof that Hall's condition implies the existence of a matching that saturates L. You may follow either proof in this chapter. You may quote results proved in Chapter 8, such as Lemma 8.5 and Proposition 8.6, if you state what they say. (d) Write a paragraph that explains the difference between a maximal and a maximum matching, and why a greedy assignment can get stuck even when a full assignment exists.

CHAPTER 10

Linearity: changing the language of a problem

Overview

Work with vectors as ordered lists of numbers, and recognize a linear map as one that preserves addition and scaling. See why a matrix, a finite table of numbers, describes such a map completely. Use the kernel to decide when a linear map is injective, prove the Cauchy–Schwarz and triangle inequalities, and see why matrices are multiplied the way they are. Throughout, notice how a change of language, such as a new basis or a matrix in place of a formula, can explain why a fact is true, not only confirm it.

10.1 Vectors are ordered measurements

Let n be a positive integer. A *vector* in $\mathbb{R}^n$ is an ordered list $x = (x_1, x_2, \ldots, x_n)$ of n real numbers, called its *coordinates*, and $\mathbb{R}^n$ is the set of all such lists. We picture a vector (x_1, x_2) of $\mathbb{R}^2$ as the point of the plane x_1 units to the right of a fixed point, the *origin*, and x_2 units above it (a negative coordinate means a step left or down). But coordinates need not describe a position: a shop could record 3 kilograms of flour and 5 kilograms of sugar as the vector $(3, 5)$. As with the ordered pairs of Section 3.1, order matters: two vectors are equal exactly when they agree in every coordinate, so $(1, 3) \neq (3, 1)$.

Vectors are added coordinate by coordinate, and multiplying a vector by a real number c multiplies every coordinate by c:

$$(x_1, \ldots, x_n) + (y_1, \ldots, y_n) = (x_1 + y_1, \ldots, x_n + y_n),$$
$$c(x_1, \ldots, x_n) = (cx_1, \ldots, cx_n).$$

For example, $(1, 3) + (2, -1) = (3, 2)$ and $2(1, 3) = (2, 6)$. In this setting a real number such as c is called a *scalar*. The *zero vector*, written simply 0, has every coordinate equal to zero; we write $-x = (-1)x$ and $x - y = x + (-y)$. The familiar rules of arithmetic, such as $x + y = y + x$, $x + 0 = x$, $x - x = 0$, $c(x + y) = cx + cy$ and $(a + b)x = ax + bx$, carry over to vectors, because each can be checked one coordinate at a time.

A *linear combination* of vectors $v_1, \ldots, v_k$ is a vector of the form

$$a_1 v_1 + a_2 v_2 + \cdots + a_k v_k,$$

where the scalars $a_1, \ldots, a_k$ are the *coefficients* of the combination. Here the subscripts number the vectors in a list; each v_j is a whole vector, not a coordinate. For example, $2(1, 3) - (2, -1) = (0, 7)$ is a linear combination of $(1, 3)$ and $(2, -1)$ with coefficients 2 and -1.

A list of vectors in $\mathbb{R}^n$ is a *basis* of $\mathbb{R}^n$ when every vector of $\mathbb{R}^n$ can be written as a linear combination of the list in exactly one way. The simplest example is the list $e_1, \ldots, e_n$, where e_j has a 1 in position j and a 0 in every other position; in the plane, $e_1 = (1, 0)$ and $e_2 = (0, 1)$. The vector $x_j e_j$ has x_j in position j and zeros elsewhere, so adding these n vectors gives

$$x_1 e_1 + x_2 e_2 + \cdots + x_n e_n = (x_1, x_2, \ldots, x_n).$$

So every vector is a combination of $e_1, \ldots, e_n$. The coefficients are forced: if $(x_1, \ldots, x_n) = a_1 e_1 + \cdots + a_n e_n$, the same calculation shows that the right side is $(a_1, \ldots, a_n)$, and comparing coordinates gives $a_j = x_j$ for every j. Hence $e_1, \ldots, e_n$ is a basis, the *standard basis* of $\mathbb{R}^n$. In $\mathbb{R}^3$, for instance, $(4, 5, 0) = 4e_1 + 5e_2 + 0e_3$. (The number of coordinates of e_j depends on n: e_1 is $(1, 0)$ in the plane but $(1, 0, 0)$ in $\mathbb{R}^3$.)

Two requirements: reaching every vector, and reaching it only once

A basis meets two separate demands. Call a way of writing a vector as a linear combination of a given list a *representation* of the vector. First, every vector must have a representation, as $(3, -2) = 3e_1 - 2e_2$ does in the standard basis; a list with this property is said to *span* $\mathbb{R}^n$. Second, the representation must be *unique*.

Neither demand implies the other. The list consisting of the single vector $(1, 1)$ passes the uniqueness test, since $a(1, 1) = b(1, 1)$ means $(a, a) = (b, b)$, so $a = b$. But it does not span the plane: every multiple (a, a) has equal coordinates, so none equals $(1, 0)$. In the other direction, the list $(1, 0), (0, 1), (1, 1)$ spans the plane, since already $(x_1, x_2) = x_1(1, 0) + x_2(0, 1) + 0(1, 1)$. But it fails the uniqueness test: the vector $(1, 1)$ is both $0(1, 0) + 0(0, 1) + 1(1, 1)$ and $1(1, 0) + 1(0, 1) + 0(1, 1)$.

Uniqueness has a convenient test. A list $v_1, \ldots, v_k$ is called *linearly independent* if the only linear combination of it that equals the zero vector is the one whose coefficients are all zero:

$$a_1 v_1 + \cdots + a_k v_k = 0 \quad \text{implies} \quad a_1 = a_2 = \cdots = a_k = 0.$$

A list is linearly independent exactly when no vector has two different representations. Indeed, suppose first that the list is linearly independent and that some vector x has two representations,

$$x = a_1 v_1 + \cdots + a_k v_k \qquad \text{and} \qquad x = b_1 v_1 + \cdots + b_k v_k.$$

Subtract the second from the first and collect the terms that contain each v_j:

$$(a_1 - b_1)v_1 + \cdots + (a_k - b_k)v_k = x - x = 0.$$

By independence, every difference $a_j - b_j$ is zero, so the two representations were the same after all. Conversely, if the list is not linearly independent, then some combination with a nonzero coefficient equals 0, and so does the combination with all coefficients zero: the zero vector has two representations. For the list $(1, 0), (0, 1), (1, 1)$, subtracting the two representations of $(1, 1)$ above gives

$$(-1)(1, 0) + (-1)(0, 1) + 1(1, 1) = (0, 0),$$

so this list is not linearly independent. In short, a basis is a list that spans and is linearly independent.

Pause and decide. Is the list $(1, 2), (2, 4)$ a basis of $\mathbb{R}^2$?

Check. No, and it fails both requirements. It is not linearly independent, because $2(1, 2) + (-1)(2, 4) = (0, 0)$. It does not span the plane either: every combination $a(1, 2) + b(2, 4) = (a + 2b,\ 2a + 4b)$ has second coordinate equal to twice its first, so $(1, 0)$ is never reached.

A basis can be chosen to fit the problem

The standard basis is not the only basis of the plane. Take $w_1 = (1, 1)$ and $w_2 = (1, -1)$. Which coefficients a and b give $aw_1 + bw_2 = (x_1, x_2)$? Since $aw_1 + bw_2 = (a + b,\ a - b)$, we need $a + b = x_1$ and $a - b = x_2$. Adding and subtracting these equations gives $2a = x_1 + x_2$ and $2b = x_1 - x_2$, so the only possible coefficients are

$$a = \frac{x_1 + x_2}{2} \qquad \text{and} \qquad b = \frac{x_1 - x_2}{2},$$

and these do work, since with them $a + b = x_1$ and $a - b = x_2$. So every vector of the plane has exactly one representation, and w_1, w_2 is a basis. For instance, $(3, 1) = 2w_1 + 1w_2$. The vectors themselves have not changed; only the way we describe them has.

This basis suits questions about totals: the shop's stock $aw_1 + bw_2 = (a + b,\ a - b)$ has total $(a + b) + (a - b) = 2a$, which depends only on the first coefficient (for $(3, 5)$, $a = 4$ and the total is 8 kilograms), while changing b moves stock from one item to the other. Choosing a description that suits the question is Habit 1 (Section 1.6) once more, now applied to vectors instead of even numbers.

10.2 A linear map is determined by its building blocks

A map $T : \mathbb{R}^n \to \mathbb{R}^m$ is *linear* when it preserves both addition, in the sense of Section 6.4, and multiplication by scalars:

$$T(x + y) = T(x) + T(y) \qquad \text{and} \qquad T(cx) = cT(x)$$

for all vectors $x, y \in \mathbb{R}^n$ and all scalars c. The two conditions can be combined into one: T is linear exactly when

$$T(ax + by) = aT(x) + bT(y) \qquad \text{for all } x, y \in \mathbb{R}^n \text{ and all } a, b \in \mathbb{R}.$$

Indeed, the two conditions give $T(ax + by) = T(ax) + T(by) = aT(x) + bT(y)$. Conversely, the combined condition with $a = b = 1$ is preservation of addition, and with $b = 0$ it says $T(ax) = aT(x)$. Taking $c = 0$ in the second condition shows that a linear map sends the zero vector to the zero vector: $T(0) = T(0x) = 0\,T(x) = 0$ for any x.

A linear map sends every linear combination to the same combination of the outputs. For three vectors, split off the last term by preservation of addition, then use the combined condition on the first piece and scaling on the second:

$$T(a_1v_1 + a_2v_2 + a_3v_3) = T(a_1v_1 + a_2v_2) + T(a_3v_3) = a_1T(v_1) + a_2T(v_2) + a_3T(v_3).$$

Splitting off one term at a time works for any number k of terms (formally, by induction on k, as in Chapter 5):

$$T(a_1v_1 + \cdots + a_kv_k) = a_1T(v_1) + \cdots + a_kT(v_k).$$

In particular, writing $x = x_1e_1 + \cdots + x_ne_n$ shows that every linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ satisfies

$$T(x) = x_1T(e_1) + x_2T(e_2) + \cdots + x_nT(e_n).$$

So the n vectors $T(e_1), \ldots, T(e_n)$ determine the output $T(x)$ at every one of the infinitely many inputs x. This is why a linear map can be recorded in a finite table of numbers.

Suppose, for example, that $T : \mathbb{R}^2 \to \mathbb{R}^2$ is linear, with $T(e_1) = (1, 1)$ and $T(e_2) = (1, -1)$, the basis vectors w_1 and w_2 of Section 10.1. Then

$$T(x_1, x_2) = x_1(1, 1) + x_2(1, -1) = (x_1 + x_2,\ x_1 - x_2),$$

where $T(x_1, x_2)$ is short for $T\big((x_1, x_2)\big)$. A *matrix* is a rectangular table of numbers. We record T by the matrix whose first column is $T(e_1)$ and whose second column is $T(e_2)$:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad Ax = \begin{pmatrix} x_1 + x_2 \\ x_1 - x_2 \end{pmatrix}.$$

Here Ax stands for $T(x)$. A vector may be written horizontally, as (x_1, x_2), or as a column, as in Ax; both name the same ordered list.

An *m-by-n matrix* has m rows and n columns. We write A_{ij} for the entry in row i and column j; the first subscript always names the row. For the matrix A above, $A_{12} = 1$ and $A_{22} = -1$. An m-by-n matrix A acts on a vector $x \in \mathbb{R}^n$ and produces the vector $Ax \in \mathbb{R}^m$ whose ith coordinate is

$$(Ax)_i = \sum_{j=1}^{n} A_{ij}x_j = A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{in}x_n \qquad (1 \le i \le m).$$

In words: to find the ith coordinate of Ax, multiply the entries of row i by the matching coordinates of x, and add the products. We call this the *row rule*. For example, take

$$M = \begin{pmatrix} 2 & -1 & 3 \\ 1 & 0 & 1 \end{pmatrix} \qquad \text{and} \qquad x = (4, 5, 0).$$

The first row gives $2 \cdot 4 + (-1) \cdot 5 + 3 \cdot 0 = 3$, and the second gives $1 \cdot 4 + 0 \cdot 5 + 1 \cdot 0 = 4$, so $Mx = (3, 4)$. The input needs three coordinates, one for each entry of a row, and the output has one coordinate for each row. A matrix with as many rows as columns is called *square*.

The row rule agrees with the way A was built from its columns. For any m-by-n matrix A, the ith coordinate of the combination

$$x_1(\text{column 1 of } A) + x_2(\text{column 2 of } A) + \cdots + x_n(\text{column } n \text{ of } A)$$

is $x_1A_{i1} + x_2A_{i2} + \cdots + x_nA_{in}$, which is exactly $(Ax)_i$. So Ax is this combination of the columns, with the coordinates of x as coefficients; in the example, $Mx = 4(2, 1) + 5(-1, 0) + 0(3, 1) = (3, 4)$. Taking $x = e_j$ shows that Ae_j is column j of A. Now, for any linear map $T : \mathbb{R}^n \to \mathbb{R}^m$, build the

m-by-n matrix A whose column j is $T(e_j)$. Then $Ax = x_1T(e_1) + \cdots + x_nT(e_n) = T(x)$. So A records T exactly; it is called the *matrix of* T.

Conversely, every m-by-n matrix A gives a linear map $x \mapsto Ax$ from $\mathbb{R}^n$ to $\mathbb{R}^m$. Take $x, y \in \mathbb{R}^n$ and scalars a, b. The jth coordinate of $ax + by$ is $ax_j + by_j$, so by the row rule the ith coordinate of $A(ax + by)$ is

$$\sum_{j=1}^{n} A_{ij}(ax_j + by_j) = a\sum_{j=1}^{n} A_{ij}x_j + b\sum_{j=1}^{n} A_{ij}y_j,$$

which is the ith coordinate of $aAx + bAy$. This holds for every i, so $A(ax + by) = aAx + bAy$.

The two constructions undo each other. The matrix of the map $x \mapsto Ax$ is A itself, since its column j is Ae_j. And the matrix of T is the only matrix B with $Bx = T(x)$ for every x, since for such B column j is $Be_j = T(e_j)$. Thus linear maps from $\mathbb{R}^n$ to $\mathbb{R}^m$ and m-by-n matrices are two descriptions of the same objects, and we often say that a matrix maps $\mathbb{R}^n$ to $\mathbb{R}^m$.

Pause and decide. Is $F(x_1, x_2) = (x_1 + 1, x_2)$ linear?

Check. No. A linear map sends the zero vector to the zero vector, but $F(0, 0) = (1, 0)$.

10.3 Kernel: the information lost at zero

The *kernel* of a linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ is the set of inputs that T sends to the zero vector:

$$\ker T = \{x \in \mathbb{R}^n : T(x) = 0\}.$$

The kernel always contains 0, because $T(0) = 0$ (Section 10.2). For a matrix A, $\ker A$ means the kernel of the map $x \mapsto Ax$. As an example, let $P : \mathbb{R}^2 \to \mathbb{R}^2$ be the map $P(x_1, x_2) = (x_1, 0)$. It is linear, because

$$P(ax + by) = (ax_1 + by_1,\ 0) = a(x_1, 0) + b(y_1, 0) = aP(x) + bP(y).$$

Its output is the zero vector exactly when $x_1 = 0$, so $\ker P = \{(0, t) : t \in \mathbb{R}\}$, the vertical axis.

Why look at the kernel? If $T(x) = T(y)$, then linearity gives $T(x - y) = T(x) - T(y) = 0$: two inputs with the same output differ by a vector of the kernel. For P, the inputs $(2, 5)$ and $(2, 7)$ have the same output and differ by $(0, -2) \in \ker P$. So the kernel collects in one place all the information that T cannot see.

Theorem 10.1. *A linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ is injective if and only if its kernel consists only of the zero vector, that is,* $\ker T = \{0\}$.

Proof. Suppose first that T is injective, and let $x \in \ker T$. Then $T(x) = 0 = T(0)$, and injectivity gives $x = 0$. So the kernel contains no vector other than 0.

Conversely, suppose that $\ker T = \{0\}$, and let $x, y \in \mathbb{R}^n$ satisfy $T(x) = T(y)$. Linearity gives $T(x - y) = T(x) - T(y) = 0$, so $x - y$ lies in the kernel. Hence $x - y = 0$, that is, $x = y$. Therefore T is injective. □

Both directions used linearity: the first needed $T(0) = 0$, and the second turned $T(x) = T(y)$ into $T(x - y) = 0$. Without linearity the theorem fails: the map $F : \mathbb{R} \to \mathbb{R}$ with $F(x) = x^2$ sends only 0 to 0, yet $F(1) = F(-1)$.

Uniqueness is not existence

Fix a target $b \in \mathbb{R}^m$. If x_0 is one solution of $T(x) = b$, then the solutions are exactly the vectors $x_0 + k$ with $k \in \ker T$. Indeed, each such vector is a solution, because $T(x_0+k) = T(x_0)+T(k) = b+0 = b$. And for every solution x, the vector $k = x - x_0$ satisfies $T(k) = T(x) - T(x_0) = b - b = 0$, so $k \in \ker T$ and $x = x_0 + k$. In particular, when $\ker T = \{0\}$, each equation $T(x) = b$ has at most one solution.

But at most one is not exactly one: a kernel equal to $\{0\}$ does not promise that every target is reached. Regarding $\mathbb{R}$ as $\mathbb{R}^1$, the map $t \mapsto (t, 0)$ from $\mathbb{R}$ to $\mathbb{R}^2$ is linear, by the same check as for P, and its kernel is $\{0\}$, so it is injective. But no input reaches $(0, 1)$, so it is not surjective.

It is tempting to think that two equations in two unknowns always have exactly one solution. The map $T(x_1, x_2) = (x_1 + x_2,\ 2x_1 + 2x_2)$ is linear, being recorded by the matrix with rows $(1, 1)$ and $(2, 2)$. Solving $T(x_1, x_2) = (b_1, b_2)$ means solving $x_1 + x_2 = b_1$ and $2x_1 + 2x_2 = b_2$, where the second left side is always twice the first; so the target $(1, 0)$, which would need $0 = 2 \cdot 1$, is never reached. Both coordinates of $T(x_1, x_2)$ are zero exactly when $x_2 = -x_1$, so $\ker T = \{(t, -t) : t \in \mathbb{R}\}$. The target $(2, 4)$ has the solution $(2, 0)$, so its solutions are exactly the vectors $(2, 0) + (t, -t) = (2 + t,\ -t)$ with $t \in \mathbb{R}$. Counting equations is not enough: what matters is that the map loses information, sending a whole line of vectors $(t, -t)$ to zero.

Inverting the matrix of sums and differences

For the matrix A of Section 10.2, apply its sum-and-difference rule twice:

$$\begin{aligned} A(Ax) &= A(x_1 + x_2,\ x_1 - x_2) \\ &= \big((x_1 + x_2) + (x_1 - x_2),\ (x_1 + x_2) - (x_1 - x_2)\big) \\ &= (2x_1,\ 2x_2) = 2x. \end{aligned}$$

This identity shows that A is a bijection and gives its inverse. If $Ax = 0$, then $2x = A(Ax) = A0 = 0$, so $x = 0$. Thus $\ker A = \{0\}$, and A is injective by Theorem 10.1. For surjectivity, take any target $b \in \mathbb{R}^2$ and try $x = \frac{1}{2}Ab$. By linearity,

$$Ax = A\big(\tfrac{1}{2}Ab\big) = \tfrac{1}{2}A(Ab) = \tfrac{1}{2} \cdot 2b = b.$$

So A is a bijection, and $b \mapsto \frac{1}{2}Ab$ is its inverse: we have just checked that $A\big(\frac{1}{2}Ab\big) = b$, and in the other order $\frac{1}{2}A(Ax) = \frac{1}{2} \cdot 2x = x$.

In coordinates, the inverse sends $b = (b_1, b_2)$ to $\big((b_1 + b_2)/2,\ (b_1 - b_2)/2\big)$, and these are exactly the coefficients of b in the basis w_1, w_2 of Section 10.1. That is no coincidence: the columns of A are w_1 and w_2, so $Ax = x_1w_1 + x_2w_2$, and solving $Ax = b$ means finding the coefficients of b in this basis. Inverting A and changing to the basis of its columns are the same problem, stated in two languages.

10.4 The geometry of an inner product

Adding and scaling vectors tells us nothing about lengths or right angles; for those we need a way to multiply two vectors. For vectors $u, v \in \mathbb{R}^n$, define their *dot product* by

$$u \cdot v = \sum_{j=1}^{n} u_j v_j = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

The result is a number, not a vector. For example, $(1, 2, 3) \cdot (4, -5, 6) = 4 - 10 + 18 = 12$. The dot product is also called the standard *inner product* on $\mathbb{R}^n$. In this language, the row rule of Section 10.2 says that $(Ax)_i$ is the dot product of row i of A with x.

The proofs in the next section use only three properties of the dot product.

- *Symmetry:* $u \cdot v = v \cdot u$, because each product $u_j v_j$ equals $v_j u_j$.
- *Linearity in each argument:* we call u the *first argument* of $u \cdot v$ and v the *second argument* (an argument here means an input, not a piece of reasoning). If one argument is held fixed, the dot product is linear in the other. For the first argument, the definition gives

$$\begin{aligned}(au + bw) \cdot v &= \sum_{j=1}^{n} (au_j + bw_j) v_j \\ &= a \sum_{j=1}^{n} u_j v_j + b \sum_{j=1}^{n} w_j v_j = a(u \cdot v) + b(w \cdot v).\end{aligned}$$

  The same expansion gives $u \cdot (av + bw) = a(u \cdot v) + b(u \cdot w)$. Also $0 \cdot v = 0$ for every v, where the 0 on the left is the zero vector.
- *Positivity:* $u \cdot u = u_1^2 + \cdots + u_n^2 \geq 0$, and $u \cdot u = 0$ only when $u = 0$, because a sum of squares is zero only if every square is zero, and $u_j^2 = 0$ only when $u_j = 0$.

The *Euclidean length*, or *norm*, of a vector u is

$$\|u\|_2 = \sqrt{u \cdot u} = \sqrt{u_1^2 + u_2^2 + \cdots + u_n^2}.$$

The subscript 2 labels this way of measuring size, built from squares (Chapter 11 meets others); it is not a coordinate of u. By positivity, the number under the root is nonnegative, and $\|u\|_2 = 0$ exactly when $u = 0$. In the plane, $\|(3, 4)\|_2 = \sqrt{9 + 16} = 5$, which by Pythagoras' theorem is the distance from the origin to the point $(3, 4)$.

Multiplying a vector by a scalar c multiplies its length by $|c|$: $\|cu\|_2 = |c| \, \|u\|_2$. Indeed, linearity in each argument gives $\|cu\|_2^2 = (cu) \cdot (cu) = c^2 (u \cdot u)$. So $\|cu\|_2$ and $|c| \, \|u\|_2$ are nonnegative numbers with the same square $c^2 \, \|u\|_2^2$, and a nonnegative number is determined by its square (Section 1.8).

Two vectors u and v are *orthogonal* when $u \cdot v = 0$. In the plane, two nonzero vectors are orthogonal exactly when they are perpendicular. To see why, look at the triangle with corners at the origin, u, and v. By Pythagoras' theorem, its sides from the origin have squared lengths $u_1^2 + u_2^2$

and $v_1^2 + v_2^2$, and the side from u to v has squared length

$$(u_1 - v_1)^2 + (u_2 - v_2)^2 = (u_1^2 + u_2^2) + (v_1^2 + v_2^2) - 2(u_1 v_1 + u_2 v_2),$$

by expanding each square. By Pythagoras' theorem and its converse, the angle at the origin is a right angle exactly when the squared length of the opposite side is the sum of the other two, that is, when $2(u_1 v_1 + u_2 v_2) = 0$, or $u \cdot v = 0$. This needs a genuine triangle, so one case remains: if u and v lie on one line through the origin, then $v = cu$ for some $c \neq 0$, and $u \cdot v = c(u \cdot u) \neq 0$ by positivity; such arrows are not perpendicular either.

For example, a quarter turn counterclockwise takes (a, b) to $(-b, a)$ (so $(1, 0)$ to $(0, 1)$), so a nonzero (a, b) is perpendicular to $(-b, a)$; and indeed $(a, b) \cdot (-b, a) = -ab + ba = 0$. The algebraic definition makes sense in $\mathbb{R}^n$ for every n, even when we can no longer draw a picture. The zero vector is orthogonal to every vector, since $0 \cdot v = 0$.

10.5 Deriving a bound by removing a component

How large can a dot product be, compared with the lengths of the two vectors? When $v = u$, the dot product $u \cdot u = \|u\|_2^2$ is exactly the product $\|u\|_2 \, \|u\|_2$ of the two lengths. A natural guess is that it can never be larger in size:

$$|u \cdot v| \leq \|u\|_2 \, \|v\|_2 \, .$$

Test it on $u = (3, 1)$ and $v = (1, 1)$: here $u \cdot v = 4$, $\|u\|_2 = \sqrt{10}$ and $\|v\|_2 = \sqrt{2}$. To compare 4 with $\sqrt{10}\sqrt{2}$ without decimals, compare squares: $4^2 = 16 \leq 20 = (\sqrt{10}\sqrt{2})^2$, so the guess holds here. This is legitimate because for nonnegative p and q, $p \leq q$ exactly when $p^2 \leq q^2$: by the squaring rules of Section 2.6, $0 \leq p \leq q$ gives $p^2 \leq q^2$, while $p > q \geq 0$ gives $p^2 > q^2$, so $p^2 \leq q^2$ rules out $p > q$.

How could we find a reason that works for all vectors? Start from the fact that $u - tv$ has a nonnegative squared length for every real number t:

$$(u - tv) \cdot (u - tv) \geq 0.$$

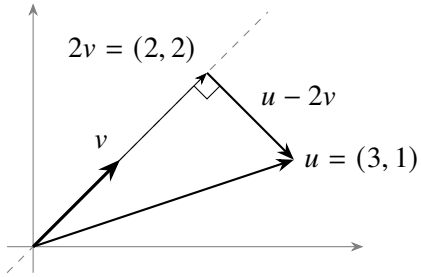
Each t gives an inequality, and it says the most when the squared length is as small as possible, so we choose t to make $u - tv$ as short as possible. Geometrically, this happens when the leftover piece $u - tv$ meets the line through v at a right angle. So we look for the t that makes $u - tv$ orthogonal to v:

$$(u - tv) \cdot v = 0.$$

By linearity in the first argument, this says $u \cdot v - t(v \cdot v) = 0$. When $v \neq 0$, positivity gives $v \cdot v > 0$, so we may divide and obtain

$$t = \frac{u \cdot v}{v \cdot v}.$$

With this t, the vector tv is called the *component* of u along v, and $u - tv$ is the *residual*: what remains of u after its component along v is removed. In our example $t = 4/2 = 2$, so the component is $2v = (2, 2)$ and the residual is $(1, -1)$, which has dot product $1 - 1 = 0$ with v, as planned; in the picture it meets the dashed line through v at a right angle.



Since a dot product can be negative, the theorem bounds $|u \cdot v|$; by Section 2.6, this says that $u \cdot v$ lies between $-\|u\|_2 \|v\|_2$ and $\|u\|_2 \|v\|_2$.

Theorem 10.2 (Cauchy–Schwarz inequality)**.** *For all $u, v \in \mathbb{R}^n$, $|u \cdot v| \le \|u\|_2 \|v\|_2$.*

Plan. The squared length of $u - tv$ is nonnegative for every real t. Expand it, and then substitute the value of t found above.

Proof. If $v = 0$, both sides of the inequality are zero. Assume from now on that $v \neq 0$, so that $v \cdot v > 0$. For any real t, expand the squared length of $u - tv$, using linearity first in the first argument and then in the second:

$$\begin{aligned}(u - tv) \cdot (u - tv) &= u \cdot (u - tv) - t\big(v \cdot (u - tv)\big) \\ &= u \cdot u - t(u \cdot v) - t(v \cdot u) + t^2(v \cdot v) \\ &= u \cdot u - 2t(u \cdot v) + t^2(v \cdot v).\end{aligned}$$

The last step uses symmetry, $v \cdot u = u \cdot v$. The left side is a squared length, so by positivity the right side is nonnegative for every real t.

Now choose $t = (u \cdot v)/(v \cdot v)$, which is defined because $v \cdot v > 0$. Then $t(u \cdot v) = (u \cdot v)^2/(v \cdot v)$, and

$$t^2(v \cdot v) = \frac{(u \cdot v)^2}{(v \cdot v)^2}\,(v \cdot v) = \frac{(u \cdot v)^2}{v \cdot v}.$$

Substituting these two expressions gives

$$\begin{aligned}0 &\le u \cdot u - 2\,\frac{(u \cdot v)^2}{v \cdot v} + \frac{(u \cdot v)^2}{v \cdot v} \\ &= u \cdot u - \frac{(u \cdot v)^2}{v \cdot v}.\end{aligned}$$

Add the fraction to both sides and multiply by the positive number $v \cdot v$:

$$(u \cdot v)^2 \le (u \cdot u)(v \cdot v) = \|u\|_2^2 \, \|v\|_2^2 \,.$$

Finally, $|u \cdot v|$ and $\|u\|_2 \|v\|_2$ are nonnegative numbers whose squares are $(u \cdot v)^2$ (because $|a|^2 = a^2$ for every real a) and $\|u\|_2^2 \|v\|_2^2$. Nonnegative numbers compare in the same way as their squares, so $|u \cdot v| \le \|u\|_2 \|v\|_2$. □

When does equality hold?

When is $|u \cdot v|$ exactly equal to $\|u\|_2 \, \|v\|_2$? For $v \neq 0$, the proof contains the answer. With the chosen t, it showed that $\|u - tv\|_2^2 = u \cdot u - (u \cdot v)^2/(v \cdot v)$. Multiplying by $v \cdot v$ gives

$$(u \cdot u)(v \cdot v) - (u \cdot v)^2 = (v \cdot v) \, \|u - tv\|_2^2 \, .$$

The left side is the gap between the two sides of the squared inequality, and since nonnegative numbers are equal exactly when their squares are, equality holds in the theorem exactly when this gap is zero. As $v \cdot v > 0$, that happens exactly when $\|u - tv\|_2 = 0$, that is, when the residual $u - tv$ is the zero vector, or $u = tv$. So if equality holds, u is a multiple of v. Conversely, if $u = sv$ for some number s, then $t = \big((sv) \cdot v\big)/(v \cdot v) = s$, so the residual $u - sv$ is zero, the gap is zero, and equality holds. For example, $u = (2, 2) = 2v$ with $v = (1, 1)$ gives $u \cdot v = 4$ and $\|u\|_2 \, \|v\|_2 = \sqrt{8}\sqrt{2} = 4$. When $v = 0$, both sides are zero for every u. Altogether, equality holds exactly when $v = 0$ or u is a scalar multiple of v.

Section 9.6 recommended asking what happens when an inequality holds with equality. Here the answer also shows that the bound cannot be improved: when $u = v \neq 0$, both sides equal $\|v\|_2^2$, so no constant factor smaller than 1, such as 0.9, can be placed in front of the right-hand side.

From the dot-product bound to a distance bound

In the plane, walking from one point to another by way of a third is never shorter than walking directly. Chapter 11 needs this fact in $\mathbb{R}^n$, and Cauchy–Schwarz gives it quickly. For any $u, v \in \mathbb{R}^n$, expand using linearity in each argument and symmetry:

$$\begin{aligned} \|u + v\|_2^2 &= (u + v) \cdot (u + v) \\ &= u \cdot u + u \cdot v + v \cdot u + v \cdot v \\ &= \|u\|_2^2 + 2(u \cdot v) + \|v\|_2^2 \\ &\leq \|u\|_2^2 + 2|u \cdot v| + \|v\|_2^2 \\ &\leq \|u\|_2^2 + 2 \, \|u\|_2 \, \|v\|_2 + \|v\|_2^2 \\ &= (\|u\|_2 + \|v\|_2)^2 . \end{aligned}$$

The first inequality holds because every real number a satisfies $a \leq |a|$ (if $a \geq 0$ they are equal, and if $a < 0$ then $a < 0 \leq |a|$), and the second is Cauchy–Schwarz. Both $\|u + v\|_2$ and $\|u\|_2 + \|v\|_2$ are nonnegative, and nonnegative numbers compare in the same way as their squares. We obtain the *triangle inequality*

$$\|u + v\|_2 \leq \|u\|_2 + \|v\|_2 \, .$$

To see where the name comes from, take three points $x, y, z \in \mathbb{R}^n$ and put $u = x - y$ and $v = y - z$. Then $u + v = x - z$, and the inequality becomes

$$\|x - z\|_2 \leq \|x - y\|_2 + \|y - z\|_2 \, .$$

In the plane, $\|x - z\|_2$ is the ordinary distance between x and z, so this says that in the triangle with corners x, y, z, the side from x to z is at most as long as the other two sides together. Exercise 10.5 asks when equality holds.

10.6 Matrix multiplication records composition

Section 10.3 applied the matrix A twice and found $A(Ax) = 2x$. When we apply one linear map after another, we want a single matrix for the combined map. So let A now be any m-by-n matrix and B any k-by-m matrix. Then A maps $\mathbb{R}^n$ to $\mathbb{R}^m$ and B maps $\mathbb{R}^m$ to $\mathbb{R}^k$, so the composition "B after A," which sends x to $B(Ax)$, maps $\mathbb{R}^n$ to $\mathbb{R}^k$. It is linear, because linearity of A and then of B gives

$$B\big(A(ax + by)\big) = B\big(aAx + bAy\big) = a\,B(Ax) + b\,B(Ay).$$

By Section 10.2, the composition is therefore recorded by a k-by-n matrix, called the *product* BA, so that $(BA)x = B(Ax)$ for every $x \in \mathbb{R}^n$. As with $g \circ f$ in Section 3.2, the factor written on the right acts first.

To find the entries of BA, recall that column j of any matrix M is Me_j. So column j of BA is

$$(BA)e_j = B(Ae_j) = B\,(\text{column } j \text{ of } A).$$

Column j of A is the vector $(A_{1j}, A_{2j}, \ldots, A_{mj})$, and $(BA)_{ij}$ is the ith coordinate of B applied to it, so the row rule for B gives

$$(BA)_{ij} = \sum_{r=1}^{m} B_{ir}A_{rj} = B_{i1}A_{1j} + B_{i2}A_{2j} + \cdots + B_{im}A_{mj}$$

for $1 \le i \le k$ and $1 \le j \le n$. In words, the entry of BA in row i and column j is the dot product of row i of B with column j of A. This makes sense only when B has as many columns as A has rows. The row-by-column rule may look strange at first, but it is exactly what composing the two maps requires.

For example, multiply the matrix A of Section 10.2 by itself:

$$\begin{aligned} AA &= \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \begin{pmatrix} 1\cdot 1 + 1\cdot 1 & 1\cdot 1 + 1\cdot(-1) \\ 1\cdot 1 + (-1)\cdot 1 & 1\cdot 1 + (-1)\cdot(-1) \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}. \end{aligned}$$

This matrix sends (x_1, x_2) to $(2x_1, 2x_2)$, in agreement with $A(Ax) = 2x$.

The order of the factors matters, just as $g \circ f$ and $f \circ g$ were different functions in Section 3.2. For

$$C = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \qquad \text{and} \qquad D = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix},$$

the row-by-column rule gives

$$CD = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \qquad \text{but} \qquad DC = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

The maps explain the difference: C sends (x_1, x_2) to $(x_1 + x_2,\ x_2)$, while D sends it to $(x_1,\ x_1 + x_2)$. Starting from $(1, 0)$, the product CD applies D first, reaching $(1, 1)$, and then C gives $(2, 1)$. The product DC applies C first, which leaves $(1, 0)$ unchanged, and then D gives $(1, 1)$. These are the first columns of CD and DC.

The transpose and orthogonal rows

The *transpose* A^{T} of an m-by-n matrix A is the n-by-m matrix obtained by exchanging rows and columns: $(A^{\mathsf{T}})_{ij} = A_{ji}$. So column j of A^{T} is row j of A. For example,

$$\begin{pmatrix} 2 & -1 & 3 \\ 1 & 0 & 1 \end{pmatrix}^{\mathsf{T}} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 3 & 1 \end{pmatrix}.$$

By the row-by-column rule, $(AA^{\mathsf{T}})_{ij}$ is the dot product of row i of A with column j of A^{T}, which is row j of A:

$$(AA^{\mathsf{T}})_{ij} = \sum_{r=1}^{n} A_{ir}A_{jr} = (\text{row } i \text{ of } A) \cdot (\text{row } j \text{ of } A).$$

The n-by-n *identity matrix* I has ones on its *diagonal*, that is, in the positions (i, i), and zeros everywhere else. By the row rule, $(Ix)_i = x_i$, because the only nonzero entry in row i is the 1 in column i. So $Ix = x$ for every $x \in \mathbb{R}^n$, and I is the matrix of the identity map. Multiplying a matrix by a number multiplies every entry, so nI has n in each diagonal position and zeros elsewhere. For an n-by-n matrix A, the equation $AA^{\mathsf{T}} = nI$ therefore says two things at once: the diagonal entries say that every row of A has squared length n, and the entries off the diagonal say that any two different rows of A are orthogonal.

The matrix A of Section 10.2 is an example with $n = 2$. Exchanging its rows and columns does not change it, so $AA^{\mathsf{T}} = AA = 2I$: its rows $(1, 1)$ and $(1, -1)$ have squared length 2 and are orthogonal. Square matrices with entries ± 1 that satisfy $AA^{\mathsf{T}} = nI$, where n is the number of rows, are called *Hadamard matrices*. They are central to Chapter 14, which follows a question about them all the way to current research.

Exercises

Exercises 10.1–10.4 are core exercises, and Exercises 10.5 and 10.6 are transfer exercises. Complete solutions are in Appendix A, Section A.10.

Exercise 10.1. For $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $x = (1, -2)$, compute Ax. Then explain the answer using the columns of A.

Exercise 10.2. Find the kernel of the linear map $T : \mathbb{R}^2 \to \mathbb{R}$ given by $T(x, y) = x + y$. Then describe all solutions of $T(x, y) = 5$.

Exercise 10.3. Section 10.3 found the inverse of the map $T : \mathbb{R}^2 \to \mathbb{R}^2$, $T(x, y) = (x + y, x - y)$, from the equation $A(Ax) = 2x$ for its matrix A. Check this answer directly: show that $(s, t) \mapsto \big((s + t)/2, (s - t)/2\big)$ is the inverse of T by computing both compositions.

Exercise 10.4. Compute the dot product and the lengths of $u = (1, 2)$ and $v = (2, -1)$. Check the Cauchy–Schwarz inequality for this pair.

Exercise 10.5. The proof of the triangle inequality $\|u + v\|_2 \leq \|u\|_2 + \|v\|_2$ in Section 10.5 passes through two inequalities. By asking when each of them is an equality, prove that for vectors $u, v \in \mathbb{R}^n$ the equation $\|u + v\|_2 = \|u\|_2 + \|v\|_2$ holds exactly when $v = 0$ or $u = sv$ for some number $s \geq 0$. Then give nonzero vectors u and v, with u a multiple of v, for which the inequality is strict.

Exercise 10.6. Let $u_1, \ldots, u_k \in \mathbb{R}^n$ be nonzero vectors that are pairwise orthogonal, which means that $u_i \cdot u_j = 0$ whenever $i \neq j$. Show that $a_1 u_1 + \cdots + a_k u_k = 0$ forces every coefficient a_j to be zero; in other words, the list $u_1, \ldots, u_k$ is linearly independent. Then give an example showing that the word "nonzero" cannot be left out.

CHAPTER 11

Distance, error, and the meaning of a limit

Overview

Describe distance by three simple rules, so that one argument works for numbers, vectors, and other objects. Replace the impression that a sequence "gets close" to a point by a precise promise covering every positive tolerance. See why steps that keep getting smaller need not make a sequence settle down, while steps that shrink by a fixed factor do. Recognize completeness as the property that guarantees a limit in the space for every sequence whose terms eventually all lie as close together as we like. Describe the edge of a set of real numbers with upper and lower bounds, suprema and infima, and prove that every nonempty set that is bounded above has a least upper bound.

11.1 Distance begins with absolute value

Recall from Section 2.6 that the absolute value $|x|$ of a real number x is x when $x \geq 0$ and $-x$ when $x < 0$. The *distance* between real numbers x and y is $|x - y|$, the length of the stretch of the number line between them; for $x = -2$ and $y = 3$ it is $|-5| = 5$. The order does not matter: $y - x = -(x - y)$, and changing the sign of a number does not change its absolute value, so $|y - x| = |x - y|$.

This chapter asks which properties of $|x - y|$ our arguments actually use. An argument that relies only on them then works for any objects with a distance that has the same properties, such as the vectors of Chapter 10.

The most important of these properties is the *triangle inequality*: for all real numbers x and y,

$$|x + y| \leq |x| + |y|.$$

For example, $x = 5$ and $y = -2$ give $|x + y| = 3 < 7 = |x| + |y|$. When x and y are both nonnegative or both nonpositive, the two sides are equal: $x = -5$ and $y = -2$ give $|x + y| = 7 = |x| + |y|$.

To prove it, recall from Section 2.6 that, for $b \geq 0$, the statement $|x| \leq b$ holds exactly when $-b \leq x \leq b$. With $b = |x|$, the true statement $|x| \leq |x|$ becomes $-|x| \leq x \leq |x|$. In the same way, $-|y| \leq y \leq |y|$. Adding the two left-hand inequalities, and then the two right-hand ones, gives

$$-(|x| + |y|) \leq x + y \leq |x| + |y|.$$

The same rule, now used in the other direction with $x + y$ in place of x and the nonnegative number $b = |x| + |y|$, turns this into $|x + y| \leq |x| + |y|$.

The triangle inequality is most often used in a second form. Given real numbers a, b and c, take $x = a - b$ and $y = b - c$. Then $x + y = a - c$, and the triangle inequality becomes

$$|a - c| \le |a - b| + |b - c|.$$

In words: going from a to c by way of a stopping point b is never shorter than going directly. For $a = 0$, $b = 5$ and $c = 2$, the direct distance is 2, while the route through b has length $5 + 3 = 8$; when b lies between a and c, the two sides are equal. This form appears in almost every error estimate in this chapter: to show that a and c are close, find a third quantity b close to each of them and add the two small distances.

Here is a first error estimate. Suppose x approximates a and y approximates b, with errors $|x - a| \le \varepsilon$ and $|y - b| \le \delta$, where δ (the Greek letter delta) is a second tolerance. Regrouping $(x + y) - (a + b) = (x - a) + (y - b)$ and applying the triangle inequality gives

$$|(x + y) - (a + b)| \le |x - a| + |y - b| \le \varepsilon + \delta.$$

So when approximations are added, their error bounds add. With $a = 2$, $b = 5$, $\varepsilon = 0.01$ and $\delta = 0.02$, the bound 0.03 is reached by $x = 2.01$ and $y = 5.02$, whose errors point the same way, while errors in opposite directions partly cancel ($x = 2.01$ and $y = 4.98$ give 6.99). Since we know only the sizes of the errors, not their directions, $\varepsilon + \delta$ is the best guarantee available.

Pause and decide. Suppose $|x-a| \le \varepsilon$ and $|y-b| \le \delta$. Does it follow that $|(x-y)-(a-b)| \le \varepsilon-\delta$? If not, what bound is correct?

Check. No. Take $a = b = 0$, $x = \varepsilon$ and $y = -\delta$. Both hypotheses hold, and $(x-y)-(a-b) = \varepsilon+\delta$, which is larger than $\varepsilon - \delta$ because $\delta > 0$. The correct bound is $\varepsilon + \delta$: write $(x - y) - (a - b) = (x - a) + \big(-(y - b)\big)$ and apply the triangle inequality, using $|-(y - b)| = |y - b|$. Subtracting two approximations does not subtract their errors; in the worst case the errors still add.

11.2 A metric keeps only the properties we use

Let X be a set, whose members we call *points*, even when they are numbers or vectors. A distance on X should be a function d from $X \times X$ to $\mathbb{R}$ (Sections 3.1 and 3.2).

Definition 11.1 (Metric)**.** A *metric* on a set X assigns a real number $d(x, y)$ to every pair of points $x, y \in X$ and satisfies these requirements for all $x, y, z \in X$:

1. $d(x, y) \ge 0$, and $d(x, y) = 0$ holds exactly when $x = y$.
2. $d(x, y) = d(y, x)$.
3. $d(x, z) \le d(x, y) + d(y, z)$.

A set together with a chosen metric is called a *metric space* and is written (X, d). The number $d(x, y)$ is called the *distance* between x and y.

The first requirement says, in particular, that two different points are a positive distance apart. The second is called *symmetry*, and the third is the triangle inequality of Section 11.1, now demanded of d. To show that a new formula is a metric, we must *recognize* it as one in the sense of Section 1.4, by checking all three requirements; calling a formula a distance does not make it one.

Pause and decide. On the real line, let $d(x, y) = (x - y)^2$. Is d a metric?

Check. No. The first two requirements hold: a square is never negative, $(x - y)^2 = 0$ exactly when $x = y$, and $(y - x)^2 = (x - y)^2$. The triangle inequality fails. Take $x = 0$, $y = 1$ and $z = 2$. Then $d(x, z) = 4$, while $d(x, y) + d(y, z) = 1 + 1 = 2$, so the route through y comes out *shorter* than the direct route. One counterexample is enough.

Examples

The real line with $d(x, y) = |x - y|$ is a metric space. The absolute value is never negative, and $|x - y| = 0$ exactly when $x = y$; symmetry and the triangle inequality were proved in Section 11.1. When we treat the real line as a metric space without naming a metric, this is the metric we mean.

The set $\mathbb{R}^n$ of vectors $x = (x_1, \ldots, x_n)$ from Section 10.1 carries several useful metrics. The first is the *Euclidean distance* $d_2(x, y) = \|x - y\|_2$, the length of the difference vector; in the plane it is the straight-line distance, so from $(0, 0)$ to $(3, 4)$ it is $\sqrt{3^2 + 4^2} = 5$. It is a metric: a length is never negative and is zero only for the zero vector (Section 10.4), which $x - y$ is exactly when $x = y$; changing the sign of every coordinate of $x - y$ does not change the squares under the root, so $d_2(y, x) = d_2(x, y)$; and the triangle inequality $\|x - z\|_2 \le \|x - y\|_2 + \|y - z\|_2$ is proved at the end of Section 10.5.

Another useful choice is

$$d_\infty(x, y) = \max_{1 \le j \le n} |x_j - y_j|,$$

the largest of the n numbers $|x_1 - y_1|, \ldots, |x_n - y_n|$; the subscript ∞ is just part of the traditional name. This metric records the worst error in any single coordinate. For $x = (1, 5)$ and $y = (4, 3)$, the coordinate differences have sizes 3 and 2, so $d_\infty(x, y) = 3$, while $d_2(x, y) = \sqrt{13} \approx 3.61$.

To prove the triangle inequality for d_∞, let $x, y, z \in \mathbb{R}^n$ and fix a coordinate j. The number $|x_j - y_j|$ is one of those whose largest is $d_\infty(x, y)$, so $|x_j - y_j| \le d_\infty(x, y)$; likewise $|y_j - z_j| \le d_\infty(y, z)$. With the triangle inequality for real numbers,

$$|x_j - z_j| \le |x_j - y_j| + |y_j - z_j| \le d_\infty(x, y) + d_\infty(y, z).$$

The right side does not depend on j, so it is at least each of the numbers $|x_j - z_j|$, and hence at least their largest: $d_\infty(x, z) \le d_\infty(x, y) + d_\infty(y, z)$. Exercise 11.3 checks the other two requirements.

Which metric to use depends on the question. If an instrument must get *every* coordinate right to within 0.01, the condition is $d_\infty(x, y) \le 0.01$; a bound on d_2 instead controls all the coordinate errors together, through the sum of their squares. The reading $x = (0.01, 0.01)$ of the true point $y = (0, 0)$ meets the first condition, since $d_\infty(x, y) = 0.01$, but fails $d_2(x, y) \le 0.01$, since $d_2(x, y) \approx 0.0141$.

Norms, subsets, and spaces without addition

Both metrics on $\mathbb{R}^n$ measure the size of a difference vector: $d_2(x, y) = \|x - y\|_2$, and $d_\infty(x, y) = \|x - y\|_\infty$ with $\|x\|_\infty = \max_j |x_j|$. Such measures of size are called norms. Precisely, a *norm* on

$\mathbb{R}^n$ assigns to each vector x a real number $\|x\|$ such that, for all vectors x, y and every real number c,

1. $\|x\| \geq 0$, with equality exactly when x is the zero vector;
2. $\|cx\| = |c|\ \|x\|$;
3. $\|x + y\| \leq \|x\| + \|y\|$.

The Euclidean length is a norm, by Sections 10.4 and 10.5. So is $\|x\|_\infty$: it is never negative and is 0 exactly when every coordinate is 0, multiplying every coordinate by c multiplies the largest coordinate size by $|c|$, and the third property is proved one coordinate at a time, like the triangle inequality for d_∞. Conversely, every norm gives a metric $d(x, y) = \|x - y\|$: the first requirement comes from the first property of the norm, since $x - y$ is the zero vector exactly when $x = y$; symmetry from the second with $c = -1$, since $y - x = (-1)(x - y)$; and the triangle inequality from the third, since $x - z = (x - y) + (y - z)$.

A metric, however, needs only one number $d(x, y)$ for each pair of points, not a way to add points. So the rest of the chapter works in a general metric space, and its arguments apply also to sets in which addition makes no sense, or can lead outside the set. The most common such set is part of a larger metric space. If (X, d) is a metric space and $C \subseteq X$, then d, used only for pairs of points of C (the restriction of d to $C \times C$, in the language of Section 3.3), is a metric on C: the three requirements hold for all points of X, so in particular for all points of C. For example, the interval $[0, 1]$ with distance $|x - y|$ is a metric space, even though a sum of its points can leave it, as $0.7 + 0.6 = 1.3$ does.

11.3 Convergence promises eventual control

A *sequence* in a metric space (X, d) is an endless list $x_0, x_1, x_2, \ldots$ of points of X, one for each natural number, as in Section 5.3 for $X = \mathbb{R}$. Here the subscript counts terms: x_n is a whole point of X, not a coordinate of a vector. The terms $x_N, x_{N+1}, x_{N+2}, \ldots$ from some index N onward form a *tail* of the sequence.

Definition 11.2 (Convergence)**.** A sequence $x_0, x_1, x_2, \ldots$ in a metric space (X, d) *converges* to a point $x_* \in X$ if, for every tolerance $\varepsilon > 0$, there is a threshold N such that every term from x_N onward lies at distance less than ε from x_*:

$$\forall \varepsilon > 0\ \exists N \in \mathbb{N}\ \forall n \geq N : \quad d(x_n, x_*) < \varepsilon.$$

The point x_* (the star is part of the name) is then called a *limit* of the sequence, and we write $x_n \to x_*$, read "x_n tends to x_*." A sequence that converges to some point of X is called *convergent*.

Read the quantifiers as the skeptic–prover game of Section 2.3, with x_* fixed in advance. The skeptic names a tolerance ε; the prover, knowing ε, names a threshold N; the skeptic then picks *any* index $n \geq N$, and the prover wins if $d(x_n, x_*) < \varepsilon$. The sequence converges to x_* when the prover can always win. A smaller tolerance may force a larger threshold, which is allowed because N is chosen after ε; but a single N must handle every $n \geq N$ at once.

Two sequences that converge

The statement $x_n \to x_*$ does not say that the sequence ever reaches its limit. Take $x_n = 1/(n+1)$ on the real line, so that the sequence is $1, \frac{1}{2}, \frac{1}{3}, \ldots$; its terms are all positive, yet they tend to 0. Let $\varepsilon > 0$ be given. By the Archimedean property (Section 2.3) we may choose an integer $N > 1/\varepsilon$; it is positive because $1/\varepsilon > 0$. Now let $n \geq N$. Then $n + 1 > N > 0$, and taking reciprocals reverses the order, so $1/(n+1) < 1/N$. Taking reciprocals in $N > 1/\varepsilon$ gives $1/N < \varepsilon$. Together,

$$d(x_n, 0) = \left| \frac{1}{n+1} - 0 \right| = \frac{1}{n+1} < \frac{1}{N} < \varepsilon.$$

This proves $x_n \to 0$. For $\varepsilon = 0.01$ the recipe asks for an integer $N > 100$, such as $N = 101$; the threshold $N = 100$ would also work, since $x_{100} = 1/101 < 0.01$. The definition asks only for *some* threshold that works, not the smallest one.

Next, the sequence $x_n = 1 - 2^{-n}$ of Section 5.3, described there as coming as close to 1 as we like, does converge to 1. The distance from x_n to 1 is 2^{-n}. To compare it with $1/(n+1)$, we show by induction that $2^n \geq n + 1$ for every $n \in \mathbb{N}$. At $n = 0$ both sides equal 1. If $2^n \geq n + 1$ for some n, then

$$2^{n+1} = 2 \cdot 2^n \geq 2(n+1) = 2n + 2 \geq n + 2,$$

which is the claim for $n + 1$. Taking reciprocals gives $2^{-n} \leq 1/(n+1)$. So if $\varepsilon > 0$ and N is an integer with $N > 1/\varepsilon$, every $n \geq N$ satisfies $|x_n - 1| \leq 1/(n+1) < \varepsilon$. This proves $x_n \to 1$.

Because the definition speaks only about tails, changing finitely many terms of a sequence affects neither whether it converges nor its limit. If the first thousand terms of $1/(n+1)$ were replaced by arbitrary numbers, a threshold that is at least 1000 and larger than $1/\varepsilon$ would still work.

What failure to converge looks like

Negating the definition with the rules of Section 2.4 shows what it means for a sequence *not* to converge to x_*:

$$\exists \varepsilon > 0 \; \forall N \in \mathbb{N} \; \exists n \geq N : \quad d(x_n, x_*) \geq \varepsilon.$$

There is one fixed tolerance such that, however late we start, some later term is at least that far from x_*.

Consider the sequence $0, 1, 0, 1, \ldots$, with $x_n = 0$ for even n and $x_n = 1$ for odd n. It returns to 0 again and again, but convergence needs a whole tail near the limit. Take $\varepsilon = 1/2$. For every threshold N, the odd index $n = 2N + 1$ is at least N, and $d(x_n, 0) = 1 \geq 1/2$. So the sequence does not converge to 0.

Pause and decide. Does the sequence $0, 1, 0, 1, \ldots$ converge to $1/2$? Does it converge to any real number at all?

Check. It does not converge to $1/2$: every term is at distance exactly $1/2$ from $1/2$, so for $\varepsilon = 1/2$ no term is close enough. In fact it converges to no real number L. Suppose it did, and take $\varepsilon = 1/2$. Some threshold N would make every term from x_N onward lie less than $1/2$ away

from L, including $x_{2N} = 0$ and $x_{2N+1} = 1$. The triangle inequality through L would then give $1 = |1 - 0| \le |1 - L| + |L - 0| < \frac{1}{2} + \frac{1}{2} = 1$, which is impossible.

11.4 Limits are unique

Can a sequence converge to two different points at once? The proof below uses only the three requirements of a metric, so it settles the question in every metric space.

Proposition 11.3. *A sequence in a metric space has at most one limit: if $x_n \to a$ and $x_n \to b$, then $a = b$.*

Proof. Suppose $x_n \to a$ and $x_n \to b$, and assume, for contradiction, that $a \neq b$. By the first requirement of a metric, $D = d(a, b)$ is then positive, and so is the tolerance $D/3$. Convergence to a supplies a threshold N_1 such that $d(x_n, a) < D/3$ for every $n \ge N_1$, and convergence to b supplies a threshold N_2 such that $d(x_n, b) < D/3$ for every $n \ge N_2$. Take a single index n at least as large as both thresholds, for instance the larger of N_1 and N_2. The triangle inequality with x_n as the stopping point, together with the symmetry $d(a, x_n) = d(x_n, a)$, gives

$$D = d(a, b) \le d(a, x_n) + d(x_n, b) < \frac{D}{3} + \frac{D}{3} = \frac{2D}{3}.$$

So $D < 2D/3$, that is, $D/3 < 0$, which contradicts $D > 0$. Therefore $a = b$. □

Any tolerance $\varepsilon > 0$ with $2\varepsilon \le D$ would do in place of $D/3$, since the proof then gives $D < 2\varepsilon \le D$. The key step is the choice of one index beyond both thresholds. The triangle inequality needs a single point x_n close to both a and b, as a stop on the route from a to b; a term close to a and a *different* term close to b would give two pieces that do not join up.

Because of the proposition, we may speak of *the* limit of a convergent sequence: $1/(n + 1)$, for example, converges to 0 and to no other real number.

11.5 Cauchy sequences do not need a known destination

To prove that a sequence converges, the definition asks us to name the limit first, and often we do not know it. In Chapter 12, for example, a sequence is produced by applying the same rule again and again, and the point it approaches is exactly the unknown we are trying to find. The following condition, named after the French mathematician Augustin-Louis Cauchy (1789–1857), speaks only about the terms themselves.

Definition 11.4 (Cauchy sequence)**.** A sequence $x_0, x_1, x_2, \ldots$ in a metric space (X, d) is a *Cauchy sequence* if, for every tolerance $\varepsilon > 0$, there is a threshold N such that any two terms from x_N onward are less than ε apart:

$$\forall \varepsilon > 0 \; \exists N \in \mathbb{N} \; \forall m, n \ge N : \quad d(x_m, x_n) < \varepsilon.$$

For convergence, the late terms had to be close to one fixed point x_*; here they have to be close to one another, and no limit appears. In the game, the skeptic, after seeing N, may now pick *any two* indices $m, n \ge N$.

Proposition 11.5. *Every convergent sequence in a metric space is a Cauchy sequence.*

Proof. Suppose $x_n \to x_*$, and let $\varepsilon > 0$ be given. Apply the definition of convergence with the smaller tolerance $\varepsilon/2$. It supplies a threshold N such that $d(x_n, x_*) < \varepsilon/2$ for every $n \geq N$. Now let $m, n \geq N$, so that both x_m and x_n are less than $\varepsilon/2$ away from x_*. The triangle inequality with x_* as the stopping point, together with symmetry, gives

$$d(x_m, x_n) \leq d(x_m, x_*) + d(x_*, x_n) < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

So the threshold N works for every pair of indices from N on, and the sequence is Cauchy. □

We used the tolerance $\varepsilon/2$ because the triangle inequality adds two distances. Splitting a tolerance into pieces in this way is a standard move that will return in later chapters.

The converse, "every Cauchy sequence converges," is not true in every metric space: the terms crowd together, but the point they crowd around may be missing from the space (Section 11.6).

Small steps are not enough

A sequence whose consecutive terms get closer and closer need not be Cauchy. Consider the *harmonic sums*

$$s_0 = 0, \qquad s_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{j=1}^{n} \frac{1}{j} \quad (n \geq 1).$$

Each step adds one more term, so $s_{n+1} - s_n = 1/(n+1)$, and Section 11.3 showed that these step sizes tend to 0. Yet the sums do not settle down. For $n \geq 1$, the difference $s_{2n} - s_n = \frac{1}{n+1} + \cdots + \frac{1}{2n}$ has n terms, each at least the smallest one, $1/(2n)$. For instance, $s_8 - s_4 = \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} \geq 4 \cdot \frac{1}{8} = \frac{1}{2}$. In general,

$$s_{2n} - s_n = \sum_{j=n+1}^{2n} \frac{1}{j} \geq n \cdot \frac{1}{2n} = \frac{1}{2}.$$

However late we start, doubling the index adds at least $1/2$. To turn this into a proof, negate the definition of a Cauchy sequence, as we did for convergence:

$$\exists \varepsilon > 0 \; \forall N \in \mathbb{N} \; \exists m, n \geq N : \quad d(x_m, x_n) \geq \varepsilon.$$

Take $\varepsilon = 1/2$, and let N be any proposed threshold. Choose $n = \max\{N, 1\}$; we need $n \geq 1$ because the estimate $s_{2n} - s_n \geq \frac{1}{2}$ holds only from $n = 1$ on (for $n = 0$ the difference is 0). Then choose $m = 2n$. Both indices are at least N, and $|s_m - s_n| = s_{2n} - s_n \geq 1/2$. So no threshold works for the tolerance $1/2$, and the sequence is not Cauchy. By the contrapositive (Section 2.5) of Proposition 11.5, the harmonic sums therefore do not converge.

11.6 Completeness excludes missing limits

Section 11.5 showed that every convergent sequence is Cauchy and noted that the converse can fail. The spaces in which it never fails have a name.

Definition 11.6 (Complete metric space)**.** A metric space (X, d) is *complete* if every Cauchy sequence in X converges to a point of X.

Informally, a complete space has no holes: whenever the terms of a sequence eventually all lie as close to one another as we like, there is a point of the space that they approach. Completeness is a property of a whole space, not of a single sequence. To use it, we prove that one particular sequence is Cauchy, and the definition hands us a limit in the space. To show that a space is *not* complete, it is enough to find one Cauchy sequence in it that has no limit in the space.

An interval with a missing endpoint

The interval $(0, 1)$, with distance $|x - y|$, is not complete. Consider the sequence $x_n = 2^{-(n+1)}$, that is, $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$. Every term lies in $(0, 1)$, because $0 < 2^{-(n+1)} \leq \frac{1}{2}$.

First, the sequence is Cauchy. On the real line it converges to 0: the inequality $2^k \geq k + 1$ of Section 11.3, with $k = n + 1$, gives $|x_n - 0| \leq 1/(n + 2) < 1/(n + 1)$, and the argument given there for $1/(n + 1)$ shows that these distances fall below any $\varepsilon > 0$ from some threshold on. So the sequence is Cauchy on the real line, by Proposition 11.5. The Cauchy condition involves only distances between terms, which are the same in $(0, 1)$ as in $\mathbb{R}$, so the sequence is Cauchy in $(0, 1)$ as well.

Second, the sequence has no limit in $(0, 1)$. A limit $L \in (0, 1)$ would also be a limit on the real line, since distances in $(0, 1)$ are distances in $\mathbb{R}$. As the sequence already converges to 0 there, Proposition 11.3 would give $L = 0$, which is not in $(0, 1)$. The only possible destination of the sequence has been removed from the space.

The real line

The real line, with distance $|x - y|$, is complete. We accept this as the last of the four starting facts about the real numbers listed in *To the reader*; a proof would require constructing the real numbers from simpler objects. The closed intervals proved complete below, and every later argument that needs a limit to exist without naming it in advance, rest on this one fact.

The rational numbers show what the assumption rules out. Cutting off the decimal expansion of $\sqrt{2} = 1.41421\ldots$ after k decimal places gives rational numbers $t_0 = 1$, $t_1 = 1.4$, $t_2 = 1.41$, $t_3 = 1.414$, and so on, which crowd around $\sqrt{2}$. Since $\sqrt{2}$ is not rational (Theorem 4.3), $\mathbb{Q}$, with distance $|x - y|$, has a hole there: it is not complete.

To make this precise, first check that $t_k \to \sqrt{2}$ on the real line. The difference $\sqrt{2} - t_k$ consists of the digits of $\sqrt{2}$ after the kth decimal place (for instance, $\sqrt{2} - t_2 = 0.00421\ldots$). Its first k decimal digits are zeros, so $0 \leq \sqrt{2} - t_k \leq 10^{-k}$. Since $10^k \geq 2^k \geq k + 1$, this is at most $1/(k + 1)$, and the argument of Section 11.3 for $1/(k + 1)$ gives $t_k \to \sqrt{2}$. Now argue as for $(0, 1)$. By Proposition 11.5, the sequence is Cauchy on the real line, and so also in $\mathbb{Q}$, since the distances

$|t_m - t_n|$ are the same. If it converged in $\mathbb{Q}$ to a rational number L, it would converge to L on the real line too, and Proposition 11.3 would give $L = \sqrt{2}$, which is not rational.

Closed subsets of a complete space

Completeness passes from a space to many of its subsets, each given the same distance as in Section 11.2.

Definition 11.7 (Closed subset)**.** A subset C of a metric space (X, d) is *closed* in X if, whenever a sequence of points of C converges to a point of X, that limit lies in C.

In words, you cannot leave C by taking limits.

Proposition 11.8. *If (X, d) is complete and C is closed in X, then C, with the same distance d, is complete.*

Proof. Take a Cauchy sequence of points of C. The distances between its terms are the same in C as in X, so it is also Cauchy in X. Since X is complete, it converges to some point of X. Since C is closed, that point lies in C, and the distances from the terms to it are again the same in C as in X. So the sequence converges to a point of C. □

The interval $(0, 1]$ shows what goes wrong without closedness. The points $1/(n + 1)$ lie in $(0, 1]$, but their limit 0 on the real line does not, so $(0, 1]$ is not closed in $\mathbb{R}$; and exactly as for $(0, 1)$, this sequence is Cauchy in $(0, 1]$ with no limit there, so $(0, 1]$ is not complete.

The closed interval $[0, 1]$, on the other hand, is closed in $\mathbb{R}$, and therefore complete. To prove that it is closed, suppose points $x_n \in [0, 1]$ converge on the real line to a number L. We show that $L < 0$ and $L > 1$ are both impossible, using the rule from Section 2.6 that, for $\varepsilon > 0$, the inequality $|x_n - L| < \varepsilon$ means $L - \varepsilon < x_n < L + \varepsilon$.

If $L < 0$, take half the distance from L to 0 as the tolerance: $\varepsilon = -L/2 > 0$. Convergence supplies a threshold N such that $|x_n - L| < -L/2$ for every $n \geq N$, and in particular

$$x_N < L + \left(-\frac{L}{2}\right) = \frac{L}{2} < 0,$$

contradicting $x_N \in [0, 1]$. If $L > 1$, the positive tolerance $\varepsilon = (L - 1)/2$ gives in the same way a term with $|x_N - L| < \varepsilon$, so

$$x_N > L - \frac{L - 1}{2} = \frac{L + 1}{2} > 1,$$

because $L > 1$ gives $L + 1 > 2$. This again contradicts $x_N \in [0, 1]$. So $L \in [0, 1]$. Hence $[0, 1]$ is closed in the complete real line, and by Proposition 11.8 it is complete.

The same argument, with a and b in place of 0 and 1 and the tolerances $(a - L)/2$ and $(L - b)/2$, shows that every closed interval $[a, b]$ with $a < b$ is complete. For instance, if $L < a$, a late term satisfies $x_N < L + (a - L)/2 = (L + a)/2 < a$.

Pause and decide. Is the set $[0, \infty)$ of nonnegative real numbers, with distance $|x - y|$, complete?

Check. Yes. If nonnegative numbers converged on the real line to some $L < 0$, the first case of the argument for $[0, 1]$ would produce a negative term. So $L \geq 0$, the set $[0, \infty)$ is closed in $\mathbb{R}$, and by

Proposition 11.8 it is complete. A complete space need not have a right-hand end; what matters is that no limit is missing.

11.7 A geometric bound controls an entire tail

The harmonic sums of Section 11.5 failed to be Cauchy because their steps, though small, added up to a lot. Steps that shrink by a fixed factor at every stage cannot add up to much, and the tool that shows this is the finite geometric sum.

The finite geometric sum

Let q be a real number with $q \neq 1$, and let r be a positive integer. Then

$$1 + q + q^2 + \cdots + q^{r-1} = \frac{1 - q^r}{1 - q}.$$

To see why, call the sum S. Multiplying by q raises every power by one:

$$\begin{aligned} S &= 1 + q + q^2 + \cdots + q^{r-1}, \\ qS &= \quad q + q^2 + \cdots + q^{r-1} + q^r. \end{aligned}$$

When the second line is subtracted from the first, the terms q through q^{r-1} cancel, leaving $(1 - q)S = 1 - q^r$. Since $1 - q \neq 0$, we may divide by it. For example, $q = 1/2$ and $r = 4$ give $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{15}{8}$, and the formula gives $\left(1 - \frac{1}{16}\right) \big/ \frac{1}{2} = \frac{15}{8}$.

From now on let $0 < q < 1$. We need sums that start at a later power. For integers $m > n \geq 0$, factor out q^n and apply the formula with $r = m - n$:

$$\sum_{j=n}^{m-1} q^j = q^n\left(1 + q + \cdots + q^{m-n-1}\right) = q^n \, \frac{1 - q^{m-n}}{1 - q} \leq \frac{q^n}{1 - q}.$$

The final inequality holds because $q^{m-n} > 0$, so $1 - q^{m-n} < 1$, and $q^n/(1 - q) > 0$. The bound does not depend on m: however many terms we add, a sum of consecutive powers starting at q^n never exceeds $q^n/(1 - q)$. We call this the *geometric tail bound*.

Powers of a number between zero and one tend to zero

The bound $q^n/(1 - q)$ is useful only if $q^n \to 0$. Section 11.3 handled $q = 1/2$ by proving $2^n \geq n + 1$; for general $0 < q < 1$, we imitate that proof with $1/q$ in place of 2. Put $c = 1/q - 1$. Since $1/q > 1$, we have $c > 0$, and $1/q = 1 + c$. We prove by induction that

$$(1 + c)^n \geq 1 + nc \qquad \text{for every } n \in \mathbb{N}.$$

This is known as *Bernoulli's inequality*. At $n = 0$ both sides equal 1. If $(1 + c)^n \geq 1 + nc$ for some n, multiply both sides by the positive number $1 + c$:

$$(1 + c)^{n+1} \geq (1 + nc)(1 + c) = 1 + c + nc + nc^2 = 1 + (n + 1)c + nc^2 \geq 1 + (n + 1)c,$$

because $nc^2 \geq 0$. This is the inequality for $n+1$. Since $q^n = 1/(1+c)^n$, taking reciprocals of positive numbers gives

$$0 < q^n \leq \frac{1}{1+nc}.$$

Now let $\varepsilon > 0$ be given, and use the Archimedean property to choose a positive integer $N > 1/(c\varepsilon)$. If $n \geq N$, then $1 + nc > nc \geq Nc > 1/\varepsilon$, where the last step multiplies $N > 1/(c\varepsilon)$ by $c > 0$. Taking reciprocals once more gives $q^n \leq 1/(1+nc) < \varepsilon$. This proves $q^n \to 0$. The closer q is to 1, the smaller c is, and the larger the threshold that this recipe gives.

Geometrically shrinking steps give a Cauchy sequence

Proposition 11.9. *Let $x_0, x_1, x_2, \ldots$ be a sequence in a metric space (X, d), and suppose there are numbers $D \geq 0$ and $0 < q < 1$ such that*

$$d(x_{j+1}, x_j) \leq Dq^j \qquad \text{for every } j \in \mathbb{N}.$$

Then $d(x_m, x_n) \leq Dq^n/(1-q)$ whenever $m > n$, and the sequence is a Cauchy sequence.

For example, the sequence $x_n = 1 - 2^{-n}$ of Section 11.3 has steps $x_{j+1} - x_j = 2^{-j} - 2^{-(j+1)} = \frac{1}{2}\left(\frac{1}{2}\right)^j$; these are positive, so they equal the distances $d(x_{j+1}, x_j)$, and the hypothesis holds with $D = q = 1/2$. The proposition promises $|x_m - x_n| \leq \frac{1}{2}\left(\frac{1}{2}\right)^n \big/ \frac{1}{2} = 2^{-n}$ for all $m > n$; directly, $|x_m - x_n| = 2^{-n} - 2^{-m}$, which is indeed less than 2^{-n}.

Proof. Step 1: from single steps to many steps. Fix n. We show by induction on m that the distance from x_n to x_m is at most the total length of the single steps between them: for every $m > n$,

$$d(x_m, x_n) \leq \sum_{j=n}^{m-1} d(x_{j+1}, x_j).$$

When $m = n+1$, the sum has the single term $d(x_{n+1}, x_n)$, and the claim is an equality. Suppose the claim holds for some $m > n$. The triangle inequality with x_m as the stopping point gives

$$d(x_{m+1}, x_n) \leq d(x_{m+1}, x_m) + d(x_m, x_n) \leq d(x_{m+1}, x_m) + \sum_{j=n}^{m-1} d(x_{j+1}, x_j) = \sum_{j=n}^{m} d(x_{j+1}, x_j),$$

which is the claim for $m+1$.

Step 2: a bound for the whole tail. For $m > n$, Step 1, the assumed bounds on the steps and the geometric tail bound give

$$d(x_m, x_n) \leq \sum_{j=n}^{m-1} d(x_{j+1}, x_j) \leq \sum_{j=n}^{m-1} Dq^j = D \sum_{j=n}^{m-1} q^j \leq \frac{Dq^n}{1-q}.$$

The last step multiplies the geometric tail bound by D, which keeps the inequality in the same direction because $D \geq 0$.

Step 3: the Cauchy condition. Let $\varepsilon > 0$. If $D = 0$, Step 2 gives $d(x_m, x_n) \leq 0$, hence $d(x_m, x_n) = 0$, for all $m > n$; so all the terms are equal, and the threshold $N = 0$ works. Now let

$D > 0$. Since $q^n \to 0$ and $\varepsilon(1-q)/D > 0$, there is a threshold N such that $q^n < \varepsilon(1-q)/D$, that is, $Dq^n/(1-q) < \varepsilon$, for every $n \geq N$. Take any two indices $m, n \geq N$. If $m = n$, their distance is $0 < \varepsilon$. Otherwise, by symmetry of d, we may name them so that $m > n$, and Step 2 gives $d(x_m, x_n) \leq Dq^n/(1-q) < \varepsilon$. So the sequence is Cauchy. □

The bound $Dq^n/(1-q)$ in Step 2 involves n but not m, so a single choice of n controls the whole tail from x_n onward. If the space is complete, as the real line and every closed interval are, the sequence also has a limit in the space. This combination of a geometric bound on the steps with completeness is the heart of the contraction theorem in Chapter 12. Before turning to that theorem, we use completeness once more to describe the edge of a set of real numbers.

11.8 The edge of a set: bounds, suprema, and infima

Section 11.6 showed that the interval $(0, 1)$ is missing the point 0: the terms $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$ crowd together in the interval but have no limit there. The missing point is an edge of the set. This section describes such edges with inequalities rather than sequences, and proves that every nonempty set of real numbers that is bounded above has a least upper bound.

Upper and lower bounds

Let A be a set of real numbers. An *upper bound* for A is a number that lies above every member of A, and a *lower bound* lies below every member. A set with an upper bound is *bounded above*, and a set with a lower bound is *bounded below*.

Definition 11.10 (Upper and lower bounds)**.** Let $A \subseteq \mathbb{R}$. A real number b is an *upper bound* for A if $a \leq b$ for every $a \in A$; then A is *bounded above*. A real number c is a *lower bound* for A if $c \leq a$ for every $a \in A$; then A is *bounded below*. A set that is both bounded above and bounded below is *bounded*.

For $A = [0, 1]$, the upper bounds are exactly the numbers $b \geq 1$, and the lower bounds are exactly the numbers $c \leq 0$. For $A = (0, 1)$, the upper and lower bounds are the same, but neither 1 nor 0 belongs to A. For $A = \{1/n : n \geq 1\}$, every upper bound satisfies $b \geq 1$ and every lower bound satisfies $c \leq 0$; the number 1 belongs to A, while 0 does not.

Suprema and infima

Two bounds deserve special names. A *maximum* is an upper bound that belongs to the set, and a *minimum* is a lower bound that belongs to the set.

Definition 11.11 (Maximum and minimum)**.** Let $A \subseteq \mathbb{R}$. A *maximum* of A is a member $M \in A$ that is an upper bound for A. A *minimum* of A is a member $m \in A$ that is a lower bound for A.

Thus $[0, 1]$ has maximum 1 and minimum 0. The interval $(0, 1)$ has neither: for every $x \in (0, 1)$, the numbers $x/2$ and $(x+1)/2$ lie in $(0, 1)$, one below x and one above it. The set $\{1/n : n \geq 1\}$ has maximum 1 but no minimum, because no member is smallest. Chapter 6 introduced a maximum for a partial order; for the usual order on $\mathbb{R}$, this is the same notion.

The empty set is bounded: every real number is vacuously both an upper bound and a lower bound. It has no maximum and no minimum, and the definitions below give it no supremum or infimum either. This is why the existence theorem below assumes a nonempty set.

The maximum, when it exists, is the most obvious upper bound: it belongs to the set. Many sets, such as $(0, 1)$, have no maximum but still have a smallest upper bound. That smallest upper bound is the supremum.

Definition 11.12 (Supremum and infimum)**.** Let $A \subseteq \mathbb{R}$. A *supremum* of A is an upper bound s for A such that $s \leq b$ for every upper bound b for A. An *infimum* of A is a lower bound t for A such that $c \leq t$ for every lower bound c for A.

In words, the supremum is the least upper bound, and the infimum is the greatest lower bound. We write $\sup A$ and $\inf A$ when they exist. The words are worth keeping alongside the symbols: "least upper bound" says exactly what the symbol means.

For $[0, 1]$, $\sup A = 1$ and $\inf A = 0$, and both belong to A. For $(0, 1)$, $\sup A = 1$ and $\inf A = 0$, but neither belongs to A; this is why the set has no maximum or minimum. For $A = \{1/n : n \geq 1\}$, $\sup A = 1$ and $\inf A = 0$; the supremum is the maximum, while the infimum is not a member. The empty set has no supremum and no infimum: every real number is an upper bound, so there is no least one, and every real number is a lower bound, so there is no greatest one.

Here is a proof in the definition's language that $\sup[0, 1] = 1$. Every $a \in [0, 1]$ satisfies $a \leq 1$, so 1 is an upper bound. If b is any upper bound, then $1 \leq b$, because $1 \in [0, 1]$. Hence 1 is the least upper bound. The same argument with the inequalities reversed gives $\inf[0, 1] = 0$.

Proposition 11.13. *A set $A \subseteq \mathbb{R}$ has at most one supremum and at most one infimum.*

Proof. Suppose that s and s' are both suprema of A. Since s is an upper bound and s' is the least upper bound, $s' \leq s$. Since s' is an upper bound and s is the least upper bound, $s \leq s'$. Hence $s = s'$. The argument for infima is the same with the inequalities reversed. Thus the notation $\sup A$ and $\inf A$ is unambiguous when the supremum or infimum exists. □

Proposition 11.14. *If A has a maximum M, then $\sup A = M$. If A has a minimum m, then $\inf A = m$.*

Proof. The member M is an upper bound by definition, and every upper bound b satisfies $M \leq b$ because $M \in A$. So M is the least upper bound. The minimum case is analogous. □

Pause and decide. If a set has a supremum, must it have a maximum?

Check. No. The interval $(0, 1)$ has supremum 1, but no member is a maximum: for every $x \in (0, 1)$, the number $(x + 1)/2$ is a larger member. The supremum is an upper bound that need not belong to the set.

The least-upper-bound property

The examples above had suprema that were easy to name. The next theorem says that, on the real line, a supremum exists whenever the set is nonempty and bounded above, even when no formula

for it is known. The idea of the proof is to trap the edge between two numbers: one that is not an upper bound, and one that is. At every step we halve the distance between them, keeping the left number below some member of the set and the right number above every member. The two numbers crowd together, and completeness supplies the point they approach.

Theorem 11.15 (Least-upper-bound property)**.** *Every nonempty set $A \subseteq \mathbb{R}$ that is bounded above has a supremum.*

Proof. If A has a maximum, Proposition 11.14 gives its supremum. So suppose that A has no maximum. Choose $a_0 \in A$ and an upper bound b_0 for A. Since a_0 belongs to A but is not a maximum, it is not an upper bound; hence $a_0 < b_0$.

We construct sequences $a_0, a_1, \ldots$ and $b_0, b_1, \ldots$ with the following properties for every n:

$$a_n \text{ is not an upper bound for } A, \qquad b_n \text{ is an upper bound for } A,$$

and

$$0 \le b_n - a_n \le \frac{b_0 - a_0}{2^n}.$$

Suppose a_n and b_n have been chosen, and let $m = (a_n + b_n)/2$. If m is an upper bound for A, set $a_{n+1} = a_n$ and $b_{n+1} = m$. If m is not an upper bound, choose $a_{n+1} \in A$ with $a_{n+1} > m$, and set $b_{n+1} = b_n$. In the first case,

$$b_{n+1} - a_{n+1} = m - a_n = \frac{b_n - a_n}{2},$$

and in the second,

$$b_{n+1} - a_{n+1} = b_n - a_{n+1} < b_n - m = \frac{b_n - a_n}{2}.$$

Thus the displayed bound holds by induction. In both cases $a_{n+1} \ge a_n$ and $b_{n+1} \le b_n$, so the intervals $[a_n, b_n]$ are nested.

The lengths $b_n - a_n$ tend to 0, because they are at most $(b_0 - a_0)/2^n$ and $2^{-n} \to 0$ (Section 11.7). Both sequences are Cauchy. Indeed, if $m \ge n$, then

$$a_n \le a_m \le b_m \le b_n,$$

so $0 \le a_m - a_n \le b_n - a_n$ and $0 \le b_n - b_m \le b_n - a_n$. Given $\varepsilon > 0$, choose N with $b_n - a_n < \varepsilon$ for every $n \ge N$; the same N works for both sequences, after exchanging the two indices if necessary. Since $\mathbb{R}$ is complete, they converge. Their limits are equal, because $b_n - a_n \to 0$; call the common limit s.

We show that s is an upper bound for A. If not, some $x \in A$ satisfies $x > s$. Since $b_n \to s$, there is an n with $b_n < x$. But b_n is an upper bound for A, and $x \in A$, a contradiction.

Finally, let b be any upper bound for A. We show that $s \le b$. If $b < s$, then, since $a_n \to s$, there is an n with $a_n > b$. But a_n is not an upper bound for A, so some $x \in A$ satisfies $x > a_n > b$. This contradicts the assumption that b is an upper bound. Hence every upper bound b satisfies $s \le b$, and s is the least upper bound of A. □

Corollary 11.16. *Every nonempty set $A \subseteq \mathbb{R}$ that is bounded below has an infimum.*

Proof. Apply Theorem 11.15 to $-A = \{-a : a \in A\}$. This set is nonempty and bounded above. A number d is an upper bound for $-A$ exactly when $-d$ is a lower bound for A. If s is the least upper bound of $-A$, then $-s$ is the greatest lower bound of A. □

Theorem 11.15 is called the *least-upper-bound property* of the real line. Some books take it as the starting fact about $\mathbb{R}$; we took the Cauchy form of completeness in Section 11.6, so here it is a theorem. The order form says that a nonempty set that does not run off to infinity has a first number above it, and that this number may or may not belong to the set.

The rational numbers do not have the property. The set $\{q \in \mathbb{Q} : q^2 < 2\}$ is nonempty and bounded above in $\mathbb{Q}$, but it has no supremum in $\mathbb{Q}$: on the real line its least upper bound is $\sqrt{2}$, which is not rational (Theorem 4.3). This is the same gap that Section 11.6 exposed with a Cauchy sequence.

Exercises

Exercises 11.1–11.4 are core exercises, and Exercises 11.5 and 11.6 are transfer exercises. Complete solutions are in Appendix A, Section A.11.

Exercise 11.1. Suppose $|x-a| \le 0.02$ and $|y-b| \le 0.03$. Find an upper bound for $|(2x-y)-(2a-b)|$ that holds for all such numbers, and give values of x, y, a and b for which your bound is attained.

Exercise 11.2. Prove that the sequence $a_n = 3/(n+1)$, for $n \in \mathbb{N}$, converges to 0. For each tolerance $\varepsilon > 0$, state explicitly the threshold N that you use.

Exercise 11.3. Show that the function d_∞ on $\mathbb{R}^n$ from Section 11.2 satisfies the first two requirements of Definition 11.1: $d_\infty(x, y) \ge 0$, with equality exactly when $x = y$, and $d_\infty(x, y) = d_\infty(y, x)$. With the triangle inequality proved there, this makes d_∞ a metric.

Exercise 11.4. A sequence $x_0, x_1, x_2, \ldots$ in a metric space satisfies $d(x_{j+1}, x_j) \le 2(1/3)^j$ for every $j \in \mathbb{N}$. Find an upper bound for $d(x_m, x_n)$ that holds for all $m > n$ and does not depend on m. Then show that the sequence is Cauchy, with an explicit threshold for each tolerance $\varepsilon > 0$.

Exercise 11.5. Give a Cauchy sequence in the interval $(2, 3)$, with distance $|x - y|$, that has no limit in $(2, 3)$. Prove both properties of your sequence.

Exercise 11.6. Let $A = (0, 1)$.

1. Show that $\sup A = 1$ and $\inf A = 0$.
2. Show that A has neither a maximum nor a minimum.
3. Theorem 11.15 states the least-upper-bound property. Explain why the statement would become false if the word “supremum” were replaced by “maximum.”

CHAPTER 12

The contraction theorem: an unfamiliar proof made readable

Overview

Prove the contraction theorem: a map of a nonempty complete metric space to itself that multiplies every distance by at most a fixed factor below one has exactly one fixed point, and repeating the map from any starting point approaches it. Prove existence, uniqueness, convergence and the error bounds as four separate claims with the tools of Chapter 11. Use the error bounds to guarantee the accuracy of an approximation without knowing the exact answer, and remove the hypotheses one at a time to see why each is needed.

12.1 From solving an equation to repeating a map

Section 4.4 proved that the equation

$$x = \frac{x+1}{2}$$

has exactly one real solution, $x = 1$. Section 5.3 reached the same number differently: starting from $x_0 = 0$ and applying the rule $x \mapsto (x+1)/2$ again and again gives

$$0,\ \tfrac{1}{2},\ \tfrac{3}{4},\ \tfrac{7}{8},\ \ldots, \qquad \text{in general } x_n = 1 - 2^{-n},$$

and Section 11.3 proved that these numbers converge to 1. So 1 is both the solution of an equation and the limit of a repeated update. This chapter turns the second point of view into a general method for solving equations.

The general setting is a metric space X, as in Chapter 11, and a map $T : X \to X$ (a map is the same thing as a function; Section 3.2). A *fixed point* of T is a point $x_* \in X$ with $T(x_*) = x_*$: a point that the map leaves where it is. Starting from any point $x_0 \in X$, we can apply T over and over: $x_1 = T(x_0)$, $x_2 = T(x_1)$, and in general $x_{n+1} = T(x_n)$ for every $n \in \mathbb{N}$. The points $x_0, x_1, x_2, \ldots$ are called the *iterates* of T starting from x_0. If they converge to a fixed point, then repeating the map solves the equation $T(x) = x$. Since in practice we stop after finitely many steps, we will also want to know how good the approximation is.

This simple method can fail in several ways.

- The iterates may never approach the fixed point. For $T(x) = 1 - x$ on $\mathbb{R}$, the fixed point is $1/2$, but starting from $x_0 = 0$ the iterates are $0, 1, 0, 1, \ldots$, each at distance $1/2$ from it.
- There can be several fixed points, and then "the solution" has no clear meaning (Section 4.4). For $T(x) = x^2$ on $[0, 1]$, both 0 and 1 are fixed points.

- The iterates can approach a point that is missing from the space. Take $T(x) = x/2$ on the interval $(0, 1)$; an interval like this, which leaves out both of its endpoints, is called an *open interval*. Starting from $x_0 = 1/2$, the iterates $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$ approach 0, which does not belong to $(0, 1)$, and in fact T has no fixed point in $(0, 1)$ at all.

We want conditions on T and X that rule out these failures in advance. The first condition says that T brings points closer together. From now on we often write Tx for $T(x)$ to save brackets.

Definition 12.1 (Contraction)**.** Let (X, d) be a metric space, and let q be a real number with $0 \le q < 1$. A map $T : X \to X$ is a *contraction with factor q* if

$$d(Tx, Ty) \le q\, d(x, y) \qquad \text{for all } x, y \in X.$$

This inequality is called the *contraction inequality*, and q is called a *contraction factor* of T.

In words: Tx and Ty are at most q times as far apart as x and y were. A contraction with factor q is also a contraction with any factor q' satisfying $q \le q' < 1$, since $q\, d(x, y) \le q'\, d(x, y)$ (distances are never negative). That is why we say "a" contraction factor rather than "the."

Two features of the definition matter. First, x and y are *any* two points of X, not just consecutive iterates or points close to a fixed point. For $T(x) = (x + 1)/2$ on $\mathbb{R}$, with the usual distance $d(x, y) = |x - y|$,

$$|T(x) - T(y)| = \left|\frac{x+1}{2} - \frac{y+1}{2}\right| = \left|\frac{(x+1) - (y+1)}{2}\right| = \frac{1}{2}|x - y|,$$

so this T is a contraction with factor $1/2$. The constant terms cancel when two outputs are compared, as in the uniqueness proof of Section 4.4. Second, one number q must work for all pairs at once. It is not enough that every pair of different points moves closer together: Section 12.6 gives a map that does this, with ratios $d(Tx, Ty)/d(x, y)$ coming closer and closer to 1, and has no fixed point.

The outputs must be allowed inputs

The second condition is built into the arrow of $T : X \to X$: every output $T(x)$ must again be a point of X. A map from a set to itself is called a *self-map*. This is what allows us to apply T to x_1, then to x_2, and so on. The condition is easy to overlook when a formula makes sense for more numbers than we intend to use; Section 12.6 shows what goes wrong without it. For $T(x) = (x + 1)/2$ on $[0, 1]$ it holds: if $0 \le x \le 1$, then $1 \le x + 1 \le 2$, so $1/2 \le T(x) \le 1$. So before using a contraction, we check two separate things: the self-map condition, and the contraction inequality for all pairs of points of X.

Pause and decide. Is $T(x) = x^2$ a contraction of the interval $[0, 1]$? Is it a contraction of the interval $[0, 1/3]$?

Check. On $[0, 1]$ it is not: for $x = 1$ and $y = 1/2$ we get $|T(x) - T(y)| = \frac{3}{4} > \frac{1}{2} = |x - y|$, so no factor $q < 1$ works for this pair. On $[0, 1/3]$ it is. It is a self-map, since $0 \le x \le 1/3$ gives $0 \le x^2 \le 1/9 \le 1/3$. Factoring the difference of two squares, $|x^2 - y^2| = |x - y|\,|x + y| \le \frac{2}{3}|x - y|$ for $x, y \in [0, 1/3]$, because then $0 \le x + y \le 2/3$. So $q = 2/3$ is a contraction factor.

12.2 The theorem and its dependency map

The theorem below says that these two conditions rule out the failures of Section 12.1, provided the space is nonempty and complete. It is also known as Banach's fixed-point theorem, after the Polish mathematician Stefan Banach (1892–1945), who published it in 1922.

Theorem 12.2 (Contraction theorem)**.** *Let (X, d) be a nonempty complete metric space, and let $T : X \to X$ be a contraction with factor q, where $0 \le q < 1$. Then the following hold.*

1. *T has exactly one fixed point x_*.*
2. *For every starting point $x_0 \in X$, the iterates defined by $x_{n+1} = T(x_n)$ converge to x_*, and for every $n \in \mathbb{N}$*
$$d(x_n, x_*) \le \frac{q^n}{1-q}\, d(x_1, x_0). \tag{12.1}$$
3. *For every point $z \in X$, whether or not it is an iterate,*
$$d(z, x_*) \le \frac{d(z, Tz)}{1-q}. \tag{12.2}$$

Section 1.3 agreed that $a^0 = 1$ for every nonzero number a; in this chapter we read q^0 as 1 even when $q = 0$, so that (12.1) makes sense for $n = 0$.

Four separate promises

The theorem makes four promises, each needing its own argument.

- *Existence*: some point of X is left unchanged by T.
- *Uniqueness*: no two different points are left unchanged by T.
- *Convergence*: from every starting point, the iterates approach the fixed point.
- *Error bounds*: after finitely many steps, we can say how far we still are from the fixed point, even though we do not know where it is.

For a general map, the first two promises do not follow from each other, and even together they do not give the third. The examples of Section 12.1 show this: $x \mapsto x^2$ on $[0, 1]$ has fixed points, but more than one; $x \mapsto x/2$ on $(0, 1)$ has at most one, but only because it has none; and $x \mapsto 1 - x$ on $\mathbb{R}$ has exactly one, yet its iterates from $x_0 = 0$ never approach it.

Pause and decide. The map $T(x) = 1 - x$ on $\mathbb{R}$ has a fixed point, but its iterates from $x_0 = 0$ do not converge. Which hypothesis of Theorem 12.2 does it fail?

Check. The space $\mathbb{R}$ is nonempty and complete, and T maps $\mathbb{R}$ to itself. The contraction inequality is what fails: $|T(x) - T(y)| = |(1 - x) - (1 - y)| = |x - y|$, so T never shrinks a distance, and no factor $q < 1$ works.

Two ways to measure the error

The *error* of a point $z \in X$ is its distance $d(z, x_*)$ from the fixed point. Parts 2 and 3 of the theorem bound it in two ways.

The bound (12.1) is called the *a priori estimate*, from the Latin for "from what comes before." It uses only q and the length $d(x_1, x_0)$ of the first step, so after a single step it tells us how many steps will be enough for a desired accuracy.

The bound (12.2) is called the *residual estimate*. The number $d(z, Tz)$ is the *residual* of z: it measures how far z is from satisfying $Tz = z$, and it is zero exactly when z is a fixed point. The point z may be an iterate, a guess, or the output of any other method; one evaluation of T gives a guaranteed upper bound on its error. A guaranteed bound of this kind, computed from known quantities, is called an *error certificate*.

Both bounds divide by $1 - q$, which is positive because $q < 1$; when q is close to 1, the factor $1/(1 - q)$ is large and the bounds are weak.

The plan of the proof

Here is the proof in outline, with the earlier facts that each step uses.

1. *The steps shrink geometrically.* Nonemptiness gives a starting point, and the self-map condition lets us form all the iterates. The contraction inequality and induction (Chapter 5) show that the step from x_n to x_{n+1} is at most q^n times as long as the first step.
2. *The iterates form a Cauchy sequence,* by Proposition 11.9, which rests on the triangle inequality and the geometric sum.
3. *A limit x_* exists in X,* by completeness (Section 11.6).
4. *The limit is a fixed point.* The triangle inequality, the contraction inequality and the definition of convergence show that $d(Tx_*, x_*)$ is smaller than every positive tolerance, hence zero.
5. *There is only one fixed point.* The contraction inequality, applied to two fixed points, shows that they coincide: the comparison argument of Section 4.4 in a new setting.
6. *The error bounds hold.* The triangle inequality, the contraction inequality and $Tx_* = x_*$ give (12.2); applied to x_n, together with Step 1, it gives (12.1).

A list like this, which records what each step of a proof depends on, is a *dependency map* of the proof. It tells you what you need to know before reading the proof, and where each hypothesis is used. Here everything comes from Chapters 5 and 11; no calculus is needed. Chapter 17 uses dependency maps to approach a proof whose ingredients are not all familiar yet.

12.3 The proof, one step at a time

The proof follows the six steps of Section 12.2.

Proof. **Step 1: the steps shrink geometrically.** Since X is nonempty, we can choose a starting

point $x_0 \in X$, any point at all. Define $x_{n+1} = T(x_n)$ for every $n \in \mathbb{N}$; because T maps X into X, each iterate is again a point of X, so T can be applied to it. Write

$$D = d(x_1, x_0)$$

for the length of the first step. We prove by induction on n that

$$d(x_{n+1}, x_n) \le q^n D \qquad \text{for every } n \in \mathbb{N}.$$

For $n = 0$ both sides equal D, because $q^0 = 1$. If the inequality holds for some n, then, since $x_{n+2} = T(x_{n+1})$ and $x_{n+1} = T(x_n)$, the contraction inequality for the points x_{n+1} and x_n gives

$$d(x_{n+2}, x_{n+1}) = d(Tx_{n+1}, Tx_n) \le q\, d(x_{n+1}, x_n) \le q \cdot q^n D = q^{n+1} D,$$

where the last inequality multiplies the induction hypothesis by $q \ge 0$. This is the claim for $n + 1$, which completes the induction.

Step 2: the iterates form a Cauchy sequence. Step 1 is exactly the hypothesis $d(x_{j+1}, x_j) \le Dq^j$ of Proposition 11.9, with $D = d(x_1, x_0) \ge 0$. The proposition assumes $0 < q < 1$. If $q = 0$, then T is also a contraction with factor $1/2$ (Section 12.1), so Step 1 with $1/2$ in place of q gives $d(x_{j+1}, x_j) \le D(1/2)^j$. Either way, the proposition shows that the iterates form a Cauchy sequence.

Step 3: completeness supplies a limit. Since X is complete, the Cauchy sequence $x_0, x_1, x_2, \ldots$ converges to some point x_* of X. This is the only place in the proof where completeness is used.

Step 4: the limit is a fixed point. We show that $d(Tx_*, x_*) = 0$. For every $n \in \mathbb{N}$, the triangle inequality with Tx_n as a stop on the way, the contraction inequality for the pair x_*, x_n, and $Tx_n = x_{n+1}$ give

$$d(Tx_*, x_*) \le d(Tx_*, Tx_n) + d(Tx_n, x_*) \le q\, d(x_*, x_n) + d(x_{n+1}, x_*).$$

Both distances on the right become small when n is large. To make this precise, let $\varepsilon > 0$. Because $x_n \to x_*$, the definition of convergence with tolerance $\varepsilon/2$ gives a threshold N such that $d(x_k, x_*) < \varepsilon/2$ for every $k \ge N$. Take $n = N$ in the display; then both n and $n + 1$ are at least N. Since $q < 1$, we have $q\, d(x_*, x_n) \le d(x_n, x_*) < \varepsilon/2$, and also $d(x_{n+1}, x_*) < \varepsilon/2$. Hence

$$d(Tx_*, x_*) < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

So the number $r = d(Tx_*, x_*)$, which does not depend on n or on ε, is smaller than every positive ε. Such a number must be zero: $r \ge 0$, and if r were positive, taking $\varepsilon = r$ would give $r < r$. So $d(Tx_*, x_*) = 0$, and by the first requirement of a metric, $Tx_* = x_*$.

Step 5: there is only one fixed point. Suppose y_* is also a fixed point of T. Since $x_* = Tx_*$ and $y_* = Ty_*$, the contraction inequality gives

$$d(x_*, y_*) = d(Tx_*, Ty_*) \le q\, d(x_*, y_*).$$

Subtracting $q\, d(x_*, y_*)$ from both sides gives $(1 - q)\, d(x_*, y_*) \le 0$, and dividing by the positive number $1 - q$ gives $d(x_*, y_*) \le 0$. Distances are never negative, so $d(x_*, y_*) = 0$, and therefore $y_* = x_*$.

This also proves convergence from *every* starting point: x_0 in Step 1 was arbitrary, Steps 1–4 showed that its iterates converge to a fixed point of T, and by Step 5 that fixed point can only be x_*.

Step 6: the error bounds. We prove the residual estimate first. Let z be any point of X. Use the triangle inequality with Tz as a stop, then $x_* = Tx_*$, then the contraction inequality:

$$d(z, x_*) \le d(z, Tz) + d(Tz, x_*) = d(z, Tz) + d(Tz, Tx_*) \le d(z, Tz) + q\, d(z, x_*).$$

Subtracting $q\, d(z, x_*)$ gives $(1 - q)\, d(z, x_*) \le d(z, Tz)$, and dividing by $1 - q > 0$ gives (12.2).

For the a priori estimate, fix n and take $z = x_n$. Its residual is $d(x_n, Tx_n) = d(x_n, x_{n+1}) = d(x_{n+1}, x_n)$, which Step 1 bounds by $q^n D$. So the residual estimate gives

$$d(x_n, x_*) \le \frac{d(x_n, x_{n+1})}{1 - q} \le \frac{q^n D}{1 - q} = \frac{q^n}{1 - q}\, d(x_1, x_0),$$

which is (12.1). This completes the proof. □

Pause and decide. Use the residual estimate (12.2) to give a second, one-line proof that T has at most one fixed point.

Check. If z is any fixed point, then $Tz = z$, so its residual $d(z, Tz)$ is 0, and the residual estimate gives $d(z, x_*) \le 0/(1 - q) = 0$. So $z = x_*$.

12.4 What the error estimates are for

Predicting how many steps are enough

Take $T(x) = (x + 1)/2$ on $\mathbb{R}$, a contraction with factor $q = 1/2$, and start from $x_0 = 0$. Then $x_1 = 1/2$, so $D = d(x_1, x_0) = 1/2$, and the a priori estimate (12.1) becomes

$$|x_n - 1| \le \frac{(1/2)^n}{1 - 1/2} \cdot \frac{1}{2} = 2^{-n}.$$

Suppose we want an error below $0.001 = 1/1000$. Since $2^9 = 512$ and $2^{10} = 1024$, the bound 2^{-n} is still larger than $1/1000$ at $n = 9$ but equals $1/1024$ at $n = 10$. So ten steps are guaranteed to be enough. Here the estimate cannot be improved: by the formula $x_n = 1 - 2^{-n}$, the error is exactly 2^{-n}, so nine steps really are too few. Usually, though, the true error is smaller than this worst-case bound, so the estimate may ask for more steps than necessary.

Certifying an approximation found by any method

To certify a point z, the residual estimate (12.2) multiplies its residual by $1/(1 - q)$, which can be large; the residual by itself is not a bound on the error. For example, $T(x) = 0.99x + 0.01$ is a contraction of $\mathbb{R}$ with factor $q = 0.99$, since $|T(x) - T(y)| = 0.99\,|x - y|$, and its fixed point is 1. The point $z = 0$ has the small residual $|0 - T(0)| = 0.01$, yet its error is $|0 - 1| = 1$, a hundred times larger. The residual estimate gives the bound $0.01/(1 - 0.99) = 1$, exactly the true error. So a small residual guarantees a small error only when q is not too close to 1.

Using the length of the latest step

For an iterate x_n with $n \geq 1$, there is a third bound, which uses the step just taken:

$$d(x_n, x_*) \leq \frac{q}{1-q}\, d(x_n, x_{n-1}).$$

To prove it, apply the residual estimate to $z = x_n$. Its residual is $d(x_n, Tx_n) = d(x_n, x_{n+1})$, and since $x_n = Tx_{n-1}$ and $x_{n+1} = Tx_n$, the contraction inequality gives

$$d(x_n, x_{n+1}) = d(Tx_{n-1}, Tx_n) \leq q\, d(x_{n-1}, x_n) = q\, d(x_n, x_{n-1}).$$

So $d(x_n, x_*) \leq d(x_n, x_{n+1})/(1-q) \leq q\, d(x_n, x_{n-1})/(1-q)$.

This is called an *a posteriori estimate*, from the Latin for "from what comes after," because it can be evaluated only after the step from x_{n-1} to x_n has been computed. It is never worse than the a priori estimate, because Step 1 of the proof gives $d(x_n, x_{n-1}) \leq q^{n-1}D$, so

$$\frac{q}{1-q}\, d(x_n, x_{n-1}) \leq \frac{q \cdot q^{n-1}D}{1-q} = \frac{q^n}{1-q}\, D,$$

which is the a priori bound. When the observed steps shrink faster than the worst case $q^{n-1}D$, the a posteriori estimate is much better.

Pause and decide. For $T(x) = (x+1)/2$ and $x_0 = 0$, compute the a posteriori bound for x_3 and compare it with the true error $|x_3 - 1|$.

Check. The iterates are $x_2 = 3/4$ and $x_3 = 7/8$, so the latest step has length $1/8$. With $q = 1/2$ we have $q/(1-q) = 1$, so the bound is $|x_3 - 1| \leq 1/8$. The true error is $1 - 7/8 = 1/8$, so the bound is attained, as the a priori bound 2^{-3} is for this map.

12.5 A quadratic example, worked by hand

For $T(x) = (x+1)/2$ we knew the fixed point from the start. Now we do not, and the theorem tells us about it before we have found it. Let

$$T(x) = \frac{x^2 + 1}{3} \qquad \text{on } X = [0, 1],$$

with the usual distance $|x - y|$. First we check each hypothesis.

- *The space.* The interval $[0, 1]$ is nonempty, and complete by Section 11.6.
- *The self-map condition.* If $0 \leq x \leq 1$, then $0 \leq x^2 \leq 1$, so $1 \leq x^2 + 1 \leq 2$ and $1/3 \leq T(x) \leq 2/3$. In particular $T(x) \in [0, 1]$.
- *The contraction inequality.* For $x, y \in [0, 1]$, factor the difference of two squares and use $|ab| = |a|\,|b|$ and $0 \leq x + y \leq 2$:

$$|T(x) - T(y)| = \frac{|x^2 - y^2|}{3} = \frac{|x - y|\,|x + y|}{3} \leq \frac{2}{3}|x - y|.$$

  So $q = 2/3$ is a contraction factor.

By Theorem 12.2, T has exactly one fixed point x_* in $[0, 1]$, and the iterates from every starting point in $[0, 1]$ converge to it.

Iterates and a certificate

Start from $x_0 = 0$, and compute with exact fractions, not rounded decimals:

$$x_1 = \frac{0^2 + 1}{3} = \frac{1}{3},$$
$$x_2 = \frac{(1/3)^2 + 1}{3} = \frac{1/9 + 9/9}{3} = \frac{10/9}{3} = \frac{10}{27},$$
$$x_3 = \frac{(10/27)^2 + 1}{3} = \frac{100/729 + 729/729}{3} = \frac{829/729}{3} = \frac{829}{2187}.$$

Take $z = x_2$. Then $Tz = x_3$, so the residual of z is $|x_2 - x_3|$. Writing $x_2 = 810/2187$ over the denominator $2187 = 27 \cdot 81$,

$$d(z, Tz) = \left| \frac{810}{2187} - \frac{829}{2187} \right| = \frac{19}{2187}.$$

Since $1/(1 - q) = 1/(1 - 2/3) = 3$, the residual estimate (12.2) gives the certificate

$$d(x_2, x_*) \le 3 \cdot \frac{19}{2187} = \frac{19}{729} = 0.02606\ldots.$$

A certificate rounded to a decimal must be rounded up, never down; otherwise it is no longer a guarantee. Rounded up, the certificate says that $x_2 = 10/27 \approx 0.370$ lies within 0.027 of the fixed point, although we do not yet know where the fixed point is.

The other two estimates of Section 12.4 give weaker bounds. The a priori estimate, with $D = d(x_1, x_0) = 1/3$ and $n = 2$, uses only the first step:

$$d(x_2, x_*) \le \frac{(2/3)^2}{1 - 2/3} \cdot \frac{1}{3} = \frac{4}{9} = 0.444\ldots.$$

The a posteriori estimate uses the step from $x_1 = 9/27$ to $x_2 = 10/27$ and gives $d(x_2, x_*) \le \frac{2/3}{1/3} \cdot \frac{1}{27} = \frac{2}{27} = 0.0740\ldots$. The residual estimate is the sharpest, because it uses the extra information in x_3.

Checking against the exact solution

Here we can also find x_* exactly. The fixed-point equation $(x^2 + 1)/3 = x$ becomes $x^2 + 1 = 3x$, that is, $x^2 - 3x + 1 = 0$. Multiply by 4 and add 5 to both sides to complete the square:

$$4x^2 - 12x + 9 = 5, \qquad \text{that is,} \qquad (2x - 3)^2 = 5.$$

If $a^2 = 5$, then $(a - \sqrt{5})(a + \sqrt{5}) = a^2 - 5 = 0$, so $a = \pm\sqrt{5}$. Hence $2x - 3 = \pm\sqrt{5}$, and $x = (3 \pm \sqrt{5})/2$. Since $4 < 5 < 9$, we have $2 < \sqrt{5} < 3$, so $(3 + \sqrt{5})/2 > 5/2$ lies outside $[0, 1]$, while $(3 - \sqrt{5})/2$ lies between 0 and $1/2$. The fixed point in $[0, 1]$ is therefore

$$x_* = \frac{3 - \sqrt{5}}{2} \approx 0.382.$$

The true error of x_2 is $x_* - 10/27 \approx 0.0116$, comfortably below the certificate.

On all of $\mathbb{R}$, the same formula has the two fixed points $(3 \pm \sqrt{5})/2$. This does not contradict uniqueness: Step 5 shows that a contraction never has two different fixed points, so T is not a contraction of $\mathbb{R}$. The uniqueness in Theorem 12.2 is about fixed points that lie in X.

12.6 Remove the hypotheses one at a time

Theorem 12.2 has four hypotheses: X is nonempty, X is complete, T maps X into itself, and T satisfies the contraction inequality with a factor $q < 1$. A good way to understand why a hypothesis is there is to remove it, keep all the others, and look for an example in which the conclusion fails. Exercise 12.4 asks you to check that each example in the first three groups below satisfies the hypotheses it keeps.

Without completeness

Let $T(x) = x/2$ on the open interval $X = (0, 1)$. This T is a self-map and a contraction with factor $1/2$, but it has no fixed point in X: the equation $x/2 = x$ forces $x = 0$, which does not belong to $(0, 1)$. The proof breaks down at Step 3. Starting from $x_0 = 1/2$, Steps 1 and 2 go through, and the iterates $x_n = 2^{-(n+1)}$ form exactly the Cauchy sequence that Section 11.6 used to show that $(0, 1)$ is not complete. Its only possible limit, 0, is missing from the space, so there is no point x_* for Steps 4–6 to work with.

Without a factor below one

Suppose we allow $q = 1$, so that the inequality only says $d(Tx, Ty) \le d(x, y)$. On $\mathbb{R}$, the identity map $T(x) = x$ satisfies this with equality, and every real number is a fixed point, so uniqueness fails. The map $T(x) = x + 1$ also satisfies $|T(x) - T(y)| = |x - y|$, but $x + 1 = x$ has no solution, so existence fails. So $q = 1$ does not merely slow convergence down; it can destroy existence or uniqueness. In the proof, Step 2 relies on the geometric tail bound $q^n/(1 - q)$, and Steps 5 and 6 divide by $1 - q$, which is now zero.

Even a map that brings every pair of different points strictly closer together can fail, if the shrinking gets weaker and weaker. Let $X = [1, \infty)$, the set of real numbers $x \ge 1$, and let

$$T(x) = x + \frac{1}{x}.$$

For $x \ge 1$ we have $T(x) > x \ge 1$, so T maps X into itself, and the set $[1, \infty)$ is complete, by an adaptation of the argument given for $[0, 1]$ in Section 11.6. Since $\frac{1}{x} - \frac{1}{y} = -\frac{x-y}{xy}$,

$$T(x) - T(y) = (x - y) + \left(\frac{1}{x} - \frac{1}{y}\right) = (x - y)\left(1 - \frac{1}{xy}\right).$$

For $x, y \ge 1$ with $x \ne y$, at least one of the two numbers is larger than 1, so $xy > 1$ and the factor $1 - 1/(xy)$ lies strictly between 0 and 1. Hence $|T(x) - T(y)| < |x - y|$. But the factor depends on

the pair, and no single $q < 1$ works for all pairs. Indeed, let $0 \le q < 1$, and use the Archimedean property (Section 2.3) to choose an integer $k \ge 1$ with $k > 1/(1-q)$. For $x = k+1$ and $y = k$ the ratio $|T(x) - T(y)|/|x - y|$ is $1 - 1/(k(k+1))$. Since $k(k+1) = k^2 + k > k > 1/(1-q)$, taking reciprocals gives $1/(k(k+1)) < 1 - q$, so the ratio is larger than q, and q is not a contraction factor. Finally, T has no fixed point: $x + 1/x = x$ would require $1/x = 0$.

Without the self-map condition

Let $T(x) = x/2 + 1$, used only for inputs in the nonempty complete space $[0, 1]$. The contraction inequality holds with factor $1/2$, but $T(1) = 3/2$ lies outside $[0, 1]$, so T is not a map from $[0, 1]$ to itself. The only real solution of $x/2 + 1 = x$ is $x = 2$, so T has no fixed point in $[0, 1]$. The proof breaks down already in Step 1: from $x_0 = 0$ we get $x_1 = 1$ and then $x_2 = 3/2$, which has left the space, so $T(x_2)$ is not even defined.

Without nonemptiness

If X is the empty set, there is still a map $T : X \to X$: it has no inputs, so there is nothing to assign. The requirements of a metric, of completeness and of the contraction inequality all hold vacuously (Section 2.5), because there are no points to check them for. But there is no fixed point either. The proof uses nonemptiness only to choose x_0 in Step 1.

Summary

The table records where the proof uses each hypothesis, and examples showing that it cannot be dropped.

Hypothesis	Used in the proof	Examples without it
X is nonempty	Step 1, to choose x_0	the empty set
X is complete	Step 3	$x \mapsto x/2$ on $(0, 1)$
T maps X into X	Step 1, to form the iterates	$x \mapsto x/2 + 1$ on $[0, 1]$
factor $q < 1$	Steps 2, 5 and 6	$x \mapsto x$ and $x \mapsto x + 1$ on $\mathbb{R}$
		$x \mapsto x + 1/x$ on $[1, \infty)$

Exercises

Exercises 12.1–12.4 are core exercises, and Exercises 12.5 and 12.6 are transfer exercises. Complete solutions are in Appendix A, Section A.12.

Exercise 12.1. Let $T(x) = (3x + 1)/4$ on $\mathbb{R}$. Find the fixed point of T and the smallest contraction factor of T. Then find a formula for the iterates x_n starting from $x_0 = 0$, and prove it by induction.

Exercise 12.2. Let T be a contraction with factor $q = 3/4$ of a nonempty complete metric space, with fixed point x_*, and let z be a point with $d(z, Tz) \le 0.002$. What upper bound on $d(z, x_*)$ does

Theorem 12.2 guarantee? Is the residual bound 0.002 itself an upper bound on $d(z, x_*)$ in every such case? Give a reason or an example.

Exercise 12.3. Let $T(x) = (x^2 + 1)/3$. Show that T maps the interval $[1/3, 2/3]$ into itself and that $4/9$ is a contraction factor of T on this interval. Starting from $x_0 = 0$, explain why every iterate from x_1 on lies in $[1/3, 2/3]$. Use the smaller factor to improve the certificate $d(x_2, x_*) \le 19/729$ obtained in Section 12.5, and explain why the improvement is justified.

Exercise 12.4. Section 12.6 gives counterexamples in three groups: without completeness, without a factor below one, and without the self-map condition. For each example in these three groups, prove that it satisfies every hypothesis of Theorem 12.2 except the one named in the heading of its group. For $T(x) = x + 1/x$, this includes proving that $[1, \infty)$ is complete.

Exercise 12.5. Let a and b be real numbers with $|a| < 1$, and let $T(x) = ax + b$ on $\mathbb{R}$. Find the fixed point x_* of T, and prove by induction that for every starting point x_0 the iterates satisfy $x_n - x_* = a^n(x_0 - x_*)$ for every $n \in \mathbb{N}$. How do the iterates move when $a < 0$, compared with when $0 < a < 1$?

Exercise 12.6. Let T and S be two self-maps of the same nonempty complete metric space (X, d), both contractions with factor q, where $0 \le q < 1$. Write x_T and x_S for their fixed points. Suppose there is a number $\eta \ge 0$ (the Greek letter eta) with $d(Tx, Sx) \le \eta$ for every $x \in X$. Prove that $d(x_T, x_S) \le \eta/(1 - q)$.

CHAPTER 13

Averaging, randomness, and existence

> **Overview**
> Treat finite probability as weighted counting: list the outcomes, give each one a weight, and add. Use averages and the union bound to prove that an object exists, without mistaking an average for a guarantee about every outcome. Turn one such existence proof into a step-by-step construction, and bound how often a nonnegative quantity can be much larger than its average.

13.1 A finite probability model

When we toss a coin or pick a student at random, we cannot predict the result. Probability does not try to: it lists everything that could happen, gives each possibility a weight, and calculates. In this chapter the list of possibilities is always finite, so every calculation is a finite sum.

Definition 13.1 (Finite probability space)**.** A *finite probability space* consists of a finite set Ω, whose members are called *outcomes*, together with a number $p(\omega)$ for each outcome $\omega \in \Omega$, called the *probability* of ω, such that

$$p(\omega) \geq 0 \quad \text{for every } \omega \in \Omega, \qquad \text{and} \qquad \sum_{\omega \in \Omega} p(\omega) = 1.$$

The set Ω is called the *sample space*. An *event* is a subset $A \subseteq \Omega$, and the *probability* of the event A is

$$\mathbb{P}(A) = \sum_{\omega \in A} p(\omega).$$

The symbols Ω and ω are the capital and lowercase Greek letter omega. As in Section 7.2, $\sum_{\omega \in A}$ means that we add one term for each member ω of A. A sum with no terms is 0 (Section 5.1), so $\mathbb{P}(\varnothing) = 0$, and the sample space cannot be empty, since the probabilities add up to 1.

Think of the experiment as producing exactly one outcome; an event A *occurs* when that outcome belongs to A. So Ω always occurs, and $\mathbb{P}(\Omega) = 1$, while $\varnothing$ never occurs.

In the simplest models, called *uniform*, every outcome has the same probability. These $|\Omega|$ equal probabilities add up to 1, so each is $1/|\Omega|$, and for every event A

$$\mathbb{P}(A) = \frac{|A|}{|\Omega|},$$

because the sum defining $\mathbb{P}(A)$ has $|A|$ terms, each equal to $1/|\Omega|$. A member of a finite set is chosen *uniformly at random* when that set is the sample space and the model is uniform; for example, if one student is chosen uniformly at random from a class of 30, each student is chosen with probability $1/30$.

Two coins

Toss two fair coins and record each result as 1 (heads) or 0 (tails). Write an outcome as two digits, the result of the first coin followed by the result of the second. The sample space is

$$\Omega = \{00,\ 01,\ 10,\ 11\}.$$

Because the coins are fair, we use the uniform model: each outcome has probability $1/4$. For example, the event "the two results differ" is $\{01, 10\}$, with probability $2/4 = 1/2$.

Which outcomes are equally likely?

The outcomes 01 and 10 are different. It is tempting to merge them into "one of each" and to give each possible number 0, 1, 2 of coins showing 1 the probability $1/3$, but this is a mistake: "one of each" covers two of the four equally likely outcomes. We may still use the three counts as outcomes, with the right weights: the sample space $\{0, 1, 2\}$ with

$$p(0) = \tfrac{1}{4}, \qquad p(1) = \tfrac{1}{2}, \qquad p(2) = \tfrac{1}{4},$$

describes the same two tosses, and is a finite probability space that is not uniform. Giving each count the probability $1/3$ also produces a finite probability space, but not one that describes two fair coins. A probability model is a piece of mathematics, and whether it fits a real experiment is a separate question, which no calculation inside the model can answer.

Disjoint events and complements

Recall from Section 3.1 that two sets are *disjoint* when they have no member in common. If two events A and B are disjoint, every outcome of $A \cup B$ lies in exactly one of them, so the sum defining $\mathbb{P}(A \cup B)$ splits into a sum over A and a sum over B:

$$\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B).$$

The *complement* of an event A is the event $\Omega \setminus A$ of the outcomes not in A; it occurs exactly when A does not. Since A and $\Omega \setminus A$ are disjoint and their union is Ω,

$$\mathbb{P}(A) + \mathbb{P}(\Omega \setminus A) = \mathbb{P}(\Omega) = 1.$$

Consequently, if $\mathbb{P}(A) < 1$, then $\mathbb{P}(\Omega \setminus A) = 1 - \mathbb{P}(A) > 0$. An event of positive probability contains at least one outcome ω with $p(\omega) > 0$, since otherwise its probability would be a sum of zeros, which is 0. Together these give a small but important rule: *if* $\mathbb{P}(A) < 1$*, then some outcome of positive probability lies outside* A.

Independent fair labels

Several experiments in this chapter give a label, 0 or 1, to each of n objects, as if a fair coin were tossed for each object. An outcome is then a list of n labels. There are 2^n such lists, by

the reasoning that counted the groups of students in Section 9.1, and we give every list the same probability $1/2^n$. The two-coin example is the case $n = 2$.

Now fix k of the objects and a label for each of them. A list giving these k objects their chosen labels may have any labels at the other $n - k$ objects, so there are 2^{n-k} such lists, and the probability that the k objects carry the chosen labels is

$$\frac{2^{n-k}}{2^n} = \frac{1}{2^k} = \underbrace{\frac{1}{2} \cdot \frac{1}{2} \cdots \frac{1}{2}}_{k \text{ factors}}.$$

So each object has each label with probability $1/2$ (the case $k = 1$), and the probabilities for different objects multiply. For example, with $n = 3$, the lists in which the first object has label 0 and the third has label 1 are 001 and 011, so this happens with probability $2/8 = \frac{1}{2} \cdot \frac{1}{2}$. This is what we mean when we say that the labels are chosen *independently and fairly*.

In general, two events A and B are called *independent* when $\mathbb{P}(A \cap B) = \mathbb{P}(A)\,\mathbb{P}(B)$. For example, with two coins, the events $A = \{10, 11\}$ (the first coin shows 1) and $B = \{00, 10\}$ (the second shows 0) are independent, since $\mathbb{P}(A \cap B) = \mathbb{P}(\{10\}) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} = \mathbb{P}(A)\,\mathbb{P}(B)$.

13.2 Expectation is a weighted average

Often we care about a number computed from the outcome: how many coins show 1, the score of the student who was picked, how many edges of a graph have some property.

Definition 13.2 (Random variable and expectation)**.** A *random variable* on a finite probability space is a function $X : \Omega \to \mathbb{R}$. Its *expectation*, or *expected value*, is the number

$$\mathbb{E}\,X = \sum_{\omega \in \Omega} p(\omega) X(\omega).$$

Despite its name, a random variable is an ordinary function, in the sense of Chapter 3; the word "random" refers to the way its input is chosen. The expectation is a *weighted average* of the values of X, in which the probabilities act as *weights*: they are nonnegative and add up to 1, and a more likely outcome counts for more. In a uniform model every weight is $1/|\Omega|$, so

$$\mathbb{E}\,X = \frac{1}{|\Omega|} \sum_{\omega \in \Omega} X(\omega),$$

the ordinary average of Section 7.3. From now on we often write $\sum_\omega$ for $\sum_{\omega \in \Omega}$. A random variable that takes the same value c at every outcome is called *constant*, and its expectation is $\sum_\omega p(\omega) c = c \sum_\omega p(\omega) = c$.

For two coins, let X count how many coins show 1. Then $X(00) = 0$, $X(01) = X(10) = 1$, and $X(11) = 2$, so

$$\mathbb{E}\,X = \tfrac{1}{4} \cdot 0 + \tfrac{1}{4} \cdot 1 + \tfrac{1}{4} \cdot 1 + \tfrac{1}{4} \cdot 2 = 1.$$

The two middle terms could have been combined into $\frac{1}{2} \cdot 1$, and this works in general. Write $\mathbb{P}(X = x)$ for the probability of the event $\{\omega \in \Omega : X(\omega) = x\}$. If the different values of X are

$x_1, \ldots, x_k$, the terms $p(\omega)X(\omega)$ with $X(\omega) = x_i$ have the common factor x_i, and the probabilities left over add up to $\mathbb{P}(X = x_i)$. Grouping the terms in this way gives

$$\mathbb{E}\, X = \sum_{i=1}^{k} x_i \; \mathbb{P}(X = x_i).$$

For two coins this is $0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 1$, the calculation we would make in the space $\{0, 1, 2\}$ of Section 13.1. Because of this formula, a random variable is often described just by listing its values and their probabilities; any finite probability space that produces these values with these probabilities gives the same expectation.

The expectation need not be a value of the variable. Toss one fair coin, and let Z be 0 if the coin shows 0 and 2 if it shows 1. Then $\mathbb{E}\, Z = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 2 = 1$, but Z never equals 1: the expectation summarizes the values, but does not predict the one you will see.

Linearity

Random variables on the same space are added, and multiplied by numbers, one outcome at a time: $X + Y$ has the value $X(\omega) + Y(\omega)$ at ω, and for a real number c, cX has the value $cX(\omega)$.

Proposition 13.3 (Linearity of expectation)**.** *Let X and Y be random variables on the same finite probability space, and let c be a real number. Then*

$$\mathbb{E}(X + Y) = \mathbb{E}\, X + \mathbb{E}\, Y \qquad \textit{and} \qquad \mathbb{E}(cX) = c\; \mathbb{E}\, X.$$

Proof. Multiply out each term, and then split the finite sum into two sums:

$$\begin{aligned} \mathbb{E}(X + Y) &= \sum_{\omega} p(\omega)\big(X(\omega) + Y(\omega)\big) \\ &= \sum_{\omega} \big(p(\omega)X(\omega) + p(\omega)Y(\omega)\big) \\ &= \sum_{\omega} p(\omega)X(\omega) + \sum_{\omega} p(\omega)Y(\omega) = \mathbb{E}\, X + \mathbb{E}\, Y. \end{aligned}$$

Splitting is allowed because a finite sum may be added up in any order. For the second identity, take the common factor c out of every term: $\mathbb{E}(cX) = \sum_\omega p(\omega)cX(\omega) = c \sum_\omega p(\omega)X(\omega) = c\; \mathbb{E}\, X$. □

By induction on m, the first identity extends to any finite number of random variables on the same space:

$$\mathbb{E}(X_1 + \cdots + X_m) = \mathbb{E}\, X_1 + \cdots + \mathbb{E}\, X_m;$$

for the induction step, write $X_1 + \cdots + X_m = (X_1 + \cdots + X_{m-1}) + X_m$ and apply the proposition.

For example, the results X_1 and X_2 of two coins each equal 1 at two of the four outcomes, so $\mathbb{E}\, X_1 = \mathbb{E}\, X_2 = 1/2$, and their sum, the number of coins showing 1, has expectation $1/2 + 1/2 = 1$, as found above.

The proof never asked how X and Y are related, so linearity holds even when they are closely linked. Toss one fair coin, let X be its result, and let $Y = 1 - X$, so that X determines Y. Still,

$X + Y$ is the constant 1, so $\mathbb{E}(X + Y) = 1 = \frac{1}{2} + \frac{1}{2} = \mathbb{E}\,X + \mathbb{E}\,Y$. This is what makes linearity so useful: to find the expectation of a complicated count, split it into simple pieces and add their expectations, without working out how the pieces affect one another.

Comparing expectations

Proposition 13.4 (Monotonicity of expectation)**.** *Let X and Y be random variables on the same finite probability space. If $X(\omega) \le Y(\omega)$ for every outcome ω, then $\mathbb{E}\,X \le \mathbb{E}\,Y$.*

Proof. For each outcome ω, multiply the inequality $X(\omega) \le Y(\omega)$ by $p(\omega)$. Since $p(\omega) \ge 0$, the inequality keeps its direction: $p(\omega)X(\omega) \le p(\omega)Y(\omega)$. Adding these inequalities over all outcomes gives $\mathbb{E}\,X \le \mathbb{E}\,Y$. □

An inequality that holds at every outcome, like the hypothesis $X(\omega) \le Y(\omega)$, is called *pointwise*; this proposition is what justifies "taking expectations of both sides" of it. The reverse step is not allowed: $\mathbb{E}\,X \le \mathbb{E}\,Y$ does not make $X \le Y$ pointwise. In the one-coin example with $Y = 1 - X$, $\mathbb{E}\,X = \mathbb{E}\,Y = 1/2$, but when the coin shows 1 we have $X = 1$ and $Y = 0$.

Some outcome is at least the average

By the averaging principle (Proposition 7.5), among $n \ge 1$ real numbers at least one is at least their average and at least one is at most it. The same is true for a weighted average.

Proposition 13.5. *Let X be a random variable on a finite probability space. Then some outcome ω with $p(\omega) > 0$ satisfies $X(\omega) \ge \mathbb{E}\,X$, and some outcome ω with $p(\omega) > 0$ satisfies $X(\omega) \le \mathbb{E}\,X$.*

Proof. Suppose, for contradiction, that $X(\omega) < \mathbb{E}\,X$ for every outcome ω with $p(\omega) > 0$. There is at least one such outcome, because the probabilities add up to 1 and so cannot all be 0. Multiplying each of these strict inequalities by its positive weight gives $p(\omega)X(\omega) < p(\omega)\ \mathbb{E}\,X$, while for an outcome with $p(\omega) = 0$ both sides are 0. Now add over all outcomes. At least one of the inequalities being added is strict, so the total is strict (if $x < y$ and $x' \le y'$, then $x + x' < y + x' \le y + y'$):

$$\mathbb{E}\,X = \sum_{\omega} p(\omega)X(\omega) < \sum_{\omega} p(\omega)\ \mathbb{E}\,X = \mathbb{E}\,X \cdot \sum_{\omega} p(\omega) = \mathbb{E}\,X,$$

which is impossible. The second claim is proved in the same way, with every inequality reversed. □

The condition $p(\omega) > 0$ makes sure that the outcome found can actually occur; an outcome of probability 0 contributes nothing to $\mathbb{E}\,X$, so the expectation carries no information about it. For the variable Z above, $\mathbb{E}\,Z = 1$, and only the outcome where the coin shows 1 has $Z \ge \mathbb{E}\,Z$: the proposition promises at least one such outcome and nothing more, and, like the pigeonhole principle, its proof does not tell us which.

13.3 Indicators convert events into quantities

Indicators connect events, which are sets of outcomes, with random variables, which assign numbers to outcomes. The *indicator* of an event A is the random variable I_A given by

$$I_A(\omega) = \begin{cases} 1 & \text{if } \omega \in A, \\ 0 & \text{if } \omega \notin A. \end{cases}$$

So I_A is 1 when A occurs and 0 when it does not. Its expectation is the probability of A:

$$\mathbb{E}\, I_A = \sum_{\omega \in A} p(\omega) \cdot 1 + \sum_{\omega \notin A} p(\omega) \cdot 0 = \mathbb{P}(A).$$

Indicators are useful for counting. For events $A_1, \ldots, A_m$, the sum $I_{A_1} + \cdots + I_{A_m}$ adds 1 at a given outcome for each event that occurs, so it counts how many of the events occur. By linearity of expectation, and because $\mathbb{E}\, I_{A_j} = \mathbb{P}(A_j)$,

$$\mathbb{E}\big(I_{A_1} + \cdots + I_{A_m}\big) = \mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m).$$

This holds even when the events overlap.

The union bound

The union $A_1 \cup \cdots \cup A_m$ is the event that at least one of the A_j occurs. Its indicator records only *whether* some event occurs, not *how many* do. To compare it with the count, let A and B be the events that the first coin and the second coin show 1:

outcome	I_A	I_B	$I_A + I_B$	$I_{A \cup B}$
00	0	0	0	0
01	0	1	1	1
10	1	0	1	1
11	1	1	2	1

At every outcome $I_{A \cup B} \le I_A + I_B$, strictly at 11, where both events occur. Taking expectations gives $\mathbb{P}(A \cup B) = 3/4 \le 1 = \mathbb{P}(A) + \mathbb{P}(B)$.

The same holds for any number of events. Where none of them occurs, both sides of

$$I_{A_1 \cup \cdots \cup A_m} \le I_{A_1} + \cdots + I_{A_m}$$

are 0; where at least one occurs, the left side is 1 and the right side is at least 1. So this pointwise inequality holds at every outcome. Taking expectations keeps it, by monotonicity (Proposition 13.4); the left side becomes $\mathbb{P}(A_1 \cup \cdots \cup A_m)$, since the expectation of an indicator is the probability of its event, and by linearity the right side becomes $\mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m)$. This proves the following inequality.

Proposition 13.6 (Union bound)**.** *For any events $A_1, \ldots, A_m$ in a finite probability space,*

$$\mathbb{P}(A_1 \cup \cdots \cup A_m) \leq \sum_{j=1}^{m} \mathbb{P}(A_j).$$

The union bound becomes an existence theorem when we think of $A_1, \ldots, A_m$ as *bad events*, the ways in which an outcome can fail to be what we want. Suppose that

$$\mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m) < 1.$$

Then the union bound gives $\mathbb{P}(A_1 \cup \cdots \cup A_m) < 1$, so by the rule of Section 13.1, some outcome of positive probability lies outside the union. That outcome avoids every bad event.

For example, a class has 7 committees of 4 students each, and a student may sit on several. Can the class be split into two teams so that every committee has members in both? Give each student the label 0 or 1, independently and fairly, and let A_j be the bad event that the four members of committee j all get the same label. By the rule for independent fair labels with $k = 4$, they all get the label 0 with probability $1/2^4 = 1/16$, and all get 1 with probability $1/16$; these two events are disjoint, so $\mathbb{P}(A_j) = 1/8$. The seven probabilities add up to $7/8 < 1$, so some labeling avoids every bad event. Put the students labeled 0 in one team and the others in the other; then no committee lies within one team. We never needed to know who sits on which committee.

Pause and decide. Suppose the probabilities of the bad events add up to exactly 1. Does it follow that some outcome avoids them all?

Check. Not necessarily. With one fair coin, the bad events "the coin shows 0" and "the coin shows 1" have probabilities adding up to 1, and every outcome is bad. With a sum of 1, the union bound says only that the union has probability at most 1, which is true of every event.

13.4 Every graph has a large cut

Suppose a class is to be split between two rooms so that as many pairs of chatty students as possible end up in different rooms. In the language of Chapter 7, with the students as vertices and the chatty pairs as edges, we want to divide the vertices into two groups with many edges running between them.

Definition 13.7 (Cut)**.** Let V be the vertex set of a graph. A *cut* of the graph is a pair of sets L and R with $L \cup R = V$ and $L \cap R = \varnothing$, so that every vertex lies in exactly one of them. An edge *crosses* the cut when one of its endpoints lies in L and the other in R. The *size* of the cut is the number of crossing edges.

The letters L and R suggest "left" and "right"; one of them may be empty. In the example graph G of Section 7.1, with edges ab, bc, ca, and cd, the cut $L = \{a, d\}$, $R = \{b, c\}$ has size 3, since only bc does not cross, and the cut $L = \{a, b, c, d\}$, $R = \varnothing$ has size 0. No cut of G has size 4, since then ab, bc, and ca would all cross; but if ab and bc cross, then a and c both lie on the side opposite to b, hence on the same side, and ca does not cross.

In a small graph we can find a largest cut by trying every possibility. But each vertex can go to either side, so a graph with 100 vertices has $2^{100} = (2^{10})^{10} > (10^3)^{10} = 10^{30}$ cuts, and no fast

method is known that finds a largest cut in every graph. The following theorem asks for less, and an averaging argument proves it in a few lines.

Theorem 13.8. *Every graph with m edges has a cut of size at least $m/2$.*

Plan. Put each vertex on the left or on the right by tossing a fair coin for it. Count the crossing edges with indicators, find their expected number by linearity, and use the fact that some outcome is at least the average.

Proof. Let V be the vertex set, E the edge set, and $n = |V|$. We use the model of independent fair labels from Section 13.1: an outcome is a labeling that gives each vertex the label 0 or 1, and each of the 2^n labelings has probability $1/2^n$. Each labeling determines a cut: L is the set of vertices labeled 0, and R is the set of vertices labeled 1.

Fix an edge $e = uv$, and let A_e be the event that e crosses the cut, that is, that u and v get different labels. Since u and v are different vertices, the rule for independent fair labels with $k = 2$ shows that u and v get the labels $0, 1$ with probability $1/4$, and the labels $1, 0$ with probability $1/4$. These two events are disjoint and make up A_e, so $\mathbb{P}(A_e) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$.

Let X be the size of the cut, viewed as a random variable on the labelings. At each labeling, X counts the crossing edges, so

$$X = \sum_{e \in E} I_{A_e}.$$

By linearity of expectation and $\mathbb{E}\, I_{A_e} = \mathbb{P}(A_e)$,

$$\mathbb{E}\, X = \sum_{e \in E} \mathbb{P}(A_e) = m \cdot \frac{1}{2} = \frac{m}{2}.$$

By Proposition 13.5, some labeling satisfies $X \geq \mathbb{E}\, X = m/2$. The cut determined by that labeling has at least $m/2$ crossing edges. □

The expected size of the cut is $m/2$, but labeling every vertex 0 gives a cut of size 0: the argument guarantees only that *some* labeling does at least as well as the average. Confusing this with a claim about every labeling is a common mistake; watch for it whenever you, or an AI assistant, summarize such a proof.

The proof used independence only to find $\mathbb{P}(A_e) = 1/2$ for a single edge; it never asked how the crossing events of different edges are related, and they can be closely linked. In the *triangle*, the graph with vertices a, b, c and edges ab, bc, ca, the three edges never all cross, as we saw for G, although each crosses with probability $1/2$; if each edge had a separate fair coin of its own, all three would cross with probability $1/8$. Yet the expected number of crossing edges is still $3/2$.

Since cut sizes are integers, for odd m the theorem gives a cut of size at least $(m + 1)/2$. For the triangle this is 2, the best possible.

13.5 Turning the average argument into a construction

The proof of Theorem 13.8 does not tell us how to find a good labeling. Instead of searching through all 2^n labelings, we can build one a vertex at a time, choosing each side so that a certain

score never goes down: the expected final size of the cut if the remaining vertices were placed at random.

The score of a partial placement

Suppose some of the vertices have been placed on the left or on the right, and the others not yet. Call an edge *settled* when both of its endpoints have been placed, and *unsettled* otherwise. Let C be the number of settled edges that cross, let U be the number of unsettled edges, and define

$$F = C + \frac{U}{2}.$$

So F has one term for each edge: 1 for a settled edge that crosses, 0 for a settled edge that does not, and $\frac{1}{2}$ for an unsettled edge.

To see where this formula comes from, imagine that each vertex not yet placed were now placed by tossing a fair coin, independently. A settled edge crosses or not, whatever the coins show, while an unsettled edge would cross with probability $1/2$: if one endpoint has been placed, it crosses when the coin sends the other endpoint to the opposite side, and if neither has, when the two coins disagree, as in the proof of Theorem 13.8. By linearity, the expected size of the final cut would be $C + U/2 = F$. We never actually toss these coins; we only compute F.

Placing one vertex

Let v be the next vertex to be placed, with ℓ neighbors already placed on the left and r on the right. If we place v on the left, the edges at v change their contributions to F as follows.

- The ℓ edges to placed neighbors on the left go from $1/2$ to 0: they become settled and do not cross.
- The r edges to placed neighbors on the right go from $1/2$ to 1: they become settled and cross.
- The edges to vertices not yet placed stay unsettled, and still contribute $1/2$ each.

Other edges are not affected. So placing v on the left changes F by

$$r \cdot \left(1 - \frac{1}{2}\right) + \ell \cdot \left(0 - \frac{1}{2}\right) = \frac{r - \ell}{2},$$

and, in the same way, placing v on the right changes F by $(\ell - r)/2$. These two changes are negatives of each other, so one of them is nonnegative. This gives a simple rule: *put v on the side opposite to the majority of its placed neighbors*, that is, on the right if $\ell > r$ and on the left if $r > \ell$; if $\ell = r$, either side will do. With this choice F changes by $|\ell - r|/2 \geq 0$.

Proposition 13.9. *Place the vertices of a graph with m edges one at a time, in any order, putting each vertex on the side opposite to the majority of its already placed neighbors, and on either side in case of a tie. The resulting cut has size at least $m/2$.*

Proof. Before any vertex is placed, every edge is unsettled, so $F = m/2$. By the calculation above, each placement made according to the rule changes F by $|\ell - r|/2 \geq 0$, so after the last placement $F \geq m/2$. At that point every edge is settled, so the size of the cut is $C = F \geq m/2$. □

This construction makes no random choices, and it is fast: placing v needs only its $\deg(v)$ neighbors, and by the handshake theorem (Theorem 7.2) these numbers add up to $2m$, far less work than checking 2^n labelings. An expected value computed with the choices made so far held fixed, and only the remaining choices left to chance, is called a *conditional expectation*; here it is F. Keeping it from decreasing while the choices are made one at a time is the *method of conditional expectations*.

A full trace on the triangle

Place the vertices of the triangle in the order a, b, c. Initially $C = 0$, $U = 3$, and $F = 3/2$.

- Vertex a has no placed neighbors, so $\ell = r = 0$; put a on the left. No edge is settled yet, so F stays $3/2$.
- Vertex b has a on the left, so $\ell = 1$ and $r = 0$; put b on the right. Now ab is settled and crosses, so $C = 1$, $U = 2$, and $F = 2$, an increase of $(\ell - r)/2 = 1/2$, as predicted.
- Vertex c has a on the left and b on the right, so $\ell = r = 1$, and either side leaves F unchanged; put c on the left. Now bc crosses and ca does not, so $C = 2$, $U = 0$, and $F = 2$.

The final cut $L = \{a, c\}$, $R = \{b\}$ has size 2, which by Section 13.4 is the largest possible for the triangle.

The construction need not find a largest cut

Take the path with vertices a, b, c, d and edges ab, bc, cd, place the vertices in the order a, d, b, c, and break ties by choosing the left side. Neither a nor d has a placed neighbor, so both go to the left; then b has a on the left, so it goes to the right; finally c has one placed neighbor on each side, so it goes to the left. The resulting cut $L = \{a, c, d\}$, $R = \{b\}$ has size 2, since cd does not cross, while all three edges cross the cut $L = \{a, c\}$, $R = \{b, d\}$.

13.6 A bound on rare large values

The expectation also limits how often a quantity that is never negative can be much larger than its average. A random variable X is *nonnegative* when $X(\omega) \geq 0$ for every outcome ω. For a real number a, we write $\mathbb{P}(X \geq a)$ for the probability of the event $\{\omega \in \Omega : X(\omega) \geq a\}$.

Proposition 13.10 (Markov's inequality)**.** *Let X be a nonnegative random variable on a finite probability space, and let $a > 0$ be a real number. Then*

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}\,X}{a}.$$

Proof. Split the sum defining the expectation into the outcomes where $X \geq a$ and the rest:

$$\mathbb{E}\,X = \sum_{\omega:\, X(\omega) \geq a} p(\omega)X(\omega) + \sum_{\omega:\, X(\omega) < a} p(\omega)X(\omega).$$

In the first sum $X(\omega) \geq a$ and $p(\omega) \geq 0$, so $p(\omega)X(\omega) \geq p(\omega)a$ for each term, and the sum is at least

$$\sum_{\omega : X(\omega) \geq a} p(\omega)\, a = a\ \mathbb{P}(X \geq a).$$

In the second sum every term is nonnegative, because $p(\omega) \geq 0$ and $X(\omega) \geq 0$, so the second sum is at least 0. Together, $\mathbb{E}\,X \geq a\ \mathbb{P}(X \geq a)$, and dividing by the positive number a gives the claim. □

For example, suppose that the scores on a test are all nonnegative and that the class average is 20. If X is the score of a student chosen uniformly at random, then $\mathbb{E}\,X = 20$, and $\mathbb{P}(X \geq 80)$ is the fraction of the class that scored 80 or more. Markov's inequality says that this fraction is at most $20/80 = 1/4$: if more than a quarter of the class had scored at least 80, those students alone would contribute more than $\frac{1}{4} \cdot 80 = 20$ to the average, and the others at least 0, so the average would exceed 20.

The bound need not be exact. For the number X of coins showing 1 in the two-coin experiment, $\mathbb{E}\,X = 1$, so Markov's inequality with $a = 2$ gives $\mathbb{P}(X \geq 2) \leq 1/2$, while in fact $\mathbb{P}(X \geq 2) = \mathbb{P}(\{11\}) = 1/4$. Exercise 13.6 asks for a case in which the bound is exact.

In terms of multiples of the average: if $\mathbb{E}\,X > 0$ and $t > 0$, taking $a = t\ \mathbb{E}\,X$ gives

$$\mathbb{P}(X \geq t\ \mathbb{E}\,X) \leq \frac{\mathbb{E}\,X}{t\ \mathbb{E}\,X} = \frac{1}{t}.$$

So a nonnegative quantity with a positive average is at least twice its average with probability at most $1/2$, and at least ten times its average with probability at most $1/10$.

The hypothesis that X is nonnegative is needed, since otherwise negative values can balance positive ones. A random variable equal to 100 or -100, each with probability $1/2$, has expectation 0, yet $\mathbb{P}(X \geq 100) = 1/2 > 0/100$. The proof used the hypothesis to make the second sum at least 0; here that sum is -50.

> **Habit 8: choose a quantity whose average is easier to find than its best value**
>
> When an object with a desired property is hard to construct directly, set up a finite random experiment whose outcomes are candidate objects. Count the features you want, or want to avoid, with indicators, and find an average you can compute exactly; linearity often makes this easy even when the features interact. Then say precisely what you have proved. Three conclusions are easy to confuse: that a suitable outcome exists, that a random outcome is unlikely to be bad, and that a suitable object can be built efficiently. The proof of Theorem 13.8 reached only the first. The second comes from a small upper bound on the probability of a bad outcome, such as the union bound or Markov's inequality can give, and the third needed the construction of Section 13.5.

Exercises

Exercises 13.1–13.4 are core exercises, and Exercises 13.5 and 13.6 are transfer exercises. Complete solutions are in Appendix A, Section A.13.

Exercise 13.1. A random variable takes the values 0, 2, and 5 with probabilities $1/2$, $1/3$, and $1/6$. Compute its expectation, and name a value of the variable that is at least as large as the expectation.

Exercise 13.2. Label each vertex of a triangle with 0 or 1, and take the cut whose left side consists of the vertices labeled 0. List the possible sizes of this cut, and compute the average cut size over all eight labelings.

Exercise 13.3. Run the construction of Section 13.5 on the graph with vertices $1, 2, 3, 4$ and edges 12, 23, 34, 41 (here 12 stands for the edge $\{1, 2\}$, and so on). Place the vertices in the order $1, 2, 3, 4$, and break ties by choosing the left side. At each step, record ℓ, r, the side chosen, and the new value of F. How large is the final cut, and how does it compare with the guarantee of Theorem 13.8?

Exercise 13.4. In a finite probability space, four bad events each have probability at most 0.2. Prove that some outcome of positive probability avoids all four. Then suppose instead that each bad event has probability at most 0.25. Does the same conclusion follow? Explain, using examples.

Exercise 13.5. Give an example of a random variable whose expectation is at least 5 but which takes the value 0 at some outcome of positive probability. This shows that a large expectation does not force every value to be large. Then state what the expectation does guarantee about the values of a random variable.

Exercise 13.6. Let X be a nonnegative random variable with $\mathbb{E}\,X = 3$. Use Markov's inequality to bound $\mathbb{P}(X \geq 12)$. Then give an example of such a random variable, on a finite probability space, for which $\mathbb{P}(X \geq 12)$ equals your bound.

CHAPTER 14

Sign sequences: from experiments to a research boundary

Overview

Measure how closely a sequence of plus and minus signs matches shifted copies of itself, and translate the search for sequences that agree in exactly half of their positions with each rotated copy into a question about matrices with orthogonal rows. Prove two conditions that the size of such a matrix must satisfy, compare them with a complete computer search over short sequences, and see why no search can replace a proof that covers every length. Finally, read a recent research claim about these matrices and say exactly what it adds to what you have proved.

14.1 A sequence with small echoes

Sign sequences, met in Section 6.5, are the main characters of this chapter. A *sign sequence* of length n, where $n \geq 1$, is a list $a = (a_0, a_1, \ldots, a_{n-1})$ in which every entry is 1 or -1. As in Python (Appendix B), we number the positions from 0, so the last entry is a_{n-1}.

Think of the sequence as a pattern of pulses, and of a copy delayed by s steps as an echo. If the pattern matches its echoes only weakly, a receiver that compares what it hears with the pattern gets a strong match only at the moment the pattern arrives, which is why such sequences are used in radar and digital communication. To measure the match, write a copy shifted s places to the right under the sequence and look at the positions where the two rows overlap. For $a = (1, 1, 1, -1)$ and $s = 1$:

$$\begin{array}{cccc} 1 & 1 & 1 & -1 \\ & 1 & 1 & 1 \quad -1 \end{array}$$

In each overlapping position, the entry a_j of the shifted copy stands below the entry a_{j+s} of the original. Their product $a_j a_{j+s}$ is 1 when the two entries agree and -1 when they differ, so adding these products gives the number of agreements minus the number of disagreements.

Definition 14.1 (Aperiodic autocorrelation)**.** Let a be a sign sequence of length n. For each shift s with $1 \leq s \leq n - 1$, the *aperiodic autocorrelation* of a at shift s is

$$A_s(a) = \sum_{j=0}^{n-1-s} a_j a_{j+s}.$$

We write A_s when the sequence is clear. “Aperiodic” means that nothing wraps around: entries pushed past the end drop off, as the last -1 in the picture does. Shifts of n or more would leave no

overlap. Shift zero is left out because there every entry meets itself, giving $a_0^2 + \cdots + a_{n-1}^2 = n$ for every sequence of length n, which tells us nothing.

Decode the summation limits before computing

The term $a_j a_{j+s}$ makes sense only when both j and $j+s$ lie in the list $0, 1, \ldots, n-1$. So the sum starts at $j = 0$ and stops when $j + s = n - 1$, that is, at $j = n - 1 - s$, giving $(n-1-s) - 0 + 1 = n - s$ terms, one for each overlapping position. For length five and shift two,

$$A_2 = a_0 a_2 + a_1 a_3 + a_2 a_4,$$

with $5 - 2 = 3$ terms; there is no term $a_3 a_5$, because a_5 is not in the list. At the largest shift, $s = n - 1$, a single term $a_0 a_{n-1}$ remains. Note that the subscript on A_s is a shift, while the subscript on a_j is a position.

A first example, and the sequences we are looking for

For $a = (1, 1, 1, -1)$ there are three shifts to check:

$$\begin{aligned} A_1 &= a_0 a_1 + a_1 a_2 + a_2 a_3 = (1)(1) + (1)(1) + (1)(-1) = 1, \\ A_2 &= a_0 a_2 + a_1 a_3 = (1)(1) + (1)(-1) = 0, \\ A_3 &= a_0 a_3 = (1)(-1) = -1. \end{aligned}$$

Definition 14.2 (Barker sequence)**.** A *Barker sequence* is a sign sequence a of length n with $|A_s(a)| \le 1$ for every shift s with $1 \le s \le n - 1$.

So $(1, 1, 1, -1)$ is a Barker sequence. These sequences are named after R. H. Barker, who studied them in 1953. A sequence of length one has no shifts to check, so the condition holds vacuously (Section 2.5), and both (1) and (-1) are Barker sequences.

Why "at most one" rather than "zero"? Because A_s often cannot be zero. If m of its $n - s$ terms equal -1, the other $n - s - m$ equal 1, and

$$A_s = (n - s - m) - m = (n - s) - 2m.$$

Subtracting the even number $2m$ does not change parity, so A_s is even exactly when $n - s$ is even; in particular, A_s can be zero only when $n - s$ is even. Since the only even integer with absolute value at most one is 0, and the only odd ones are ± 1, a Barker sequence has $A_s = 0$ whenever $n - s$ is even and $A_s = \pm 1$ whenever $n - s$ is odd. Its echoes are as weak as they can possibly be; for $(1, 1, 1, -1)$, where $n - s = 3, 2, 1$, the correlations are $1, 0, -1$.

Pause and decide. Is $(1, 1, -1, -1)$ a Barker sequence?

Check. No. At shift two, $n - s = 2$ is even, so a Barker sequence would need $A_2 = 0$. But $A_2 = a_0 a_2 + a_1 a_3 = (1)(-1) + (1)(-1) = -2$. The correlations $A_1 = 1$ and $A_3 = -1$ are fine, but one bad shift is enough to fail the definition.

14.2 Periodic correlation is a different quantity

Instead of letting entries fall off the end, we can write the n entries around a circle, like the hours on a clock face, and rotate the copy by s places. Now a position that runs past the end of the list comes back in at the start: the position $j + s$ is replaced by its remainder on division by n, written $(j + s) \bmod n$ (Section 5.5). For $a = (1, 1, 1, -1)$ and $s = 1$, the picture of Section 14.1 becomes

$$\begin{array}{cccc} 1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 \end{array}$$

The -1 that dropped off the end comes back in at the front, so all four columns overlap.

Definition 14.3 (Periodic autocorrelation)**.** Let a be a sign sequence of length n. For each shift s with $0 \le s \le n - 1$, the *periodic autocorrelation* of a at shift s is

$$C_s(a) = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n}.$$

We write C_s when the sequence is clear. Unlike A_s, this sum always has n terms. At shift zero every entry meets itself, so $C_0 = n$; this says nothing about the sequence, but we keep shift zero because the value is needed in Section 14.4.

For $a = (1, 1, 1, -1)$ and $s = 1$, the last position $3 + 1 = 4$ wraps around to $4 \bmod 4 = 0$:

$$C_1 = a_0a_1 + a_1a_2 + a_2a_3 + a_3a_0 = 1 + 1 - 1 - 1 = 0.$$

In the same way,

$$C_2 = a_0a_2 + a_1a_3 + a_2a_0 + a_3a_1 = 1 - 1 + 1 - 1 = 0,$$
$$C_3 = a_0a_3 + a_1a_0 + a_2a_1 + a_3a_2 = -1 + 1 + 1 - 1 = 0,$$

and $C_0 = 4$. Like A_s, C_s is the number of agreements minus the number of disagreements, so $C_s = 0$ says that the sequence agrees with its rotated copy in exactly half of the n positions. Sequences with this property at every nonzero shift, like this one, are what the rest of the chapter is about.

How the two kinds of correlation are related

Start with $n = 5$ and $s = 2$. The periodic sum has five terms, and the last two are the ones that wrap around:

$$C_2 = \underbrace{a_0a_2 + a_1a_3 + a_2a_4}_{A_2} + \underbrace{a_3a_0 + a_4a_1}_{A_3}.$$

Swapping the factors turns the last two terms into $a_0a_3 + a_1a_4$, which is A_3. The same split works for every shift s with $1 \le s \le n - 1$. If $0 \le j \le n - 1 - s$, then $j + s \le n - 1$: nothing wraps, and these terms a_ja_{j+s} are exactly those of A_s. If $n - s \le j \le n - 1$, then $n \le j + s < 2n$, so subtracting n once brings the position back into the list: $(j + s) \bmod n = j + s - n$. Thus

$$C_s = \sum_{j=0}^{n-1-s} a_ja_{j+s} + \sum_{j=n-s}^{n-1} a_ja_{j+s-n}.$$

The first sum is A_s. In the second, put $k = j + s - n$, so $j = k + n - s$; the first term $j = n - s$ has $k = 0$, and the last term $j = n - 1$ has $k = s - 1$. Substituting,

$$\begin{aligned}\sum_{j=n-s}^{n-1} a_j a_{j+s-n} &= \sum_{k=0}^{s-1} a_{k+n-s} a_k \\ &= \sum_{k=0}^{s-1} a_k a_{k+(n-s)} \\ &= A_{n-s}.\end{aligned}$$

The middle step swaps the factors. The last is the definition of A_{n-s}: the shift $n - s$ lies between 1 and $n - 1$, and the index of A_{n-s} runs from 0 to $n - 1 - (n - s) = s - 1$. So, for every shift s with $1 \le s \le n - 1$,

$$C_s = A_s + A_{n-s}. \tag{14.1}$$

For $(1, 1, 1, -1)$ this gives $C_1 = A_1 + A_3 = 1 + (-1) = 0$, as computed directly. But small aperiodic correlations do not make the periodic ones zero. The Barker sequence $(1, 1, 1, -1, 1)$ has $A_1 = 0$, $A_2 = 1$, $A_3 = 0$, and $A_4 = 1$ (check them), so by (14.1) its periodic correlations $C_1 = A_1 + A_4 = 1$ and $C_2 = A_2 + A_3 = 1$ are not zero.

Pause and decide. Explain why, when the length n is odd, no periodic correlation C_s can be zero.

Check. The sum C_s has n terms, each 1 or -1. By the parity argument of Section 14.1, if m terms equal -1, the sum is $n - 2m$, which has the same parity as n. So when n is odd, C_s is odd, and therefore not zero, as for $(1, 1, 1, -1, 1)$ above.

14.3 Circulant matrices turn correlations into dot products

Periodic correlations can be read off from a matrix. Write the sequence a as the first row, and obtain each further row from the one above it by moving every entry one place to the right, the last entry wrapping around to the front. One more step after n rows would bring back the first row, so we stop with an n-by-n matrix, its rows and columns numbered from 0 to $n - 1$ (not from 1 to n as in Chapter 10).

Definition 14.4 (Circulant matrix)**.** Let a be a sign sequence of length n. The *circulant matrix* of a is the n-by-n matrix H with entries

$$H_{ij} = a_{(j-i) \bmod n}, \qquad 0 \le i, j \le n - 1.$$

A matrix of this form is called *circulant*.

Row i is the sequence rotated i places to the right. Such a rotation moves position k to $(k + i) \bmod n$, so the entry in column j started at position $j - i$, or $j - i + n$ if $j - i$ is negative; in both cases that is $(j - i) \bmod n$. For instance, in row 1 the entry in column 0 is $a_{(0-1) \bmod n} = a_{n-1}$, because $-1 = (-1) \cdot n + (n - 1)$ and remainders are never negative (Section 5.5). Row 0 is the sequence itself, because $H_{0j} = a_j$, so every circulant matrix is the circulant matrix of its own first

row. For $a = (1, 1, 1, -1)$,

$$H = \begin{pmatrix} 1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \end{pmatrix}.$$

As j runs through $0, 1, \ldots, n-1$, the index $(j - i) \bmod n$ also runs through $0, 1, \ldots, n-1$, each value exactly once; with $n = 4$ and $i = 1$, the columns $j = 0, 1, 2, 3$ give the indices $3, 0, 1, 2$. In the language of Section 3.3, the map $j \mapsto (j - i) \bmod n$ is a bijection from $\{0, 1, \ldots, n-1\}$ to itself, with inverse $k \mapsto (k + i) \bmod n$. Indeed, $(j - i) \bmod n + i$ is congruent to j modulo n (Section 6.2), so it has the same remainder as j, which is j itself because $0 \le j \le n - 1$; the other composition works in the same way. So each row of H contains every entry of a exactly once, and has squared length $a_0^2 + \cdots + a_{n-1}^2 = n$.

Dot products of rows are periodic correlations

The dot product (Section 10.4) of row i with row ℓ is

$$\sum_{j=0}^{n-1} a_{(j-i) \bmod n}\, a_{(j-\ell) \bmod n}.$$

For example, when $n = 4$, row 0 is (a_0, a_1, a_2, a_3) and row 1 is (a_3, a_0, a_1, a_2), so their dot product is $a_0a_3 + a_1a_0 + a_2a_1 + a_3a_2$, which is exactly C_3 as written out in Section 14.2. For the matrix H above it is $-1 + 1 + 1 - 1 = 0$.

In general, put $k = (j - i) \bmod n$. By the bijection above, we may sum over k instead of j: the terms are the same, in a different order. For the second factor, $k \equiv j - i \pmod n$ gives

$$j - \ell = (j - i) + (i - \ell) \equiv k + (i - \ell) \pmod n.$$

Congruent integers have the same remainder, so $(j - \ell) \bmod n = (k + i - \ell) \bmod n$. Now let $s = (i - \ell) \bmod n$. Then $s \equiv i - \ell \pmod n$, so $(k + i - \ell) \bmod n = (k + s) \bmod n$ for the same reason, and the dot product becomes

$$\sum_{k=0}^{n-1} a_k\, a_{(k+s) \bmod n} = C_s.$$

Proposition 14.5. *Let a be a sign sequence of length n, and let H be its circulant matrix.*

(a) For all rows i and ℓ, the dot product of row i with row ℓ equals $C_{(i-\ell) \bmod n}(a)$.

(b) Any two different rows of H are orthogonal exactly when $C_s(a) = 0$ for every shift s with $1 \le s \le n - 1$.

Proof. Part (a) is the calculation above. For part (b), note first that if $i \neq \ell$, then $i - \ell$ lies between $-(n-1)$ and $n - 1$ and is not zero, so it is not a multiple of n, and the shift $(i - \ell) \bmod n$ is not zero. So by part (a), the dot product of two different rows is some C_s with $1 \le s \le n - 1$, and if all these C_s are zero, different rows are orthogonal. Conversely, if any two different rows are orthogonal, then each C_s with $1 \le s \le n - 1$ is zero, because by part (a) it is the dot product of the different rows $i = s$ and $\ell = 0$, as $(s - 0) \bmod n = s$. □

Circulant Hadamard matrices

Recall from Section 10.6 that a square matrix H with entries ± 1 is a *Hadamard matrix* when $HH^{\mathsf{T}} = nI$, where n is the number of rows; this number n is called the *order* of the matrix. The diagonal entries of this identity say that every row has squared length n, which is automatic for entries ± 1, and the others say that different rows are orthogonal. So a matrix with entries ± 1 is a Hadamard matrix exactly when any two different rows are orthogonal.

A *circulant Hadamard matrix* is a Hadamard matrix that is also circulant. Since every circulant matrix is the circulant matrix of its own first row, Proposition 14.5(b) and the previous paragraph give:

> A circulant Hadamard matrix of order n exists exactly when some sign sequence of length n has $C_s = 0$ for every shift s with $1 \le s \le n-1$.

Since $(1, 1, 1, -1)$ has $C_1 = C_2 = C_3 = 0$, the matrix H above is a circulant Hadamard matrix of order four.

Habit 6 (Section 6.6) asks what a change of description preserves. Here nothing is lost: we have the same question in two languages, one suggesting sums and shifts, the other dot products and orthogonality. The next two sections use each in turn.

14.4 The sum of all correlations reveals a square

Which lengths n allow a sign sequence whose periodic correlations at all nonzero shifts are zero? A first restriction comes from adding up all the periodic correlations, including C_0: the total depends only on the sum of the entries,

$$r = \sum_{j=0}^{n-1} a_j = a_0 + a_1 + \cdots + a_{n-1}.$$

Each row of the circulant matrix contains every entry of a once, so r is also the sum of the entries in any row, and we call it the *row sum*. For $(1, 1, 1, -1)$, the row sum is $r = 2$.

Look first at $n = 3$, with unknown entries a_0, a_1, a_2:

$$\begin{aligned} C_0 &= a_0a_0 + a_1a_1 + a_2a_2, \\ C_1 &= a_0a_1 + a_1a_2 + a_2a_0, \\ C_2 &= a_0a_2 + a_1a_0 + a_2a_1. \end{aligned}$$

Together these contain each of the nine products a_ja_k, with j, k between 0 and 2, exactly once (with a_0a_1 and a_1a_0 counted as different products). Adding the display row by row gives $C_0 + C_1 + C_2$. Adding it column by column instead, the first column is $a_0a_0 + a_0a_1 + a_0a_2 = a_0(a_0 + a_1 + a_2)$, and the other two are $a_1(a_0 + a_1 + a_2)$ and $a_2(a_0 + a_1 + a_2)$. So

$$C_0 + C_1 + C_2 = (a_0 + a_1 + a_2)^2 = r^2.$$

The same idea works for every n: the total is a table of n^2 products, one row for each shift s and one column for each position j, and we may add it column by column instead of row by row:

$$\begin{aligned}\sum_{s=0}^{n-1} C_s &= \sum_{s=0}^{n-1}\sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n} = \sum_{j=0}^{n-1}\sum_{s=0}^{n-1} a_j a_{(j+s) \bmod n} \\ &= \sum_{j=0}^{n-1} a_j \left(\sum_{s=0}^{n-1} a_{(j+s) \bmod n}\right) \\ &= \sum_{j=0}^{n-1} a_j \left(\sum_{k=0}^{n-1} a_k\right) = \sum_{j=0}^{n-1} a_j\, r = r\sum_{j=0}^{n-1} a_j = r^2.\end{aligned}$$

- The first line writes out each C_s and changes the order of summation.
- The second pulls out a_j, which does not depend on s.
- The third is the key step. Fix j. As s runs through $0, 1, \ldots, n-1$, the position $(j+s) \bmod n$ runs through $j, \ldots, n-1$ and then wraps around to $0, \ldots, j-1$, visiting every position once. So the inner sum is the sum of all the entries in a different order, which is r.
- Finally, r does not depend on j, so it comes out of the outer sum.

For $(1, 1, 1, -1)$ the identity says $4 + 0 + 0 + 0 = 2^2$. It holds for every sequence: $(1, 1, -1, -1)$ has $r = 0$ and $C_0 = 4$, $C_1 = 0$, $C_2 = -4$, $C_3 = 0$, which add up to $0 = 0^2$.

The identity holds for any real numbers $a_0, \ldots, a_{n-1}$; the signs are used only afterward, to know that $C_0 = n$ and that r is an integer. Tracking which hypothesis is used where, as Habit 3 (Section 3.4) asks, shows which parts of an argument survive in a more general setting.

Now suppose that $C_s = 0$ for every shift s with $1 \le s \le n-1$. Then the left side is just $C_0 = n$, so

$$n = r^2. \tag{14.2}$$

Since r is a sum of signs, it is an integer, so n must be a perfect square. By Proposition 14.5(b), the first row of a circulant Hadamard matrix of order n is such a sequence, so the order of every circulant Hadamard matrix is a perfect square: in the language of Section 2.1, this is a *necessary* condition, and it covers every length at once.

Pause and decide. The computer search of Section 14.6 covers the lengths $1 \le n \le 13$. Which of these lengths pass the square test? Has one of them already been ruled out by another argument?

Check. The perfect squares in this range are 1, 4, and 9. Length 9 is odd, so by the pause at the end of Section 14.2 no periodic correlation there can be zero: passing the test only means that this test cannot rule the length out. Lengths 1 and 4 do have such sequences: (1), vacuously, as there are no shifts s with $1 \le s \le 0$, and $(1, 1, 1, -1)$.

14.5 Orthogonality also forces divisibility by four

The square condition of Section 14.4 relied on the circulant pattern. The next condition holds for every Hadamard matrix of order greater than 2, circulant or not. The restriction on the order is

needed: the 1-by-1 matrix (1) and the matrix

$$\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

of Section 10.6 are Hadamard matrices of orders 1 and 2, and neither order is divisible by four.

Lemma 14.6. *If a Hadamard matrix has order $n > 2$, then 4 divides n.*

Proof. We replace the matrix by a simpler Hadamard matrix of the same order and look closely at three of its rows. Two kinds of change keep a matrix Hadamard.

- *Multiplying a column by -1.* In the dot product of two rows, the term from that column is a product of two entries that have both changed sign, so it is unchanged, because $(-x)(-y) = xy$; the other terms are not touched. So every dot product of two rows stays the same, and the entries are still ± 1.
- *Rearranging the columns.* This applies the same rearrangement to every row, so the dot product of two rows consists of the same terms as before, added in a different order.

Neither change alters n, so it is enough to prove that 4 divides n for the new matrix. The new matrix is usually not circulant, but the proof uses only that the entries are ± 1 and that different rows are orthogonal.

Step 1: make the first row consist of ones. For each column whose entry in the first row is -1, multiply that column by -1.

Step 2: the second row has as many plus signs as minus signs. Since $n > 2$, the matrix has at least three rows; this step uses the second and Step 4 the third. The second row is orthogonal to the first, which consists of ones, so their dot product, which is just the sum of the entries of the second row, is zero. If the second row has p entries equal to 1 and $n - p$ equal to -1, this sum is $p - (n - p) = 2p - n$, so $n = 2p$ is even, and the second row has $h = n/2$ entries of each sign.

Step 3: sort the second row. Rearrange the columns so that the h columns in which the second row has a 1 come first. The first row is unchanged, and the second row is now

$$(\underbrace{1, \ldots, 1}_{h}, \underbrace{-1, \ldots, -1}_{h}).$$

Step 4: look at the third row. Let x be the sum of the first h entries of the third row, and y the sum of its last h entries. Its dot product with the first row is $x + y$, and with the second row it is $x - y$, because the second row multiplies the first h entries by 1 and the last h by -1. The third row is orthogonal to both, so

$$x + y = 0 \qquad \text{and} \qquad x - y = 0.$$

Adding the two equations gives $2x = 0$, so $x = 0$.

Step 5: count again. So the first h entries of the third row are signs with sum zero. As in Step 2, if q of them equal 1, their sum is $q - (h - q) = 2q - h$, so $h = 2q$. Therefore $n = 2h = 4q$, and 4 divides n. □

To see the proof in action, apply it to the circulant Hadamard matrix H of Section 14.3. Its first row is $(1, 1, 1, -1)$, so Step 1 multiplies the last column by -1, giving

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{pmatrix}.$$

The second row, $(-1, 1, 1, -1)$, has its plus signs in columns 1 and 2 (counting from 0), so Step 3 uses the column order $1, 2, 0, 3$:

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix}.$$

The third row is $(-1, 1, 1, -1)$; its halves $(-1, 1)$ and $(1, -1)$ both add up to zero, as Step 4 predicts, and the first half contains $q = 1$ plus sign, so $n = 4q = 4$. The rows of the final matrix are still orthogonal, but the matrix is no longer circulant: its second row is not its first row moved one place to the right.

We now combine the two necessary conditions.

Proposition 14.7. *If a circulant Hadamard matrix has order n, then $n = 1$ or $n = 4u^2$ for some positive integer u.*

Proof. Let a be the first row of the matrix. The matrix is the circulant matrix of a, so by Proposition 14.5(b), $C_s(a) = 0$ for every shift s with $1 \le s \le n - 1$. Equation (14.2) therefore gives $n = r^2$, where the row sum r is an integer.

If $n = 1$, there is nothing more to prove. Otherwise $n \ge 2$, and $n \ne 2$, because 2 is not the square of an integer: if $|r| \le 1$ then $r^2 \le 1$, and if $|r| \ge 2$ then $r^2 \ge 4$. Hence $n > 2$, and Lemma 14.6 shows that 4 divides $n = r^2$. In particular r^2 is even, so r is even: if r were odd, say $r = 2k + 1$ for an integer k, then $r^2 = 4k^2 + 4k + 1$ would be odd. Write $|r| = 2u$ with u a nonnegative integer. Since $n = r^2$ is positive, $r \ne 0$, so $u \ge 1$. Then $n = |r|^2 = 4u^2$. □

The proposition allows the orders

$$1,\ 4,\ 16,\ 36,\ 64,\ 100,\ \ldots.$$

For the first two we have examples, (1) and H. About $16, 36, 64, \ldots$ it says nothing either way, since a necessary condition can only rule orders out, never produce a matrix. These orders mark the boundary between what our elementary arguments settle and what needs new ideas.

14.6 An exact finite experiment

A sign sequence of length n is fixed by n independent choices of sign, so there are 2^n of them, and for small n a computer can examine every one. The table below shows the result of such an exhaustive search for $1 \le n \le 13$, produced directly from the definitions by the short program in Section B.3; only integers are involved, so the arithmetic is exact. For each length the search counts

- the Barker sequences;
- the sequences with $C_s = 0$ for every shift s with $1 \le s \le n - 1$, which by Section 14.3 are exactly the first rows of circulant Hadamard matrices.

Each sequence is counted separately: a sequence, its negative, its reversal (the same entries in the opposite order), and its rotations all count as different sequences whenever they are different lists.

Length n	Barker sequences	Sequences with $C_s = 0$ for $1 \le s \le n - 1$
1	2	2
2	4	0
3	4	0
4	8	8
5	4	0
6	0	0
7	4	0
8	0	0
9	0	0
10	0	0
11	4	0
12	0	0
13	4	0

For these thirteen lengths the search examines every case, so, provided the program is correct, the table is complete. About any other length it says nothing, and the search cannot be pushed much further, because each additional position doubles the work. Length 64, which Proposition 14.7 leaves open, has $2^{64} > 1.8 \cdot 10^{19}$ sign sequences; at a billion per second, that single length would take more than 580 years. And even a far faster computer could check only finitely many of the infinitely many lengths. Only a proof can settle every length.

Every count agrees with what we have proved, and several patterns are explained by our results.

- The last column is zero at every length except 1 and 4, as Proposition 14.7 predicts: below 16 it allows only the orders 1 and 4.
- For the odd lengths greater than 1 there is a simpler explanation (the pause at the end of Section 14.2): every periodic correlation is odd, so none can be zero.
- Every count is even. Negating every sign changes no correlation, because each is a sum of pair products and negation does not change pair products (Section 6.5; Exercise 14.2). So whenever a is counted in a column, so is $-a$, and $-a \neq a$, since $a_0 = -a_0$ would force $a_0 = 0$. Hence the sequences counted in each entry split into pairs $\{a, -a\}$.

Other features do not follow from anything proved so far, such as the absence of Barker sequences of lengths 6, 8, 9, 10, and 12; for the even lengths among these, Section 14.7 quotes a known result that accounts for the zeros.

14.7 What current research adds

The question of this chapter is a famous research problem. The *circulant Hadamard conjecture*, usually attributed to H. J. Ryser and open since the 1960s, states that there is no circulant Hadamard matrix of order greater than 4: apart from the lengths 1 and 4, no sign sequence has all of its periodic correlations at nonzero shifts equal to zero. Researchers have since proved restrictions on the possible orders much stronger than Proposition 14.7, but none rules out all of them.

Barker sequences have a related history. Since the early 1960s it has been known that a Barker sequence of odd length has length at most 13; in our table, the odd lengths with Barker sequences are exactly 1, 3, 5, 7, 11, and 13.

For even lengths, the Barker question becomes a question about circulant Hadamard matrices. It has long been known that when $n > 2$ is even, every Barker sequence of length n also has $C_s = 0$ for every shift s with $1 \le s \le n - 1$, so its circulant matrix is a Hadamard matrix. We do not prove this, but you can check it at length four: the eight Barker sequences of length four ($(1, 1, 1, -1)$, its three other rotations, and the negatives of these four) are exactly the eight sequences in the last column. This result explains the zeros in the Barker column at the even lengths 6, 8, 10, and 12: a Barker sequence of such a length would be the first row of a circulant Hadamard matrix of that order, which Proposition 14.7 rules out because none of these lengths is 1 or of the form $4u^2$. Likewise, a Barker sequence of even length greater than 4 would give a circulant Hadamard matrix of order greater than 4, so the conjecture, if true, rules out such sequences too; whether they exist has also been open for decades.

In September 2026, a preprint (a research paper made public before a journal has reviewed it, Section 7.7) claimed a proof of the circulant Hadamard conjecture. It was released in OpenAI's public mathematics repository, together with a formalization in Lean [4]. Lean, introduced in Chapters 15 and 16, is a *proof assistant*: a computer program that checks every step of a proof written in its own precise language. A *formalization* of a theorem is a version of its statement and proof written in such a language.

According to the repository's description, the formalized statement says that a real circulant Hadamard matrix of positive order n exists exactly when $n = 1$ or $n = 4$. The word "real" separates the matrices of this chapter, with entries the real numbers ± 1, from a related family with complex entries. A second formalized statement, described there as a consequence, says that a sign sequence of positive even length whose aperiodic correlations at all nonzero shifts have absolute value at most one, that is, a Barker sequence of even length, must have length 2 or 4. The description also notes that the preprint's classification of Barker sequences of odd length is not part of the formalized statements.

Nothing in this chapter proves these claims: our experiment stops at length 13, and Proposition 14.7 still permits the orders $16, 36, 64, \ldots$, whose exclusion requires far more mathematics than this book covers. What the chapter gives you is the vocabulary to read the claims precisely, and the exact list of orders that a complete proof must rule out. When you meet a summary of a result like this, whether in a news report or from an AI assistant, first check which statement was proved: which matrices, which kind of correlation, which lengths. Chapter 17 returns to this example and describes how to begin reading a proof whose background you do not yet have.

Exercises

Exercises 14.1–14.4 are core exercises, and Exercises 14.5 and 14.6 are transfer exercises. Complete solutions are in Appendix A, Section A.14.

Exercise 14.1. For the sequence $(1, 1, -1)$, compute the aperiodic correlations A_1 and A_2 and the periodic correlations C_1 and C_2 directly from their definitions. Check that your answers satisfy equation (14.1). Is $(1, 1, -1)$ a Barker sequence?

Exercise 14.2. Let a be a sign sequence of length n, and let $-a = (-a_0, -a_1, \ldots, -a_{n-1})$ be its negative. Prove that $A_s(-a) = A_s(a)$ for every shift s with $1 \le s \le n-1$, and that $C_s(-a) = C_s(a)$ for every shift s with $0 \le s \le n-1$.

Exercise 14.3. Let H be the circulant matrix of $(1, 1, 1, -1)$ from Section 14.3, with rows and columns numbered from 0 to 3. Compute the entries of HH^{T} in positions $(0, 2)$, $(1, 3)$, and $(2, 2)$ directly, as dot products of rows. Check that they agree with $HH^{\mathsf{T}} = 4I$, and name the periodic correlation C_s that each of the two off-diagonal entries equals.

Exercise 14.4. Let a be a sign sequence of length n. For which shifts s are both sides of the equation $C_{n-s} = C_s$ defined? For those shifts, prove the equation by renaming the summation index in the sum that defines C_{n-s}.

Exercise 14.5. Use the two necessary conditions of this chapter (the order of a circulant Hadamard matrix is a perfect square, and a Hadamard matrix of order greater than 2 has order divisible by four) to decide which of 8, 9, 16, 25, 36 could be the order of a circulant Hadamard matrix. Does your answer show that a circulant Hadamard matrix of any of these orders exists?

Exercise 14.6. A computer examines all 2^n sign sequences of one fixed length n, looking for sequences with $C_s = 0$ for every shift s with $1 \le s \le n-1$. Explain why finding one such sequence proves that a circulant Hadamard matrix of order n exists, while finding none proves only that no circulant Hadamard matrix of this one order n exists. What would it take to prove that there is no circulant Hadamard matrix of any order greater than 4?

CHAPTER 15

Lean I: reading the state of a proof

> **Overview**
> Meet the proof assistant Lean and see the proof states of Chapter 4 written in a formal language. Before reading the steps of a Lean proof, read what the theorem says: its parameters, including its hypotheses, and the claim after its colon. Write small Lean proofs in which the ordinary argument stays visible.

15.1 A checker is not the same as an explanation

Lean is a programming language and a *proof assistant*: a program that checks proofs. You write a statement and its proof in a precise formal language, and Lean reports an error at any step that does not follow from what came before. If Lean accepts the whole proof without any error or warning, every step has been checked.

Lean is built around one idea: every expression has a *type*, which says what kind of object it is. The number 3 has the type of natural numbers. Propositions are types too, and a proof of a proposition is an object of that type, so checking a proof comes down to checking that an object really has the type claimed for it. Proofs are usually written with *tactics*, instructions that build the proof object step by step, much as the sentences of a written proof build an argument [10, 12].

Checking a proof is not the same as explaining it. When Lean accepts a proof, you know that every step is valid, but not why the argument works, which step carries the main idea, or whether the statement says what you meant. That understanding is still your job, especially when a formal proof comes from an AI assistant: a proof that Lean accepts can still be a proof of the wrong statement (Section 16.5). So every Lean proof in this chapter is placed beside the ordinary argument it encodes.

The aim is not a catalogue of tactics but the proof state of Section 4.1, made explicit: at each moment, which objects and facts are available, and what remains to be shown? The examples prove facts you already know from Chapters 2 and 3, so that a difficulty with Lean's notation can be told apart from a difficulty with the mathematics.

You can read this chapter without installing anything. To run the examples, install Lean by following the official instructions [9], which set up the free editor VS Code with its Lean 4 extension. The examples in Chapters 15 and 16 are written for Lean 4.19.0 and use only the libraries that come with Lean, not the large mathematical library Mathlib.

What a Lean file looks like

A Lean file is a text file whose name ends in `.lean`, such as `proofs.lean`. It contains *declarations*, which define new objects or state and prove results. All the examples in Chapters 15 and 16 can go into one file of this shape:

```
import Std

namespace MathCourse

-- the definitions and theorems go here

end MathCourse
```

The line `import Std` makes Lean's standard library available. Every name declared between `namespace MathCourse` and `end MathCourse` receives the prefix `MathCourse`, which keeps our names from colliding with names in Lean's libraries; this `end` has nothing to do with the end of a proof. Everything from `--` to the end of a line is a *comment*, which Lean ignores.

In the editor, Lean checks the file as you type. When the cursor is inside a proof, a panel beside the file, called the *infoview*, shows the proof state at that point: the facts that are available and what remains to be proved. Appendix B explains how to set up such a file and how to check all of it at once from a terminal.

15.2 Reading Lean's basic notation

One symbol appears in almost every line of Lean: the colon. An expression `a : T` says that `a` is an object of type `T`; read the colon as "is a" or "has type." Because propositions are types and their proofs are objects of those types, the one symbol serves three purposes:

- `n : Nat` says that n is a natural number. The type `Nat` is Lean's name for $\mathbb{N}$, and, as in this book, it includes zero.
- `P : Prop` says that P is a proposition, in the second sense of Section 1.5: a statement that is either true or false, whether or not anyone has proved it.
- `hp : P` says that `hp` is a proof of P, also called *evidence* for P. When it is assumed rather than constructed, it is a *hypothesis*; by convention, names of hypotheses begin with `h`.

Lean writes function application with a space instead of parentheses: `f x` means $f(x)$. Parentheses only group, so `f (x + 1)` means $f(x+1)$, while `f x + 1` means $f(x)+1$. A function can also be written without a name: `fun x => x + 1` is the function $x \mapsto x+1$, and `(fun x => x + 1) 2` gives `2 + 1`, which is 3.

Do not confuse `:=` with `=`. The symbol `:=` means "is defined as" or "is proved by": it attaches a definition to a name, or a proof to a theorem. The sign `=` means equality inside statements, as on paper.

Reading a declaration by its parts

Here is a definition:

```
def twice (n : Nat) : Nat := n + n
```

The word `def` announces a definition, named `twice`. The part `(n : Nat)` declares the input, a natural number `n`, the `: Nat` after it declares the type of the output, and after `:=` comes the rule. So `twice` is the function $\mathbb{N} \to \mathbb{N}$ given by $n \mapsto n + n$.

A theorem is declared in the same way: the word `theorem` and the theorem's name, then its *parameters*, if any, which are the objects and hypotheses it talks about, each in parentheses like `(n : Nat)`. A colon follows, and after it the *claim*, the proposition that the theorem asserts. Last come `:=` and the proof; the word `by` says that the proof is given by tactics on the lines that follow. This theorem has no parameters:

```
theorem twice_two : twice 2 = 4 := by
  rfl
```

To check such a claim, Lean can *reduce* expressions: it replaces a defined name by its definition and carries out arithmetic on specific numbers. Here `twice 2` reduces to `2 + 2` and then to `4`. The tactic `rfl` (short for *reflexivity*, the fact that every object equals itself) proves an equality whose two sides reduce to the same expression, as both sides of `twice 2 = 4` do. The definition is a choice, with nothing to prove; the theorem is a claim about that choice. Had we defined `twice n` to be `n + 1`, the claim would amount to the false equation 3 = 4, and `rfl` would fail.

The tactic `rfl` has a limit. The equation `twice n = 2 * n` holds for every natural number n, and each instance, such as `twice 5 = 2 * 5`, is proved by `rfl`, because both sides reduce to `10`. But for a variable n, the left side reduces only to `n + n` and the right side stays `2 * n`. These are different expressions, so `rfl` fails; a proof for every n needs an argument that works for a variable, such as induction (Chapter 5).

Renaming `twice` everywhere would not change what is proved, because names carry no mathematical content: Lean would accept a theorem called `all_problems_solved` whose claim is only `0 = 0`. To know what a theorem says, read its parameters, which include its hypotheses, and the claim after its colon.

A proposition and a proof of it are different objects

The expression `2 + 2 = 5` is a perfectly good proposition, an object of type `Prop`; but it is false, so nothing has the type `2 + 2 = 5`. A proof `hp : P` can exist only when P is true, so `hp : P` among a theorem's parameters is the assumption that P holds.

An implication $P \Rightarrow Q$ can be used as a rule: give it a proof of P, and it returns a proof of Q. Lean writes it as `P -> Q`, with the same arrow as in $f : A \to B$, because a proof of `P -> Q` is a function that turns proofs of P into proofs of Q. So if `h : P -> Q` and `hp : P`, then `h hp` is a proof of Q: the inference "P holds, and P implies Q; therefore Q holds." This way of reading statements and proofs is called *propositions as types*, and it is all the theory this chapter needs.

Pause and decide. Suppose `hpq : P -> Q`, `hqr : Q -> R`, and `hp : P`. What is the type of `hpq hp`? Which expression built from these three is a proof of R? Why does Lean reject `hqr hp`?

Check. The application `hpq hp` has type `Q`. Feeding it to `hqr` gives `hqr (hpq hp)`, a proof of R; the parentheses make `hpq hp` a single input. Lean rejects `hqr hp` with an "application type mismatch" error, because `hqr` accepts only a proof of Q, while `hp` is a proof of P.

15.3 An implication as a change of goal

Our first proof with more than one step proves an implication. Suppose P, Q and R are propositions such that $P \Rightarrow Q$ and $Q \Rightarrow R$; we want to prove $P \Rightarrow R$. On paper: assume P; then Q holds, because $P \Rightarrow Q$; then R holds, because $Q \Rightarrow R$. In Lean:

```
theorem implication_chain (P Q R : Prop)
    (hpq : P -> Q) (hqr : Q -> R) : P -> R := by
  intro hp
  have hq : Q := hpq hp
  exact hqr hq
```

Read the statement first. Several names before one colon, as in `(P Q R : Prop)`, are declared at once, so the parameters are three propositions and two hypotheses, `hpq` and `hqr`, and the claim is `P -> R`. Like the results of Section 4.5, the theorem is a promise: supply three propositions and proofs of the two implications, and it delivers a proof of `P -> R`.

The statement that remains to be proved is called the *goal*, the "still to show" entry of Section 4.1. At the start, it is the claim `P -> R`.

- `intro hp` does what "assume P" does on paper: it adds a proof `hp` of P to the available facts and leaves the goal `R`.
- `have hq : Q := hpq hp` records an intermediate fact: its name, its type, and, after `:=`, its proof, the rule `hpq` applied to `hp`.
- `exact hqr hq` says that `hqr hq`, the rule $Q \Rightarrow R$ applied to `hq`, proves exactly the current goal `R`. Lean accepts it, and no goal remains.

The table records the proof state after each step.

Step	Facts added	Goal afterwards
Start	`P Q R : Prop`, `hpq : P -> Q`, and `hqr : Q -> R`	`P -> R`
`intro hp`	`hp : P`	`R`
`have hq`	`hq : Q`	`R`
`exact hqr hq`	none	none: the proof is complete

The infoview shows each of these states in Lean's own notation. After `intro hp`, for example, it shows

```
P Q R : Prop
hpq :  P → Q
hqr :  Q → R
hp :  P
⊢ R
```

Each line above the symbol ⊢, called the *turnstile*, lists something available: one or more names, a colon, and a type. The line that begins with ⊢ shows the goal. This is the proof state of Section 4.1, with the "known" list above the "still to show" list. Lean prints `->` as →; in the editor you may type either, and `\to` followed by a space produces →. If you have Lean installed, place the cursor at the end of each line of the proof in turn and compare the infoview with the matching row of the table.

15.4 Conjunction means two obligations

To prove a conjunction $P \wedge Q$ you must prove both P and Q, and from a hypothesis $P \wedge Q$ you may take either half (Section 2.2). The next theorem does both: it proves $Q \wedge P$ from $P \wedge Q$ by taking the two halves apart and putting them back together in the opposite order. On paper: if P and Q both hold, then Q holds and P holds, so $Q \wedge P$.

```
theorem and_swap (P Q : Prop) : And P Q -> And Q P := by
  intro h
  exact And.intro h.right h.left
```

Lean's name for $P \wedge Q$ is `And P Q`; the infoview displays it as $P \wedge Q$, and in the editor `\and` followed by a space produces ∧. After `intro h`, the fact `h : And P Q` is available and the goal is `And Q P`. A proof of a conjunction is a pair of proofs, one for each half: `h.left`, the first component, is a proof of P, and `h.right` is a proof of Q. In the other direction, `And.intro` builds a pair: if `a` is a proof of R and `b` is a proof of S, then `And.intro a b` is a proof of `And R S`. The order of the inputs must match the conjunction being built: `And Q P` needs a proof of Q first, hence `And.intro h.right h.left`.

A second version makes the two obligations visible as two separate goals:

```
theorem and_swap_two_goals (P Q : Prop) :
    And P Q -> And Q P := by
  intro h
  constructor
  exact h.right
  exact h.left
```

The tactic `constructor` applies `And.intro` without supplying its inputs, and each missing input becomes a goal of its own: `And Q P` is replaced by the goals Q and P, which the infoview labels `case left` and `case right`. Each `exact` closes the first goal still open. Without the last line, Lean reports the error "unsolved goals" at the word `by`, where the proof begins, and displays the open goal P.

This is the formal counterpart of a paper proof that must establish two set containments

(Section 3.1), or both directions of an "if and only if" (Section 2.1). After a long first half it is easy to feel finished. Lean tracks every open goal; on paper we must keep that list ourselves.

15.5 Composition of injective functions in Lean

Our next example is Theorem 3.2: if $f : A \to B$ and $g : B \to C$ are injective, then so is $g \circ f$. Stating it needs a little more notation for functions.

Arrows and implicit parameters

If `A` and `B` are types, then `A -> B` is the type of functions from `A` to `B`, and `f : A -> B` is the Lean form of $f : A \to B$. Do not confuse this arrow, which builds a type, with the `=>` in `fun x => x + 1`, which separates the input of one particular function from its rule.

A function of two inputs has two arrows in its type: `A -> B -> C` means `A -> (B -> C)`, so such a function takes an input of type `A` and returns a function from `B` to `C`. For example, Lean's addition `Nat.add` has type `Nat -> Nat -> Nat`; `Nat.add 2` is the function $n \mapsto 2 + n$, and `Nat.add 2 3` is 5. This is why several inputs are written one after another, and why `f x y` means `(f x) y`.

The sets A, B, C of a mathematical statement become types: just as `P : Prop` says that P is a proposition, `A : Type` says that `A` is a type of ordinary mathematical objects, such as `Nat`. Below, such parameters are written in curly braces, as in `{A B : Type}`. Braces mark *implicit* parameters, which Lean works out from the other inputs: when we write `Inj f`, Lean already knows that `f : A -> B`, so we need not write `A` and `B`. Parameters in round brackets are *explicit* and must be written at every use. Implicit parameters are still part of the statement, just not written out each time.

The definition and the theorem

We write the definition of injectivity from Section 3.3 ourselves, so that every part of it is visible.

```
def Inj {A B : Type} (f : A -> B) : Prop :=
  forall x y : A, f x = f y -> x = y

theorem inj_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hf : Inj f) (hg : Inj g) :
    Inj (fun x => g (f x)) := by
  intro x y h
  have hxy : f x = f y := hg (f x) (f y) h
  exact hf x y hxy
```

The definition makes `Inj f` a proposition, as the `: Prop` says. The word `forall` is Lean's keyboard spelling of ∀, which the infoview prints, so `forall x y : A` means "for all x and y in A"; the comma ends the list of variables. Lean reads each equation as a unit before the arrow, so

`f x = f y -> x = y` means `(f x = f y) -> (x = y)`. Altogether, `Inj f` is the statement $\forall x, y \in A,\ f(x) = f(y) \Rightarrow x = y$ used in Chapter 3.

The theorem has implicit parameters `A`, `B`, `C`, explicit functions `f` and `g`, and hypotheses `hf : Inj f` and `hg : Inj g`. Its claim is `Inj (fun x => g (f x))`, where `fun x => g (f x)`, which sends x to $g(f(x))$, is $g \circ f$ written without a name.

The proof uses one more fact: a proof of a "for all" statement is used like a function. It accepts any element and returns a proof of the statement for that element, just as a proof of `P -> Q` turns proofs of P into proofs of Q. Since Lean can replace `Inj g` by its definition, `hg` is such a rule. Applied to two elements `b1` and `b2` of `B`, it gives a proof of `g b1 = g b2 -> b1 = b2`, and applying that to a proof `e` of `g b1 = g b2` gives `hg b1 b2 e`, a proof of `b1 = b2`. So `hg` takes three inputs, two elements of `B` and evidence that their values under g agree, and returns evidence that the two elements are equal. The same holds for `hf`, with elements of `A`.

Now the proof, line by line.

- Replaced by its definition, the goal `Inj (fun x => g (f x))` is a "for all" statement followed by an implication, so `intro` can look inside it. On a "for all" statement, `intro x` does what "take any x" does on paper: it adds an arbitrary element `x` and removes the quantifier from the goal. On an implication, `intro` assumes the hypothesis, as in Section 15.3. So `intro x y h` introduces arbitrary `x` and `y` in `A` and a hypothesis `h` that their outputs under the composition agree, and the goal becomes `x = y`. On paper: "Take any $x, y \in A$ and assume $g(f(x)) = g(f(y))$."
- In `hg (f x) (f y) h`, the two elements of `B` are `f x` and `f y`, not `x` and `y`, and `h` is the evidence that their values under g agree. The result, recorded as `hxy`, proves `f x = f y`. On paper: "Injectivity of g gives $f(x) = f(y)$."
- Finally, `hf x y hxy` proves exactly the goal `x = y`. On paper: "Injectivity of f then gives $x = y$."

The Lean proof takes the same two steps as the proof of Theorem 3.2, in the same order.

When Lean rejects an application, compare the types of its pieces. Skipping the middle step and writing `exact hf x y h` gives an "application type mismatch" error, because `hf` needs evidence that `f x = f y`, while `h` is evidence about outputs of g. The missing step is exactly what injectivity of g supplies.

In the editor, Lean displays the hypothesis `h` as

```
h : (fun x => g (f x)) x = (fun x => g (f x)) y
```

rather than as `g (f x) = g (f y)`. The expression `(fun x => g (f x)) x` means "apply the rule $x \mapsto g(f(x))$ to the input x," which evaluates to `g (f x)`. Inside `fun x => g (f x)`, the letter `x` is only a placeholder, like the x in $x \mapsto g(f(x))$; the `x` after the closing parenthesis is the element of `A` that `intro` introduced. Evaluating an application of a `fun` expression by putting the actual input in place of the placeholder is called *beta-reduction*, one more kind of the reduction of Section 15.2. When Lean checks that a proof has the right type, it treats a proposition and its beta-reduced form as the same, which is why `hg` accepts `h`. Some tactics, however, look for the

literal written form of an expression; Section 16.2 shows a case where we must first restate the goal in its beta-reduced form.

15.6 Existence statements, and the type of a number

To prove an existence statement, we name a witness and check that it has the required property (Section 4.1). To prove that some natural number n satisfies $n + 1 = 4$, we offer the witness 3 and check that $3 + 1 = 4$. In Lean:

```
theorem a_witness : Exists (fun n : Nat => n + 1 = 4) := by
  exact Exists.intro 3 rfl
```

Lean writes $\exists n \in \mathbb{N} : n + 1 = 4$ as `Exists (fun n : Nat => n + 1 = 4)`, which the infoview displays as `∃ n, n + 1 = 4`. The input to `Exists` is a function that turns each natural number n (that is what `n : Nat` says) into the proposition $n + 1 = 4$, true for $n = 3$ and false for $n = 2$. The whole expression says that this proposition is true for at least one natural number.

To prove the statement, `Exists.intro` takes two inputs: a witness, here 3, and a proof that the witness has the property, which is a proof of `3 + 1 = 4`. That proof is `rfl`, used this time inside an expression rather than as a tactic; there it names the proof that an object equals itself. Lean accepts it because `3 + 1` reduces to `4`. With the witness 2, Lean would ask for a proof of the false equation `2 + 1 = 4`, and `rfl` would fail.

To use a hypothesis that something exists, we must first name the object, so that we can refer to it, and record its property, as a paper proof does when it says "let n be such a number, so that $n + 1 = 4$"; Section 16.2 does this with the tactic `cases`.

The type of a number decides what subtraction means

The witness 3 had to be a natural number, because the statement declared `n : Nat`. The type also decides what the arithmetic symbols mean. A natural number cannot be negative, so Lean's natural-number subtraction is *truncated* at zero: `a - b` is the usual difference if $a \geq b$, and 0 if $a < b$. The notation `(e : T)` tells Lean to read `e` as an object of type `T`, and so decides which subtraction is meant: `(7 - 10 : Nat)` is 0, while `(7 - 10 : Int)` is -3, because `Int`, Lean's name for $\mathbb{Z}$, contains negative numbers. To see this, put the line `#eval (7 - 10 : Nat)` in your file, outside any proof: the command `#eval` asks Lean to compute an expression, and the result `0` appears in the infoview when the cursor is on that line. The line `#eval (7 - 10 : Int)` gives `-3`.

Truncated subtraction breaks familiar laws: $(a - b) + b = a$ holds for all integers, but in `Nat` it fails whenever $a < b$, since `(7 - 10) + 10` is $0 + 10 = 10$. So an argument that subtracts a larger number from a smaller one cannot be copied unchanged into natural-number subtraction. This is one reason why the statement contract of Habit 2 (Section 2.6) records the domain of every variable. When an assistant offers to repair a formal proof, check whether it has quietly changed the kind of number being used.

15.7 Three ways to use an equation

The first way to use an equation is to apply the same function to both sides: if $x = y$, then $f(x) = f(y)$. In Lean, if `h` proves `x = y`, then `congrArg f h` proves `f x = f y`; the name is short for "congruence in the argument," the *argument* of a function being its input. This works for every function. Injectivity goes the other way, from $f(x) = f(y)$ to $x = y$, and holds only for some functions: squaring on $\mathbb{R}$ sends 1 and -1 to the same output (Section 3.3).

The second way is to substitute. If `h : a = b`, the tactic `rw [h]` rewrites the goal, replacing every occurrence of the left side `a` by the right side `b`. Afterwards `rw` tries `rfl` by itself, so a goal that has become an equation with identical sides is closed at once:

```
theorem add_one_of_eq (a b : Nat) (h : a = b) :
    a + 1 = b + 1 := by
  rw [h]
```

The rewrite turns the goal `a + 1 = b + 1` into `b + 1 = b + 1`, which `rw` then closes; on paper, "since $a = b$, substitute b for a." The brackets may also hold several equations, separated by commas, which `rw` uses one after another.

The third way, for a chain of equalities, is Lean's `calc` block [11]. Each line states one link of the chain, with its justification after `:=`, and Lean combines the links by transitivity: if $a = b$ and $b = c$, then $a = c$. Here is the first half of Exercise 3.5: a function with a left inverse is injective.

```
theorem left_inverse_inj {A B : Type}
    (f : A -> B) (g : B -> A)
    (h : forall x, g (f x) = x) : Inj f := by
  intro x y hxy
  calc
    x = g (f x) := (h x).symm
    _ = g (f y) := congrArg g hxy
    _ = y := h y
```

The hypothesis `h` says that $g(f(x)) = x$ for every x; Lean infers that x ranges over `A`, because `f` is applied to it. Since functions are equal when they agree at every input (Section 3.2), this is the condition $g \circ f = \mathrm{id}_A$ of Exercise 3.5, stated one input at a time. (The exercise calls the left inverse h; here `h` names the hypothesis.) As in Section 15.5, `intro x y hxy` introduces `x`, `y` and `hxy : f x = f y`, leaving the goal `x = y`. The `calc` block is the Lean form of the chain

$$x = g(f(x)) = g(f(y)) = y,$$

where the underscore at the start of a line stands for the right side of the line before.

- The first link, `x = g (f x)`, comes from `h x`, which proves `g (f x) = x`, with the sides in the other order. The suffix `.symm` (for *symmetry*) reverses an equation: if `e` proves `a = b`, then `e.symm` proves `b = a`. The parentheses in `(h x).symm` are needed: Lean would read `h x.symm` as `h` applied to `x.symm` and report an error.
- The second link is `congrArg g hxy`: apply g to both sides of $f(x) = f(y)$.

- The third link, `g (f y) = y`, is `h y`.

Exercises

Exercises 15.1–15.4 are core exercises, and Exercises 15.5 and 15.6 are transfer exercises. Complete solutions are in Appendix A, Section A.15.

Exercise 15.1. In `h : P -> Q`, explain the roles of `h`, `P`, and `Q`. If `hp : P`, what does `h hp` provide?

Exercise 15.2. Write a Lean proof of $P \wedge Q \Rightarrow P$, and explain the one mathematical inference it uses.

Exercise 15.3. Write a Lean theorem stating that some natural number n satisfies $n + 2 = 7$, and prove it by giving a witness.

Exercise 15.4. Explain why `congrArg g h` cannot, in general, recover $x = y$ from `h : g x = g y`. Which hypothesis is missing?

Exercise 15.5. Using `Inj`, state and prove in Lean the first part of Proposition 3.3: if $g \circ f$ is injective, then f is injective. Do not assume that g itself is injective.

Exercise 15.6. Explain why `(7 - 10 : Nat)` and `(7 - 10 : Int)` describe different computations. What lesson does this give for translating a theorem from one kind of number to another?

CHAPTER 16

Lean II: definitions, dependencies, and trust

> **Overview**
>
> Use Lean to bring out the choices hidden in a mathematical statement. Before trusting an accepted proof, read the theorem's hypotheses and the definitions it uses, and find out whether the proof depends on unfinished steps. Learn to repair a failing proof without losing the statement you meant to prove.

16.1 The statement must be right before the proof can help

When Lean accepts a proof without any error or warning, it guarantees one thing: the conclusion, exactly as written in Lean, follows from the hypotheses, exactly as written in Lean. It does not guarantee that the Lean statement says what anyone meant. That statement translates a mathematical sentence, from a paper, an exercise, or a request to an AI assistant, and Lean never sees the original, so it cannot notice a mistranslation. Comparing the formal statement with the intended one is the reader's job, not the proof assistant's [14].

For example, Section 7.7 defined the second out-neighborhood $N_2^+(v)$ as the set of vertices at directed distance exactly two from v. A tempting shortcut is to take all the endpoints w of directed walks $v \to u \to w$. Lean would accept correct proofs about either description, but they describe different sets. In the graph of that section, with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$, the two-step walks from a end at c and d, but the edge $a \to c$ puts c at distance one, so $N_2^+(a) = \{d\}$. Since $|N_1^+(a)| = 2$, the vertex a fails the inequality $|N_2^+(v)| \geq |N_1^+(v)|$ of Seymour's conjecture but passes it with the shortcut set $\{c, d\}$. In general the shortcut set contains $N_2^+(v)$, so it can only make the inequality easier to satisfy, and a "proof of Seymour's conjecture" that used it would prove a weaker statement, as that section warned.

So before reading any tactics, compare the two statements in a table: in one column, list the objects, hypotheses, and conclusion of the mathematical statement; in the other, write what represents each of them in Lean. This is the statement contract of Habit 2 (Section 2.6), written out for a formal statement; Section 16.2 gives a complete example. While you fill it in, check the places where translations most often go wrong; most can be seen in Seymour's conjecture: *every nonempty finite oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.*

- *Domains.* Does each variable range over the intended set? In Section 15.6, the expression $7 - 10$ meant 0 for natural numbers and -3 for integers.

- *Strict and non-strict inequalities.* With $>$ in place of $\geq$, the conjecture would be false: a graph with one vertex and no edges is a nonempty finite oriented graph, and its only vertex has $|N_2^+(v)| = 0 = |N_1^+(v)|$.
- *Nonemptiness.* A graph with no vertices has no vertex with the required property, so without the word "nonempty" the conjecture would be false.
- *Excluded cases.* The conditions $w \neq v$ and $w \notin N_1^+(v)$ in the definition of $N_2^+(v)$ are exactly what the shortcut leaves out.
- *The order of quantifiers.* The conjecture claims that *some* vertex satisfies the inequality, not that every vertex does: in the graph above, a fails it, while the vertex d, which has no out-neighbors, satisfies it with $0 \geq 0$. Likewise, "for every $y \in B$ there is an $x \in A$ with $f(x) = y$" says that f is surjective, whereas "there is an $x \in A$ with $f(x) = y$ for every $y \in B$" asks a single input to be sent to every element of B at once, which is impossible once B has two elements (Section 2.3).

Finding the statement and its definitions

To fill in the table for an unfamiliar Lean theorem, read its parameters and the claim after its colon (Section 15.2). Then look up each definition that the statement uses and read its *body*, the part after its own `:=`. Only then turn to the proof. The name of a definition says no more than the name `all_problems_solved` of a theorem whose claim is only `0 = 0`: a definition called `Second` could describe the second out-neighborhood with or without the conditions $w \neq v$ and $w \notin N_1^+(v)$, and only its body tells you which.

16.2 A complete existence proof in Lean

In Section 15.6 the witness for an existence statement was easy to spot; often it has to be built from other information. Our example is Exercise 3.2: if $f : A \to B$ and $g : B \to C$ are surjective, then so is $g \circ f$. Recall from Section 3.3 that f is surjective when every target has a preimage:

$$\forall b \in B\ \exists a \in A : \qquad f(a) = b.$$

Here is this definition in Lean, followed by the theorem and its proof.

```
def Surj {A B : Type} (f : A -> B) : Prop :=
  forall y : B, Exists (fun x : A => f x = y)

theorem surj_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hf : Surj f) (hg : Surj g) :
    Surj (fun x => g (f x)) := by
  intro c
  cases hg c with
  | intro b hb =>
    cases hf b with
    | intro a ha =>
      refine Exists.intro a ?_
      show g (f a) = c
      rw [ha, hb]
```

The statement

Following Section 16.1, we read the statement first. The body of `Surj` shows the domain `A`, the codomain `B`, and the order of the quantifiers: the target `y` is chosen first, and the input `x` is found afterward, so it may depend on `y`. The table compares the theorem with the mathematical statement.

Mathematics	Lean
sets A, B, C	types `A B C : Type`, as implicit parameters
functions $f : A \to B$ and $g : B \to C$	`f : A -> B` and `g : B -> C`
hypothesis: f is surjective	`hf : Surj f`
hypothesis: g is surjective	`hg : Surj g`
the composition $g \circ f$	`fun x => g (f x)`
conclusion: $g \circ f$ is surjective	`Surj (fun x => g (f x))`

Every part of the mathematical statement has a partner in Lean, and nothing extra has been assumed (Section 16.5 shows what an extra assumption can do).

The proof, line by line

Because `Surj` is defined as a "for all" statement, `intro` can look inside it, as it did with `Inj` in Section 15.5: `intro c` introduces an arbitrary target `c` in `C`, and the infoview shows the goal

```
∃ x, (fun x => g (f x)) x = c
```

As in Section 15.5, the composition is shown applied to `x` but not evaluated: the goal asks for some x in A with $g(f(x)) = c$. Our only sources of elements are the two hypotheses; we start with the one about g, because the outputs of g lie in C, where c lives.

Applied to c, the hypothesis hg gives hg c, a proof that some element of B is sent to c by g. Like all existence evidence, it consists of a witness and a proof that the witness has the property. The lines

```
cases hg c with
| intro b hb =>
```

use the tactic cases, which takes evidence apart. They say: unpack this evidence, call the witness b and the proof of its property hb, and continue with the indented lines below, which form a *branch* of the proof. Inside this branch the facts b : B and hb : g b = c are available, and the goal is unchanged. The word intro after the vertical bar is not the tactic intro. It refers to Exists.intro, the only way to build existence evidence (Section 15.6): | intro b hb says that the evidence was built as Exists.intro b hb, and takes it apart into those two pieces.

The second unpacking does the same with hf b: since b lies in B, the surjectivity of f supplies a : A and ha : f a = b. Inside this second branch we know

$$b \in B, \quad g(b) = c, \qquad a \in A, \quad f(a) = b.$$

The line refine Exists.intro a ?_ proposes a as the witness. The tactic refine works like exact, except that its argument may contain the placeholder ?_ for a part of the proof not yet supplied; that part becomes the new goal. Here it is the proof that a has the required property:

```
(fun x => g (f x)) a = c
```

The line show g (f a) = c replaces this goal by its beta-reduced form (Section 15.5); show accepts a new form of the goal only when Lean can check, by reducing expressions, that it is the same proposition as the old one.

The show line is needed because rw [ha, hb] searches the goal for the written expression f a, the left side of ha, and (fun x => g (f x)) a contains only f x, with the variable x. Without the show line, Lean stops at the rw line with the error

```
tactic 'rewrite' failed, did not find instance of the pattern in the target
    expression
  f a
```

After show, rewriting with ha : f a = b turns the goal into g b = c, and rewriting with hb : g b = c turns it into c = c, which rw closes by itself (Section 15.7).

Pause and decide. Could the proof begin by unpacking hf instead of hg?

Check. No. The hypothesis hf can only be applied to an element of B, and at the start the only element available is c, which lies in C. Lean rejects hf c with an "application type mismatch" error. An element of B becomes available only when unpacking hg c supplies b. The types of the hypotheses force the order of the steps: the composition applies f first, but the proof finds an input to g first.

The ordinary proof beside the Lean proof

Here is the same argument written on paper, as in the solution of Exercise 3.2.

> Let $c \in C$. Since g is surjective, there is $b \in B$ with $g(b) = c$. Since f is surjective, there is $a \in A$ with $f(a) = b$. Then $(g \circ f)(a) = g(f(a)) = g(b) = c$. Since c was arbitrary, $g \circ f$ is surjective.

"Let $c \in C$" is `intro c`, the next two sentences are the two unpackings, and the chain of equalities is `refine`, `show`, and `rw`. The last sentence needs no line in Lean, because `c` was introduced as an arbitrary element. The indentation shows where each name may be used: `b` and `hb` exist only inside the branch that follows `| intro b hb =>`, and `a` and `ha` only inside the branch nested within it. In the written proof, too, "there is $b \in B$ with $g(b) = c$" introduces the name b together with the one fact known about it. Since c may have several preimages under g, the element b need not be unique; the proof uses only $g(b) = c$, so it works whichever of them b stands for. This is the care that Section 4.4 asked for when it distinguished "suppose x is a solution" from "let x be the solution."

16.3 Sets as membership predicates

A *predicate* on a set A is a rule that turns each element of A into a proposition. For example, "n is even" is a predicate on the integers, giving statements such as "6 is even" (true) and "7 is even" (false). In Lean, a predicate on a type `A` is a function of type `A -> Prop`; you met one in Section 15.6, where the input to `Exists` was `fun n : Nat => n + 1 = 4`.

A predicate P on A describes the set $\{x \in A : P(x)\}$ of Section 3.1, and conversely a set $S \subseteq A$ gives the predicate "$x \in S$." So a subset of a type `A` can be represented in Lean by a predicate `s : A -> Prop`, where `s x` means that x belongs to the set; nothing is lost, because for every x the question "does x belong?" has the same answer in both descriptions. For example, the set of even natural numbers is represented by `fun n : Nat => Exists (fun k : Nat => n = 2 * k)`.

Here are intersection and preimage, defined by their membership conditions, with a theorem that connects them:

```
def meet {A : Type} (s t : A -> Prop) : A -> Prop :=
  fun x => And (s x) (t x)

def preimage {A B : Type} (f : A -> B)
    (s : B -> Prop) : A -> Prop :=
  fun x => s (f x)

theorem preimage_meet {A B : Type} (f : A -> B)
    (s t : B -> Prop) (x : A) :
    Iff (preimage f (meet s t) x)
      (meet (preimage f s) (preimage f t) x) := by
  rfl
```

If `s` and `t` describe the sets S and T, then `meet s t` holds at `x` when both `s x` and `t x` hold, so it describes $S \cap T$; and `preimage f s` holds at `x` when $f(x) \in S$, so it describes the preimage $f^{-1}(S)$. Since `Iff P Q` is Lean's way of writing $P \Leftrightarrow Q$, the theorem `preimage_meet` is the identity

$$f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T)$$

from Section 3.4, stated one element at a time: for each `x`, membership in the left side is equivalent to membership in the right side, which is what it means for the two sets to be equal.

Why does `rfl` prove it? The left side `preimage f (meet s t) x` unfolds to `meet s t (f x)` and then to `And (s (f x)) (t (f x))`. The right side `meet (preimage f s) (preimage f t) x` unfolds to `And (preimage f s x) (preimage f t x)` and then to the very same `And (s (f x)) (t (f x))`. Just as `rfl` proves an equality whose two sides reduce to the same expression (Section 15.2), it proves an equivalence whose two sides reduce to the same proposition, unfolding the definitions by itself. These unfoldings are exactly the steps "by definition" of the chain of equivalences in Section 3.4.

Pause and decide. Suppose the right side of `preimage_meet` were written `meet (preimage f t) (preimage f s) x`, with `s` and `t` exchanged. The statement is still true. Would `rfl` still prove it?

Check. No. The right side now unfolds to `And (t (f x)) (s (f x))`, which is equivalent to `And (s (f x)) (t (f x))` but not the same expression, so `rfl` fails, with an error saying that the two sides are "not definitionally equal." A proof would have to swap the two halves of the conjunction in each direction of the equivalence, for instance with the theorem `and_swap` of Section 15.4.

Lean writes the disjunction $P \vee Q$ of Section 2.2 as `Or P Q`. Had `meet` been defined with `Or` in place of `And`, the theorem `preimage_meet` would keep its name and its proof `rfl`, and Lean would accept it; but it would be the statement about unions from Exercise 16.2. The bodies of the definitions, not their names, decide what a statement says.

16.4 Encoding the exclusions in a second out-neighborhood

A directed graph whose vertices form a type `V` can be described in Lean by its edge relation `r : V -> V -> Prop`, a function of two inputs (Section 15.5): it takes vertices `v` and `w` and returns the proposition `r v w`, read as $v \to w$. Nothing in such a relation rules out a loop $v \to v$, which Definition 7.7 forbids, or opposite edges $v \to w$ and $w \to v$, which an oriented graph forbids, so a statement that needs to exclude them must say so in extra hypotheses.

Recall from Section 7.7 that w belongs to $N_2^+(v)$ when three conditions hold: $w \neq v$; w is not an out-neighbor of v; and $v \to u \to w$ for some vertex u. In Lean they become a definition:

```
def Second {V : Type} (r : V -> V -> Prop)
    (v w : V) : Prop :=
  And (Not (w = v)) (And (Not (r v w))
    (Exists (fun u : V => And (r v u) (r u w))))
```

Here `Not P` is the negation of `P`, so `Not (w = v)` expresses $w \neq v$ and `Not (r v w)` says that there is no edge $v \to w$; the last part asks for at least one route $v \to u \to w$, not a unique one.

When the definition is unfolded, for instance with the tactic `unfold Second`, the infoview displays the body with the usual symbols:

```
¬ w = v ∧ ¬ r v w ∧ ∃ u, r v u ∧ r u w
```

Three conventions replace the parentheses: ¬ applies only to the proposition right after it, such as the whole equation `w = v`, and stops at the next ∧; `∃ u,` reaches as far to the right as possible, so it covers both `r v u` and `r u w`; and ∧ groups to the right, so $P \wedge Q \wedge R$ means $P \wedge (Q \wedge R)$. In this book's Lean code we write `And`, `Not`, and `Exists` instead, with parentheses that make the grouping visible.

To read the nesting, give the three conditions temporary names:

$$P:\ w \neq v, \qquad Q:\ \text{there is no edge } v \to w, \qquad R:\ v \to u \to w \text{ for some vertex } u.$$

The definition has the shape `And P (And Q R)`. A proof `h` of it is a pair (Section 15.4) whose first component proves P and whose second component is itself a pair of proofs of Q and R. So `h.left` proves P and `h.right` proves `And Q R`; taking components once more, `h.right.left` proves Q and `h.right.right` proves R. Taking a component of a pair in this way is called a *projection*.

Pause and decide. Take the oriented graph of Section 7.7 with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$, and let $v = a$. For each vertex w, decide which of the conditions P, Q, R hold. For which w does `Second r a w` hold?

Check. The out-neighbors of a are b and c, and neither has an edge to a or to b, so R fails for $w = a$ and for $w = b$. For $w = a$, P fails and Q holds; for $w = b$, P holds and Q fails, because of the edge $a \to b$. For $w = c$, P and R hold (through $a \to b \to c$), but Q fails because of the edge $a \to c$. For $w = d$, all three hold: $d \neq a$, there is no edge $a \to d$, and $a \to c \to d$ is a route. So `Second r a w` holds only for $w = d$, in agreement with $N_2^+(a) = \{d\}$; the vertex c is excluded by Q alone, a condition that the shortcut of Section 16.1 leaves out.

A useful consequence is that a second out-neighbor is never a first out-neighbor:

```
theorem second_not_first {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) : Not (r v w) := by
  exact h.right.left
```

The proof is the projection `h.right.left`, which extracts the proof of Q, exactly the claim `Not (r v w)`. Exercise 16.3 asks you to extract the other two conditions.

Besides a type `V` of vertices and an edge relation `r`, the formal statement of Seymour's conjecture in the repository of Section 7.7 [3] has hypotheses saying that there are finitely many vertices, that there is at least one, and that `r` is oriented. Its condition for second out-neighbors is exactly the one above; its statement file [6], named in the repository's configuration [8], even joins the three conditions by ∧ in the same order. So the piece of the statement that the shortcut gets wrong can be checked on its own, before anyone studies the long proof (Section 17.8 reads the full statement).

16.5 A successful proof can still be irrelevant

Lean accepts the following theorem without any error or warning:

```
theorem impossible_context (h : False) : (0 : Nat) = 1 := by
  exact False.elim h
```

Its conclusion, $0 = 1$, is false, yet the theorem is correct. The hypothesis `h : False` asks for a proof of `False`, the proposition that is never true, so the theorem says: *if* we had a proof of `False`, then $0 = 1$ would follow. This implication is vacuously true (Section 2.5), since its hypothesis can never be satisfied. The proof uses the rule `False.elim`, which turns a proof of `False` into a proof of any proposition. So the theorem can never be used to conclude $0 = 1$, and a reader who looked only at the conclusion would misunderstand what was proved.

The same thing can happen less visibly when an assistant "repairs" a proof that Lean rejects by changing the statement, most often by adding a hypothesis: the new statement may be true and correctly proved, yet not be the one that was asked for. Here are two examples.

- The claim "for all real numbers x and y, if $x^2 = y^2$, then $x = y$" is false: take $x = 1$ and $y = -1$. Adding the hypotheses $x \geq 0$ and $y \geq 0$ makes it true, because the square map is injective on $[0, \infty)$ (Section 3.3), but the repaired claim says nothing about negative numbers. Whether the repair is acceptable depends on whether those hypotheses were part of the intended statement.
- An existence theorem can be made trivial by assuming the very thing to be proved. Here is `surj_comp` with one extra hypothesis:

  ```
  theorem surj_comp_assumed {A B C : Type}
      (f : A -> B) (g : B -> C)
      (hf : Surj f) (hg : Surj g)
      (h : Surj (fun x => g (f x))) :
      Surj (fun x => g (f x)) := by
    exact h
  ```

  The new hypothesis `h` is the conclusion itself, so the proof only repeats it. Lean accepts this, with a warning that `hf` and `hg` are unused, a useful hint that the proof never needed the real hypotheses. But a warning is not an error, and a changed statement does not always leave such a trace: `impossible_context` produces no warning at all.

In both examples the mathematical task has changed. So when a proof has been repaired, first compare the new statement with the old one, hypothesis by hypothesis and then conclusion with conclusion, as in the table of Section 16.2; only then look at how the proof changed.

16.6 Placeholders and axiom dependencies

While a proof is being developed, a gap can be filled temporarily with the word `sorry`. Lean then accepts the declaration with the warning `declaration uses 'sorry'`, treating the missing step as if it had been assumed; the proof is not complete. In fact `sorry` closes any goal, even a false

one: Lean accepts `theorem bad : (0 : Nat) = 1 := by sorry`, with the same warning.

The warning appears only at the declaration that contains `sorry`. If the last line of `surj_comp` is replaced by `sorry`, Lean warns at `surj_comp`, but a later theorem whose proof applies `surj_comp` gets no warning of its own, although it rests on the same gap. To find out whether a theorem depends on a gap anywhere, directly or through other theorems, use the command

```
#print axioms theoremName
```

with the theorem's name in place of `theoremName`. It lists every *axiom*, that is, every statement accepted without proof, on which the theorem's proof ultimately relies. A `sorry` anywhere among its dependencies shows up as the axiom `sorryAx`.

Not every axiom signals a problem. Lean's foundations include three standard axioms that ordinary mathematics uses all the time:

- `propext`, short for *propositional extensionality*: two propositions that are logically equivalent are equal, so that if $P \Leftrightarrow Q$, then P may be replaced by Q anywhere;
- `Quot.sound`: in a quotient, the set of equivalence classes of Section 6.2, two related elements have the same class, as in $[1]_4 = [5]_4$, since 4 divides $5 - 1$;
- `Classical.choice`: once it has been proved that an object of some kind exists, Lean may pick one, even when no particular one can be described. From it, with the other two axioms, Lean derives the principle that every proposition is either true or false, which proof by contradiction uses in its last step (Section 4.2); so a proof by contradiction about an arbitrary proposition usually depends on `Classical.choice`.

Seeing these in the list is normal. What matters is to know what each listed axiom says [13], and to notice one that does not belong, such as `sorryAx`.

For three of the examples in Chapters 15 and 16, the commands

```
#print axioms MathCourse.inj_comp
#print axioms MathCourse.surj_comp
#print axioms MathCourse.preimage_meet
```

print the three lines

```
'MathCourse.inj_comp' does not depend on any axioms
'MathCourse.surj_comp' does not depend on any axioms
'MathCourse.preimage_meet' does not depend on any axioms
```

So these proofs use only the rules of Lean's logic and our own definitions. (Appendix B.4 explains where such commands go in the file.)

An axiom list is no substitute for reading the statement. The theorems `impossible_context` and `surj_comp_assumed` of Section 16.5 depend on no axioms at all, yet the first assumes `False` and the second its own conclusion. To trust a theorem, you need to know both the assumptions in its statement and the axioms its proof depends on.

When you check a file, read everything Lean reports, warnings included, and the output of any

`#print axioms` commands; and wait until Lean has finished the whole file, since a line it has not yet reached shows no error. Proofs from sources you do not trust call for the stronger procedures in the official documentation [14]; Section 17.8 describes the check used for Seymour's conjecture.

How far a check reaches

A different question is how many cases a check covers, even when it finishes without any problem. Compare three kinds of check.

- A Python program, such as the one in Section B.3, can run through all $2^4 = 16$ sign sequences of length 4 and report that exactly 8 of them have $C_1 = C_2 = C_3 = 0$ (Section 14.6). This covers 16 cases and says nothing about other lengths, and no checker compares the code with the definition of C_s.
- A Lean proof about specific objects, such as `twice_two` in Section 15.2, is checked step by step, but it covers only the case it states: `twice 2 = 4`, and nothing about `twice 3`.
- A Lean proof of a universal statement, such as `surj_comp`, covers every case at once: all types A, B, C and all surjective functions between them, infinitely many cases that no program could test.

In all three, the result concerns the statement as written: code or a definition that encodes the wrong quantity, through a slip in the wrap-around or by using the aperiodic correlation where the periodic one was meant, gives a correct answer to the wrong question.

16.7 A debugging sequence that does not lose the mathematics

When Lean rejects a line, it is tempting to try one tactic after another, or to ask an assistant for anything that Lean will accept. Either habit can end with a file that Lean accepts and a statement that nobody wanted. Finding and removing the cause of an error is called *debugging*; the following sequence keeps the mathematics in view while you do it.

1. *Keep the statement fixed.* Do not edit the theorem's statement while you look for the problem.
2. *Read the proof state.* Look at the goal, the available facts, and the error message.
3. *Translate into ordinary mathematics.* Say in words what the goal claims and what each fact says.
4. *Diagnose, then respond.* Decide which of four situations you are in.
    - A *difference of form*: the goal follows from the available facts, but something is written in a different form from the one the tactic expects. A one-line change, such as a `show` line, is usually enough.
    - A *missing lemma*: the goal is true, but the proof needs a fact that is not yet available. Find one, read its exact statement with the command `#check` (described below), and make sure that its hypotheses hold in your situation before you use it.
    - A *mismatch of domains*: a variable ranges over a different set from the intended one. Go

back to the statement contract and decide which domain was intended. If the statement has to change, change it on purpose and record the change, for instance in a comment above the theorem.

- A *goal that does not follow*: in some situation where all the available facts hold, the goal fails. Stop looking for tactics, since none can derive it. First ask whether an earlier step caused the problem: in Section 15.6, offering the witness 2 would have required a proof of the false equation $2 + 1 = 4$, and the remedy was a better witness. If the goal already fails to follow at the start, or after steps such as `intro` that only name arbitrary objects and assume the hypotheses, then the statement itself is false. Look for a *counterexample*, an instance in which every hypothesis holds and the conclusion fails; what to do next is then a decision about the statement.

Two worked examples

A difference of form. Leave out the line `show g (f a) = c` in the proof of `surj_comp`, keeping the statement (step 1). The proof state (step 2) shows that `rw` did not find `f a` in the goal `(fun x => g (f x)) a = c`. Translated (step 3), this goal says $(g \circ f)(a) = c$, which follows from $f(a) = b$ and $g(b) = c$. So the diagnosis (step 4) is a difference of form, and the response is the `show` line, which changes the form of the goal but not its meaning (Section 16.2).

A false statement. Suppose we set out to prove `Surj (fun n : Nat => n + 1)`, that $n \mapsto n + 1$ is surjective on the natural numbers. After `intro y`, the goal is

```
∃ x, (fun n => n + 1) x = y
```

that is, for this arbitrary y we must find a natural number x with $x + 1 = y$. Suppose that every tactic we try fails. Instead of trying more, translate the goal and test a small case: for $y = 0$ we would need $x + 1 = 0$, which no natural number satisfies. Since `intro y` only named an arbitrary target, the problem cannot come from an earlier step: the statement itself is false, with the counterexample $y = 0$, and no tactic will ever close this goal. An assistant asked only to make Lean accept the file could succeed only by changing the statement (Section 16.5) or by leaving a `sorry` (Section 16.6).

What should happen next depends on what was intended, which only a person can decide. If the claim was meant to be about natural numbers, it is false, and Lean can prove that it is false:

```
theorem not_surj_succ :
    Not (Surj (fun n : Nat => n + 1)) := by
  intro h
  cases h 0 with
  | intro n hn =>
    exact Nat.succ_ne_zero n hn
```

In Lean, `Not P` is defined to mean `P -> False`, so `intro h` assumes that the map is surjective and leaves the goal `False`. Applying `h` to the target `0` and unpacking gives a natural number `n` and a proof `hn` that $n + 1 = 0$; the infoview shows it unreduced, as `(fun n => n + 1) n = 0`, which Lean treats as the same proposition. The last step uses the library lemma `Nat.succ_ne_zero`,

whose exact statement step 4 tells us to read first. To see the statement of any named theorem or lemma, put `#check` followed by its name on a line of its own, outside every proof; like `#eval` (Section 15.6), it shows its output in the infoview. Here `#check Nat.succ_ne_zero` prints

```
Nat.succ_ne_zero (n : Nat) : n.succ ≠ 0
```

It reads like a declaration, with the parameter `(n : Nat)` before the colon and the claim after it. Here `n.succ`, the *successor* of `n`, is another way of writing `n + 1`, and `a ≠ b` means `Not (a = b)`. So the lemma says that $n + 1 \neq 0$ for every natural number n, with no further hypotheses. Applied to our `n`, it proves `Not (n + 1 = 0)`, that is, `n + 1 = 0 -> False`, and applying this to `hn` gives the required proof of `False`.

If instead the claim was meant to be about integers, the domain was mistranslated: over $\mathbb{Z}$, the target y is reached by the input $y - 1$. Changing `Nat` to `Int` is a deliberate change of the statement, recorded as such in the comment on the first line:

```
-- Statement changed on purpose: Int in place of Nat.
theorem succ_surj_int : Surj (fun n : Int => n + 1) := by
  intro y
  refine Exists.intro (y - 1) ?_
  show y - 1 + 1 = y
  exact Int.sub_add_cancel y 1
```

As in `surj_comp`, we introduce a target, propose a witness, restate the goal in reduced form, and prove it. The library lemma `Int.sub_add_cancel a b` proves $(a - b) + b = a$ for all integers a and b; with $a = y$ and $b = 1$ it is exactly the goal. The same step would fail for natural numbers: with truncated subtraction (Section 15.6), the witness for the target 0 would be $0 - 1 = 0$, and $(0 - 1) + 1 = 1 \neq 0$.

For these two theorems, `#print axioms` lists `propext`, which the library lemmas `Nat.succ_ne_zero` and `Int.sub_add_cancel` use; as Section 16.6 explained, this standard axiom is no cause for concern.

Exercises

Exercises 16.1–16.4 are core exercises, and Exercises 16.5 and 16.6 are transfer exercises. Complete solutions are in Appendix A, Section A.16.

Exercise 16.1. In the proof of `surj_comp` (Section 16.2), explain what `hb` and `ha` state, and why the witness that the proof finally proposes must be an element of the domain A of f.

Exercise 16.2. Define a predicate `join s t` that describes the union of the sets described by `s` and `t`, as `meet s t` describes their intersection. Then state and prove in Lean the corresponding fact about preimages: for each `x`, `x` belongs to the preimage of the union exactly when it belongs to the union of the preimages.

Exercise 16.3. Suppose `h : Second r v w`. Write a Lean proof that $w \neq v$, and a Lean proof that some vertex u gives a route $v \to u \to w$.

Exercise 16.4. An assistant cannot prove a theorem, so it adds the hypothesis `h : False` and then proves the result. Explain precisely what has changed, and why the fact that Lean accepts the new file does not answer the original question.

Exercise 16.5. Design a review in three steps that detects, in turn, a changed domain, a missing exclusion (such as the condition that a second out-neighbor is not a first out-neighbor), and an unfinished proof containing `sorry`. For each step, say what you examine: which part of the Lean file, or which output of Lean.

Exercise 16.6. Explain the difference between three kinds of check: running a finite test in Python, proving a statement about specific objects in Lean, and proving a universal theorem in Lean. Give an example of each, different from the examples in Section 16.6, and say which cases your example covers.

CHAPTER 17

Reading an unfamiliar proof one piece at a time

> **Overview**
>
> Read a proof whose background you only partly have, without first studying a whole subject. For a proof that every sequence in a complete, totally bounded metric space has a convergent subsequence, map what this one proof needs, learn only the pieces that are new to you, and rebuild the whole argument. Then use the same method to begin reading a formalized research proof, whose short final file rests on many other files that hold most of the mathematics.

17.1 A new theorem on a familiar foundation

Imagine meeting the following sentence in a research paper:

> Every sequence in a complete, totally bounded metric space has a convergent subsequence.

Most of its words are familiar: Chapter 11 defined metric spaces, convergence, Cauchy sequences and completeness, and Chapter 12 used them all. Two expressions are new: "totally bounded" and "subsequence."

The conclusion is a form of what mathematicians call *compactness*, which topology studies in great generality. Instead of first taking a course on that theory, ask what the two new ideas do in this particular argument, and learn exactly that much. A first exchange with an AI assistant might go like this.

Reader: I know metric spaces, Cauchy sequences and completeness. Describe how the proof of this statement works, without quoting any theorem I may not know.

Assistant: Keep infinitely many terms of the sequence inside a small region, then inside a smaller one, and so on. Pick later and later terms from these nested selections. The picked terms form a Cauchy sequence, and completeness supplies their limit.

Reader: Which facts do those steps rely on that I may not have met yet?

Assistant: Three. You need to know that the space can be covered by finitely many small balls; that if infinitely many indices are split into finitely many groups, some group still contains infinitely many of them; and how to choose indices that keep increasing.

The reader's side of this exchange is a *minimal-background request*: it says what the reader already knows and asks only for what one particular proof needs. The words "those steps" pin

down which proof, and that matters: a different proof of the same theorem could need different concepts. A good request also asks to see, right after each new concept, the step of the proof that uses it, which keeps the conversation tied to one argument instead of drifting into a survey of a subject.

You need not follow every phrase of the assistant's compressed answers yet. Together they propose a *dependency map* of the proof (Section 12.2): the steps of the proof and the facts that each step relies on. Once we mark which entries we already know, we call it a *prerequisite map*; its new entries are all we need to learn. This proposed map is incomplete, as Section 17.7 will show. Sections 17.2, 17.3 and 17.4 supply the three missing pieces, each in a short self-contained lesson that we call a *capsule*; the last also sets out the plan of the proof, which Section 17.5 carries out.

17.2 Capsule one: small regions that cover the space

The first missing piece is a way to describe a whole metric space, at any desired precision, by finitely many small regions called balls.

Definition 17.1 (Ball)**.** Let (X, d) be a metric space, let $c \in X$, and let $r > 0$. The *ball* with center c and radius r is the set of points of X at distance less than r from c:

$$B(c, r) = \{x \in X : d(x, c) < r\}.$$

The center belongs to its own ball, because $d(c, c) = 0 < r$. On the real line, $B(c, r)$ is the open interval $(c - r, c + r)$, because $|x - c| < r$ means $c - r < x < c + r$ (Section 2.6). In the space $[0, 1]$, the ball $B(0, 1/4)$ is $[0, 1/4)$: points outside the space do not count.

The assistant's list did not mention it, but the proof also needs one property of balls. If x and y both lie in $B(c, r)$, then the triangle inequality, with c as the stopping point, and symmetry give

$$d(x, y) \leq d(x, c) + d(c, y) < r + r = 2r.$$

So any two points of a ball of radius r are less than $2r$ apart. We call this the *diameter bound* for balls. In the proof, it turns "many terms lie in one small ball" into "these terms are close to one another."

A collection of sets *covers* X if every point of X lies in at least one set of the collection.

Definition 17.2 (Totally bounded)**.** A metric space (X, d) is *totally bounded* if, for every $r > 0$, there are finitely many points $c_1, \ldots, c_k$ of X such that the balls $B(c_1, r), \ldots, B(c_k, r)$ cover X. In symbols:

$$\forall r > 0\ \exists k \in \mathbb{N}\ \exists c_1, \ldots, c_k \in X\ \forall x \in X\ \exists i \in \{1, \ldots, k\} : \quad d(x, c_i) < r.$$

Read the quantifiers as a game (Section 2.3): a skeptic names a radius $r > 0$; you answer with a finite list of centers in the space, which may depend on r; the skeptic picks any point x of the space; and you win if x is at distance less than r from one of your centers. A smaller radius may need more centers. The definition does not ask for one finite list that works for every radius at once, and for $[0, 1]$ none does.

Two examples on the interval $[0, 1]$

The interval $[0, 1]$ *is totally bounded.* Let $r > 0$. By the Archimedean property (Section 2.3), there is an integer $N > 1/r$; it is positive, and $1/N < r$. Use the $N + 1$ centers $0, \frac{1}{N}, \frac{2}{N}, \ldots, 1$, which are spaced $1/N$ apart. A point $x \in [0, 1]$ that is a center lies in its own ball; otherwise $j/N < x < (j + 1)/N$ for two consecutive centers, and

$$d\Big(x, \frac{j}{N}\Big) = x - \frac{j}{N} < \frac{j+1}{N} - \frac{j}{N} = \frac{1}{N} < r.$$

So the $N + 1$ balls of radius r cover $[0, 1]$. For instance, $r = 0.3$ allows $N = 4$, and the five balls of radius 0.3 centered at 0, 0.25, 0.5, 0.75 and 1 cover $[0, 1]$.

No single finite list works for every radius. An empty list covers nothing, so fix a list of $k \geq 1$ points $c_1, \ldots, c_k$ of $[0, 1]$, and take the radius $r = 1/(2k)$. The $k + 1$ points $0, \frac{1}{k}, \frac{2}{k}, \ldots, 1$ are at least $1/k = 2r$ apart. If the k balls $B(c_i, r)$ covered them, we could assign each of these points to one ball containing it. By the pigeonhole principle (Proposition 7.4), two of the points would share a ball, and by the diameter bound they would be less than $2r$ apart, which is impossible. So the list fails for this radius.

Totally bounded spaces are bounded

A metric space is *bounded* if there is a real number M such that $d(x, y) \leq M$ for all points x and y of the space. Every totally bounded space is bounded. If the space is empty, there is nothing to prove; otherwise, cover it by balls of radius 1 with centers $c_1, \ldots, c_k$, where $k \geq 1$, and let M be the largest of the finitely many distances $d(c_i, c_j)$. If x lies in the ball about c_i and y in the ball about c_j, the triangle inequality used twice, together with $d(c_j, y) = d(y, c_j) < 1$, gives

$$d(x, y) \leq d(x, c_i) + d(c_i, c_j) + d(c_j, y) < 1 + M + 1.$$

So $M + 2$ bounds every distance in the space.

The converse is false. On any set X, let

$$d(x, y) = \begin{cases} 0 & \text{if } x = y, \\ 1 & \text{if } x \neq y. \end{cases}$$

This is a metric, the *discrete metric* on X. The first two requirements of Definition 11.1 hold because d is 0 exactly when $x = y$, is otherwise 1, and gives $d(x, y)$ and $d(y, x)$ by the same case. For the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$: if $x = z$, the left side is 0; if $x \neq z$, the left side is 1, and the point y cannot equal both x and z, so at least one distance on the right is 1.

On $\mathbb{N}$ with the discrete metric, every distance is at most 1, so the space is bounded. But a ball of radius $1/2$ contains only its center, $B(n, 1/2) = \{n\}$, since every other number is at distance 1 from n; so finitely many such balls cannot cover $\mathbb{N}$: the space is not totally bounded. Boundedness asks for a single bound on all distances; total boundedness asks for a finite description of the space at every scale.

Pause and decide. Is the real line, with distance $|x - y|$, totally bounded?

Check. No. A totally bounded space is bounded, but the real line is not: for any proposed bound M, the points 0 and $|M| + 1$ are at distance $|M| + 1 > M$.

17.3 Capsule two: keeping infinitely many indices

A sequence $x_0, x_1, x_2, \ldots$ has one term for each index $n \in \mathbb{N}$, so it has infinitely many indices even when its values repeat: $x_n = (-1)^n$ takes only the values 1 and -1, yet the value 1 occurs at the infinitely many indices $0, 2, 4, \ldots$. Counting points instead would break the assistant's plan for a constant sequence: no ball contains infinitely many different terms, yet a constant sequence converges at once. So the proof keeps track of sets of *indices*: a ball contains infinitely many terms of a sequence if it contains x_n for infinitely many indices n, whether or not these terms are different points.

Recall from Section 3.1 that a set is *finite* if its members can be written down in a list that ends, and *infinite* otherwise.

Lemma 17.3 (Infinite pigeonhole principle)**.** *Let S be an infinite set, and let $A_1, \ldots, A_k$ be finitely many sets whose union contains S. Then at least one of the sets $S \cap A_1, \ldots, S \cap A_k$ is infinite.*

Proof. Every member of S lies in at least one of the sets A_i, so

$$S = (S \cap A_1) \cup (S \cap A_2) \cup \cdots \cup (S \cap A_k).$$

Suppose, for contradiction, that each of the sets $S \cap A_i$ is finite. Write down the members of $S \cap A_1$, then those of $S \cap A_2$, and so on up to $S \cap A_k$. The result is a list that ends, and every member of S appears in it at least once. Crossing out repetitions, which occur for members lying in several of the sets A_i, leaves a list of the members of S that still ends. So S is finite, a contradiction. □

This is the pigeonhole principle of Proposition 7.4 for infinitely many objects: if the members of S are placed in finitely many boxes $A_1, \ldots, A_k$, some box holds infinitely many of them. Here, though, the boxes may overlap; an object in two boxes simply appears twice in the list.

For example, take $x_n = (-1)^n$ and the balls $B(1, 1/2)$ and $B(-1, 1/2)$ on the real line. The indices n with $x_n \in B(1, 1/2)$ form the set A_1 of even numbers, and those with $x_n \in B(-1, 1/2)$ form the set A_2 of odd numbers. Their union is $\mathbb{N}$, and the lemma with $S = \mathbb{N}$ promises that at least one of them is infinite, without saying which. Here both are.

In the proof of the theorem, S will be the current infinite set of indices and A_i the set of indices n for which x_n lies in the ith ball of a finite cover; the lemma then supplies one ball containing x_n for infinitely many $n \in S$.

17.4 Capsule three: subsequences, and a plan for the proof

Definition 17.4 (Subsequence)**.** Let $x_0, x_1, x_2, \ldots$ be a sequence. A *subsequence* of it is a sequence of the form

$$x_{n_1}, x_{n_2}, x_{n_3}, \ldots, \qquad \text{where } n_1 < n_2 < n_3 < \cdots$$

are natural numbers.

In words: we may skip terms, but the terms we keep must stay in their original order, and no index may be used twice. For $x_n = (-1)^n$, the indices $n_k = 2k$ give the subsequence $x_2, x_4, x_6, \ldots$, which is the constant sequence $1, 1, 1, \ldots$. By contrast, $x_1, x_0, x_3, x_2, \ldots$ is not a subsequence, because its indices do not increase, and neither is $x_0, x_0, x_0, \ldots$, which uses the index 0 again and again.

Keep the two counters apart: k is the position in the subsequence, while n_k is the position of the same term in the original sequence; in the example, the third term of the subsequence is x_6. We number the terms of a subsequence from $k = 1$, not 0. This makes no difference: with $y_k = x_{n_k}$, the definitions of convergence and of Cauchy sequences apply to $y_1, y_2, y_3, \ldots$ as before, with thresholds that are at least 1.

To make the chosen indices increase, we need one fact. *An infinite set S of natural numbers contains a number larger than any given natural number M.* Otherwise every member of S would lie in $\{0, 1, \ldots, M\}$, and crossing out of the list $0, 1, \ldots, M$ every number not in S would leave a list of the members of S that ends, so S would be finite. So if each stage of a construction offers an infinite set of allowed indices, we can always choose an allowed index larger than the one chosen at the previous stage.

A plan with three kinds of objects

The construction involves three kinds of objects that must not be mixed up: points of the space, such as the term x_7; balls, which are sets of points; and sets of indices, which are sets of natural numbers. If $x_7 = x_9$ lies in a chosen ball, it is one point of the ball but contributes two indices, 7 and 9.

Here is the plan. Start with every index allowed: $S_0 = \mathbb{N}$. At stage 1, cover the space by finitely many balls of radius $1/4$, as total boundedness allows. By Lemma 17.3, one of these balls contains x_n for infinitely many indices $n \in S_0$; keep exactly those indices, as the set S_1. By the diameter bound, any two terms with indices in S_1 are less than $2 \cdot \frac{1}{4} = \frac{1}{2}$ apart. In general, stage k uses balls of radius $2^{-(k+1)}$, considers only the indices kept at stage $k-1$, and keeps infinitely many indices whose terms are less than $2 \cdot 2^{-(k+1)} = 2^{-k}$ apart. The radius is half the desired distance because the diameter bound doubles it; any radii shrinking towards zero would work, and this choice keeps the arithmetic tidy. Since each stage only throws indices away,

$$S_0 \supseteq S_1 \supseteq S_2 \supseteq \cdots,$$

where $A \supseteq B$, read "A contains B," means $B \subseteq A$. We say that the index sets are *nested*. The chosen balls need not be nested: the ball of stage 2 may stick out of the ball of stage 1. That does no harm, because a term whose index lies in S_2 also has its index in S_1, so it lies in both chosen balls. The nesting of the index sets carries the distance bound of each stage forward to all later stages.

17.5 The complete proof

Theorem 17.5. *Let (X, d) be a complete, totally bounded metric space. Then every sequence $x_0, x_1, x_2, \ldots$ in X has a subsequence that converges to a point of X.*

Unlike the contraction theorem (Theorem 12.2), this theorem does not assume that X is nonempty: if X has no points, there is no sequence in X, and the statement is vacuously true (Section 2.5).

Proof. Let $x_0, x_1, x_2, \ldots$ be a sequence in X.

Step 1: nested infinite sets of indices. We construct infinite sets of indices $S_0 \supseteq S_1 \supseteq S_2 \supseteq \cdots$ such that, for every $k \geq 1$,

$$d(x_i, x_j) < 2^{-k} \qquad \text{for all } i, j \in S_k. \tag{17.1}$$

Start with $S_0 = \mathbb{N}$. Now let $k \geq 1$, and suppose that an infinite set S_{k-1} has been constructed. Since X is totally bounded, finitely many balls of radius $2^{-(k+1)}$ cover X. For each of these balls, take the set of indices n with x_n in that ball; since every x_n lies in some ball, every index lies in one of these finitely many sets. By Lemma 17.3, applied to the infinite set S_{k-1}, there is a ball B among them such that the set

$$S_k = \{n \in S_{k-1} : x_n \in B\}$$

is infinite. By its definition, $S_k \subseteq S_{k-1}$. If $i, j \in S_k$, then x_i and x_j both lie in B, so the diameter bound gives $d(x_i, x_j) < 2 \cdot 2^{-(k+1)} = 2^{-k}$, which is (17.1). Since S_k is again infinite, the construction continues for every k.

Step 2: increasing indices. Choose any $n_1 \in S_1$. For $k \geq 2$, once n_{k-1} has been chosen, choose $n_k \in S_k$ with $n_k > n_{k-1}$. Such an index exists because S_k is infinite, and an infinite set of natural numbers contains a number larger than any given one (Section 17.4). Then $n_1 < n_2 < n_3 < \cdots$, so $x_{n_1}, x_{n_2}, x_{n_3}, \ldots$ is a subsequence, and $n_k \in S_k$ for every $k \geq 1$.

Step 3: later terms keep the earlier bounds. Let $1 \leq k \leq \ell$. The sets are nested, so $n_\ell \in S_\ell \subseteq S_k$: every term chosen at stage k or later has its index in S_k. So if $\ell \geq k$ and $m \geq k$, then $n_\ell, n_m \in S_k$, and (17.1) gives

$$d(x_{n_\ell}, x_{n_m}) < 2^{-k}.$$

Step 4: the subsequence is a Cauchy sequence. Let $\varepsilon > 0$. By the Archimedean property (Section 2.3), choose an integer $K > 1/\varepsilon$. It is positive, so $K \geq 1$; this matters, because (17.1) holds only from stage 1 on. Since $2^K \geq K + 1$ (Section 11.3),

$$2^{-K} \leq \frac{1}{K+1} < \frac{1}{K} < \varepsilon.$$

For any $\ell, m \geq K$, Step 3 with $k = K$ gives $d(x_{n_\ell}, x_{n_m}) < 2^{-K} < \varepsilon$. Since $\varepsilon > 0$ was arbitrary, the subsequence is a Cauchy sequence.

Step 5: completeness supplies the limit. The space X is complete, so the Cauchy sequence $x_{n_1}, x_{n_2}, x_{n_3}, \ldots$ converges to some point x_* of X. □

Watching the construction on an example

The interval $[-1, 2]$, with distance $|x - y|$, is complete, being a closed interval (Section 11.6), and totally bounded by the argument used for $[0, 1]$ (Exercise 17.4 asks you to write it out for every interval $[a, b]$). In this space consider

$$x_n = (-1)^n + \frac{1}{n+1},$$

whose first terms are $2, -\frac{1}{2}, \frac{4}{3}, -\frac{3}{4}, \frac{6}{5}$. The even-indexed terms crowd towards 1 from above, and the odd-indexed terms crowd towards -1, never rising above $-\frac{1}{2}$.

Stage k can cover $[-1, 2]$, as we covered $[0, 1]$, by balls of radius $2^{-(k+1)}$ whose centers run from -1 to 2 in steps of $2^{-(k+1)}$; one of the centers is 1. At stage 1 these are thirteen balls of radius $1/4$. The ball $B(1, 1/4)$ contains x_n exactly when n is even and $\frac{1}{n+1} < \frac{1}{4}$, that is, when n is even and $n \geq 4$, so the construction may take $S_1 = \{4, 6, 8, \ldots\}$. Choosing the ball around 1 at every stage gives the set S_k of even numbers n with $\frac{1}{n+1} < 2^{-(k+1)}$, that is, with $n \geq 2^{k+1}$: $S_2 = \{8, 10, 12, \ldots\}$, $S_3 = \{16, 18, 20, \ldots\}$, and so on.

Taking in Step 2 the smallest allowed index each time gives $n_1 = 4$, $n_2 = 8$, $n_3 = 16$, and in general $n_k = 2^{k+1}$. The subsequence $x_4, x_8, x_{16}, \ldots$, with terms $1 + \frac{1}{2^{k+1}+1}$, converges to 1. Stage 1 could instead have chosen $B(-1, 1/4)$, which contains x_n for every odd $n \geq 5$; then every later index would have been odd, and the subsequence would converge to -1: the theorem promises *some* convergent subsequence, not a particular one. In general the indices follow no simple formula; each n_k need only lie in S_k and exceed n_{k-1}.

Why the proof picks a new index at every stage

Why not look for a single index n in every set S_k, so that the one term x_n lies in every chosen ball? In general there is none: nested infinite sets can have no common member. For example, the sets $\{k, k+1, k+2, \ldots\}$ for $k = 1, 2, 3, \ldots$ are infinite and nested, but no natural number n belongs to all of them, since $n \notin \{n+1, n+2, \ldots\}$. And even when such an index exists, as for a constant sequence, it gives one term, not a subsequence. So the proof picks a new, later index from each set, and Step 3 shows that any two sufficiently late picks lie together in one earlier set, which is exactly what the Cauchy condition asks for.

Pause and decide. Where does the proof use each hypothesis of Theorem 17.5?

Check. Total boundedness is used at every stage of Step 1, to cover X by finitely many balls of radius $2^{-(k+1)}$; the finiteness of that cover is what allows Lemma 17.3 to keep infinitely many indices. Completeness is used once, in Step 5, to give the Cauchy subsequence a limit in X. The diameter bound, the nesting of the index sets and the increasing choice of indices are not hypotheses but facts proved along the way.

17.6 Why neither hypothesis can be dropped

As in Section 12.6, we remove one hypothesis at a time, keep the other, and look for an example in which the conclusion fails.

Without total boundedness

Give $\mathbb{N}$ the discrete metric of Section 17.2. This space is not totally bounded, as we saw there, but it is complete. Indeed, for a Cauchy sequence in it, the tolerance $1/2$ gives a threshold N such that

any two terms from x_N onward are less than $1/2$ apart. Distances here are only 0 or 1, so every term from x_N onward equals x_N, and the sequence converges to x_N.

Yet the conclusion fails for the sequence $x_n = n$. In any subsequence the indices increase, so the terms are different numbers, any two of them at distance 1. So no subsequence is a Cauchy sequence, since no threshold works for the tolerance $1/2$, and none converges (Proposition 11.5). In the proof, Step 1 breaks down: no finite collection of balls of radius $1/4$ covers the space, so the selection cannot even begin.

Without completeness

The interval $(0, 1)$, with distance $|x - y|$, is totally bounded, although the centers 0 and 1 used for $[0, 1]$ are not points of it; we use midpoints instead. Given $r > 0$, the Archimedean property supplies a positive integer $N > 1/(2r)$, so that $1/(2N) < r$. Split $[0, 1]$ into the N pieces $\left[\frac{j-1}{N}, \frac{j}{N}\right]$ for $j = 1, \ldots, N$. Their midpoints $\frac{2j-1}{2N}$ lie in $(0, 1)$. Every $x \in (0, 1)$ lies in one of the pieces, and so is at distance at most half its length, $1/(2N) < r$, from that piece's midpoint.

But $(0, 1)$ is not complete (Section 11.6), and the conclusion fails for the sequence $x_n = 1/(n+2)$, that is, $\frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \ldots$. On the real line it converges to 0: given $\varepsilon > 0$, choose an integer $N > 1/\varepsilon$; then every $n \geq N$ gives $x_n = \frac{1}{n+2} < \frac{1}{N} < \varepsilon$. Every subsequence also converges to 0 on the real line: the indices increase by at least one at every step, starting from $n_1 \geq 0$, so induction gives $n_k \geq k - 1$, and with ε and N as before, every $k \geq N + 1$ has $n_k \geq N$ and therefore $x_{n_k} < \varepsilon$.

If some subsequence converged to a point L of $(0, 1)$, it would converge to L on the real line too, since distances in $(0, 1)$ are distances on the real line, and uniqueness of limits (Proposition 11.3) would give $L = 0 \notin (0, 1)$. So no subsequence converges in $(0, 1)$. In the proof, Steps 1–4 go through, and Step 5 fails: the Cauchy subsequence exists, but the point it crowds around is missing from the space.

When you meet a new hypothesis, ask not only "What does it mean?" but also "Which step fails without it?"

17.7 The prerequisite map after the work is done

The assistant's list of prerequisites in Section 17.1 was incomplete. It did not mention the diameter bound for balls, used in Step 1, or explain why nested selections make the chosen terms a Cauchy sequence, the argument of Step 3. Such gaps turn up when you rebuild a proof line by line, and finding them is part of the method: add each missing step to the map, learn it, and return to the argument. A list of prerequisites deserves your trust only once you have used it to rebuild the proof.

Here is the completed map.

Where	Facts used	Conclusion
Capsule one	triangle inequality and symmetry	diameter bound for balls
Capsule two	definition of a finite set	infinite pigeonhole principle (Lemma 17.3)
Capsule three	definition of a finite set	an infinite set of natural numbers contains a number larger than any given one
Step 1	total boundedness, infinite pigeonhole principle, diameter bound	nested infinite index sets S_k satisfying (17.1)
Step 2	the sets S_k are infinite, and an infinite set of natural numbers contains a number larger than any given one	increasing indices $n_k \in S_k$, hence a subsequence
Step 3	Step 2 (each n_ℓ lies in S_ℓ), nesting of the sets S_k, the bound (17.1)	terms from position k on are less than 2^{-k} apart
Step 4	Step 3, Archimedean property, $2^K \geq K + 1$	a Cauchy subsequence
Step 5	Step 4, completeness	a subsequence that converges to a point of the space

The facts not proved here came from Chapter 11 (the triangle inequality, Cauchy sequences, completeness, $2^K \geq K + 1$), Chapter 2 (the Archimedean property) and Chapter 3 (finite sets). Each new ingredient was put to work in the proof soon after it was learned. That matters: a concept you have seen doing its job stays with you, while one learned and left unused becomes disconnected vocabulary.

Check what you have learned, not only what you can name

Here is a four-part check of whether you have learned a new concept well enough for the proof at hand, carried out for total boundedness.

1. *State the problem the concept solves.* At every scale, the proof must split the infinitely many indices into finitely many groups whose terms are close together. Total boundedness provides, for any radius, finitely many balls of that radius covering the space, and the balls give the groups.
2. *State the definition with its quantifiers in the right order.* The radius comes first. The list of centers may depend on the radius, but for that radius one list must cover every point.
3. *Test it on examples near the boundary.* The discrete metric on $\mathbb{N}$ shows that a bound on all distances does not give finite covers by small balls. The interval $(0, 1)$ shows that finite covers by small balls do not provide the missing limits.
4. *Return to the exact line of the proof where the concept is used,* here the beginning of each stage of Step 1, and explain it. Why must the cover be finite? Because Lemma 17.3 applies only to finitely many sets: with infinitely many balls, each might contain just one term, as in the discrete metric on $\mathbb{N}$, and no ball would keep infinitely many indices.

If you can recite the definition but cannot explain that line, you have not yet learned the concept well enough for this proof.

Labels for the entries of the map

For an unfamiliar paper, mark each entry of the map with its status, using the labels of "Keeping track of evidence" in *To the reader* and one new label, *borrowed*.

- *Understood:* the entry passes the four-part check above, and you can use it in a slightly different setting. After studying $[0, 1]$, proving that every interval $[a, b]$ is totally bounded (Exercise 17.4) is a small test of this kind.
- *Can follow:* you can check each step but could not yet explain the entry, or use it in a new setting, on your own. You might check every line of the proof of Lemma 17.3 and still be unable to say, without looking, which line would fail with infinitely many sets A_i.
- *Borrowed:* you use the result without having studied its proof. You must still check that its hypotheses hold and that its conclusion is exactly what you need; learning its proof can be a separate task. Here the completeness of the real line is borrowed: Chapter 11 accepted it as a starting fact, and the completeness of $[-1, 2]$ in Section 17.5 rests on it.
- *Open:* you cannot yet follow the entry, which records exactly what remains to be learned, like much of the research record of Section 17.8.

A map may contain borrowed results and open entries, as long as it labels them clearly.

17.8 Reading a formalized research proof with the same method

Claimed proofs of Seymour's second-neighborhood conjecture (Section 7.7) and of the circulant Hadamard conjecture (Section 14.7) appear in OpenAI's public mathematics repository [2], which collects preprints with formal proofs in Lean and describes exactly which statements were formalized. A paper's presence there does not vouch for every claim in it. The right unit of reading is one exact theorem, with its definitions and the results it depends on, not a title or an impressive final line.

Decoding the statement

The repository's statement of Seymour's conjecture asks, in a nonempty finite oriented graph, for a vertex whose second out-neighborhood has at least as many vertices as its first [3]. The official statement lives in a file of its own [6], where the theorem is called `exists_goodVertex`. Collecting its assumptions in one place, writing `->` and `Exists` for the file's $\to$ and $\exists$ as in Chapter 16, and putting `Type` where the file has `Type*` (a technical detail about the sizes of types that makes no difference here), we can display the statement as follows.

```
theorem exists_goodVertex {V : Type}
    [Fintype V] [DecidableEq V] [Nonempty V]
    (r : V -> V -> Prop) (hr : IsOriented r) :
    Exists (fun v => GoodVertex r v) := by
  sorry
```

The repository builds on Mathlib (Chapter 15), which provides finite sets and ways of counting their members. Piece by piece:

- `V` is the type of vertices. `[Fintype V]` says that there are only finitely many vertices, and `[Nonempty V]` that there is at least one. `[DecidableEq V]`, that equality of vertices can be decided, is a technical requirement of Mathlib's finite sets that excludes no graph: with the standard axiom `Classical.choice` of Section 16.6, Lean can supply it for every type.
- The edge relation `r` has type `V -> V -> Prop`, so `r v w` is the proposition $v \to w$, as in Section 16.4.
- The hypothesis `(hr : IsOriented r)` says that `r` has no loops and no pair of opposite edges; the definition of `IsOriented` has exactly these two parts, named `loopless` and `asymmetric`.
- The conclusion says that some vertex `v` satisfies `GoodVertex r v`, defined to mean that the set `firstNeighbors r v` has at most as many members as `secondNeighbors r v`. The first set consists of the out-neighbors of v, the vertices `w` with `r v w`. The second is defined by exactly the three conditions of Section 7.7: $w \neq v$, $v \to w$ fails, and $v \to u \to w$ for some vertex u.

No condition of the informal statement is missing, and decoding it needed nothing beyond Chapters 7, 15 and 16. The `sorry` at the end belongs there: this file only records what must be proved.

Locating the main steps

The proof is in another file [5], which imports three further files, named `Reduction`, `Pruning` and `ExtremalContradiction`, and whose own proof is only a few lines long. It assumes that no vertex is good, so that the graph is a counterexample; uses a theorem from the reduction file to turn the counterexample into a new oriented graph with additional properties; and applies results from the other two files to reach a contradiction.

The reduction theorem [7] says that from any counterexample one can build, for some positive integer n, an oriented graph whose vertices are the pairs (i, j) of natural numbers with $i < n$ and $j \leq n$, and which has two further properties, named `PositiveIndegree` and `StrictSubsetGrowth`. The reduction file imports three more files, so the map keeps branching. The name `PositiveIndegree` suggests that every vertex has an incoming edge, but Lean accepts any name for any statement (Section 15.2): to understand these properties, you must read their definitions.

Keeping a reading record

A *reading record* is a written account of a proof you are studying: what you have read, what you have understood, and what remains open (Section C.5 describes a finished one). So far, a reading record of the Seymour proof has entries of three kinds.

- *Understood:* the final statement has been decoded, and its definitions match the informal conjecture.
- *Borrowed:* the logical role of the reduction is clear, since it turns a counterexample into a graph with extra properties, but its proof has not been read.
- *Open:* the meaning of `PositiveIndegree` and `StrictSubsetGrowth`, and the mathematics of the pruning step and of the final contradiction.

A sketch like this one is a way into the proof, which uses ideas well beyond this book, not a replacement for it.

What the formal check compares

A configuration file [8] lists what the repository's formal check compares: the file containing the official statement, the file containing the proof, the name of the theorem, and the permitted axioms, which are `propext`, `Quot.sound` and `Classical.choice`, the three standard axioms of Section 16.6. The check is designed to confirm that the proof file proves exactly the official statement using no other axioms. It compares the statements, not just their names: a theorem in the proof file with the official name would, by itself, say nothing about whether the two statements agree.

The configuration file tells you what the check is designed to compare, not that it passed. Unless you run the check yourself, its passing is a claim made by the repository, and your reading record should attribute it: "the repository reports that the check passes," not "the check passes." Likewise, the record can describe the role of a file whose proofs you have not read, but cannot vouch for them; on your own authority it reports only what you decoded or reconstructed yourself.

The circulant Hadamard conjecture

Section 14.7 has already looked at which statements about circulant Hadamard matrices the repository formalized [4], and you have proved a necessary condition yourself: by Proposition 14.7, a circulant Hadamard matrix of order greater than 4 would need an order $4u^2$ with $u \geq 2$, such as 16, 36 or 64. That gives a precise next question: which lemma of the proof rules out these orders? A question informed by mathematics you have done yourself gets you much further than asking an assistant to "explain the whole repository."

Exercises

Exercises 17.1–17.4 are core exercises, and Exercises 17.5 and 17.6 are transfer exercises. Complete solutions are in Appendix A, Section A.17.

Exercise 17.1. Suppose a ball contains x_n for infinitely many indices n. Explain why it does not follow that the sequence takes infinitely many different values in that ball. Does the proof of Theorem 17.5 need the sequence to take infinitely many different values?

Exercise 17.2. An assistant writing out the proof of Theorem 17.5 chooses an arbitrary $n_k \in S_k$ at each stage, without requiring $n_k > n_{k-1}$. Identify the gap and repair the choice. Explain why the repair is always possible.

Exercise 17.3. Let X be any set with the discrete metric of Section 17.2. (a) Prove that X is complete. (b) Prove that X is totally bounded if and only if X is finite. (c) Prove that a sequence $x_0, x_1, x_2, \ldots$ in X has a subsequence that converges in X exactly when there is a point c of X with $x_n = c$ for infinitely many indices n. Use this to explain, for $X = \mathbb{N}$ and the sequence $x_n = n$, the difference between completeness and the conclusion of Theorem 17.5.

Exercise 17.4. Let $a < b$ be real numbers. Give a complete proof that the interval $[a, b]$, with distance $|x - y|$, is totally bounded. State exactly what the number of balls in your cover may depend on, and what it may not depend on.

Exercise 17.5. A directed graph has the three vertices v, u, w and the edges $v \to u$, $u \to w$ and $v \to w$. Let $R_2(v)$ be the set of vertices z for which there is a route $v \to y \to z$ through some vertex y. Find $N_1^+(v)$, $N_2^+(v)$ and $R_2(v)$. Then explain what would go wrong if a formal statement of Seymour's conjecture used $R_2(v)$ in place of $N_2^+(v)$.

Exercise 17.6. Write a minimal-background request, in the sense of Section 17.1, for the proof of the contraction theorem (Theorem 12.2), and another for the proof of Theorem 17.5. For each of the two proofs, name a concept that it needs but the other proof does not. Then state what a reading record of a formalized research proof may honestly claim if you have not run its formal check yourself.

CHAPTER 18

A complete AI-assisted investigation

> **Overview**
>
> Change one assumption of the contraction theorem of Chapter 12: suppose that every update is made with a small error. Test a first guess, find a statement that is actually true, prove it, ask whether its constant can be improved, and check it against an exact computation. Then write the investigation up so that another reader can check every claim in it, and plan an investigation of your own.

18.1 The question that starts the project

The contraction theorem of Chapter 12 is about exact updates: each new point is $x_{n+1} = T(x_n)$, computed without any error. Real computations are rarely that clean. A computer keeps only finitely many digits, so it rounds each value of T before storing it; a calculator that keeps two decimal places might store 0.87 where the exact value is 0.875. Measurements, too, have limited precision. In each case the new point is close to T of the old point, but not equal to it. Does the iteration still approach the fixed point? And if not, how far from it can it end up?

This makes a good first investigation: it changes one feature of a theorem we already understand and keeps everything else. The bounds we reach are standard; what matters is how the investigation proceeds.

The setting

Throughout the chapter, (X, d) is a nonempty complete metric space, and $T : X \to X$ is a contraction with factor q, where $0 < q < 1$:

$$d(Tx, Ty) \le q\, d(x, y) \qquad \text{for all } x, y \in X.$$

Asking for $q > 0$ costs nothing, because a contraction with factor 0 satisfies $d(Tx, Ty) \le 0 \le \frac{1}{2} d(x, y)$ and so is also one with factor $1/2$; in return, every power q^k below is positive. By Theorem 12.2, T has exactly one fixed point, which we call x_*.

In place of the exact iterates we now have points $y_0, y_1, y_2, \ldots$ of X, starting anywhere, and nonnegative numbers $\varepsilon_0, \varepsilon_1, \varepsilon_2, \ldots$ such that

$$d(y_{n+1}, T(y_n)) \le \varepsilon_n \qquad \text{for every } n \in \mathbb{N}.$$

The point y_{n+1} is what the update actually produced when it should have produced $T(y_n)$, and ε_n bounds the error made in that update. We call ε_n the *update error* at step n (strictly, it is an upper

bound for that error, which is all we need). If every ε_n is zero, we are back to the exact iteration. Apart from these bounds we assume nothing: the errors need not follow any pattern, and they are not assumed to be random.

One assumption is easy to overlook: every y_n must be a point of X, which an error bound alone cannot guarantee. If $X = [0, 1]$ and a computed value is -0.001, then $d(y_{n+1}, T(y_n))$ is not even defined, and T cannot be applied at the next step. In an application, $y_n \in X$ must be checked separately, like the self-map condition of Section 12.1.

18.2 Testing the first guess

A natural first guess is that small update errors do no lasting harm: if every ε_n is small, the points y_n should still converge to x_*. Before trying to prove a guess, it pays to try to break it. A short exchange with an AI assistant might go like this.

Reader: I expect that small update errors still give convergence to x_*. Before I try to prove this, try to disprove it.

Assistant: A constant bias can shift the limit. Take $T(x) = qx$ on $\mathbb{R}$, whose fixed point is 0, and add the same small number $\eta > 0$ at every update: $y_{n+1} = qy_n + \eta$. Then y_n approaches $\eta/(1-q)$, not 0, however small η is.

Reader: Then convergence is the wrong target. Is there an error bound that still holds? And can it tell apart errors that stay bounded from errors that tend to zero?

Assistant: Start by bounding the distance from the next point to x_* by q times the present distance plus the update error. Then apply that bound over and over.

Neither reply has been checked yet. The first is a counterexample built on a *constant bias*, an error that is the same at every update. Its hypotheses are easy to check: $|qx - qy| = q|x - y|$, so T is a contraction with factor q; its only fixed point is 0, because $qx = x$ gives $(1-q)x = 0$; and every update error is $|y_{n+1} - T(y_n)| = |qy_n + \eta - qy_n| = \eta$, which can be as small as we like. With $q = 1/2$, $\eta = 1$ and $y_0 = 0$, the points

$$0,\ 1,\ \tfrac{3}{2},\ \tfrac{7}{4},\ \tfrac{15}{8},\ \tfrac{31}{16},\ \ldots$$

seem to approach $2 = \eta/(1-q)$ rather than 0. A few terms do not prove a limit, so Section 18.4 proves an exact formula for y_n. The second reply is a plan for a proof, which Section 18.3 carries out.

The counterexample has changed the question. We now ask how far from x_* the points can be, when that distance still tends to zero, and, when it does not, where the points end up. The constant bias shows that these questions are separate: there the distance stays bounded but does not tend to zero, and the points converge to a different point. Habit 2 (Section 2.6) asks us to compare such a change with the statement contract: the hypotheses are the same, but the conclusion is now a more modest one, an error bound.

Pause and decide. A calculator keeps two decimal places and always rounds down. It runs the update $x \mapsto (x+1)/2$, whose fixed point is 1, starting from 0. Each rounding error is less than 0.01. Do the stored values converge to 1?

Check. No. The stored values are 0, 0.5, 0.75, 0.87, 0.93, 0.96, 0.98, 0.99, 0.99, …; for instance, $(0.75 + 1)/2 = 0.875$ is stored as 0.87. From 0.99 the exact update gives 0.995, which is stored as 0.99 again, so the values stay at distance 0.01 from the fixed point. Rounding down acts like a small bias that always pushes in the same direction.

18.3 The one-step estimate

Write

$$e_n = d(y_n, x_*)$$

for the distance from the nth point to the fixed point, and call it the *error* at step n. Keep the two kinds of error apart: e_n measures how far y_n is from x_*, while the update error ε_n measures how badly a single update went wrong.

To compare e_{n+1} with e_n, insert the exact update $T(y_n)$ as a stop between y_{n+1} and x_*:

$$\begin{aligned}
e_{n+1} &= d(y_{n+1}, x_*) \\
&\le d(y_{n+1}, T(y_n)) + d(T(y_n), x_*) && \text{triangle inequality} \\
&= d(y_{n+1}, T(y_n)) + d(T(y_n), T(x_*)) && \text{since } T(x_*) = x_* \\
&\le \varepsilon_n + q\, d(y_n, x_*) && \text{update error and contraction inequality} \\
&= q e_n + \varepsilon_n.
\end{aligned}$$

So for every $n \in \mathbb{N}$ we have the *one-step estimate*

$$e_{n+1} \le q e_n + \varepsilon_n.$$

In the spirit of Habit 3 (Section 3.4), notice how little the argument uses: the triangle inequality, the fixed-point equation, the bound on the update error and the contraction inequality; completeness enters only through the existence of x_*. So X need not be a set of numbers, and the update errors need not follow any pattern.

Unfolding the estimate three times

To see how an early update error affects later steps, substitute each bound into the next. Every substitution multiplies an inequality by q, which keeps its direction because $q > 0$:

$$\begin{aligned}
&e_1 \le q e_0 + \varepsilon_0, \\
&e_2 \le q e_1 + \varepsilon_1 \le q(q e_0 + \varepsilon_0) + \varepsilon_1 = q^2 e_0 + q\varepsilon_0 + \varepsilon_1, \\
&e_3 \le q e_2 + \varepsilon_2 \le q(q^2 e_0 + q\varepsilon_0 + \varepsilon_1) + \varepsilon_2 = q^3 e_0 + q^2 \varepsilon_0 + q\varepsilon_1 + \varepsilon_2.
\end{aligned}$$

Each term has been multiplied by q once for every update that came after it: e_0 three times, ε_0 twice, and the latest error ε_2 not at all. The same count works at every step: ε_j is made while producing y_{j+1}, and after it come the $n - (j + 1) = n - 1 - j$ updates that produce $y_{j+2}, \dots, y_n$. So in the bound for e_n, ε_j carries the weight q^{n-1-j}, from q^{n-1} for ε_0 down to $q^0 = 1$ for ε_{n-1}.

Two assumptions about the update errors

Parts 2 and 3 of the theorem below make different assumptions. Part 2 assumes that the update errors are *bounded*: $\varepsilon_j \le \eta$ for every j, for a fixed number $\eta \ge 0$, as for the constant bias. Part 3 assumes that they *tend to zero* in the sense of Definition 11.2: for every tolerance $\delta > 0$, all the update errors from some index onward are smaller than δ. (We write δ because ε already names the update errors.) This is the stronger assumption, because errors that tend to zero are bounded: with the tolerance 1, every update error from some index J on is below 1, so the largest of $1, \varepsilon_0, \dots, \varepsilon_{J-1}$ bounds them all. Yet such errors can add up to a lot: $\varepsilon_j = 1/(j+1)$ tends to zero (Section 11.3), but $\varepsilon_0 + \cdots + \varepsilon_{n-1} = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ is a harmonic sum, which does not converge (Section 11.5). We will see that $y_n \to x_*$ all the same.

Theorem 18.1 (Iteration with update errors)**.** *Let (X, d) be a nonempty complete metric space, let $T : X \to X$ be a contraction with factor q, where $0 < q < 1$, and let x_* be its fixed point. Let $y_0, y_1, y_2, \dots$ be points of X and let $\varepsilon_0, \varepsilon_1, \varepsilon_2, \dots$ be nonnegative numbers such that $d(y_{n+1}, T(y_n)) \le \varepsilon_n$ for every $n \in \mathbb{N}$. Write $e_n = d(y_n, x_*)$.*

1. *For every integer $n \ge 1$,*
$$e_n \le q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j.$$
2. *If there is a number $\eta \ge 0$ with $\varepsilon_j \le \eta$ for every $j \in \mathbb{N}$, then for every integer $n \ge 1$,*
$$e_n \le q^n e_0 + \eta \, \frac{1 - q^n}{1 - q}.$$
3. *If $\varepsilon_n \to 0$, then $y_n \to x_*$.*

In part 2, the term $q^n e_0$, what remains of the starting error, tends to zero, while the second term, the combined effect of the update errors, never exceeds $\eta/(1-q)$ and approaches it as n grows. So after many steps the error is at most a little more than $\eta/(1-q)$.

Proof. **Part 1.** We use induction on n. For $n = 1$, the sum has the single term $j = 0$, with weight $q^0 = 1$, so the claim is the one-step estimate $e_1 \le q e_0 + \varepsilon_0$. Now suppose the claim holds for some $n \ge 1$. Apply the one-step estimate, and then the induction hypothesis multiplied by $q > 0$:

$$\begin{aligned} e_{n+1} &\le q e_n + \varepsilon_n \\ &\le q \left(q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j \right) + \varepsilon_n \\ &= q^{n+1} e_0 + \sum_{j=0}^{n-1} q^{n-j} \varepsilon_j + q^0 \varepsilon_n \\ &= q^{n+1} e_0 + \sum_{j=0}^{n} q^{n-j} \varepsilon_j. \end{aligned}$$

In the third line, $q \cdot q^{n-1-j} = q^{n-j}$, and ε_n is written as $q^0 \varepsilon_n$ so that it becomes the term $j = n$ of

the new sum. Since $n - j = (n + 1) - 1 - j$, the last line is the claim for $n + 1$, which completes the induction.

Part 2. Suppose $\varepsilon_j \le \eta$ for every j. Every weight is positive, so replacing each ε_j by the larger number η cannot make the sum smaller. Then put $k = n - 1 - j$; as j runs from 0 up to $n - 1$, the exponent k runs from $n - 1$ down to 0:

$$\sum_{j=0}^{n-1} q^{n-1-j}\varepsilon_j \le \eta \sum_{j=0}^{n-1} q^{n-1-j} = \eta \sum_{k=0}^{n-1} q^k = \eta\,\frac{1-q^n}{1-q},$$

by the finite geometric sum of Section 11.7. Together with part 1, this proves part 2.

Part 3: the plan. Now suppose $\varepsilon_n \to 0$. We must find, for every $\delta > 0$, a threshold N with $e_n < \delta$ for every $n \ge N$. The early update errors in the sum of part 1 may be large, but they are finitely many, and each later update multiplies their effect by another factor q. The late ones may be many, but each is small, and their weights add up to less than $1/(1 - q)$. So we split the sum at an index J. First δ is given, then J is chosen depending on δ, and only then N. The index J must not change with n, so that the early part is one fixed collection of finitely many terms, which a single N can make small for every $n \ge N$ at once.

Part 3: choosing J. Let $\delta > 0$. The number $(1 - q)\delta/2$ is positive, and $\varepsilon_j \to 0$, so the definition of convergence, with this tolerance and the distance $|\varepsilon_j - 0| = \varepsilon_j$, gives an index J such that

$$\varepsilon_j < \frac{(1-q)\delta}{2} \qquad \text{for every } j \ge J.$$

A larger index works as well, so we may take $J \ge 1$; then the early part below contains at least ε_0. For every $n > J$, split the sum of part 1 into the terms with $j < J$ and those with $j \ge J$:

$$e_n \le \left(q^n e_0 + \sum_{j=0}^{J-1} q^{n-1-j}\varepsilon_j \right) + \sum_{j=J}^{n-1} q^{n-1-j}\varepsilon_j.$$

Part 3: the early part. Written out, the expression in parentheses is

$$q^n e_0 + q^{n-1}\varepsilon_0 + q^{n-2}\varepsilon_1 + \cdots + q^{n-J}\varepsilon_{J-1}.$$

The smallest exponent is $n - J$, and factoring out q^{n-J} leaves

$$C_J = q^J e_0 + q^{J-1}\varepsilon_0 + q^{J-2}\varepsilon_1 + \cdots + \varepsilon_{J-1},$$

so the early part equals $q^{n-J}C_J$, where $C_J \ge 0$ depends on J but not on n.

Part 3: the late part. Every ε_j with $j \ge J$ is less than $(1 - q)\delta/2$, the weights are positive, and the sum has at least the term $j = n - 1$, because $n > J$. Substituting $k = n - 1 - j$ again,

$$\sum_{j=J}^{n-1} q^{n-1-j}\varepsilon_j < \frac{(1-q)\delta}{2} \sum_{k=0}^{n-1-J} q^k = \frac{(1-q)\delta}{2} \cdot \frac{1-q^{n-J}}{1-q} < \frac{(1-q)\delta}{2} \cdot \frac{1}{1-q} = \frac{\delta}{2}.$$

This is why we asked for the tolerance $(1 - q)\delta/2$: its factor $1 - q$ cancels the $1/(1 - q)$ that comes from adding up the weights, leaving half of δ. The other half is kept for the early part.

Part 3: choosing N. So far, for every $n > J$,

$$e_n < q^{n-J} C_J + \frac{\delta}{2}.$$

If $C_J = 0$, this gives $e_n < \delta$ for every $n > J$, and $N = J + 1$ will do. If $C_J > 0$, then $q^m \to 0$ (Section 11.7) gives a threshold $M \geq 1$ such that $q^m < \delta/(2C_J)$ for every $m \geq M$. Put $N = J + M$. For every $n \geq N$ we have $n > J$ and $n - J \geq M$, so $q^{n-J} C_J < \delta/2$, and therefore

$$e_n < \frac{\delta}{2} + \frac{\delta}{2} = \delta.$$

In both cases $d(y_n, x_*) = e_n < \delta$ for every $n \geq N$. Since $\delta > 0$ was arbitrary, $y_n \to x_*$. □

Splitting a sum into finitely many early terms, damped by the powers of q, and late terms that are all small is a technique worth remembering; it combines the split of a tolerance into halves (Section 11.5) with a split of the sum. In particular, the proof covers $\varepsilon_j = 1/(j+1)$, whose total is unbounded: it controls only the weighted sum, in which older errors carry smaller powers of q.

Pause and decide. Part 3 of Theorem 18.1 assumes only that $\varepsilon_n \to 0$. Does it tell us how many steps are needed to get within 0.001 of x_*?

Check. No. The threshold N depends on J, and J on how quickly the update errors shrink, which part 3 does not specify; slowly shrinking errors can mean slow convergence. Take $T(x) = qx$, with fixed point 0, $y_0 = 1$ and $y_{n+1} = qy_n + \varepsilon_n$. Each update multiplies by $q > 0$ and adds $\varepsilon_n \geq 0$, so every y_n is nonnegative, and for $n \geq 1$ the error is $e_n = y_n = qy_{n-1} + \varepsilon_{n-1} \geq \varepsilon_{n-1}$. With $\varepsilon_j = 1/(j+1)$ this gives $e_n \geq 1/n$, so the error is still at least 0.001 after 1000 steps. Exercise 18.4 gives an explicit bound when $\varepsilon_j \leq aq^j$ for some fixed $a \geq 0$.

18.4 Can the constant be improved?

Our proof used the triangle inequality, which may give away more than necessary. Could a cleverer proof give a smaller factor than $1/(1-q)$ in part 2 of Theorem 18.1? To find out, we test the bound on the assistant's example: $X = \mathbb{R}$, $T(x) = qx$, $y_0 = 0$, a number $\eta \geq 0$, and

$$y_{n+1} = qy_n + \eta \qquad \text{for every } n \in \mathbb{N}.$$

As Section 18.2 checked, T is a contraction with factor q and fixed point $x_* = 0$, and every update error is η. The first points $y_1 = \eta$, $y_2 = q\eta + \eta$ and $y_3 = q^2\eta + q\eta + \eta$ suggest

$$y_n = \eta \sum_{k=0}^{n-1} q^k = \eta \, \frac{1 - q^n}{1 - q} \qquad \text{for every } n \in \mathbb{N}.$$

A pattern is only a guess, so we prove the formula by induction. At $n = 0$ it gives $0 = y_0$. If it holds for some n, then

$$\begin{aligned} y_{n+1} &= q y_n + \eta \\ &= q\,\frac{\eta(1-q^n)}{1-q} + \eta \\ &= \frac{\eta(q-q^{n+1}) + \eta(1-q)}{1-q} \\ &= \frac{\eta(1-q^{n+1})}{1-q}, \end{aligned}$$

which is the formula for $n + 1$. The induction never used the sign of η, so the formula holds for every real number η, negative ones included. Since $y_n - \eta/(1-q) = -\eta q^n/(1-q)$ and $q^n \to 0$, the points approach $\eta/(1-q)$, as the assistant claimed. This is no accident: the biased map $y \mapsto qy + \eta$ is itself a contraction, and its fixed point, the solution of $y = qy + \eta$, is $\eta/(1-q)$, the limit that Theorem 12.2 predicts.

Now compare with the theorem. Here $e_0 = 0$, and since $\eta \geq 0$ and $0 < q^n \leq 1$, every y_n is nonnegative, so

$$e_n = y_n = \eta\,\frac{1-q^n}{1-q}.$$

This is exactly the bound of part 2, $q^n \cdot 0 + \eta(1-q^n)/(1-q)$: the bound holds with equality at every step. So the factor $1/(1-q)$ cannot be replaced by a smaller number. Suppose someone claims that for some $K < 1/(1-q)$, every iteration satisfying our assumptions, with all update errors at most η, obeys $e_n \leq q^n e_0 + K\eta$ for every $n \geq 1$. In the example with $\eta > 0$, the claim says $\eta(1-q^n)/(1-q) \leq K\eta$. Dividing by η and multiplying by $1-q$, both positive, gives $1 - q^n \leq K(1-q)$, that is,

$$q^n \geq 1 - K(1-q).$$

The right side is positive, because $K(1-q) < 1$, and $q^n \to 0$, so the inequality fails for every sufficiently large n.

An upper bound that holds for every example in some class is called *sharp* if it cannot be lowered: every smaller proposed bound fails for some example in the class. So the factor $1/(1-q)$ in part 2 is sharp for the class of all iterations that satisfy our assumptions.

A sharp bound on the distance to x_* still does not tell us whether the points converge. Let $T(x) = 0$ on $\mathbb{R}$, a contraction with every factor q in $(0, 1)$ and fixed point 0. Let $\eta > 0$ and $y_n = (-1)^n\eta$. Every update error is $|(-1)^{n+1}\eta - 0| = \eta$, and every error e_n equals η, within the bound of part 2. Yet the points jump back and forth between η and $-\eta$ and do not converge (Exercise 18.3). That is why part 3 needs the stronger hypothesis $\varepsilon_n \to 0$.

18.5 A numerical experiment with exact arithmetic

To test the theorem on an example computed exactly, take $T(x) = (3x + 1)/4$ on $\mathbb{R}$, the map of Exercise 12.1. It is a contraction with factor $q = 3/4$, because $|T(x) - T(y)| = \frac{3}{4}|x - y|$, and its

fixed point is 1. Add a bias of $1/100$ at every update:

$$y_{n+1} = \frac{3y_n + 1}{4} + \frac{1}{100}, \qquad y_0 = 0.$$

Every update error is $\eta = 1/100$. Since $\frac{1}{4} + \frac{1}{100} = \frac{13}{50}$, the recurrence reads

$$y_{n+1} = \frac{3}{4}y_n + \frac{13}{50}.$$

This biased update has its own fixed point p: the equation $p = \frac{3}{4}p + \frac{13}{50}$ gives $p = \frac{26}{25}$. As in Section 5.3, we track the difference from it: put $z_n = y_n - \frac{26}{25}$. Subtracting the equation $\frac{26}{25} = \frac{3}{4} \cdot \frac{26}{25} + \frac{13}{50}$ from the recurrence gives

$$z_{n+1} = \frac{3}{4}z_n, \qquad z_0 = -\frac{26}{25},$$

so $z_n = -\frac{26}{25}\left(\frac{3}{4}\right)^n$ by induction (this is the formula of Exercise 12.5, with $a = 3/4$ and $b = 13/50$). Therefore

$$y_n = \frac{26}{25}\left(1 - \left(\frac{3}{4}\right)^n\right) \qquad \text{for every } n \in \mathbb{N}.$$

Such a formula, which gives y_n directly from n without computing the earlier points, is called a *closed form*. Since $(3/4)^n \to 0$, the points converge to $26/25$, not to the fixed point 1 of T.

In Theorem 18.1 we have $e_0 = |0 - 1| = 1$, $q = 3/4$ and $\eta = 1/100$, so $\eta/(1 - q) = 1/25$, and part 2 gives a guaranteed bound computed from known quantities, an *error certificate* in the sense of Section 12.2:

$$|y_n - 1| \le \left(\frac{3}{4}\right)^n + \frac{1}{25}\left(1 - \left(\frac{3}{4}\right)^n\right) \qquad \text{for every } n \in \mathbb{N}.$$

(Part 2 gives this for $n \ge 1$; at $n = 0$ it reads $1 \le 1$.) The closed form gives the true error exactly: $y_n - 1 = \frac{1}{25} - \frac{26}{25}\left(\frac{3}{4}\right)^n$. The table compares the two, rounded to four decimal places. Some certificates therefore appear rounded down, such as $0.13610\ldots$ as 0.1361; a certificate used as a guarantee must be rounded up instead (Section 12.5).

n	y_n	true error $\lvert y_n - 1\rvert$	certificate
0	0.0000	1.0000	1.0000
1	0.2600	0.7400	0.7600
2	0.4550	0.5450	0.5800
4	0.7109	0.2891	0.3438
8	0.9359	0.0641	0.1361
11	0.9961	0.0039	0.0805
12	1.0071	0.0071	0.0704
16	1.0296	0.0296	0.0496
20	1.0367	0.0367	0.0430
30	1.0398	0.0398	0.0402

The true error first decreases, to about 0.0039 at $n = 11$. Between $n = 11$ and $n = 12$ the points pass the fixed point 1, and from then on the error grows again, towards $1/25 = 0.04$. Someone who stopped at $n = 11$ might conclude that the iteration converges to 1; the closed form shows that the points overshoot and settle at $26/25$. The theorem adds something no table can: before any computing, it guarantees that the error never exceeds the certificate.

The last program in Section B.3 of Appendix B checks the closed form and the certificate in exact fractions for $n = 0, \ldots, 20$; changing `range(21)` to `range(31)` extends the run to $n = 30$.

Other patterns of update errors give further experiments, again from $y_0 = 0$. With alternating signs, $y_{n+1} = \frac{3y_n+1}{4} + \frac{(-1)^n}{100}$, the computed points soon jump back and forth around 1: those with even n appear to approach $1 - \frac{1}{175}$, those with odd n to approach $1 + \frac{1}{175}$. To see where $\frac{1}{175}$ comes from, suppose the even-indexed points settle at $1 + u$ and the odd-indexed ones at $1 + v$. Since $T(1 + u) = 1 + \frac{3}{4}u$, and an update from an even index adds $\frac{1}{100}$ while one from an odd index subtracts it, this gives $v = \frac{3}{4}u + \frac{1}{100}$ and $u = \frac{3}{4}v - \frac{1}{100}$. Substituting the first equation into the second gives $u = \frac{9}{16}u + \frac{3}{400} - \frac{1}{100}$, so $\frac{7}{16}u = -\frac{1}{400}$ and $u = -\frac{1}{175}$; then $v = -\frac{3}{700} + \frac{1}{100} = \frac{1}{175}$. This calculation assumes that the points settle; it does not prove that they do.

With update errors that tend to zero, such as $\frac{1}{100(n+1)}$ at step n, the points converge to 1. All three patterns, constant, alternating and shrinking, have update errors at most $1/100$, so the certificate holds for each, but Theorem 18.1 guarantees convergence to 1 only for the shrinking errors. For the alternating errors it says nothing either way about convergence: a computation can suggest the answer, but only a separate argument can prove it.

Sharp, but not always attained

Sharpness does not mean that the certificate equals the true error in every example; here it does so only at $n = 0$. To see why, split the true error into its two sources:

$$y_n - 1 = -\left(\frac{3}{4}\right)^n + \frac{1}{25}\left(1 - \left(\frac{3}{4}\right)^n\right).$$

The first term, what remains of the starting error, is negative, because $y_0 = 0$ lies below 1. The second, the accumulated bias, is positive for $n \geq 1$, because the bias pushes upward. The certificate is exactly the sum of their sizes, because the triangle inequality ignores signs. For $n \geq 1$ the two terms have opposite signs and partly cancel, so the true error is strictly smaller than the certificate. A bound that is larger than the true error in a particular example is called *conservative* for that example.

So a bound can be sharp for a whole class and still be conservative for one of its members, and for a smaller class with extra structure a better bound may be possible. Which extra assumption rules out the examples that force the factor $1/(1 - q)$, and what is the best factor under it? The alternating errors suggest a candidate: their points appear to settle only $\frac{1}{175}$ away from 1, not $\frac{1}{25}$, because errors of opposite signs partly cancel.

Alternating signs alone are not enough, however; the assumption must also say something about the map. Take $T(x) = -qx$ on $\mathbb{R}$, a contraction with factor q and fixed point 0 that sends every nonzero point to the other side of 0. Let $\eta > 0$, $y_0 = 0$ and $y_{n+1} = -qy_n + (-1)^n\eta$, so that the

update errors are η with alternating signs. Put $w_n = (-1)^n y_n$ and multiply the recurrence by $(-1)^{n+1}$; since $(-1)^{n+1}(-q)y_n = qw_n$ and $(-1)^{n+1}(-1)^n = -1$, this gives

$$w_{n+1} = qw_n - \eta, \qquad w_0 = 0.$$

This is the constant-bias iteration of Section 18.4 with $-\eta$ in place of η, so $w_n = -\eta(1-q^n)/(1-q)$. Hence $e_n = |y_n| = |w_n| = \eta(1-q^n)/(1-q)$, and part 2 holds with equality at every step. From the second update on, each update error has the same sign as $T(y_n)$, so it pushes the point further from 0, and nothing cancels.

18.6 The finished investigation as a short paper

An investigation is finished only when it has been written up so that another reader can check it without seeing your notes or your conversations with an assistant. For a project like this one, a short paper in four parts works well.

- *Introduction.* State the question, explain with the constant bias why the first guess is false, and say which questions replace it: how far from x_* the points can be, and when they still converge.
- *Theory.* State the setting precisely, including the model of the update errors and the requirement that every y_n lies in X. Then give Theorem 18.1 with its proof, and the example showing that the factor $1/(1-q)$ cannot be lowered.
- *Experiments.* Present the exact computation of Section 18.5, with the program and enough detail that anyone can run it again and get the same output; such a computation is called *reproducible*. Experiments illustrate the proved statements; they do not replace the proofs.
- *Conclusion.* Say what the results do not cover. The points must stay in X. The factor q must be less than 1: for $q = 1$ the map need not have a fixed point, as $x \mapsto x + 1$ on $\mathbb{R}$ shows, and even when it has one, the sum $1 + q + \cdots + q^{n-1}$ behind part 2 equals n, so the bound $e_0 + n\eta$ grows without limit when $\eta > 0$. And bounded update errors that do not tend to zero may leave the points converging to a point other than x_*, or not converging at all (Section 18.4).

A paper should also say what is new in it. Here the honest statement is that the project reconstructs and analyzes a standard argument. It must not claim novelty merely because the student, or the assistant, had not met the result before: as *To the reader* pointed out, a result you discover yourself may be correct and still be well known. Before claiming that a result is new in research, compare its exact statement and assumptions with the published literature, especially the *primary sources*, the papers and books in which results first appeared. An unsuccessful search, in a library catalogue or by an assistant, shows only that this search did not find the result.

18.7 Your independent capstone

Now it is your turn. Choose one of the three problems that run through the book (assigning tasks, iterating a map, or sign sequences) and start with a question narrow enough that you can check every claim you make. Here are three possible starting points; none is expected to lead to a new theorem, but each leaves room to go further.

- *Assignments with capacities.* Allow a task to be given to several students, up to a fixed capacity, and find the right version of Hall's condition; Exercise 18.5 takes the first step. Then ask how the search procedure of Chapter 8 changes.
- *Iteration with a specified pattern of errors.* Pick a pattern, such as $\varepsilon_n = 1/(n+1)$ or $\varepsilon_n = s^n$ with $0 < s < 1$ and $s \neq q$, and find out as precisely as you can how fast e_n goes to zero; Exercise 18.4 treats the case $\varepsilon_n \leq aq^n$.
- *Sign sequences at a new length.* Extend the table of Section 14.6 by one more length, with a program that anyone can run to reproduce your counts. Then find out which operations, such as negating every sign, reversing the order, or rotating, preserve each of the two properties the table counts.

Your write-up should contain:

- a precise statement of your question and your result, and, if the question changed while you worked on it, both versions, with the difference explained;
- a map of the background your argument needs, like the prerequisite map of Section 17.7;
- a proof, or a counterexample;
- computer checks where relevant, with the program, the software it runs on, and the range of cases it covers;
- a record of the assumptions you used and of the evidence for each claim: proved, cited, checked by computer for finitely many cases, or not established, as in "Keeping track of evidence" in *To the reader*;
- credit to every source you used, an honest account of how you used an AI assistant, and a list of what remains unsettled;
- a short explanation of what is new compared with the published literature, and what is new only to you.

The investigation ends with a short oral discussion in which you explain one delicate step of your argument, change one of your assumptions, and predict what fails; your answers matter more than how polished your saved conversation with an assistant looks. Section C.6 of Appendix C suggests a grading rubric.

Before you hand in your work, ask an assistant, one request at a time, to restate your argument in its own words (a step it gets wrong is often one you explained badly), to find counterexamples to stronger versions of your statement and say whether each hypothesis is needed, and to check that your prose, formulas, code and any Lean statements agree. All three answers come from the same assistant and may share its blind spots, so they do not replace a careful human reader such as a classmate or an instructor; check each point they raise yourself.

Exercises

Exercises 18.1–18.4 are core exercises, and Exercises 18.5 and 18.6 are transfer exercises. Complete solutions are in Appendix A, Section A.18.

Exercise 18.1. Starting from the one-step estimate $e_{n+1} \le q e_n + \varepsilon_n$, derive a bound for e_4 in terms of e_0 and the update errors, without summation notation, and check that it agrees with part 1 of Theorem 18.1 for $n = 4$. Then find the power of q that multiplies ε_6 in the bound for e_{10}, and explain it by counting the updates that come after ε_6 is made.

Exercise 18.2. Let $0 < q < 1$, $\eta \ge 0$ and $a \ge 0$, and define $y_0 = a$ and $y_{n+1} = q y_n + \eta$ for every $n \in \mathbb{N}$. Prove by induction that $y_n = q^n a + \eta(1 - q^n)/(1 - q)$ for every $n \in \mathbb{N}$. Deduce that, for $T(x) = qx$ on $\mathbb{R}$, these points satisfy the bound of part 2 of Theorem 18.1 with equality at every step, whatever the starting error $e_0 = a$. Then take $a = 0$ and $\eta > 0$, and explain why the factor $1/(1 - q)$ in part 2 cannot be replaced by any smaller number K.

Exercise 18.3. Let $\eta > 0$, let $T(x) = 0$ for every $x \in \mathbb{R}$, and let $y_n = (-1)^n \eta$ for every $n \in \mathbb{N}$. Check that all the hypotheses of Theorem 18.1 hold with update errors $\varepsilon_n = \eta$, and check directly that the bound of part 2 holds. Then prove that the sequence $y_0, y_1, y_2, \ldots$ does not converge, and explain why a valid error bound does not settle whether a sequence converges.

Exercise 18.4. Suppose that the update errors in Theorem 18.1 satisfy $\varepsilon_j \le a q^j$ for every $j \in \mathbb{N}$, where $a \ge 0$ and q is the contraction factor. Derive the bound $e_n \le q^n e_0 + a n q^{n-1}$ for every $n \ge 1$, and prove that this upper bound tends to zero as n grows. (Hint: the factor n grows, so it is not enough that $q^{n-1} \to 0$. Put $r = (1 + q)/2$ and $t = q/r$, and compare $n t^{n-1}$ with $1 + t + \cdots + t^{n-1}$.)

Exercise 18.5. As in Chapter 9, L is a finite set of students, R a finite set of tasks, each student accepts some of the tasks, and for $S \subseteq L$, $N(S)$ is the set of tasks accepted by at least one student in S. Each task $r \in R$ now has a *capacity* c_r, which is a nonnegative integer. An *assignment with capacities* gives every student one task that the student accepts, in such a way that each task r is given to at most c_r students. Prove that an assignment with capacities exists if and only if

$$|S| \le \sum_{r \in N(S)} c_r \qquad \text{for every set } S \subseteq L.$$

When every capacity equals 1, this is Hall's theorem. (Hint: replace each task r by c_r separate slots, each of which can be given to only one student, and apply Hall's theorem to the new graph.)

Exercise 18.6. Write a short record of the investigation of the biased iteration $y_{n+1} = (3y_n + 1)/4 + 1/100$, $y_0 = 0$, of Section 18.5. It should contain the exact question, the background the argument needs, an outline of the proof, the counterexample that refutes the first guess, one numerical check done in exact arithmetic, and an honest statement of what is new. Then state one further question whose answer does not follow from the proofs in this chapter.

APPENDIX A

Solutions

This appendix contains complete solutions to every exercise in the book, grouped by chapter: the section headed "Chapter N" holds the six solutions for Chapter N.

Make a serious attempt before you read a solution, and then read it to learn from it, not only to confirm an answer. If your attempt succeeded, compare it with the solution anyway: did you prove exactly what was asked, in every case, including boundary cases such as zero or the empty set? If it stalled, find where your argument stopped and the idea that carries the solution past that point; then close the book and write the whole argument in your own words. Either way, finish by trying a small variation, such as different numbers or one hypothesis removed, and see whether the idea still works.

Where an exercise asks you to invent an example or explain something in your own words, the solution gives one model answer; a different answer is just as correct if it meets every requirement of the exercise.

Several solutions for Chapters 15 and 16 are written in Lean; Section B.4 explains how to have Lean check them on your own computer. The independent capstone of Section 18.7 is your own project, so it has no model solution; Section C.6 suggests how to assess it.

A.1 Chapter 1

Solution to Exercise 1.1

At $x = 4$,

$$2(x + 3) = 2(4 + 3) = 2 \cdot 7 = 14, \qquad \text{whereas} \qquad 2x + 3 = 2 \cdot 4 + 3 = 8 + 3 = 11.$$

In $2(x + 3)$ we add three to the input and then double the result, so the three is doubled as well. In $2x + 3$ we double only the input and add three afterwards. Indeed $2(x + 3) = 2x + 6$, which is always exactly 3 more than $2x + 3$.

Solution to Exercise 1.2

Let a be an even integer and b an odd integer. By the definitions of even and odd, there are integers k and ℓ with $a = 2k$ and $b = 2\ell + 1$; we use two different letters because nothing says the two witnesses are equal. Substitute and regroup:

$$\begin{aligned} a + b &= 2k + (2\ell + 1) \\ &= 2k + 2\ell + 1 \\ &= 2(k + \ell) + 1. \end{aligned}$$

The number $k + \ell$ is an integer, because a sum of integers is an integer. So $a + b$ is twice an integer plus one, which by the definition of odd means that $a + b$ is odd. The witness for the oddness of the sum is $k + \ell$.

Solution to Exercise 1.3

Writing both factors as $2k + 1$, with the same k, forces them to be equal. So the argument covers only products of an odd integer with itself, such as $3 \cdot 3$ or $(-5) \cdot (-5)$, and not products such as $3 \cdot 5$. The fact it relies on, that $(2k + 1)^2$ is odd, is true, but narrower than the claim.

For the general claim, let a and b be odd integers. By the definition of odd, there are integers k and ℓ, possibly different, with $a = 2k + 1$ and $b = 2\ell + 1$. Multiply out one bracket at a time, as in Section 1.3:

$$\begin{aligned} ab &= (2k + 1)(2\ell + 1) \\ &= 2k(2\ell + 1) + (2\ell + 1) \\ &= 4k\ell + 2k + 2\ell + 1 \\ &= 2(2k\ell + k + \ell) + 1. \end{aligned}$$

The number $2k\ell + k + \ell$ is an integer, because products and sums of integers are integers. So ab is twice an integer plus one, and therefore odd. For instance, $3 = 2 \cdot 1 + 1$ and $5 = 2 \cdot 2 + 1$ give $k = 1$ and $\ell = 2$, so the witness is $2 \cdot 1 \cdot 2 + 1 + 2 = 7$, and indeed $3 \cdot 5 = 2 \cdot 7 + 1$. Setting $\ell = k$ recovers the assistant's special case.

Solution to Exercise 1.4

The index j takes the values 1, 2, and 3, so the sum has three terms:

$$\begin{aligned} \sum_{j=1}^{3} (2a_j + 1) &= (2a_1 + 1) + (2a_2 + 1) + (2a_3 + 1) \\ &= (2 \cdot 2 + 1) + (2 \cdot (-1) + 1) + (2 \cdot 4 + 1) \\ &= 5 + (-1) + 9 = 13. \end{aligned}$$

As a check, the same sum regroups as $2(a_1 + a_2 + a_3) + 3 = 2 \cdot 5 + 3 = 13$.

Solution to Exercise 1.5

By the fact quoted in the exercise, $x(x - 2) = 0$ holds only if $x = 0$ or $x - 2 = 0$, that is, only if $x = 0$ or $x = 2$. Both values work: $0 \cdot (0 - 2) = 0$ and $2 \cdot (2 - 2) = 2 \cdot 0 = 0$. So the real solutions are exactly 0 and 2.

Dividing both sides by x gives $x - 2 = 0$, so $x = 2$. But this division is allowed only when $x \neq 0$, so it says nothing about $x = 0$, which is in fact a solution. The case $x = 0$ must be checked separately, as for $x^2 = x$ in Section 1.2.

Solution to Exercise 1.6

Use

$$f(n) = (n-1)(n-2)\cdots(n-10),$$

the product of the ten factors $n-1, n-2, \ldots, n-10$. If n is one of the integers $1, 2, \ldots, 10$, then one of the factors is $n - n = 0$, so the product is 0. At $n = 11$ the factors are $10, 9, \ldots, 1$, so

$$f(11) = 10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 3\,628\,800 \neq 0.$$

This f is zero at all ten checked values but not at $n = 11$, so checking finitely many values cannot, on its own, establish a statement about every positive integer.

A.2 Chapter 2

Solution to Exercise 2.1

The original implication is true: if $x = 3$, then $x^2 = 3 \cdot 3 = 9$.

The converse is "If $x^2 = 9$, then $x = 3$." It is false, and $x = -3$ is a counterexample: $(-3)^2 = 9$, but $-3 \neq 3$.

The contrapositive is "If $x^2 \neq 9$, then $x \neq 3$." It is true, because it is equivalent to the original implication, which is true.

Solution to Exercise 2.2

Work from the outside in. The statement says that *every* integer n has a successful response m, so it fails if *some* integer n has none. The negation therefore begins "There is an integer n such that, for every integer m, . . . ," with both domains unchanged.

A response m succeeds when $m > n$ *and* m is even. By De Morgan's law, it fails when $m \leq n$ (which includes $m = n$) or m is not even. In symbols, the original statement is

$$\forall n \in \mathbb{Z}\ \exists m \in \mathbb{Z} : \quad (m > n) \wedge (m \text{ is even}),$$

and its negation is

$$\exists n \in \mathbb{Z}\ \forall m \in \mathbb{Z} : \quad (m \leq n) \vee \neg(m \text{ is even}).$$

In words: there is an integer n such that every integer m is at most n or is not even. The "or" is inclusive: an m that fails both requirements, such as an odd integer smaller than n, is still an unsuccessful response.

Writing down the negation does not claim that it is true. In fact it is false, because the original statement is true. For any integer n, take $m = 2|n| + 2 = 2(|n| + 1)$, which is an even integer because $|n|$ is n or $-n$. Then $m \geq |n| + 2 > |n| \geq n$: the first step holds because $|n| \geq 0$, and the last because $|n| = n$ when $n \geq 0$, while $|n| > 0 > n$ when $n < 0$.

Solution to Exercise 2.3

The first statement is true. The skeptic chooses x first, and the prover, already knowing x, chooses $y = -x$, since $x + (-x) = 0$.

The second statement is false. It asks for a single y, chosen before x, with $x + y = 0$ for every real number x. Whatever y is proposed, the skeptic can choose $x = 1 - y$, and then $x + y = 1 \neq 0$. So no choice of y works. The two statements differ only in the order of their quantifiers, like statements (A) and (B) in Section 2.3.

Solution to Exercise 2.4

The number $x = -3$ is a counterexample: it satisfies the hypothesis, since $(-3)^2 = 9 > 4$, but -3 is not greater than 2.

The repaired claim is "If a real number x satisfies $x^2 > 4$, then $x < -2$ or $x > 2$," or equivalently "then $|x| > 2$." Section 2.6 proves that $x^2 > 4$ holds exactly when $|x| > 2$, that is, when $x < -2$ or $x > 2$. One direction of this equivalence says that the repaired claim is true for every real x. The other says that every real number satisfying the new conclusion satisfies $x^2 > 4$, so the new conclusion describes exactly the real numbers with $x^2 > 4$.

Other repairs can be true but lose information: "if $x^2 > 4$, then $x \neq 0$" is true, but $x = 1$ satisfies $x \neq 0$ although $1^2 > 4$ fails.

Solution to Exercise 2.5

Let Ada and Ben be the two students, with tasks 1 and 2 available. Suppose both students accept task 1 and neither accepts task 2. Each student accepts at least one task, but task 1 is the only task either of them accepts, so they cannot be given two different acceptable tasks.

Following the hint, consider the condition

for every group of students, the number of tasks acceptable to at least one member of the group is at least the number of students in the group.

In the example, the group made up of Ada and Ben has two students but only one acceptable task between them, so the condition fails.

The condition must hold whenever all the students can be given different acceptable tasks. Suppose such an assignment exists, and take any group of k students. The assignment gives them k different tasks, each acceptable to the member of the group who receives it. Hence at least k tasks are acceptable to members of the group.

The condition is stronger than "each student accepts at least one task": for a group of one student, it says exactly that, and it also checks every larger group, which is where the example fails. Hall's theorem (Theorem 9.1) proves the converse: with finitely many students and tasks, the group condition guarantees that all the students can be given different acceptable tasks.

Solution to Exercise 2.6

Let ε be a positive real number. By the Archimedean property, choose an integer $N > 1/\varepsilon$; as in Example 2.1, N is positive and $1/N < \varepsilon$. Now let n be any integer with $n \geq N$. Since $0 < N \leq n$, taking reciprocals gives $1/n \leq 1/N$. Together,

$$\frac{1}{n} \leq \frac{1}{N} < \varepsilon.$$

The choices are made in this order: first ε is given, then we choose N, and then any integer $n \geq N$ may be tested. Since ε comes first, N may depend on it, and it must: $N = 101$ works for $\varepsilon = 1/100$ but not for $\varepsilon = 1/1000$, and no single N works for every ε, since testing $n = N$ would give $1/N < \varepsilon$ for every positive ε, which the Pause and decide question at the end of Section 2.3 rules out.

The integer n comes after N: the phrase "every integer $n \geq N$" only makes sense once N is fixed. If N could be chosen after seeing n, we could take $N = n + 1$; then $n \geq N$ would be false, so "if $n \geq N$, then $1/n < \varepsilon$" would hold vacuously (Section 2.5), whatever condition followed. With the choices in that order the statement would promise nothing. With N fixed before n, it makes a real promise: from N onward, every number $1/n$ is less than ε. Chapter 11 defines limits by statements of exactly this shape.

A.3 Chapter 3

Solution to Exercise 3.1

$A \cup B = \{1, 2, 3, 4\}$, with 2 and 4 listed once although they lie in both sets; $A \cap B = \{2, 4\}$; and $A \setminus B = \{1\}$, because the members 2 and 4 of A also lie in B.

Solution to Exercise 3.2

To prove that $g \circ f$ is surjective, we start from the *output* end. Take an arbitrary target $c \in C$; we must find a witness $a \in A$ with $g(f(a)) = c$.

Since g is surjective and c belongs to its codomain C, there is some $b \in B$ with $g(b) = c$. Since f is surjective and this b belongs to its codomain B, there is some $a \in A$ with $f(a) = b$. Then

$$(g \circ f)(a) = g(f(a)) = g(b) = c.$$

Since c was an arbitrary target, the composition is surjective.

Notice the order. The functions are applied with f first and g second, but the witnesses are found the other way round: first b for g, then a for f. We could not start by choosing a, because until we know b we do not know which target of f to aim for.

Solution to Exercise 3.3

Injective. Let $m, n \in \mathbb{Z}$ with $f(m) = f(n)$, that is, $2m = 2n$. Dividing both sides by 2 gives $m = n$. So f is injective.

Not surjective. If $2n = 1$, then $n = 1/2$, which is not an integer. So the target 1 is never reached, and f is not surjective.

A bijection. Let $2\mathbb{Z} = \{2k : k \in \mathbb{Z}\}$ be the set of even integers, and use the rule $n \mapsto 2n$ as a function from $\mathbb{Z}$ to $2\mathbb{Z}$; this is allowed, because every output $2n$ is even. It is still injective by the same argument, since changing the codomain does not change any output. It is now surjective: each member of $2\mathbb{Z}$ has the form $2k$ for some $k \in \mathbb{Z}$, and the input k has output $2k$. Hence it is a bijection. The new codomain is exactly the image of the original function.

Solution to Exercise 3.4

Let $x \in A$. Then

$$\begin{aligned} x \in f^{-1}(S \cup T) &\iff f(x) \in S \cup T && \text{definition of preimage} \\ &\iff f(x) \in S \text{ or } f(x) \in T && \text{definition of union} \\ &\iff x \in f^{-1}(S) \text{ or } x \in f^{-1}(T) && \text{definition of preimage} \\ &\iff x \in f^{-1}(S) \cup f^{-1}(T) && \text{definition of union.} \end{aligned}$$

As for intersections in Section 3.4, reading this chain of equivalences downward and then upward gives the two containments, so the sets are equal. No injectivity, surjectivity, or inverse function is needed.

Solution to Exercise 3.5

Left inverse. Here $h(f(a)) = a$ for every $a \in A$. To prove that f is injective, take $x, y \in A$ with $f(x) = f(y)$. Since $f(x)$ and $f(y)$ are equal elements of B, the domain of h, applying h gives

$$h(f(x)) = h(f(y)).$$

By the left-inverse identity, the left side equals x and the right side equals y. So $x = y$.

Right inverse. Here $f(h(b)) = b$ for every $b \in B$. To prove that f is surjective, take any target $b \in B$. The element $h(b) \in A$ is a witness, because $f(h(b)) = b$.

An inverse satisfies both identities, so by the two parts, every function that has an inverse is bijective. A function can, however, have one kind of one-sided inverse without the other. For example, let $A = \{0\}$, $B = \{0, 1\}$, $f(0) = 0$, and $h(0) = h(1) = 0$. Then $h \circ f = \mathrm{id}_A$, so h is a left inverse of f. But no input reaches 1, so f is not surjective and, by the second part, has no right inverse at all.

Solution to Exercise 3.6

No, equality can fail, but one containment always holds.

The containment $f(S \cap T) \subseteq f(S) \cap f(T)$. Let $z \in f(S \cap T)$. Then $z = f(x)$ for some $x \in S \cap T$. That x belongs to S, so $z \in f(S)$, and it belongs to T, so $z \in f(T)$. Hence $z \in f(S) \cap f(T)$.

A counterexample to equality. Let $f : \mathbb{R} \to \mathbb{R}$ be $f(x) = x^2$, with $S = \{1\}$ and $T = \{-1\}$. Then $S \cap T = \varnothing$, and the image of the empty set is empty, since there are no inputs to produce outputs; so $f(S \cap T) = \varnothing$. But $f(S) = f(T) = \{1\}$, so $f(S) \cap f(T) = \{1\}$.

The containment proof cannot be reversed: a member z of $f(S) \cap f(T)$ has preimages $s \in S$ and $t \in T$, but they may be different inputs, like 1 and -1 here, and then neither needs to lie in $S \cap T$. If f is injective, however, $f(s) = z = f(t)$ forces $s = t \in S \cap T$, so the equality holds.

A.4 Chapter 4

Solution to Exercise 4.1

Let a and b be even integers. By the definition of even, $a = 2k$ and $b = 2\ell$ for some integers k and ℓ; we use two letters because a and b may be different. Then

$$ab = (2k)(2\ell) = 4k\ell.$$

Since $k\ell$ is an integer, ab is four times an integer, which is what "divisible by four" means (Section 2.1).

Solution to Exercise 4.2

The contrapositive of the statement is: *if n is not odd, then $3n + 1$ is not even.* We prove it.

Suppose the integer n is not odd. Since every integer is even or odd (Lemma 4.1), n is even, so $n = 2k$ for some integer k. Then

$$3n + 1 = 3(2k) + 1 = 6k + 1 = 2(3k) + 1.$$

Since $3k$ is an integer, $3n + 1$ is odd, and since no integer is both even and odd (Lemma 4.1 again), $3n + 1$ is not even. This proves the contrapositive, which is equivalent to the original statement (Section 2.5).

Solution to Exercise 4.3

Suppose, for contradiction, that $s = r + u$ is rational. Then $u = s - r$ is a difference of two rational numbers, and such a difference is always rational. Write $s = a/b$ and $r = c/d$ with integers a, b, c, d, where $b \neq 0$ and $d \neq 0$. Over the common denominator bd,

$$s - r = \frac{a}{b} - \frac{c}{d} = \frac{ad}{bd} - \frac{bc}{bd} = \frac{ad - bc}{bd}.$$

Here $ad - bc$ is an integer, and bd is a nonzero integer, because a product of two nonzero numbers is nonzero. So $s - r$ is rational.

Hence u would be rational, contradicting the hypothesis that u is irrational. So $r + u$ is irrational.

Solution to Exercise 4.4

Existence. The number 2 is a solution, since $3 \cdot 2 - 4 = 2$.

Uniqueness. Suppose that x and y are real solutions, so $x = 3x - 4$ and $y = 3y - 4$. Subtracting the second equation from the first cancels the constants and gives $x - y = 3(x - y)$, so $0 = 2(x - y)$, and $x = y$.

So the equation has exactly one real solution, namely 2. Rearranging also proves uniqueness: if $x = 3x - 4$, then $2x = 4$, so $x = 2$. But this shows only that a solution must equal 2; the existence check shows that 2 is one.

Solution to Exercise 4.5

The assistant has missed the negative solution $-\sqrt{2}$. The symbol $\sqrt{2}$ denotes only the *nonnegative* number whose square is 2.

Both numbers are solutions. By definition $(\sqrt{2})^2 = 2$, and $(-\sqrt{2})^2 = (-1)^2(\sqrt{2})^2 = 2$.

There are no others. Let x be a real number with $x^2 = 2$. Then

$$(x - \sqrt{2})(x + \sqrt{2}) = x^2 + \sqrt{2}\,x - \sqrt{2}\,x - (\sqrt{2})^2 = x^2 - 2 = 0.$$

A product of two real numbers is zero only when a factor is zero (the fact used for $x^2 = 1$ in Section 4.4), so $x = \sqrt{2}$ or $x = -\sqrt{2}$.

So the real solutions of $x^2 = 2$ are exactly $\sqrt{2}$ and $-\sqrt{2}$. The assistant's answer is correct only under the extra assumption $x \geq 0$.

Solution to Exercise 4.6

First proof: using Theorem 4.3. We have $\sqrt{8} = 2\sqrt{2}$, because $2\sqrt{2}$ is nonnegative and $(2\sqrt{2})^2 = 4 \cdot 2 = 8$, and $\sqrt{8}$ is the only nonnegative number whose square is 8 (Section 1.8). Now suppose, for contradiction, that $\sqrt{8} = a/b$ with integers a and b and $b \neq 0$. Then

$$\sqrt{2} = \frac{\sqrt{8}}{2} = \frac{a}{2b},$$

and $2b$ is a nonzero integer, so $\sqrt{2}$ would be rational, contradicting Theorem 4.3. So $\sqrt{8}$ is irrational.

Second proof: adapting the argument. Suppose, for contradiction, that $\sqrt{8}$ is rational. Exactly as in the proof of Theorem 4.3, write $\sqrt{8} = a/b$ with integers a and b, where b is the smallest positive denominator for which such a fraction exists. Squaring and multiplying by b^2 gives $a^2 = 8b^2 = 2(4b^2)$, so a^2 is even, and Lemma 4.2 shows that a is even, say $a = 2c$. Then $4c^2 = 8b^2$, so $c^2 = 2b^2$, which is even; by the same lemma c is even, say $c = 2e$. Then $4e^2 = 2b^2$, so $b^2 = 2e^2$, which is even; so b is even, say $b = 2d$, and since $b > 0$, we have $0 < d < b$. Now

$$\sqrt{8} = \frac{a}{b} = \frac{2c}{2d} = \frac{c}{d}$$

is a fraction of integers with a positive denominator smaller than b, contradicting the choice of b. Hence $\sqrt{8}$ is irrational.

How the parity steps change. For $\sqrt{2}$, one halving of the numerator, $a = 2c$, already gives $b^2 = 2c^2$, so b is even. For $\sqrt{8}$, since $8 = 4 \cdot 2$, replacing a by $2c$ removes only one factor of 4 and leaves $c^2 = 2b^2$. One extra halving, $c = 2e$, is needed before we can conclude that b is even.

A.5 Chapter 5

Solution to Exercise 5.1

We use induction on n.

Base case. For $n = 0$ the sum $\sum_{j=1}^{0} j$ is the empty sum, which is 0, and the right side is $0 \cdot (0+1)/2 = 0$.

Induction step. Let k be a natural number, and assume the induction hypothesis $\sum_{j=1}^{k} j = k(k+1)/2$. Split off the last term of the sum for $k+1$:

$$\begin{aligned}
\sum_{j=1}^{k+1} j &= \sum_{j=1}^{k} j + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) && \text{by the induction hypothesis} \\
&= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\
&= \frac{k(k+1) + 2(k+1)}{2} \\
&= \frac{(k+1)(k+2)}{2}.
\end{aligned}$$

The last line factors out $k+1$. Since $(k+1)\big((k+1)+1\big)/2 = (k+1)(k+2)/2$, this is the formula with $k+1$ in place of n. By induction, the formula holds for every $n \in \mathbb{N}$.

Solution to Exercise 5.2

The first terms are $y_1 = 2 \cdot 3 = 6$, $y_2 = 2 \cdot 6 = 12$ and $y_3 = 2 \cdot 12 = 24$. Each term is twice the one before, so y_n should be 3 doubled n times: $y_n = 3 \cdot 2^n$.

Base case. At $n = 0$ the formula gives $3 \cdot 2^0 = 3$, which is y_0.

Induction step. Let n be a natural number, and assume that $y_n = 3 \cdot 2^n$. Then

$$y_{n+1} = 2y_n = 2 \cdot (3 \cdot 2^n) = 3 \cdot 2^{n+1}.$$

This is the formula at $n+1$. By induction, $y_n = 3 \cdot 2^n$ for every $n \in \mathbb{N}$.

Solution to Exercise 5.3

The divisions are

$$1071 = 2 \cdot 462 + 147, \qquad 462 = 3 \cdot 147 + 21, \qquad 147 = 7 \cdot 21 + 0.$$

The quotients are $2, 3, 7$, and the remainders are $147, 21, 0$. The successive pairs are

$$(1071, 462), \quad (462, 147), \quad (147, 21), \quad (21, 0).$$

The second number of the last pair is 0, so the algorithm stops and outputs 21. By Lemma 5.3, no step changes the greatest common divisor, so

$$\gcd(1071, 462) = \gcd(462, 147) = \gcd(147, 21) = \gcd(21, 0) = 21.$$

As a check, $1071 = 21 \cdot 51$ and $462 = 21 \cdot 22$. Notice that the answer is the last nonzero remainder, not the last quotient.

Solution to Exercise 5.4

No. As in Section 5.4, replace each plus sign by 1 and each minus sign by -1, and consider the product of the five numbers. A legal move flips two signs, so it multiplies the product by $(-1) \cdot (-1) = 1$. By induction on the number of moves, every row that can be reached has the starting product $1^5 = 1$. A row with exactly three minus signs and two plus signs has product $(-1)^3 \cdot 1^2 = -1$, so no sequence of legal moves, however long, produces such a row.

Solution to Exercise 5.5

As in Section 5.3, subtracting 1 from both sides of the rule gives the halving rule $x_{n+1} - 1 = (x_n - 1)/2$ for every $n \in \mathbb{N}$. We prove $x_n - 1 = 2^{-n}(x_0 - 1)$ by induction on n.

Base case. At $n = 0$ the right side is $2^0(x_0 - 1) = x_0 - 1$, which is the left side.

Induction step. Let n be a natural number, and assume that $x_n - 1 = 2^{-n}(x_0 - 1)$. Then

$$x_{n+1} - 1 = \frac{x_n - 1}{2} = \frac{2^{-n}(x_0 - 1)}{2} = 2^{-(n+1)}(x_0 - 1),$$

using the halving rule above, then the induction hypothesis, then $2^{-n}/2 = 2^{-(n+1)}$. This is the formula at $n + 1$, which completes the induction.

The error is never zero. Suppose that $x_0 \neq 1$, and let n be any natural number. The number $2^{-n} = 1/2^n$ is positive, so it is not zero, and $x_0 - 1$ is not zero by assumption. A product of two nonzero real numbers is never zero, so $x_n - 1 = 2^{-n}(x_0 - 1) \neq 0$: the error halves at every step but never reaches zero.

Solution to Exercise 5.6

The base case gives $P(0)$, the step with $k = 0$ gives $P(2)$, the step with $k = 2$ gives $P(4)$, and so on. To make "and so on" precise, apply ordinary induction on m to the statement $P(2m)$: the base case $m = 0$ is $P(0)$, and the step with $k = 2m$ turns $P(2m)$ into $P(2m + 2) = P\big(2(m + 1)\big)$. So $P(2m)$ holds for every $m \in \mathbb{N}$. An even natural number n is $2m$ with $m = n/2 \geq 0$, so $P(n)$ holds for every even natural number n.

Nothing follows about the odd numbers, because each use of the step moves from an even number to the next even number. Indeed, take $P(n)$ to be "n is even." Then $P(0)$ is true, and the step holds, because if k is even, so is $k + 2$; yet $P(n)$ is false for every odd n. So the answer to the first question is the even natural numbers, and only those.

The additional base case $P(1)$ is enough. The same induction on m, applied to $P(2m + 1)$ with base case $P(1)$ and the step with $k = 2m+1$, which turns $P(2m+1)$ into $P(2m+3) = P\big(2(m+1)+1\big)$, gives $P(2m + 1)$ for every $m \in \mathbb{N}$. Every natural number n is even or odd (Lemma 4.1). If $n = 2m + 1$ is odd, then $2m = n - 1 \geq -1$, so the integer m is at least $-\frac{1}{2}$ and hence at least 0. So $m \in \mathbb{N}$, and $P(n)$ holds. Together with the even case, $P(n)$ is true for every natural number n.

A.6 Chapter 6

Solution to Exercise 6.1

The class $[-2]_5$ consists of the integers $-2 + 5k$ with k an integer. Taking $k = -2, -1, 0, 1, 2$ gives

$$-12, \quad -7, \quad -2, \quad 3, \quad 8.$$

These are all the members between -12 and 12: the next members on either side, -17 and 13, lie outside the interval. The member of the class in $\{0, 1, 2, 3, 4\}$ is 3, since $3 - (-2) = 5$; equivalently, $-2 = (-1) \cdot 5 + 3$ with $0 \leq 3 < 5$. So $[-2]_5 = [3]_5$.

Solution to Exercise 6.2

Suppose $[a']_m = [a]_m$ and $[b']_m = [b]_m$. By Proposition 6.2(b), $a' \equiv a$ and $b' \equiv b \pmod{m}$, so

$$a' = a + mk, \qquad b' = b + m\ell$$

for integers k and ℓ. The output classes $[a'b']_m$ and $[ab]_m$ are equal exactly when m divides $a'b' - ab$. Compute:

$$\begin{aligned} a'b' - ab &= (a + mk)(b + m\ell) - ab \\ &= ab + am\ell + mkb + m^2k\ell - ab \\ &= m(a\ell + bk + mk\ell). \end{aligned}$$

The expression in parentheses is an integer, so m divides $a'b' - ab$ and $[a'b']_m = [ab]_m$, although the products $a'b'$ and ab themselves can differ. Since both representatives were allowed to change at once, the argument covers every choice of names for the two classes.

Solution to Exercise 6.3

The rotation r_2 sends seat j to seat $(j + 2) \bmod 4$, so applying it twice sends

$$0 \to 2 \to 0, \qquad 1 \to 3 \to 1, \qquad 2 \to 0 \to 2, \qquad 3 \to 1 \to 3.$$

Thus $r_2 \circ r_2 = r_0 = \mathrm{id}$. By the definition in Section 3.2, a map h is an inverse of r_2 when $h \circ r_2 = \mathrm{id}$ and $r_2 \circ h = \mathrm{id}$; for $h = r_2$, both conditions say $r_2 \circ r_2 = \mathrm{id}$. So r_2 is its own inverse. It is not the identity, because the identity leaves every seat in place, while r_2 moves seat 0 to seat 2.

Solution to Exercise 6.4

Yes. Write $X \sim Y$ when $|X| = |Y|$, and let X, Y, Z be sets in the collection.

- *Reflexivity:* $|X| = |X|$, so $X \sim X$.
- *Symmetry:* if $|X| = |Y|$, then $|Y| = |X|$, so $X \sim Y$ implies $Y \sim X$.
- *Transitivity:* if $|X| = |Y|$ and $|Y| = |Z|$, then $|X| = |Z|$, so $X \sim Y$ and $Y \sim Z$ imply $X \sim Z$.

Each property comes from the same property of equality of numbers, so $\sim$ is an equivalence relation. The class of a set X consists of all sets in the collection with as many elements as X; for instance, $\{1, 2\}$ and $\{p, q\}$, with $p \neq q$, lie in the same class if both belong to the collection.

Solution to Exercise 6.5

The rule is well-defined when $[a]_m = [b]_m$ always implies $[a]_d = [b]_d$. We prove the two directions of "exactly when" separately.

First suppose d divides m, say $m = dt$ with t an integer. Let $[a]_m = [b]_m$. By Proposition 6.2(b), $a - b = mk$ for an integer k, so $a - b = d(tk)$, and d divides $a - b$. By the same proposition, $[a]_d = [b]_d$, so the rule is well-defined.

Conversely, suppose the rule is well-defined. The integers 0 and m represent the same class modulo m, because $m - 0 = m \cdot 1$, so their output classes agree: $[0]_d = [m]_d$. By Proposition 6.2(b) once more, d divides $m - 0 = m$.

Solution to Exercise 6.6

Use the family $\{\{1\}, \{1, 2\}, \{3\}\}$, ordered by inclusion. The sets $\{1, 2\}$ and $\{3\}$ are maximal, because no other member contains either of them. The set $\{1\}$ is not maximal, because $\{1\} \subseteq \{1, 2\}$ and $\{1\} \neq \{1, 2\}$. So there are exactly two maximal members. There is no maximum: a maximum is always maximal (Section 6.6), so it would have to be $\{1, 2\}$ or $\{3\}$, but neither of these contains the other.

Now add the set $\{1, 2, 3\}$. Every member of the new family $\{\{1\}, \{1, 2\}, \{3\}, \{1, 2, 3\}\}$ is a subset of $\{1, 2, 3\}$, so $\{1, 2, 3\}$ is a maximum. Notice that $\{1, 2\}$ and $\{3\}$ are no longer maximal, because $\{1, 2, 3\}$ lies strictly above each of them. Whether an element is maximal, or a maximum, depends on the family it is compared with.

A.7 Chapter 7

Solution to Exercise 7.1

(a) The new graph has the five edges ab, bc, ca, cd, and bd. Since a is joined to b, c; b to a, c, d; c to a, b, d; and d to b, c, the degrees are

$$\deg(a) = 2, \qquad \deg(b) = 3, \qquad \deg(c) = 3, \qquad \deg(d) = 2.$$

The degree sum is $2 + 3 + 3 + 2 = 10 = 2 \cdot 5$, twice the number of edges, as Theorem 7.2 requires. The vertices of odd degree are b and c: two of them, an even number, as Proposition 7.3 requires.

(b) Deleting cd from G leaves the three edges ab, bc, and ca, and the degrees are

$$\deg(a) = 2, \qquad \deg(b) = 2, \qquad \deg(c) = 2, \qquad \deg(d) = 0.$$

The vertex d is still in the graph, with no neighbors. The degree sum is $2 + 2 + 2 + 0 = 6 = 2 \cdot 3$, again twice the number of edges. No vertex has odd degree, and their number, 0, is even.

Solution to Exercise 7.2

Recall from Section 5.5 that the remainder of an integer on division by four is one of $0, 1, 2, 3$; this holds for negative integers too, for example $-7 = (-2) \cdot 4 + 1$ has remainder 1. Use these four remainders as boxes, and place each index $i \in \{1, 2, 3, 4, 5\}$ in the box of the remainder of x_i. Five indices are placed in four boxes, and $5 > 4 \cdot 1$, so by the pigeonhole principle (Proposition 7.4 with $k = 1$) some box contains two different indices $i \neq j$. Then x_i and x_j leave the same remainder.

We placed the indices, not the values, in the boxes. This matters when two of the integers are equal: if $x_1 = x_2 = 7$, the indices 1 and 2 land in the same box, and the conclusion holds with $i = 1$ and $j = 2$.

Four integers are not enough: $0, 1, 2, 3$ leave four different remainders. Here the pigeonhole principle does not apply, because 4 is not larger than $4 \cdot 1$.

Solution to Exercise 7.3

Take the directed cycle $a \to b \to c \to a$. It is an oriented graph: there are no loops, and each pair of vertices is joined in only one direction. It is nonempty and finite, and every vertex has an out-neighbor, so there is no sink. The only hypothesis of Theorem 7.8 that fails is acyclicity.

Every nonempty oriented graph with fewer vertices has a sink. (The graph with no vertices has none, which is why Theorem 7.8 assumes nonemptiness.) With one vertex there can be no edges, because a directed edge needs two different vertices, so the vertex is a sink. With two vertices u and v, there is at most one edge. If there is none, both vertices are sinks. If the edge is $u \to v$, then $v \to u$ is forbidden, and u is the only other vertex, so v is a sink; the case $v \to u$ is the same with the names exchanged. Hence three is the smallest possible number of vertices.

In the proof of Theorem 7.8, a longest directed path is, for example, $a \to b \to c$, since no directed path has more than three vertices. Its last vertex c has the out-neighbor a on the path: this

is the second case of the proof, where $c \to a$ closes a directed cycle. There the proof reached a contradiction only by using acyclicity, so for this graph it cannot conclude that c is a sink.

Solution to Exercise 7.4

At a, the first out-neighborhood is $N_1^+(a) = \{b, c\}$. The two-step routes from a are $a \to b \to d$ and $a \to c \to d$. Both end at d, which is neither a nor a first out-neighbor of a, so $N_2^+(a) = \{d\}$ and $|N_2^+(a)| = 1$: the vertex d is counted once, even though two routes reach it.

At b, we have $N_1^+(b) = \{d\}$. Since d has no out-neighbors, there is no two-step route from b, and $N_2^+(b) = \varnothing$. In the same way, $N_1^+(c) = \{d\}$ and $N_2^+(c) = \varnothing$. Finally, d is a sink, so $N_1^+(d) = N_2^+(d) = \varnothing$.

Comparing sizes: at a we get $1 < 2$, at b and c we get $0 < 1$, and at d we get $0 \geq 0$. So only d satisfies $|N_2^+(v)| \geq |N_1^+(v)|$.

This agrees with Proposition 7.11: d is a sink, and the graph is acyclic. Every vertex on a directed cycle has an arrow entering it and one leaving it; no arrow enters a and none leaves d, so a directed cycle could use only b and c, and its arrows would join b and c. But no arrow joins them.

Solution to Exercise 7.5

By the handshake theorem, the n degrees add up to $2m$, so their average is $2m/n$. By the second half of the averaging principle (Proposition 7.5), some vertex has degree at most this average.

This says nothing about how far the other degrees may be from the average. For example, let $n \geq 3$ and take a *star*: one vertex, the center, is joined to each of the other $n - 1$ vertices, called leaves, and there are no other edges. The star has $m = n - 1$ edges, so its average degree is

$$\frac{2(n-1)}{n} = 2 - \frac{2}{n},$$

which is less than 2. Every leaf has degree 1, below the average, since $2 - 2/n \geq 2 - 2/3 = 4/3 > 1$ for $n \geq 3$. The center, however, has degree $n - 1$, which grows without bound as n grows, while the average stays below 2.

Solution to Exercise 7.6

No. Take the tournament on the vertices a, b, c with $a \to b$, $a \to c$, and $b \to c$; each of the three pairs is joined by exactly one directed edge. The vertex a is a king, because it reaches both other vertices in one step.

Its first out-neighborhood is $N_1^+(a) = \{b, c\}$. The only two-step route from a is $a \to b \to c$, since c has no out-neighbors, and it ends at c, which is already a first out-neighbor of a. So $N_2^+(a) = \varnothing$, and $|N_2^+(a)| = 0 < 2 = |N_1^+(a)|$: the king a fails the inequality.

The two properties measure different things: being a king means that every other vertex is within directed distance two, while the inequality compares the numbers of vertices at directed distance exactly one and exactly two. A king that beats almost everyone directly leaves few vertices for its second out-neighborhood.

A.8 Chapter 8

Solution to Exercise 8.1

The matchings. The empty set is a matching, and so is each of the three single edges Ada–1, Ada–2 and Ben–1. Of the three sets with two edges, {Ada–1, Ada–2} is not a matching, because both edges have the endpoint Ada, and {Ada–1, Ben–1} is not, because both have the endpoint 1; the remaining one, {Ada–2, Ben–1}, is a matching. The set of all three edges contains Ada–1 and Ada–2, so it is not a matching. Hence the graph has exactly five matchings.

Which are maximal. Any edge can be added to the empty matching, Ben–1 to {Ada–2}, and Ada–2 to {Ben–1}, so these three are not maximal. The matching {Ada–1} is maximal, because Ada–2 shares the endpoint Ada with it and Ben–1 shares the endpoint 1. So is {Ada–2, Ben–1}, because adding the only remaining edge, Ada–1, would give Ada two tasks.

Which are maximum. The largest number of edges is two, so {Ada–2, Ben–1} is the only maximum matching. Hence {Ada–1} is the only matching that is maximal but not maximum.

Solution to Exercise 8.2

In the matching $\{a\text{–}2,\ b\text{–}1\}$, the student c is unmatched, tasks 1 and 2 are used, and task 3 is unused. Search from c.

- Inspect c. Its only edge, c–2, is unmatched, so task 2 is reached with predecessor c. Task 2 is held by a, so a is reached with predecessor 2.
- Inspect a. The edge a–2 is matched; the edge a–1 is unmatched, so task 1 is reached with predecessor a. Task 1 is held by b, so b is reached with predecessor 1.
- Inspect b. The edge b–1 is matched; the edge b–3 is unmatched, so task 3 is reached with predecessor b. Task 3 is unused, and the search succeeds.

Following the predecessors back from task 3 gives the augmenting path

$$c - 2 - a - 1 - b - 3.$$

Flipping removes its matched edges a–2 and b–1 and adds its unmatched edges c–2, a–1, b–3, which gives the matching

$$\{a\text{–}1,\ b\text{–}3,\ c\text{–}2\}.$$

Each student now has an acceptable task, and the three tasks are different.

Solution to Exercise 8.3

Let S be the set of all three students. Then $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$, and S is a shortage set. It certifies that no full assignment exists, since a full assignment would give the three students three different tasks from $N(S)$, which has only two members.

Each student has two options, but adding them up, $2 + 2 + 2 = 6$, counts the same two tasks again and again; what matters is the number of *different* tasks available to the group, which is 2.

No smaller group is a shortage set: the empty group accepts 0 tasks, which is not fewer than its 0 members; one student accepts $2 \geq 1$ tasks; and any two students together accept $2 \geq 2$ tasks.

Solution to Exercise 8.4

(a) The proof uses twice that the vertices of the path are all different: to conclude that an inner vertex lies on exactly two path edges, one matched and one unmatched, and an end on exactly one, so that after the flip each lies on exactly one edge of the new matching; and to count the path's edges as $r + 1$ unmatched and r matched ones, all different. A vertex appearing twice could lie on two unmatched edges of the walk, both added by the flip, and would then have two partners.

For an example, take students x, s, u and tasks 1, 2, 3, with edges x–1, s–1, s–2, s–3, u–1, u–2, and the matching $M = \{s\text{–}1,\ u\text{–}2\}$. Student x is unmatched and task 3 is unused. The walk

$$x,\ 1,\ s,\ 2,\ u,\ 1,\ s,\ 3$$

uses the edges x–1, s–1, s–2, u–2, u–1, s–1, s–3, which are unmatched, matched, unmatched, matched, unmatched, matched, unmatched in turn. So it is an alternating walk from an unmatched student to an unused task, but it visits task 1 and student s twice. Flipping along it removes s–1 and u–2 and adds x–1, s–2, u–1, s–3. The result

$$\{x\text{–}1,\ s\text{–}2,\ u\text{–}1,\ s\text{–}3\}$$

is not a matching: task 1 lies on two of its edges, and so does student s. Cutting out the part of the walk between the two visits to task 1 leaves $x, 1, s, 3$, which is an augmenting path; flipping along it gives the matching $\{x\text{–}1,\ s\text{–}3,\ u\text{–}2\}$.

(b) The lengths of the alternating walks from an unmatched student to an unused task form a nonempty set of natural numbers, so by the well-ordering property there is a shortest such walk, say $v_0, v_1, \ldots, v_k$, with $x = v_0$. Since x is a student and v_k is a task, $k \geq 1$. The graph is bipartite, so v_i is a student when i is even and a task when i is odd. The first edge is unmatched, because x is unmatched, and the edges alternate, so the mth edge is unmatched when m is odd and matched when m is even.

Suppose, for contradiction, that $v_i = v_j$ for some $i < j$. Then i and j have the same parity, because v_i and v_j lie on the same side, so $j - i$ is even. Cut out the entries $v_{i+1}, \ldots, v_j$, as in the proof of Lemma 7.6. What remains, $v_0, \ldots, v_i, v_{j+1}, \ldots, v_k$, is a walk from x to the same unused task, for the reason given in that proof. Its first i edges are unchanged. If $j < k$, each later edge is an edge of the old walk whose position number has dropped by $j - i$: for example, the new $(i + 1)$st edge $v_i v_{j+1}$ is the old $(j + 1)$st edge $v_j v_{j+1}$. Since $j - i$ is even, each of these edges keeps the parity of its position, so it is still unmatched exactly when its position is odd, and the shorter walk is still alternating. Its length $k - (j - i) < k$ contradicts the choice of a shortest walk. Hence a shortest walk repeats no vertex: it is an alternating path with at least one step whose ends are both unmatched, that is, an augmenting path.

Solution to Exercise 8.5

A quantity that increases by one at every step can still do so forever, like the counter $0, 1, 2, 3, \ldots$. To conclude that a procedure stops, we also need a ceiling that the quantity can never pass.

Finiteness of L provides this ceiling: as shown at the end of Section 8.3, sending each edge of a matching to its student is an injective map from the matching to L, so a matching has at most $|L|$ edges. Starting from k edges, after m steps the matching has $k + m \leq |L|$ edges. Hence at most $|L| - k$ steps can occur, and at most $|L|$ from the empty matching.

Solution to Exercise 8.6

(a) The search reaches a task only by a move from a reached student s along an unmatched edge s–t, and a student other than x only by a move from a reached used task to its owner.

Every task t in B was reached from some student s in A along an edge s–t, so s accepts t, and $t \in N(A)$. Thus $B \subseteq N(A)$. Conversely, let $t \in N(A)$, and choose a student $s \in A$ that accepts t.

- *The edge s–t is unmatched.* Because the search is complete, s has been inspected, and inspecting s follows every unmatched edge at s. So $t \in B$.
- *The edge s–t is matched.* Then s is matched, so $s \neq x$, and s was reached by a move from the task it holds, which is t. So $t \in B$.

Hence $N(A) = B$. Completeness is used only in the unmatched case.

(b) The containment $B \subseteq N(A)$ still holds, with the same proof, and so does the matched case of $N(A) \subseteq B$, which uses only the way students are reached. Only the unmatched case can fail, because a student that has not been completely inspected may have an unmatched edge that the search never followed. So $N(A) \subseteq B$ need not hold.

For example, let Ada accept tasks 1 and 2, let Ben accept only task 1, and let $M = \{\text{Ben–1}\}$. Search from Ada, and stop right after reaching task 1 and its owner Ben, before following the edge Ada–2. Then $A = \{\text{Ada}, \text{Ben}\}$ and $B = \{1\}$, but $N(A) = \{1, 2\} \not\subseteq B$.

Now take any early-stopped search in which $N(A) \subseteq B$ fails. Step 1 of the proof of Proposition 8.6 still works: every reached task is used, because the search has reached no unused task; its owner has been reached, because the owner of a used task is marked as reached at the moment the task is; and every reached student other than x was reached from the task it holds. So $|B| = |A| - 1$. Since $B \subseteq N(A)$ and $N(A)$ contains a task outside B, we get $|N(A)| \geq |B| + 1 = |A|$, so A is not a shortage set. In the example, $|N(A)| = 2 = |A|$.

A.9 Chapter 9

Solution to Exercise 9.1

Let

$$N(a) = N(b) = N(c) = \{1, 2\}, \qquad N(d) = \{3, 4\}.$$

Every student accepts two tasks, and $N(L) = \{1, 2, 3, 4\}$, so $|N(L)| = 4 = |L|$. The group $S = \{a, b, c\}$ has $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$. A full assignment would give a, b and c three different tasks, all lying in $N(S)$, which has only two members. So no matching saturates L.

The tests for single students read $2 \geq 1$, and the test for the whole class reads $4 \geq 4$, so all of them pass. The shortage lies in the group of three students who compete for the same two tasks, and only the test for that group detects it.

Solution to Exercise 9.2

The complete check is

S	$N(S)$	$\lvert N(S)\rvert \geq \lvert S\rvert$
$\varnothing$	$\varnothing$	$0 \geq 0$
$\{a\}$	$\{1\}$	$1 \geq 1$
$\{b\}$	$\{1, 2\}$	$2 \geq 1$
$\{c\}$	$\{2, 3\}$	$2 \geq 1$
$\{a, b\}$	$\{1, 2\}$	$2 \geq 2$
$\{a, c\}$	$\{1, 2, 3\}$	$3 \geq 2$
$\{b, c\}$	$\{1, 2, 3\}$	$3 \geq 2$
$\{a, b, c\}$	$\{1, 2, 3\}$	$3 \geq 3$

Every inequality holds, so the graph satisfies Hall's condition. The tight groups, where the two sides are equal, are $\varnothing$, $\{a\}$, $\{a, b\}$ and $\{a, b, c\}$. Leaving out the empty group and the whole class, the nonempty proper tight subsets are $\{a\}$ and $\{a, b\}$.

A matching that saturates L is a–1, b–2, c–3: each pair is an edge, because $1 \in N(a)$, $2 \in N(b)$ and $3 \in N(c)$, and the three tasks are different. Once this matching is known, the necessary direction of Hall's theorem (Section 9.2) guarantees all eight inequalities without the table.

Solution to Exercise 9.3

Write $X = N(S)$ and $Y = N(T) \setminus N(S)$. We show that $N(S) \cup N(T) = X \cup Y$ and that X and Y are disjoint.

First, $X \cup Y \subseteq N(S) \cup N(T)$, because $X = N(S)$ and $Y \subseteq N(T)$. Conversely, let $t \in N(S) \cup N(T)$. If $t \in N(S)$, then $t \in X$. If $t \notin N(S)$, then $t \in N(T)$, because it lies in the union, so $t \in Y$. Hence $N(S) \cup N(T) = X \cup Y$. Moreover, every member of Y lies outside $N(S) = X$, so X and Y are disjoint.

For disjoint finite sets, the size of the union is the sum of the sizes (Section 3.1). Therefore

$$|N(S) \cup N(T)| = |X| + |Y| = |N(S)| + |N(T) \setminus N(S)|.$$

Nowhere did we assume that $N(S)$ and $N(T)$ are disjoint: a common task is counted once, as a member of $N(S)$. In the example of Section 9.4, $N(S) = \{1, 2\}$ and $N(\{c\}) = \{2, 3\}$ share task 2, and their union $\{1, 2, 3\}$ has $3 = 2 + |\{3\}|$ members.

Solution to Exercise 9.4

The sizes $|N(S)|$ and $|S|$ are natural numbers, so $k = |N(S)| - |S|$ is an integer, and $k > 0$ because $|N(S)| > |S|$. There is no integer strictly between 0 and 1, so $k \geq 1$, that is, $|N(S)| \geq |S| + 1$.

For arbitrary real numbers the step is not valid, because a positive real number can be smaller than 1: for example, $1.2 > 1$, but $1.2 \geq 1 + 1$ is false. The extra unit in Case 2 comes from the fact that the sizes of finite sets are integers.

Solution to Exercise 9.5

By Hall's theorem, it is enough to show that $|N(S)| \geq |S|$ for every set S of students. Fix such a set S, let E_S be the set of edges whose student endpoint lies in S, and count E_S in two ways.

By students. Each student of S lies on exactly d edges, and each edge has only one student endpoint, so $|E_S| = d|S|$.

By tasks. The task endpoint of each edge in E_S is acceptable to a member of S, so it lies in $N(S)$. Group the edges of E_S by their task endpoint, one group for each of the $|N(S)|$ tasks of $N(S)$; every edge lies in exactly one group. Each task is acceptable to at most d students, so its group has at most d edges. Hence $|E_S| \leq d|N(S)|$.

Comparing the two counts gives $d|S| \leq d|N(S)|$. Since $d > 0$, dividing by d keeps the direction of the inequality and gives $|S| \leq |N(S)|$. This holds for every $S \subseteq L$, including $S = \varnothing$, so the graph satisfies Hall's condition, and Hall's theorem gives a full assignment.

The proof uses $d > 0$ at exactly one point, the division. If $d = 0$, the inequality reads $0 \leq 0$ and says nothing about $|S|$ and $|N(S)|$. The conclusion fails too: when $d = 0$, no student accepts any task, so there is no full assignment unless there are no students at all.

Solution to Exercise 9.6

Here is a model portfolio.

(a) An assignment certificate. For the graph of Exercise 9.2, with $N(a) = \{1\}$, $N(b) = \{1, 2\}$ and $N(c) = \{2, 3\}$, the matching a–1, b–2, c–3 saturates L. A reader checks that each pair is an edge ($1 \in N(a)$, $2 \in N(b)$, $3 \in N(c)$), that each student appears in exactly one pair, and that the three tasks are different.

(b) A shortage certificate. For the graph of Exercise 9.1, with $N(a) = N(b) = N(c) = \{1, 2\}$ and $N(d) = \{3, 4\}$, take $S = \{a, b, c\}$. A reader checks it by collecting the tasks that the members of S accept: $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$. This proves that no full assignment exists, because such an assignment would give the three students of S three different tasks, all in $N(S)$, which has only two members. This is the necessary direction of Hall's theorem (Section 9.2).

(c) A proof of sufficiency. *Claim: let G be a finite bipartite graph with parts L and R such that $|N(S)| \geq |S|$ for every $S \subseteq L$. Then G has a matching that saturates L.*

The proof follows Section 9.3 and quotes results of Chapter 8.

Proof. Start with the empty matching M. As long as some student u is unmatched, carry out

a round: run the complete search of Section 8.4 from u. The graph is finite, so the search ends, either by reaching an unused task or by inspecting every reached student without finding one.

Suppose the search fails, and let A and B be the sets of students and tasks it reached. Proposition 8.6 says that then $N(A) = B$ and $|N(A)| = |A| - 1$. So $|N(A)| < |A|$, contradicting Hall's condition for A. Hence the search reaches an unused task. Following the predecessors back to u then gives an augmenting path (Section 8.4), and by Lemma 8.5, flipping M along an augmenting path gives a matching with one more edge.

Each edge of a matching contains exactly one student, and different edges contain different students, so M never has more than $|L|$ edges. Since M starts with no edges and gains one in every round, there are at most $|L|$ rounds. The rounds stop only when every student is matched, so the final matching saturates L. (If $L = \varnothing$, there are no rounds, and the empty matching saturates L.) This completes the proof.

(d) Maximal and maximum matchings. A matching is *maximal* if no edge of the graph can be added to it without creating a shared endpoint, and *maximum* if no matching in the graph has more edges. Every maximum matching is maximal (an edge that could be added would give a larger matching), but not conversely. If Ada accepts tasks 1 and 2 and Ben accepts only task 1, the matching {Ada–1} is maximal, since adding Ada–2 or Ben–1 would create a shared endpoint, but it is not maximum, because {Ada–2, Ben–1} has two edges (Exercise 8.1). A greedy procedure hands out tasks one at a time and never takes one back. When it finishes, each student left without a task finds every acceptable task already taken, so its matching is maximal, but it need not be maximum: if it starts by giving Ada task 1, it leaves Ben without a task, although a full assignment exists. So getting stuck proves nothing about impossibility; an augmenting-path repair, here along Ben–1–Ada–2, revises earlier choices. Only a shortage certificate, as in part (b), proves that no full assignment exists.

A.10 Chapter 10

Solution to Exercise 10.1

The row rule gives

$$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 + 1 \cdot (-2) \\ 0 \cdot 1 + 3 \cdot (-2) \end{pmatrix} = \begin{pmatrix} 0 \\ -6 \end{pmatrix}.$$

The columns of A are $Ae_1 = (2, 0)$ and $Ae_2 = (1, 3)$, and $x = 1e_1 - 2e_2$. By linearity,

$$Ax = Ae_1 - 2Ae_2 = (2, 0) + (-2, -6) = (0, -6),$$

the same answer: the coordinates of x say how much of each column to take.

Solution to Exercise 10.2

The kernel consists of the pairs with $x + y = 0$, that is, with $y = -x$, so

$$\ker T = \{(t, -t) : t \in \mathbb{R}\}.$$

Similarly, $x + y = 5$ exactly when $y = 5 - x$, so the solutions of $T(x, y) = 5$ are the pairs $(t, 5 - t)$ with $t \in \mathbb{R}$. As Section 10.3 predicts, this is one particular solution plus a vector in the kernel:

$$(t, 5 - t) = (0, 5) + (t, -t).$$

Solution to Exercise 10.3

Write $U(s, t) = \big((s + t)/2, (s - t)/2\big)$. By the definition of an inverse in Section 3.2, we must check that $U(T(x, y)) = (x, y)$ and $T(U(s, t)) = (s, t)$ for all real x, y, s, t. For the first, apply U with $s = x + y$ and $t = x - y$:

$$U(T(x, y)) = \left(\frac{(x + y) + (x - y)}{2},\ \frac{(x + y) - (x - y)}{2}\right) = \left(\frac{2x}{2},\ \frac{2y}{2}\right) = (x, y).$$

For the second,

$$T(U(s, t)) = \left(\frac{s + t}{2} + \frac{s - t}{2},\ \frac{s + t}{2} - \frac{s - t}{2}\right) = \left(\frac{2s}{2},\ \frac{2t}{2}\right) = (s, t).$$

Both compositions are identity maps, so U is the inverse of T.

Solution to Exercise 10.4

The dot product is $u \cdot v = 1 \cdot 2 + 2 \cdot (-1) = 0$. The squared lengths are $1^2 + 2^2 = 5$ and $2^2 + (-1)^2 = 5$, so both lengths equal $\sqrt{5}$. The Cauchy–Schwarz inequality reads $|0| \le \sqrt{5} \cdot \sqrt{5} = 5$, which is true and strict. This agrees with the equality case of Section 10.5: since $v \neq 0$, equality would require u to be a multiple of v, but $(1, 2) = c(2, -1)$ would need both $2c = 1$ and $-c = 2$, which is impossible.

Solution to Exercise 10.5

Both sides of $\|u + v\|_2 = \|u\|_2 + \|v\|_2$ are nonnegative, and nonnegative numbers are equal exactly when their squares are, so we must decide when $\|u + v\|_2^2 = (\|u\|_2 + \|v\|_2)^2$. The proof in Section 10.5 gives the chain

$$\begin{aligned} \|u + v\|_2^2 &= \|u\|_2^2 + 2(u \cdot v) + \|v\|_2^2 \\ &\le \|u\|_2^2 + 2|u \cdot v| + \|v\|_2^2 \\ &\le \|u\|_2^2 + 2\,\|u\|_2\,\|v\|_2 + \|v\|_2^2 = (\|u\|_2 + \|v\|_2)^2. \end{aligned}$$

If either inequality were strict, the first number would be strictly smaller than the last, so the two ends are equal exactly when both inequalities are equalities.

The first inequality is an equality exactly when $u \cdot v = |u \cdot v|$, that is, when $u \cdot v \ge 0$. The second is an equality exactly when $|u \cdot v| = \|u\|_2\,\|v\|_2$, which by the equality case of the Cauchy–Schwarz inequality happens exactly when $v = 0$ or u is a multiple of v.

Now combine the two conditions. If $v = 0$, both hold, since then $u \cdot v = 0$. If $v \neq 0$ and $u = sv$, then $u \cdot v = s(v \cdot v)$, and since $v \cdot v > 0$ this is nonnegative exactly when $s \geq 0$. So equality holds exactly when $v = 0$ or $u = sv$ with $s \geq 0$.

For a strict example, take $v = (1, 1)$ and $u = -v$. Then $u + v = 0$, so $\|u + v\|_2 = 0$, while $\|u\|_2 + \|v\|_2 = 2\sqrt{2}$; here u is a multiple of v, but with the negative factor $s = -1$.

Solution to Exercise 10.6

Assume that $a_1u_1 + \cdots + a_ku_k = 0$, pick any index j, and take the dot product of both sides with u_j. By linearity in the first argument, applied one term at a time as in Section 10.2, the left side becomes

$$a_1(u_1 \cdot u_j) + \cdots + a_k(u_k \cdot u_j),$$

and the right side becomes $0 \cdot u_j = 0$. For every $i \neq j$, orthogonality gives $u_i \cdot u_j = 0$, so only the term with $i = j$ remains:

$$a_j(u_j \cdot u_j) = a_j \left\|u_j\right\|_2^2 = 0.$$

The vector u_j is nonzero, so its squared length is positive, and dividing by it gives $a_j = 0$. Because j was arbitrary, every coefficient is zero, which is the definition of linear independence in Section 10.1.

The word "nonzero" cannot be left out: $u_1 = (1, 0)$ and $u_2 = (0, 0)$ are orthogonal, yet $0u_1 + 1u_2 = (0, 0)$ is a combination equal to zero with a nonzero coefficient, so the list is not linearly independent. In the proof, this is where dividing by $\|u_2\|_2^2 = 0$ would be impossible.

A.11 Chapter 11

Solution to Exercise 11.1

Regroup the difference as $(2x - y) - (2a - b) = 2(x - a) - (y - b)$. The triangle inequality and the rule $|cx| = |c|\,|x|$ from Section 2.6 give

$$\begin{aligned} |(2x - y) - (2a - b)| &\leq |2(x - a)| + |-(y - b)| \\ &= 2|x - a| + |y - b| \\ &\leq 2(0.02) + 0.03 = 0.07. \end{aligned}$$

The bound 0.07 is attained: $a = b = 0$, $x = 0.02$ and $y = -0.03$ satisfy both hypotheses and give $(2x - y) - (2a - b) = 0.04 + 0.03 = 0.07$. Because the second error is subtracted, the worst case comes from errors of opposite signs. Since 0.07 is attained, no smaller bound holds in every case.

Solution to Exercise 11.2

Let $\varepsilon > 0$. By the Archimedean property, choose an integer $N > 3/\varepsilon$; it is positive because $3/\varepsilon > 0$. Let $n \geq N$. Then $n + 1 > N > 0$, so $1/(n + 1) < 1/N$, and multiplying by 3 gives $3/(n + 1) < 3/N$.

Multiplying $N > 3/\varepsilon$ by the positive number ε/N gives $\varepsilon > 3/N$. Together,

$$|a_n - 0| = \frac{3}{n+1} < \frac{3}{N} < \varepsilon \qquad \text{for every } n \geq N.$$

So the threshold is any integer $N > 3/\varepsilon$, and $a_n \to 0$. For example, $\varepsilon = 0.01$ calls for $N > 300$, so $N = 301$ works; indeed $a_{301} = 3/302 \approx 0.00993$.

Solution to Exercise 11.3

Recall that $d_\infty(x, y)$ is the largest of the n numbers $|x_1 - y_1|, \ldots, |x_n - y_n|$.

Nonnegativity. Each of these numbers is an absolute value and so is at least 0; their largest is one of them, so $d_\infty(x, y) \geq 0$.

Zero exactly for equal vectors. If $x = y$, all n numbers are 0, and so is their largest. Conversely, suppose $d_\infty(x, y) = 0$. Every number $|x_j - y_j|$ is at most the largest one, which is 0, and at least 0, so $|x_j - y_j| = 0$, that is, $x_j = y_j$, for every j. Hence $x = y$.

Symmetry. For each j, $|x_j - y_j| = |y_j - x_j|$, because $y_j - x_j = -(x_j - y_j)$. So $d_\infty(x, y)$ and $d_\infty(y, x)$ are the largest of the same list of numbers, and they are equal.

Solution to Exercise 11.4

The hypothesis of Proposition 11.9 holds with $D = 2$ and $q = 1/3$, so for all $m > n$

$$d(x_m, x_n) \leq \frac{2(1/3)^n}{1 - 1/3} = 3\left(\frac{1}{3}\right)^n = 3^{1-n},$$

a bound that does not depend on m.

For an explicit threshold, we need to know when 3^{1-n} becomes small. With $q = 1/3$ we have $c = 1/q - 1 = 2$, and Bernoulli's inequality gives $3^n \geq 1 + 2n$, so $3^{1-n} \leq 3/(1 + 2n)$. Let $\varepsilon > 0$, and choose an integer $N > 3/(2\varepsilon)$. For $n \geq N$ we get $1 + 2n > 2n \geq 2N > 3/\varepsilon$, hence $3^{1-n} \leq 3/(1 + 2n) < \varepsilon$. Now take any $m, n \geq N$. If $m = n$, their distance is $0 < \varepsilon$. Otherwise, since d is symmetric, we may name them so that $m > n$; then $d(x_m, x_n) \leq 3^{1-n} < \varepsilon$. So the sequence is Cauchy, with threshold any integer $N > 3/(2\varepsilon)$. (To conclude that it converges, we would also need the space to be complete.)

Solution to Exercise 11.5

Take $x_n = 2 + 2^{-(n+1)}$ for $n \in \mathbb{N}$, that is, $2.5,\ 2.25,\ 2.125, \ldots$. Since $0 < 2^{-(n+1)} \leq 1/2$, every term satisfies $2 < x_n \leq 2.5$, so the sequence lies in $(2, 3)$.

The sequence is Cauchy. On the real line it converges to 2. Indeed, $|x_n - 2| = 2^{-(n+1)}$, and $2^{n+1} \geq n + 2$ (Section 11.3), so $|x_n - 2| \leq 1/(n + 2) < 1/(n + 1)$. Given $\varepsilon > 0$, choose an integer $N > 1/\varepsilon$; then every $n \geq N$ satisfies $|x_n - 2| < 1/(n + 1) < 1/N < \varepsilon$. By Proposition 11.5, the sequence is Cauchy on the real line. Distances between its terms are the same in $(2, 3)$ as in $\mathbb{R}$, so it is Cauchy in $(2, 3)$.

The sequence has no limit in $(2, 3)$. A limit $L \in (2, 3)$ would also be a limit on the real line, since distances in $(2, 3)$ are distances in $\mathbb{R}$. As the sequence converges to 2 there, Proposition 11.3 would force $L = 2$, which is not in $(2, 3)$. In particular, $(2, 3)$ is not complete.

Solution to Exercise 11.6

1. The supremum and infimum. Every $x \in (0, 1)$ satisfies $x < 1$, so 1 is an upper bound for $(0, 1)$. Now let b be any upper bound, and suppose $b < 1$. Since $1/2 \in (0, 1)$, we have $1/2 \le b$, so $b > 0$. The midpoint $t = (b + 1)/2$ of b and 1 satisfies $0 < b < t < 1$, so $t \in (0, 1)$ and $t > b$. Then b is not an upper bound, a contradiction. Hence every upper bound b satisfies $b \ge 1$, and 1 is the supremum of $(0, 1)$.

For the infimum, every $x \in (0, 1)$ satisfies $x > 0$, so 0 is a lower bound. Let c be any lower bound. If $c > 0$, then $t = \min(c, 1)/2$ satisfies $0 < t < c$ and $t \in (0, 1)$, contradicting the fact that c is a lower bound. Hence every lower bound c satisfies $c \le 0$, and 0 is the infimum.

2. No maximum or minimum. A maximum would be a member x of $(0, 1)$ that is also an upper bound. For any $x \in (0, 1)$, the number $t = (x + 1)/2$ satisfies $x < t < 1$, so t is a member of $(0, 1)$ larger than x. Thus x is not an upper bound, and no member of $(0, 1)$ is a maximum. Similarly, a minimum would be a member x of $(0, 1)$ that is also a lower bound. For any $x \in (0, 1)$, the number $t = x/2$ satisfies $0 < t < x$, so t is a member of $(0, 1)$ smaller than x; thus x is not a lower bound, and no member of $(0, 1)$ is a minimum.

3. Supremum versus maximum. The interval $(0, 1)$ is nonempty and has the upper bound 1, but by part 2 it has no maximum. So it is a counterexample to "every nonempty set of real numbers that has an upper bound has a maximum." A maximum must belong to the set, while a supremum need not: the supremum 1 of $(0, 1)$ is exactly the point missing from the interval, just as the limit 0 was missing from $(0, 1)$ in Section 11.6.

A.12 Chapter 12

Solution to Exercise 12.1

Contraction factor. For all real x and y,

$$T(x) - T(y) = \frac{3x + 1}{4} - \frac{3y + 1}{4} = \frac{3(x - y)}{4}, \qquad \text{so} \qquad |T(x) - T(y)| = \frac{3}{4}|x - y|.$$

Hence T is a contraction of $\mathbb{R}$ with factor $3/4$. No smaller factor works: if $q < 3/4$, the pair $x = 1$, $y = 0$ gives $|T(1) - T(0)| = 3/4 > q\,|1 - 0|$. So the smallest contraction factor is $3/4$.

Fixed point. The equation $(3x + 1)/4 = x$ becomes $3x + 1 = 4x$, so $x = 1$.

Iterates. From $x_0 = 0$ we get $x_1 = 1/4$, $x_2 = 7/16$ and $x_3 = 37/64$. The gaps $1 - x_n$ are $1,\ 3/4,\ 9/16,\ 27/64$, each three quarters of the one before, which suggests $x_n = 1 - (3/4)^n$. We prove this by induction on n. For $n = 0$, the formula gives $1 - 1 = 0 = x_0$. If $x_n = 1 - (3/4)^n$ for some n, then

$$x_{n+1} = \frac{3x_n + 1}{4} = \frac{3 - 3(3/4)^n + 1}{4} = \frac{4 - 3(3/4)^n}{4} = 1 - \frac{3}{4}\left(\frac{3}{4}\right)^n = 1 - \left(\frac{3}{4}\right)^{n+1},$$

which is the formula for $n + 1$.

Solution to Exercise 12.2

By the residual estimate (12.2),

$$d(z, x_*) \le \frac{d(z, Tz)}{1 - q} \le \frac{0.002}{1 - 3/4} = 4 \cdot 0.002 = 0.008.$$

The residual bound 0.002 is not always an upper bound on the error. For example, $T(x) = 3x/4$ is a contraction of the nonempty complete space $\mathbb{R}$ with factor $3/4$ and fixed point 0. The point $z = 0.008$ has $Tz = 0.006$, so its residual is exactly 0.002, while its distance to the fixed point is 0.008, four times larger.

Solution to Exercise 12.3

Self-map. Let $1/3 \le x \le 2/3$. Squaring nonnegative numbers preserves their order, so $1/9 \le x^2 \le 4/9$. Hence $10/9 \le x^2 + 1 \le 13/9$ and $10/27 \le T(x) \le 13/27$. Since $1/3 = 9/27$ and $2/3 = 18/27$, the output $T(x)$ lies in $[1/3, 2/3]$.

Contraction factor. For $x, y \in [1/3, 2/3]$ we have $0 \le x + y \le 4/3$, so

$$|T(x) - T(y)| = \frac{|x - y|\,|x + y|}{3} \le \frac{4/3}{3}\,|x - y| = \frac{4}{9}|x - y|.$$

Where the iterates lie. The first iterate $x_1 = T(0) = 1/3$ lies in $[1/3, 2/3]$, and if x_n does, then so does $x_{n+1} = T(x_n)$, by the self-map property. By induction, every iterate from x_1 on lies in $[1/3, 2/3]$. The starting point $x_0 = 0$ does not, so the smaller factor becomes available only after the first step.

The improved certificate and why it is justified. The interval $[1/3, 2/3]$ is nonempty and complete, because Section 11.6 showed that every closed interval $[a, b]$ is complete. With the two facts above, Theorem 12.2 applies to T on $[1/3, 2/3]$ with $q = 4/9$ and gives a fixed point there. Since $[1/3, 2/3]$ is part of $[0, 1]$ and T has only one fixed point in $[0, 1]$, this is the same point x_* as in Section 12.5. The point $z = x_2 = 10/27$ also lies in $[1/3, 2/3]$, with residual $19/2187$ (Section 12.5). So the residual estimate in the space $[1/3, 2/3]$, where $1/(1 - 4/9) = 9/5$, gives

$$d(x_2, x_*) \le \frac{9}{5} \cdot \frac{19}{2187} = \frac{19}{5 \cdot 243} = \frac{19}{1215} = 0.01563\ldots,$$

which improves the earlier certificate $19/729 = 0.02606\ldots$; the true error is about 0.0116.

Solution to Exercise 12.4

The map $x \mapsto x/2$ on $(0, 1)$: completeness fails. The space is nonempty, since it contains $1/2$. It is a self-map, because $0 < x < 1$ gives $0 < x/2 < 1/2$. The contraction inequality holds with factor $1/2$, because $|x/2 - y/2| = |x - y|/2$. Completeness fails: by Section 11.6, the sequence $2^{-(n+1)}$ is a Cauchy sequence in $(0, 1)$ whose only possible limit, 0, does not belong to $(0, 1)$.

The maps $x \mapsto x$ and $x \mapsto x + 1$ on $\mathbb{R}$: no factor below one. The space $\mathbb{R}$ is nonempty and complete (Section 11.6), and both maps send real numbers to real numbers. For both, $|T(x) - T(y)| = |x - y|$, so the inequality holds with $q = 1$ but with no $q < 1$: for $x = 1$ and $y = 0$ we would need $1 \le q$.

The map $x \mapsto x + 1/x$ on $[1, \infty)$: no factor below one. The space is nonempty, since it contains 1, and T is a self-map, because $x \ge 1$ gives $x + 1/x > x \ge 1$. Section 12.6 showed that every pair of different points moves strictly closer, so the inequality holds with $q = 1$, but that no single factor $q < 1$ works. It remains to prove that $[1, \infty)$ is complete. As Section 11.6 did for $[0, 1]$, we show that it is closed in $\mathbb{R}$. Suppose that numbers $x_n \ge 1$ converge in $\mathbb{R}$ to a number L, and suppose $L < 1$. The tolerance $(1 - L)/2$ is positive, so for a late enough index $|x_n - L| < (1 - L)/2$, and then

$$x_n < L + \frac{1 - L}{2} = \frac{1 + L}{2} < 1,$$

which contradicts $x_n \ge 1$. So $L \ge 1$. Thus $[1, \infty)$ is closed in the complete space $\mathbb{R}$, and by Proposition 11.8 it is complete.

The map $x \mapsto x/2 + 1$ on $[0, 1]$: the self-map condition fails. The interval $[0, 1]$ is nonempty and complete (Section 11.6), and $|T(x) - T(y)| = |x - y|/2$, so the contraction inequality holds with factor $1/2$. But $T(1) = 3/2$ does not lie in $[0, 1]$.

Solution to Exercise 12.5

Fixed point. The equation $x = ax + b$ becomes $(1 - a)x = b$. Since $|a| < 1$, we have $a \ne 1$, so we may divide: $x_* = b/(1 - a)$.

The error formula. Subtracting $x_* = ax_* + b$ from $x_{n+1} = ax_n + b$ cancels the constants b:

$$x_{n+1} - x_* = a(x_n - x_*).$$

Now induct on n. For $n = 0$ the formula reads $x_0 - x_* = a^0(x_0 - x_*)$, which is true because $a^0 = 1$ (when $a = 0$, read a^0 as 1, as agreed after Theorem 12.2). If $x_n - x_* = a^n(x_0 - x_*)$ for some n, then

$$x_{n+1} - x_* = a(x_n - x_*) = a \cdot a^n(x_0 - x_*) = a^{n+1}(x_0 - x_*),$$

which is the formula for $n + 1$.

How the iterates move. The distance $|x_n - x_*| = |a|^n\,|x_0 - x_*|$ shrinks by the factor $|a|$ at each step, and for $x_0 \ne x_*$ the sign of a^n decides on which side of x_* the iterate x_n lies. If $0 < a < 1$, every a^n is positive, so all iterates lie on the same side of x_* as x_0 and approach it from that side. If $a < 0$, then a^n is positive for even n and negative for odd n, so the iterates jump from one side of x_* to the other at every step, with ever shorter jumps. For example, $a = -1/2$ and $b = 3/2$ give the fixed point 1, and the iterates from $x_0 = 0$ are $0,\ 3/2,\ 3/4,\ 9/8,\ 15/16, \ldots$, alternately below and above 1.

Solution to Exercise 12.6

By Theorem 12.2, applied to each map separately, the fixed points exist: $T(x_T) = x_T$ and $S(x_S) = x_S$. The contraction inequality compares two outputs of the *same* map, so insert the

intermediate point $T(x_S)$:

$$\begin{aligned} d(x_T, x_S) &= d(T(x_T), S(x_S)) \\ &\le d(T(x_T), T(x_S)) + d(T(x_S), S(x_S)) \\ &\le q\, d(x_T, x_S) + \eta. \end{aligned}$$

The first term is bounded by the contraction inequality for T, the second by the hypothesis $d(Tx, Sx) \le \eta$ with $x = x_S$. Write $r = d(x_T, x_S)$. Then $r \le qr + \eta$, so $(1 - q)r \le \eta$, and dividing by $1 - q > 0$ gives $r \le \eta/(1 - q)$.

So a small change in the map causes only a small change in the fixed point, as long as q is not close to 1.

A.13 Chapter 13

Solution to Exercise 13.1

The probabilities add up to 1: $\frac{1}{2} + \frac{1}{3} + \frac{1}{6} = \frac{3}{6} + \frac{2}{6} + \frac{1}{6} = 1$. By the formula $\mathbb{E}\, X = \sum_i x_i\, \mathbb{P}(X = x_i)$ of Section 13.2,

$$\mathbb{E}\, X = 0 \cdot \tfrac{1}{2} + 2 \cdot \tfrac{1}{3} + 5 \cdot \tfrac{1}{6} = \tfrac{4}{6} + \tfrac{5}{6} = \tfrac{9}{6} = \tfrac{3}{2}.$$

Both 2 and 5 are at least $3/2$, as Proposition 13.5 guarantees for some value of positive probability.

Solution to Exercise 13.2

Write a labeling by listing the labels of the three vertices in a fixed order; there are $2^3 = 8$ labelings. In 000 and 111 all three vertices are on the same side, and the cut has size 0. In each of the other six, two vertices share a label and the third has the other label; the two edges at the third vertex cross, and the edge joining the other two does not, so the cut has size 2. The possible cut sizes are therefore 0 and 2, and the average over all eight labelings is

$$\frac{2 \cdot 0 + 6 \cdot 2}{8} = \frac{12}{8} = \frac{3}{2},$$

half of the three edges, as in the proof of Theorem 13.8. The average is not a possible cut size, but the labelings with cut size 2 do at least as well as the average.

Solution to Exercise 13.3

There are $m = 4$ edges, so at the start $C = 0$, $U = 4$, and $F = 2$.

- Vertex 1 has no placed neighbors, so $\ell = r = 0$, and the tie puts it on the left. No edge is settled yet, so $F = 2$.
- Vertex 2 has 1 on the left (3 is not yet placed), so $\ell = 1$, $r = 0$; place 2 on the right. Edge 12 crosses, so $C = 1$, $U = 3$, and $F = 1 + 3/2 = 5/2$.

- Vertex 3 has 2 on the right (4 is not yet placed), so $\ell = 0$, $r = 1$; place 3 on the left. Edge 23 crosses, so $C = 2$, $U = 2$, and $F = 3$.
- Vertex 4 has 3 and 1 on the left, so $\ell = 2$, $r = 0$; place 4 on the right. Edges 34 and 41 cross, so $C = 4$, $U = 0$, and $F = 4$.

Each increase of F equals $|\ell - r|/2$, namely 0, $1/2$, $1/2$, and 1. The final cut $L = \{1, 3\}$, $R = \{2, 4\}$ has size 4, while Theorem 13.8 guarantees only $m/2 = 2$. This is a largest possible cut, because no cut can have more than $m = 4$ crossing edges; the path example in Section 13.5 shows that the construction does not always do this well.

Solution to Exercise 13.4

Let A_1, A_2, A_3, A_4 be the bad events. The union bound (Proposition 13.6) gives

$$\mathbb{P}(A_1 \cup A_2 \cup A_3 \cup A_4) \leq \sum_{j=1}^{4} \mathbb{P}(A_j) \leq 4 \cdot 0.2 = 0.8.$$

So the complement of the union, the event that all four bad events are avoided, has probability at least $1 - 0.8 = 0.2 > 0$, and therefore contains an outcome of positive probability (Section 13.1). That outcome belongs to none of the four bad events.

With the bound 0.25, the same calculation gives only $\mathbb{P}(A_1 \cup \cdots \cup A_4) \leq 4 \cdot 0.25 = 1$, which is true of every event, so the argument proves nothing. In fact the conclusion can fail: in a uniform space with four outcomes $\omega_1, \omega_2, \omega_3, \omega_4$, the events $A_j = \{\omega_j\}$ each have probability 0.25, but their union is the whole sample space, so no outcome avoids them all. It can also hold: in the same space, the events $A_1 = A_2 = A_3 = A_4 = \{\omega_1\}$ each have probability 0.25, and ω_2 avoids them all. So the information given does not decide the question.

Solution to Exercise 13.5

Toss one fair coin, and let X be 0 if it shows 0 and 10 if it shows 1. Then $\mathbb{E}\, X = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 10 = 5$, but X takes the value 0 at an outcome of probability $1/2$.

What the expectation does guarantee is Proposition 13.5: in a finite probability space, some outcome ω of positive probability has $X(\omega) \geq \mathbb{E}\, X$, and some outcome ω of positive probability has $X(\omega) \leq \mathbb{E}\, X$. Here these are the outcomes where the coin shows 1 and 0, respectively.

Solution to Exercise 13.6

Markov's inequality (Proposition 13.10) with $a = 12$ gives

$$\mathbb{P}(X \geq 12) \leq \frac{\mathbb{E}\, X}{12} = \frac{3}{12} = \frac{1}{4}.$$

For an example where equality holds, take a uniform space with four outcomes $\omega_1, \omega_2, \omega_3, \omega_4$, and let $X(\omega_1) = 12$ and $X(\omega_2) = X(\omega_3) = X(\omega_4) = 0$. Then X is nonnegative, $\mathbb{E}\, X = 12/4 = 3$, and

$\mathbb{P}(X \geq 12) = \mathbb{P}(\{\omega_1\}) = 1/4$, exactly the bound. So Markov's inequality cannot be improved in general.

To find such an example, ask when both estimates in the proof of Markov's inequality are equalities: at outcomes of positive probability, the first needs $X = a$ wherever $X \geq a$, and the second needs $X = 0$ wherever $X < a$. So X should take only the values 0 and 12 there, and then $\mathbb{E}\, X = 12\ \mathbb{P}(X = 12) = 3$ forces $\mathbb{P}(X = 12) = 1/4$.

A.14 Chapter 14

Solution to Exercise 14.1

Write $a = (a_0, a_1, a_2) = (1, 1, -1)$, so $n = 3$. The aperiodic correlations are

$$A_1 = a_0a_1 + a_1a_2 = 1 \cdot 1 + 1 \cdot (-1) = 0, \qquad A_2 = a_0a_2 = 1 \cdot (-1) = -1.$$

For the periodic correlations, the positions wrap around modulo 3 (for instance 3 mod 3 = 0 and 4 mod 3 = 1):

$$C_1 = a_0a_1 + a_1a_2 + a_2a_0 = 1 \cdot 1 + 1 \cdot (-1) + (-1) \cdot 1 = -1,$$
$$C_2 = a_0a_2 + a_1a_0 + a_2a_1 = 1 \cdot (-1) + 1 \cdot 1 + (-1) \cdot 1 = -1.$$

Equation (14.1) predicts $C_1 = A_1 + A_2 = -1$ and $C_2 = A_2 + A_1 = -1$, which agrees. Since $|A_1| = 0$ and $|A_2| = 1$ are at most one, $(1, 1, -1)$ is a Barker sequence.

Solution to Exercise 14.2

Every term of a correlation is a product of two entries, and $(-x)(-y) = xy$. For $1 \leq s \leq n - 1$,

$$A_s(-a) = \sum_{j=0}^{n-1-s} (-a_j)(-a_{j+s}) = \sum_{j=0}^{n-1-s} a_j a_{j+s} = A_s(a),$$

and for $0 \leq s \leq n - 1$, in the same way,

$$C_s(-a) = \sum_{j=0}^{n-1} (-a_j)(-a_{(j+s) \bmod n}) = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n} = C_s(a).$$

Solution to Exercise 14.3

The rows of H are

$$\text{row } 0 = (1, 1, 1, -1), \quad \text{row } 1 = (-1, 1, 1, 1), \quad \text{row } 2 = (1, -1, 1, 1), \quad \text{row } 3 = (1, 1, -1, 1).$$

The entry in position (i, ℓ) of HH^{T} is the dot product of row i with row ℓ:

$$(HH^{\mathsf{T}})_{02} = 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 + (-1) \cdot 1 = 1 - 1 + 1 - 1 = 0,$$
$$(HH^{\mathsf{T}})_{13} = (-1) \cdot 1 + 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 = -1 + 1 - 1 + 1 = 0,$$
$$(HH^{\mathsf{T}})_{22} = 1 \cdot 1 + (-1) \cdot (-1) + 1 \cdot 1 + 1 \cdot 1 = 4.$$

These agree with $HH^{\mathsf{T}} = 4I$, which has 4 on the diagonal and 0 elsewhere. By Proposition 14.5(a), the entries in positions $(0, 2)$ and $(1, 3)$ equal $C_{(0-2) \bmod 4} = C_2$ and $C_{(1-3) \bmod 4} = C_2$, in agreement with the value $C_2 = 0$ found in Section 14.2.

Solution to Exercise 14.4

The definition of C_s requires $0 \le s \le n-1$, and that of C_{n-s} requires $0 \le n-s \le n-1$, that is, $1 \le s \le n$. So both sides are defined exactly when $1 \le s \le n-1$ (for $n = 1$, never).

Let $1 \le s \le n-1$. By definition,

$$C_{n-s} = \sum_{j=0}^{n-1} a_j a_{(j+n-s) \bmod n}.$$

The integers $j + n - s$ and $j - s$ differ by n, so they have the same remainder on division by n. Hence

$$C_{n-s} = \sum_{j=0}^{n-1} a_j a_{(j-s) \bmod n}.$$

Now set $k = (j - s) \bmod n$. As j runs through $0, 1, \ldots, n-1$, so does k, each value exactly once, because $j \mapsto (j - s) \bmod n$ is a bijection of $\{0, 1, \ldots, n-1\}$ (Section 14.3). Moreover $j \equiv k + s \pmod{n}$, and j lies between 0 and $n-1$, so $j = (k + s) \bmod n$. Replacing both occurrences of the old index,

$$\begin{aligned} C_{n-s} &= \sum_{k=0}^{n-1} a_{(k+s) \bmod n}\, a_k \\ &= \sum_{k=0}^{n-1} a_k\, a_{(k+s) \bmod n} \\ &= C_s. \end{aligned}$$

The first equality lists the same terms in a different order, the second swaps the two factors in each product, and the last is the definition of C_s.

Equation (14.1) gives a second proof: it says $C_s = A_s + A_{n-s}$, and applied to the shift $n - s$ it says $C_{n-s} = A_{n-s} + A_{n-(n-s)} = A_{n-s} + A_s$.

Solution to Exercise 14.5

By Section 14.4, the order must be a perfect square, and by Lemma 14.6, an order greater than 2 must be divisible by four.

- 8 is not a perfect square: for an integer r, if $|r| \le 2$ then $r^2 \le 4$, and if $|r| \ge 3$ then $r^2 \ge 9$. So 8 is ruled out.
- $9 = 3^2$ and $25 = 5^2$ are perfect squares, but neither is divisible by four, so both are ruled out.

- $16 = 4 \cdot 2^2$ and $36 = 4 \cdot 3^2$ satisfy both conditions. They remain possible orders, of the form $4u^2$ with $u = 2$ and $u = 3$.

The answer does not show that a circulant Hadamard matrix of order 16 or 36 exists: necessary conditions can only rule orders out. In fact, both orders were ruled out long ago by the stronger restrictions mentioned in Section 14.7. You can even settle order 16 yourself: extending the search of Section 14.6 to the $2^{16} = 65536$ sequences of length 16 finds none with $C_s = 0$ for every shift s with $1 \le s \le 15$.

Only the divisibility condition applies to all Hadamard matrices; the square condition needs the circulant pattern. For example, Hadamard matrices of order 8 exist, although none is circulant, since 8 is not a perfect square. Write h_0, h_1, h_2, h_3 for the rows of the matrix H of Section 14.3, and form the 8-by-8 matrix with the rows

$$(h_0, h_0),\ (h_1, h_1),\ (h_2, h_2),\ (h_3, h_3),\ (h_0, -h_0),\ (h_1, -h_1),\ (h_2, -h_2),\ (h_3, -h_3),$$

where $(h_i, \pm h_i)$ means row h_i followed by $\pm h_i$. The dot product of two of these rows is the dot product of their first halves plus that of their second halves. Since different rows of H are orthogonal, for $i \ne \ell$

$$(h_i, h_i) \cdot (h_\ell, h_\ell) = (h_i, -h_i) \cdot (h_\ell, -h_\ell) = 2\, h_i \cdot h_\ell = 0,$$

and for all i and ℓ, including $i = \ell$,

$$(h_i, h_i) \cdot (h_\ell, -h_\ell) = h_i \cdot h_\ell - h_i \cdot h_\ell = 0.$$

So different rows are orthogonal, and by Section 14.3 this matrix with entries ± 1 is a Hadamard matrix of order 8.

Solution to Exercise 14.6

Existence. Suppose the search finds a sequence a of length n with $C_s(a) = 0$ for every s with $1 \le s \le n - 1$. By Proposition 14.5(b), the circulant matrix of a is a circulant Hadamard matrix of order n. This does not depend on trusting the search: the sequence is a *certificate* of existence, whose $n - 1$ correlations anyone can recompute, by hand or with an independent program.

Nonexistence at one length. Suppose instead that the search correctly examines all 2^n sequences and finds none that qualifies. The first row of a circulant Hadamard matrix of order n would be one of them, so no such matrix exists. This conclusion has no short certificate: to accept it, we must trust that the program examined every case and computed every correlation correctly, or run an independent search ourselves. And since only sequences of length n were examined, it says nothing about any other length.

All orders greater than 4. There are infinitely many such orders, so no finite search can cover them; what is needed is a proof that works for an arbitrary order n. The proof of Proposition 14.7 is of this kind, but not strong enough: it leaves the orders $4u^2$ with $u \ge 2$ open. Alternatively, one could prove that any example, if one existed, would have to appear among finitely many cases, and then search all of those cases.

A.15 Chapter 15

To run the Lean solutions for Chapters 15 and 16, add them to the file of Section B.4, after the definitions they use from Chapters 15 and 16, such as `Inj`, `preimage`, and `Second`.

Solution to Exercise 15.1

Here `P` and `Q` are propositions, and `h` is a proof of the implication `P -> Q`, that is, a function that turns any proof of P into a proof of Q. So when `hp : P`, the application `h hp` is a proof of Q: it carries out the inference "P holds, and P implies Q; therefore Q holds."

Solution to Exercise 15.2

A proof of a conjunction is a pair of proofs, so we take the first component:

```
theorem and_left (P Q : Prop) : And P Q -> P := by
  intro h
  exact h.left
```

After `intro h`, the hypothesis is `h : And P Q` and the goal is `P`, which `h.left` proves. The only inference is: if P and Q both hold, then in particular P holds. The theorem does not claim that P is true on its own, only that it is true whenever $P \wedge Q$ is.

Solution to Exercise 15.3

Offer the witness 5:

```
theorem witness_five : Exists (fun n : Nat => n + 2 = 7) := by
  exact Exists.intro 5 rfl
```

The statement is the Lean form of $\exists n \in \mathbb{N} : n + 2 = 7$. The first input of `Exists.intro` is the witness 5; the second must be a proof of `5 + 2 = 7`, and `rfl` supplies it, because `5 + 2` reduces to `7`. Since `n` is declared a `Nat`, the witness must be a natural number; 5 is the only one that works.

Solution to Exercise 15.4

Equal inputs give equal outputs, but equal outputs need not come from equal inputs. The term `congrArg g h` applies g once more to both sides of `h : g x = g y`. This makes sense only when g can be applied to its own outputs, as for `g : Nat -> Nat`, and then it proves `g (g x) = g (g y)`. Since `congrArg` moves forward through a function, never backward, it cannot remove g from both sides. The missing hypothesis is injectivity of g: with `hg : Inj g`, the term `hg x y h` proves `x = y`. Without it the conclusion can be false: the constant function $g : \mathbb{N} \to \mathbb{N}$ with $g(n) = 0$ satisfies $g(1) = g(2)$, yet $1 \neq 2$.

Solution to Exercise 15.5

Given $f(x) = f(y)$, apply g to both sides and then use injectivity of the composition:

```
theorem inj_of_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hcomp : Inj (fun x => g (f x))) : Inj f := by
  intro x y hxy
  exact hcomp x y (congrArg g hxy)
```

After `intro x y hxy`, we have `hxy : f x = f y` and the goal `x = y`. The term `congrArg g hxy` proves `g (f x) = g (f y)`, which Lean accepts as evidence that the composition takes equal values at `x` and `y`, because `(fun x => g (f x)) x` beta-reduces to `g (f x)` (Section 15.5). Then `hcomp x y` turns that evidence into a proof of `x = y`.

The only hypothesis is about the composition; nothing is assumed about g, and rightly so, because by the second part of Proposition 3.3, g need not be injective. In the example there, $g(1) = g(2)$, but 2 lies outside the image of f, so the composition never detects the collision.

Solution to Exercise 15.6

Natural-number subtraction is truncated at zero, so `(7 - 10 : Nat)` computes to 0, while `(7 - 10 : Int)` computes to -3, because the integers contain negative numbers. The same symbols name two different operations, chosen by the type. The lesson: changing the domain of a theorem can change the meaning of its operations, not only the values its variables may take, so check every subtraction before translating a statement about integers into one about natural numbers. For instance, the law $(a - b) + b = a$ fails in `Nat` whenever $a < b$: `(7 - 10) + 10` is 10, not 7.

A.16 Chapter 16

Solution to Exercise 16.1

The fact `hb : g b = c`, obtained by unpacking `hg c`, says that b is a preimage of the target c under g. The fact `ha : f a = b`, obtained by unpacking `hf b`, says that a is a preimage of b under f.

The witness must lie in A because the composition `fun x => g (f x)` takes its inputs from `A`, the domain of f: after `intro c`, the goal asks for some `x` in `A` with $g(f(x)) = c$. The witness a has this type because it came from `hf b`, which provides an element of `A` sent to b by f; proposing b, which lies in B, would be a type error.

Solution to Exercise 16.2

An element lies in the union $S \cup T$ when it lies in S or in T, so union is represented by the disjunction of the two membership predicates:

```
def join {A : Type} (s t : A -> Prop) : A -> Prop :=
  fun x => Or (s x) (t x)

theorem preimage_join {A B : Type} (f : A -> B)
    (s t : B -> Prop) (x : A) :
    Iff (preimage f (join s t) x)
      (join (preimage f s) (preimage f t) x) := by
  rfl
```

As for `preimage_meet`, the left side unfolds to `join s t (f x)` and the right side to `Or (preimage f s x) (preimage f t x)`, and both then to `Or (s (f x)) (t (f x))`, so `rfl` proves the equivalence. In ordinary notation, this is the identity $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$ of Exercise 3.4.

Solution to Exercise 16.3

As in Section 16.4, write `Second r v w` as `And P (And Q R)`. The first claim is P, the first component of `h`, and the second is R, the second component of the second component:

```
theorem second_not_self {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) : Not (w = v) := by
  exact h.left

theorem second_route {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) :
    Exists (fun u : V => And (r v u) (r u w)) := by
  exact h.right.right
```

Had `Second` been defined with the shape `And (And P Q) R`, the same two facts would be `h.left.left` and `h.right`.

Solution to Exercise 16.4

The new theorem has the original conclusion and one more hypothesis, `h : False`. Since `False` has no proof, nobody can supply this hypothesis, so the new theorem can never be applied. It says only that the conclusion would follow *if* `False` were provable, which is true of every conclusion, by `False.elim`; it says nothing about whether the conclusion follows from the original hypotheses, which was the question. Nor need Lean's output reveal the change: the new proof needs no `sorry`, `#print axioms` may report no axioms, and a warning about unused hypotheses, as for `surj_comp_assumed`, is easy to overlook and absent when the original theorem had no hypotheses, as for `impossible_context`. Only a comparison of the new statement with the original, hypothesis by hypothesis, is sure to reveal the change.

Solution to Exercise 16.5

Step 1: compare the statement with the intended one. Fill in the two-column table of Section 16.1 from the theorem's parameters and conclusion, including the type of every variable. A changed domain, such as `Nat` where integers were meant, shows up here as a mismatch of types.

Step 2: read the definitions. Compare the body of each definition that the statement uses with the mathematical definition. A missing exclusion, such as a version of `Second` without `Not (r v w)`, shows up here and nowhere else, because the name of the definition stays the same.

Step 3: check the proof. Run Lean on the whole file, wait until it has finished, read every warning, and run `#print axioms` on the theorem. An unfinished proof shows up as the warning `declaration uses 'sorry'` at the declaration that contains the gap, and as `sorryAx` in the axiom list of every theorem that depends on it.

All three steps are needed, because each problem passes the other two: a complete proof about `Nat` where integers were meant, with correct definitions, passes Steps 2 and 3, a complete proof about a definition with a missing exclusion passes Steps 1 and 3, and a correct statement with an unfinished proof passes Steps 1 and 2.

Solution to Exercise 16.6

The three kinds of check differ in what is checked and in how many cases they cover.

- *A finite test in Python.* At length 5, the program of Section B.3 runs through all $2^5 = 32$ sign sequences and finds exactly 4 Barker sequences (Section 14.6). This covers 32 cases and no others, and the answer is only as reliable as the program's translation of the definitions, which no checker compares with the code.
- *A Lean proof about specific objects.* The periodic correlation at shift 1 of the sequence $(1, 1, 1, -1)$ is $a_0a_1 + a_1a_2 + a_2a_3 + a_3a_0 = 0$:

  ```
  theorem corr_shift_one :
      (1 : Int) * 1 + 1 * 1 + 1 * (-1) + (-1) * 1 = 0 := by
    rfl
  ```

  Lean checks this proof completely, but it covers one case: one sequence and one shift. (Even here the encoding must match the definition: the last product, a_3a_0, is the wrap-around term.)
- *A universal theorem in Lean.* The theorem `preimage_meet` holds for all types A and B, every function f, all predicates `s` and `t`, and every element `x`: one proof covers infinitely many cases.

A.17 Chapter 17

Solution to Exercise 17.1

Take any point c of the space and the constant sequence $x_n = c$. Every ball $B(c, r)$ contains x_n for every index n, yet the sequence takes only one value, c, however many other points the ball contains.

The proof does not need different values. Lemma 17.3 counts indices, not values, and Step 2 needs infinitely many indices only to choose ever larger ones. For a constant sequence, every subsequence is constant and converges at once to its value.

Solution to Exercise 17.2

Without the requirement $n_k > n_{k-1}$, the chosen indices need not increase, so the terms $x_{n_1}, x_{n_2}, \ldots$ need not form a subsequence. For example, for a constant sequence $x_n = c$, each chosen ball must contain c to give an infinite set S_k (a ball without c contains no term), and then $S_k = \mathbb{N}$ for every k. The rule "take the smallest member of S_k" then gives $n_k = 0$ for every k, and $x_0, x_0, x_0, \ldots$ is not a subsequence. The distance estimates of Steps 3 and 4 still hold, because they use only $n_k \in S_k$; what fails is the subsequence that the theorem promises.

The repair: after choosing n_{k-1}, choose n_k from $S_k \setminus \{0, 1, \ldots, n_{k-1}\}$, whose members lie in S_k and are larger than n_{k-1}. This set is never empty: otherwise S_k would lie inside the finite set $\{0, 1, \ldots, n_{k-1}\}$ and be finite, but S_k is infinite by Step 1.

Solution to Exercise 17.3

(a) The argument is the one given for $\mathbb{N}$ in Section 17.6, which uses only that every distance is 0 or 1. For a Cauchy sequence in X, the tolerance $1/2$ supplies a threshold N with $d(x_m, x_n) < 1/2$, and hence $d(x_m, x_n) = 0$, for all $m, n \geq N$. So $x_n = x_N$ for every $n \geq N$, and the sequence converges to the point x_N of X: for every $\varepsilon > 0$ and $n \geq N$, $d(x_n, x_N) = 0 < \varepsilon$.

(b) Suppose X is finite, with members $c_1, \ldots, c_k$ (if X is empty, $k = 0$ and the empty list covers it). For every $r > 0$, each point x of X is one of the c_i, and $d(x, x) = 0 < r$, so the balls $B(c_1, r), \ldots, B(c_k, r)$ cover X, and X is totally bounded. Conversely, suppose X is totally bounded, and take $r = 1/2$. Finitely many balls $B(c_1, 1/2), \ldots, B(c_k, 1/2)$ cover X, and each contains only its center, because every other point is at distance 1 from it. So $X = \{c_1, \ldots, c_k\}$ is finite.

(c) Suppose first that $x_n = c$ for every n in an infinite set S of indices. As in Step 2 of the proof of Theorem 17.5, choose $n_1 < n_2 < \cdots$ in S, which is possible because S is infinite. The subsequence $x_{n_1}, x_{n_2}, \ldots$ is the constant sequence $c, c, c, \ldots$, which converges to c.

Conversely, suppose that a subsequence $x_{n_1}, x_{n_2}, \ldots$ converges to a point c of X. For the tolerance $1/2$ there is a threshold K with $d(x_{n_k}, c) < 1/2$, and hence $x_{n_k} = c$, for every $k \geq K$. The indices $n_K < n_{K+1} < n_{K+2} < \cdots$ are infinitely many natural numbers, and the sequence takes the value c at each of them.

Now take $X = \mathbb{N}$. By (a) and (b), it is complete but not totally bounded. The sequence $x_n = n$ takes each value at only one index, so by (c) no subsequence of it converges, although X is complete. Completeness gives a limit only to a sequence that is already a Cauchy sequence; producing a Cauchy subsequence is the job that total boundedness does in Theorem 17.5.

Solution to Exercise 17.4

Fix $r > 0$. Since $b - a > 0$, the Archimedean property gives a positive integer N with $N > (b-a)/r$, so the spacing $h = (b-a)/N$ satisfies $0 < h < r$. Use the $N + 1$ centers

$$c_k = a + kh, \qquad k = 0, 1, \ldots, N.$$

They run from $c_0 = a$ to $c_N = b$, so all of them are points of $[a, b]$, as the definition requires.

Let $x \in [a, b]$. If $x = b$, then x is the center c_N. Otherwise $a \le x < b$. Let k be the largest index with $c_k \le x$; there is one, because $c_0 = a \le x$. Then $k < N$, because $c_N = b > x$, and $x < c_{k+1}$. Hence

$$d(x, c_k) = x - c_k < c_{k+1} - c_k = h < r,$$

so x lies in the ball of radius r about c_k. Thus the $N + 1$ balls of radius r cover $[a, b]$.

The number $N + 1$ of balls, and the centers, depend on the radius r and on the endpoints a and b, but not on the point x: the list was chosen as soon as r was given, before any x was considered, and that one list covers every $x \in [a, b]$.

Solution to Exercise 17.5

The out-neighbors of v are u and w, so $N_1^+(v) = \{u, w\}$. The only route of length two from v is $v \to u \to w$, because w has no out-neighbors, so $R_2(v) = \{w\}$. But w is a first out-neighbor of v, because of the edge $v \to w$, so it is excluded from the second out-neighborhood, and $N_2^+(v) = \varnothing$.

In every directed graph, $N_2^+(v)$ is $R_2(v)$ with v and the first out-neighbors of v removed, so $|R_2(v)| \ge |N_2^+(v)|$. A statement asking for a vertex with $|R_2(v)| \ge |N_1^+(v)|$ would therefore be easier to satisfy, and the difference is real. Add a fourth vertex z and the edge $w \to z$; the graph is still oriented. Now $N_1^+(v) = \{u, w\}$, $R_2(v) = \{w, z\}$ and $N_2^+(v) = \{z\}$, so at v the altered inequality holds ($2 \ge 2$) while Seymour's fails ($1 < 2$).

So a formal statement using $R_2(v)$ would state a weaker claim, and a proof of it would not prove Seymour's conjecture. In fact the weaker claim has a short proof by an extremal choice, as in Chapter 7. In a nonempty finite directed graph, choose a vertex v of smallest outdegree $\delta = |N_1^+(v)|$. If $\delta = 0$, there is nothing to prove. Otherwise pick an out-neighbor u of v. Every out-neighbor z of u is reached by the route $v \to u \to z$, so $N_1^+(u) \subseteq R_2(v)$, and u has outdegree at least δ; hence

$$|R_2(v)| \ge |N_1^+(u)| \ge \delta = |N_1^+(v)|.$$

A few lines settle the altered statement, while Seymour's conjecture stayed open for more than thirty years. In an oriented graph v itself never lies in $R_2(v)$, so all of the difficulty lies in excluding the first out-neighbors. The replacement changes what the statement means; it is not a harmless simplification of notation.

Solution to Exercise 17.6

Each request should say what the reader knows, name one particular proof, and ask only for what that proof needs. For the contraction theorem, from a reader who has met metric spaces but not the

rest of Chapter 11: “I know metric spaces, the triangle inequality and induction. I want to read the proof of the contraction theorem that iterates the map from any starting point, shows that the iterates form a Cauchy sequence, and takes their limit. Teach me convergence, Cauchy sequences, completeness and the bound for a sum of geometrically shrinking steps, only as far as this proof needs them, and after each concept show me the step that uses it.” For Theorem 17.5: “I know metric spaces, Cauchy sequences and completeness. I want to read the proof that every sequence in a complete, totally bounded metric space has a convergent subsequence by keeping infinitely many terms inside smaller and smaller balls. Teach me finite covers by small balls, the infinite pigeonhole principle and subsequences, only as far as this proof needs them, and after each concept show me the step that uses it.”

Both proofs use convergence, Cauchy sequences and completeness. Only the contraction proof needs fixed points, the contraction factor q and the geometric-sum bound of Proposition 11.9; only the subsequence proof needs balls, total boundedness, the infinite pigeonhole principle and subsequences.

If you have not run the formal check, a reading record may report on your own authority what you decoded, inspected or reconstructed yourself: statements and definitions, the files and theorems you read, the roles of the dependencies, and any proofs you rebuilt. It may report the repository’s claim about its check only as the repository’s claim: “the repository reports that the check passes,” not “the check passes.” It may not claim that you compiled the project, ran the check, or verified the imported proofs (Section 17.8).

A.18 Chapter 18

Solution to Exercise 18.1

Section 18.3 applied the one-step estimate with $n = 0, 1, 2$, each time substituting the bound already found, multiplied by $q > 0$, and reached $e_3 \le q^3 e_0 + q^2 \varepsilon_0 + q\varepsilon_1 + \varepsilon_2$. One more step, with $n = 3$, gives

$$\begin{aligned} e_4 &\le q e_3 + \varepsilon_3 \le q(q^3 e_0 + q^2 \varepsilon_0 + q\varepsilon_1 + \varepsilon_2) + \varepsilon_3 \\ &= q^4 e_0 + q^3 \varepsilon_0 + q^2 \varepsilon_1 + q\varepsilon_2 + \varepsilon_3. \end{aligned}$$

Part 1 of Theorem 18.1 with $n = 4$ reads $e_4 \le q^4 e_0 + \sum_{j=0}^{3} q^{3-j} \varepsilon_j$, and writing out the sum for $j = 0, 1, 2, 3$ gives $q^3 \varepsilon_0 + q^2 \varepsilon_1 + q\varepsilon_2 + q^0 \varepsilon_3$, so the two bounds are the same.

In the bound for e_{10}, the update error ε_6 carries the weight $q^{10-1-6} = q^3$. Indeed, ε_6 is made in the update that produces y_7, and after it come the three updates that produce y_8, y_9 and y_{10}, each of which multiplies its effect by q.

Solution to Exercise 18.2

The formula. At $n = 0$ the formula gives $q^0 a + \eta(1 - q^0)/(1 - q) = a = y_0$. If it holds for some n, then

$$y_{n+1} = q\left(q^n a + \eta \frac{1 - q^n}{1 - q}\right) + \eta = q^{n+1} a + \eta \frac{q - q^{n+1} + 1 - q}{1 - q} = q^{n+1} a + \eta \frac{1 - q^{n+1}}{1 - q},$$

which is the formula for $n + 1$. By induction it holds for every $n \in \mathbb{N}$.

Equality in part 2. The map $T(x) = qx$ is a contraction of $\mathbb{R}$ with factor q and fixed point 0 (Section 18.2), and every update error is $|y_{n+1} - q y_n| = \eta$, so part 2 of Theorem 18.1 applies. The starting error is $e_0 = a$. Since $a, \eta \geq 0$ and $0 < q^n \leq 1$, every y_n is nonnegative, so

$$e_n = y_n = q^n e_0 + \eta \frac{1 - q^n}{1 - q},$$

which is exactly the bound of part 2, for every $n \geq 1$.

Sharpness. With $a = 0$ and $\eta > 0$ this is the example of Section 18.4. If the factor $1/(1 - q)$ could be replaced by some $K < 1/(1 - q)$, we would have $\eta(1 - q^n)/(1 - q) \leq K\eta$ for every $n \geq 1$; dividing by $\eta > 0$ and multiplying by $1 - q > 0$ gives $1 - q^n \leq K(1 - q)$, that is, $q^n \geq 1 - K(1 - q)$. The right side is positive, because $K(1 - q) < 1$, and $q^n \to 0$, so this fails for every large n. Hence no smaller factor works for every iteration satisfying the hypotheses: $1/(1 - q)$ is sharp.

Solution to Exercise 18.3

The hypotheses. The real line with the distance $|x - y|$ is a nonempty complete metric space (Chapter 11). The map $T(x) = 0$ maps $\mathbb{R}$ into itself, and $|T(x) - T(y)| = 0 \leq q|x - y|$ for every q with $0 < q < 1$, so T is a contraction with factor q; its fixed point is 0. Every y_n lies in $\mathbb{R}$, and $|y_{n+1} - T(y_n)| = |(-1)^{n+1}\eta| = \eta$, so the update errors are $\varepsilon_n = \eta \geq 0$, and the hypothesis of part 2 holds with this same η.

The bound of part 2. Every error is $e_n = |(-1)^n \eta| = \eta$. For $n \geq 1$, the bound of part 2 is $q^n \eta + \eta\left(1 + q + \cdots + q^{n-1}\right)$, a sum of nonnegative terms that includes $\eta \cdot 1$, so it is at least $\eta = e_n$.

No convergence. Consecutive terms are always 2η apart: $|y_{n+1} - y_n| = |-2(-1)^n \eta| = 2\eta$. Take the tolerance $\eta > 0$. For every proposed threshold N, the indices N and $N + 1$ are both at least N, and $|y_{N+1} - y_N| = 2\eta \geq \eta$. So the sequence is not a Cauchy sequence, and by Proposition 11.5 it does not converge.

Why the bound does not settle convergence. The bound controls only the distance from y_n to x_*, and different points, here η and $-\eta$, can lie at the same distance, so the points may keep jumping between them. Distances to x_* force convergence only when they tend to zero, which the bound of part 2 does not do when the update errors stay at η. That is why part 3 needs the hypothesis $\varepsilon_n \to 0$.

Solution to Exercise 18.4

The bound. Every weight q^{n-1-j} in part 1 of Theorem 18.1 is positive, so we may replace each ε_j by the larger number aq^j:

$$\begin{aligned}\sum_{j=0}^{n-1} q^{n-1-j}\varepsilon_j &\le \sum_{j=0}^{n-1} aq^{n-1-j}q^j \\ &= \sum_{j=0}^{n-1} aq^{n-1} \\ &= anq^{n-1},\end{aligned}$$

because every term has the exponent $n-1-j+j = n-1$, and there are n terms. Part 1 therefore gives $e_n \le q^n e_0 + anq^{n-1}$ for every $n \ge 1$.

The factor n. It is not enough that $q^{n-1} \to 0$, because it is multiplied by the growing number n. Put $r = (1+q)/2$. Since $q < 1$, we have $q = \frac{q+q}{2} < \frac{1+q}{2} = r < \frac{1+1}{2} = 1$. Put $t = q/r$; then $0 < t < 1$. The powers $1, t, \ldots, t^{n-1}$ decrease, so each of these n numbers is at least t^{n-1}, and adding them up gives

$$nt^{n-1} \le 1 + t + \cdots + t^{n-1} = \frac{1-t^n}{1-t} \le \frac{1}{1-t}.$$

Multiply by the positive number r^{n-1} and use $q^{n-1} = r^{n-1}t^{n-1}$:

$$0 \le nq^{n-1} = r^{n-1}\, nt^{n-1} \le \frac{r^{n-1}}{1-t}.$$

Here $1-t$ is fixed and positive, and r^{n-1} becomes as small as we like, because $0 < r < 1$.

The whole bound tends to zero. Write $b_n = q^n e_0 + anq^{n-1}$ and let $\delta > 0$; we make each term less than $\delta/2$.

- If $e_0 = 0$, put $N_1 = 1$. Otherwise, $q^n \to 0$ (Section 11.7) gives N_1 with $q^n < \delta/(2e_0)$, so $q^n e_0 < \delta/2$, for every $n \ge N_1$.
- If $a = 0$, put $N_2 = 1$. Otherwise, $r^m \to 0$ gives M with $r^m < (1-t)\delta/(2a)$ for every $m \ge M$; put $N_2 = M+1$. For $n \ge N_2$ we have $n-1 \ge M$, so $anq^{n-1} \le ar^{n-1}/(1-t) < \delta/2$.

For every n that is at least both N_1 and N_2, we get $0 \le e_n \le b_n < \delta$. So $b_n \to 0$ and $e_n \to 0$, that is, $y_n \to x_*$. This also follows from part 3, because $aq^j \to 0$; what the bound b_n adds is how small the error is after a given number of steps.

Solution to Exercise 18.5

Since several students may receive the same task, an assignment with capacities is not a matching; we reduce the problem to matchings by giving every unit of capacity its own vertex, called a *slot*.

The condition is necessary. Suppose an assignment with capacities exists, and let $S \subseteq L$. Every student in S receives a task they accept, which lies in $N(S)$. If m_r students of S receive the task r,

then $m_r \le c_r$, and every student of S is counted in exactly one of the numbers m_r, so

$$|S| = \sum_{r \in N(S)} m_r \le \sum_{r \in N(S)} c_r .$$

The condition is sufficient: a new graph. Now suppose the inequality holds for every $S \subseteq L$. Build a new bipartite graph with the same students and, in place of each task r, the c_r slots $(r,1), \dots, (r,c_r)$ (none if $c_r = 0$); a student s is joined to (r,i) exactly when s accepts r. For example, if task 1 has capacity 2 and task 2 capacity 1, the slots are $(1,1)$, $(1,2)$ and $(2,1)$, and a student who accepts task 1 is joined to both $(1,1)$ and $(1,2)$. The new graph is finite, because L and R are finite and the capacities are integers.

Hall's condition in the new graph. Let $S \subseteq L$. A slot (r,i) is joined to some student of S exactly when $r \in N(S)$, so the neighbors of S in the new graph are the slots of the tasks in $N(S)$. Slots of different tasks are different, and task r has c_r slots, so S has exactly $\sum_{r \in N(S)} c_r \ge |S|$ neighbors: Hall's condition (H) holds for the new graph.

Back to tasks. By Hall's theorem (Theorem 9.1), the new graph has a matching that saturates L: every student receives a slot they are joined to, and no slot goes to two students. Give each student the task r of their slot (r,i). The student accepts r, and task r goes to at most c_r students, because different students received different slots and r has only c_r slots. So this is an assignment with capacities. (When every capacity equals 1, the new graph is the original one, and the condition reads $|S| \le |N(S)|$.)

Solution to Exercise 18.6

Here is one possible record.

Question. The map $T(x) = (3x+1)/4$ is a contraction of $\mathbb{R}$ with factor $3/4$ and fixed point 1. If every update is computed with an error of at most $1/100$, do the points still converge to 1? If not, how far from 1 can they be?

Background. Arithmetic with fractions; induction (Chapter 5); distance, the triangle inequality, convergence, and the finite geometric sum (Chapter 11); the contraction theorem (Chapter 12).

Outline of the proof. (1) Insert $T(y_n)$ between y_{n+1} and the fixed point; the triangle inequality, $T(1) = 1$ and the contraction inequality give $e_{n+1} \le q e_n + \varepsilon_n$. (2) Unfold this by induction to $e_n \le q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j$. (3) Bound every update error by η and add up the geometric weights, which here gives $|y_n - 1| \le (3/4)^n + \frac{1}{25}\bigl(1 - (3/4)^n\bigr)$.

Counterexample to the first guess. With the constant bias $1/100$, the points are $y_n = \frac{26}{25}\bigl(1 - (3/4)^n\bigr)$ (proved in Section 18.5), which converge to $26/25$, not to the fixed point 1.

An exact check. At $n = 1$, $y_1 = \frac{3\cdot 0+1}{4} + \frac{1}{100} = \frac{13}{50}$, in agreement with the closed form $\frac{26}{25}\bigl(1 - \frac{3}{4}\bigr) = \frac{26}{100}$. The true error is $\bigl|\frac{13}{50} - 1\bigr| = \frac{37}{50}$, and the certificate is $\frac{3}{4} + \frac{1}{25} \cdot \frac{1}{4} = \frac{19}{25} = \frac{38}{50}$, so $\frac{37}{50} \le \frac{38}{50}$, computed exactly. This confirms the formulas at one index only; they hold for every n because they were proved.

What is new. The project reconstructs a standard bound for contractions whose updates are computed with errors, with exact examples; it establishes no new theorem compared with the published literature.

A further question. Let $T(x) = qx + b$ on $\mathbb{R}$, with $0 < q < 1$, and let the update errors $y_{n+1} - T(y_n)$ be $\eta, -\eta, \eta, -\eta, \ldots$ in turn. How far from the fixed point do the points end up? For $q = 3/4$ and $\eta = 1/100$, Section 18.5 suggests $1/175 = \eta/(1 + q)$, much less than the $\eta/(1 - q)$ of part 2 of Theorem 18.1, so a natural guess is $\eta/(1 + q)$ for every such map. (The restriction $q > 0$ matters: for $T(x) = -qx$, alternating errors do not cancel at all.) The proofs in this chapter cannot answer the question, because the triangle inequality ignores the signs of the update errors and so cannot exploit their cancellation.

APPENDIX B

Running the examples on a computer

None of the proofs in this book needs a computer, but a computer can help in two ways: it can carry out a finite experiment exactly, and it can check a formal proof. This appendix explains the small part of the programming language Python that the book's experiments use, including exact arithmetic with fractions (Sections B.1 and B.2); gives three complete programs, for Chapters 14, 8 and 18 (Section B.3); and shows how to check the Lean examples of Chapters 15 and 16 (Section B.4).

B.1 The smallest amount of Python needed

The programs in this appendix are written in Python, a free programming language that runs on all common operating systems; any version from Python 3.10 on will do. They use only Python's *standard library*, the collection of tools that comes with every Python installation, so nothing else has to be installed.

Running a program

A Python program is a plain text file whose name ends in `.py`, for example `signs.py`. Write it in a plain text editor; a word processor adds formatting that Python cannot read. To run the program, open a *terminal*, a window for typing commands to the computer. Move to the folder that contains the file by typing `cd` followed by the location of that folder; for example, `cd Documents/math` moves into the folder `math` inside `Documents`. Then type

```
python3 signs.py
```

On some systems the command is `python` instead of `python3`; `python3 --version` shows which version is installed.

Short calculations can also be typed directly into Python. Type `python3` in the terminal with no file name, and Python shows its *prompt*, usually >>>, to say that it is waiting for a line of Python (not a terminal command such as `python3 signs.py`); typing `2 + 2` and pressing Enter displays `4`, and `exit()` returns to the terminal. The characters >>> belong to the prompt, so never type them or copy them into a program file.

Names, numbers and lists

The line `x = 5` stores the value 5 under the name `x`, for later lines to use; a later line `x = x + 1` replaces the stored value by 6. So the sign = in Python is an instruction to store a value, not a claim that two things are equal; read as an equation, $x = x + 1$ would be false for every number. To *test*

whether two values are equal, Python uses ==: `2 + 2 == 4` is `True`, and `2 + 2 == 5` is `False`. The other comparisons are written <, <=, >, >=, and `!=` for "is not equal to."

Arithmetic uses +, -, * for multiplication, and / for division. Two stars denote a power, so `2 ** 10` is $2^{10} = 1024$, and `abs(-3)` is the absolute value 3. The sign `%` gives the remainder of Section 5.5: `17 % 5` is 17 mod 5 = 2, and `-7 % 5` is 3, because $-7 = (-2) \cdot 5 + 3$ with $0 \le 3 < 5$. The command `print` displays values separated by spaces: `print(x, x + 1)` displays `6 7` when `x` holds 6.

A *list* is an ordered collection of values, written in square brackets. In `a = [1, 1, 1, -1]`, the first entry is `a[0]`, not `a[1]`: Python numbers positions from zero, just as Chapter 14 numbers the entries of a sign sequence. So `a[3]` is the last entry, −1, and the *length* `len(a)`, the number of entries, is 4. The line `a.append(9)` adds the entry 9 at the end of `a`, and `a.pop()` removes the last entry of `a` and hands it back as its value.

Loops

A `for` loop repeats a block of lines once for each value in a range. The block consists of the indented lines below the `for` line; Python uses this indentation, not brackets, to tell where the block ends. For example:

```
a = [1, 1, 1, -1]
value = 0
for j in range(3):
    value = value + a[j] * a[j + 1]
print(value)
```

Here `range(3)` produces the values 0, 1, 2: it starts at zero and stops just before 3. So the indented line runs three times, with $j = 0$, 1 and 2, adding the products $a_0a_1 = 1$, $a_1a_2 = 1$ and $a_2a_3 = -1$ to the running total in `value`, which goes from 0 to 1, 2 and back to 1; the program prints 1, the aperiodic correlation A_1 of $(1, 1, 1, -1)$ computed in Section 14.1.

With two numbers, `range(1, 14)` produces $1, 2, \ldots, 13$: it starts at the first number and stops just before the second. A loop can also run through a list: a block that begins with `for x in [2, 3, 5]:` is carried out with `x` equal to 2, then 3, then 5.

Functions

A *function* packages a calculation so that it can be used again with different inputs.

```
def correlation(a, shift):
    value = 0
    for j in range(len(a) - shift):
        value = value + a[j] * a[j + shift]
    return value
```

The `def` line names the function, `correlation`, and its two inputs, `a` and `shift`; the indented lines below it carry out the calculation, and `return` gives the result. For a list of length n and a

shift s, the loop runs over `range(n - s)`, so j takes the values $0, 1, \ldots, n-1-s$, exactly the summation range in the definition of A_s (Definition 14.1). For example, `correlation([1, 1, 1, -1], 3)` is $a_0 a_3 = -1$, the value of A_3 found in Section 14.1. The function checks neither that `a` is a sign sequence nor that $1 \le s \le n-1$, as the definition requires: given n signs and the shift 0, it simply returns n.

Decisions and other loops

A line `if` *condition*`:` is followed by an indented block, which is carried out only when the condition is true. Conditions can be combined with the words `and`, `or` and `not`, which have the meanings of Section 2.2. A `while` loop repeats its block as long as its condition is true. After the following lines, for example, `n` holds 8, the smallest natural number whose square is at least 50:

```
n = 0
while n * n < 50:
    n = n + 1
```

Inside a loop, `break` leaves the innermost loop containing it at once, and `continue` skips the rest of the block and goes on with the next pass. The special value `None` means "no value here"; the test `x is None` asks whether `x` holds this value, and `x is not None` asks the opposite.

Tuples, sets and dictionaries

A *tuple* is like a list, with entries numbered from zero, but is written with round brackets, as in `(1, -1, 1)`, and cannot be changed once made. A tuple is a natural way to store a sign sequence $(a_0, a_1, \ldots, a_{n-1})$, and a function can return several values at once as a tuple. A piece of text is written between quotation marks, as in `"shortage"`; when displayed as an entry of a tuple, it appears between single quotes, as in `('shortage', 1)`.

A *set* records membership only, as in Section 3.1: order and repetition do not matter, so `{1, 2, 2}` is the same set as `{1, 2}`. The test `2 in s` is `True` when 2 is a member of `s`, and `not in` tests the opposite. The union $s \cup t$ is written `s | t`, so `{1, 2} | {2, 3}` is `{1, 2, 3}`. The empty set is written `set()`, because empty braces `{}` mean an empty dictionary, described below. A loop `for x in s:` visits the members of `s` in no promised order; when the order matters, loop over `sorted(s)`, the list of the members in increasing order. For example, `sorted({3, 1, 2})` is `[1, 2, 3]`.

A *dictionary* is a table that pairs each *key* with a value. After `owner = {}` creates an empty table, `owner[3] = 0` records the pair "key 3, value 0," and `owner[3]` then looks the value up. The test `3 in owner` asks whether the key 3 has an entry. Asking for `owner[5]` when the key 5 has no entry stops the program with an error, whereas `owner.get(5)` simply gives `None`. Python prints the table as `{3: 0}`: each key, a colon, and its value. The official Python tutorial explains lists, tuples, sets and dictionaries in more detail [16].

Adding up and checking many cases at once

Python can write a sum much as mathematics does. The loop that computed A_1 above can be replaced by the single expression

```
sum(a[j] * a[j + 1] for j in range(3))
```

which reads like $\sum_{j=0}^{2} a_j a_{j+1}$ and has the same value, 1. Similarly, `all(`*test* `for s in range(1, n))` is `True` when the test holds for every s with $1 \le s \le n-1$, and `False` as soon as it fails for one. For example, `all(abs(correlation(a, s)) <= 1 for s in range(1, 4))` is `True` for `a = [1, 1, 1, -1]`, and `False` for `a = [1, 1, -1, -1]`, whose correlation at shift two is $A_2 = -2$ (Section 14.1). When the range is empty, as `range(1, 1)` is, the result is `True`: a statement about every member of an empty collection is vacuously true (Section 2.5). Finally, `sum(1 for x in xs if` *test*`)` adds 1 for each member `x` of the list `xs` that passes the test, so it counts those members; for instance, `sum(1 for x in [3, 8, 5, 10] if x > 4)` is 3.

Two more conventions appear below. Python ignores everything on a line after the sign `#`, so this sign starts a comment for human readers; and a line such as `from fractions import Fraction` makes one tool from the standard library available to the program, here `Fraction` for exact rational arithmetic (Section B.2).

B.2 An exact fraction is not a rounded decimal

When a program contains a number such as `0.1`, Python does not store one tenth exactly. It stores a *floating-point number*: a binary approximation that is correct to about sixteen significant digits. The two lines

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

display `0.30000000000000004` and `False`. So in an experiment that tests an equation or an inequality, a tiny rounding error can make a true statement look false, or a false one look true.

When the numbers involved are rational, the standard library offers exact arithmetic instead [17]. A `Fraction` stores an integer numerator and a positive integer denominator, reduced to lowest terms, and it computes by the usual rules for fractions, with no rounding at any stage. For example, `Fraction(1, 3)` is the fraction $1/3$, and `Fraction(0)`, with the denominator 1 left out, is $0/1$, that is, zero. Python's integers are exact as well, however large they become.

```
from fractions import Fraction
print(Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10))
x = Fraction(1, 3)
y = (x * x + 1) / 3
print(y)
```

This program displays `True` and then `10/27`. The last three lines apply the map $T(x) = (x^2+1)/3$

of Section 12.5 to the iterate $x_1 = 1/3$, so `y` is $x_2 = T(x_1)$, and Python finds exactly the value $10/27$ computed by hand in that section. (Typed at Python's prompt without `print`, the same value is shown as `Fraction(10, 27)`.)

Two details matter. First, dividing one ordinary integer by another with `/` gives a floating-point number, so `1 / 3` is `0.3333333333333333`. But a `Fraction` combined with an integer or with another `Fraction` gives again a `Fraction`, which is why the line `y = (x * x + 1) / 3` can use the plain integers 1 and 3 and still be exact. A `Fraction` combined with a floating-point number such as `0.5`, on the other hand, gives a floating-point number, so do not mix the two kinds. Second, one tenth must be written `Fraction(1, 10)`. Writing `Fraction(0.1)` instead converts the floating-point number `0.1` into a fraction exactly as it is stored, and `print(Fraction(0.1))` displays

```
3602879701896397/36028797018963968
```

The denominator is 2^{55}, and this fraction is close to one tenth but not equal to it.

The experiments of Section B.3 keep every comparison exact: the sign-sequence counts use only integers, the iteration uses fractions, and decimals appear only when a result is displayed.

B.3 Three experiments

Each of the following programs is complete: save it in a file of its own and run it as described in Section B.1.

Counting sign sequences

The following program produces the table of Section 14.6.

```
from itertools import product

def aperiodic(a, s):
    n = len(a)
    return sum(a[j] * a[j + s] for j in range(n - s))

def periodic(a, s):
    n = len(a)
    return sum(a[j] * a[(j + s) % n] for j in range(n))

def is_barker(a):
    return all(abs(aperiodic(a, s)) <= 1 for s in range(1, len(a)))

def is_circulant_row(a):
    return all(periodic(a, s) == 0 for s in range(1, len(a)))

for n in range(1, 14):
    seqs = list(product([1, -1], repeat=n))
```

```
    barker = sum(1 for a in seqs if is_barker(a))
    circ = sum(1 for a in seqs if is_circulant_row(a))
    print(n, barker, circ)
```

The first line makes the tool `product`, from the part of the standard library called `itertools`, available to the program. Called as `product([1, -1], repeat=n)`, it fills n positions with entries of the list `[1, -1]` in every possible way, so it produces all 2^n sign sequences of length n, one at a time, each as a tuple; for $n = 2$ these are `(1, 1)`, `(1, -1)`, `(-1, 1)` and `(-1, -1)`. The line `seqs = list(...)` collects them into one list.

Exact arithmetic cannot show that a program computes the quantity the text defines, or that its loops run over exactly the intended objects. So before trusting the output, read each function against the definition it is meant to carry out.

- The function `aperiodic` is the function `correlation` of Section B.1, written more compactly with `sum`; as checked there, j runs through the summation range of A_s in Definition 14.1.
- In `periodic`, the index j runs through all of $0, 1, \ldots, n-1$, and `(j + s) % n` is the position $(j + s) \bmod n$ of Definition 14.3. This is where the wrap-around happens.
- The test `is_barker(a)` is `True` exactly when $|A_s| \le 1$ for every shift s with $1 \le s \le n-1$, the shifts that `range(1, len(a))` produces; this is Definition 14.2.
- Likewise, `is_circulant_row(a)` is `True` exactly when $C_s = 0$ for every shift s with $1 \le s \le n-1$. By Section 14.3, these are exactly the sequences that form the first row of a circulant Hadamard matrix, hence the name.

For each length n from 1 to 13, the main loop counts in `barker` and `circ` the sequences that pass each test and prints one line with n and the two counts; the thirteen lines agree with the rows of the table. The first line is `1 2 2`: for $n = 1$ there are no shifts to test, so `all(...)` is vacuously `True`, and both sequences of length one pass both tests, just as in Chapter 14. The fourth line is `4 8 8`: of the 16 sign sequences of length four, eight are Barker sequences, and eight have $C_1 = C_2 = C_3 = 0$.

To go further, raise the limit `14` in the line `for n in range(1, 14):`. Each additional length doubles the number of sequences, each a little longer to check, so the running time slightly more than doubles and the memory doubles, since `seqs` holds all 2^n sequences of one length at once. With the limit `21` (lengths up to 20, about a million sequences) the program finishes well within a minute on an ordinary computer, but at length 25 the list alone has more than 33 million entries and needs several gigabytes.

An assignment search that returns a certificate

The next function carries out the procedure of Section 8.4. Students are numbered $0, 1, 2, \ldots$, and `options[s]` is the set $N(s)$ of tasks acceptable to student `s`.

```
def assign(options):
    # options[s] is the set of tasks acceptable to student s.
    # Returns ("assignment", task_of) or ("shortage", A, N(A)).
    owner = {}      # task -> student who currently holds it
```

```
    task_of = {}   # student -> task that student currently holds
    for root in range(len(options)):
        came_from = {}        # task -> student from which it was reached
        reached = [root]      # students reached by the search
        to_explore = [root]   # reached students not yet inspected
        free_task = None
        while to_explore and free_task is None:
            student = to_explore.pop()
            for task in sorted(options[student]):
                if task in came_from or task_of.get(student) == task:
                    continue
                came_from[task] = student
                if task not in owner:
                    free_task = task
                    break
                reached.append(owner[task])
                to_explore.append(owner[task])
        if free_task is None:     # the search failed
            needed = set()        # will become N(A)
            for s in reached:
                needed = needed | options[s]
            return ("shortage", reached, needed)
        task = free_task          # flip the augmenting path back to the root
        while task is not None:
            student = came_from[task]
            old_task = task_of.get(student)
            owner[task] = student
            task_of[student] = task
            task = old_task
    return ("assignment", task_of)

print(assign([{1, 2}, {1}, {2, 3}]))
print(assign([{1, 2}, {1}, {2}]))
```

The matching. Two dictionaries describe the current matching M: `owner[t]` is the student who holds task `t`, and `task_of[s]` is the task that student `s` holds. A task without an entry in `owner` is unused, and a student without an entry in `task_of` is unmatched. Both start empty, so M starts as the empty matching. The students then take turns as the root of a search, in the order $0, 1, 2, \ldots$, and each is still unmatched when its turn comes, as the procedure requires: a flip changes the task of students who already hold one, and gives a first task only to the root.

The search. The dictionary `came_from` records each reached task with its predecessor, the student from which it was reached; the list `reached` holds the reached students, and `to_explore` those not yet inspected. A list used as a condition counts as true when it is not empty, so the `while` loop runs as long as some reached student awaits inspection and no unused task has been found. Each pass takes a student from `to_explore` and inspects it, taking its acceptable tasks in increasing order thanks to `sorted`, as in the trace of Section 8.6. The `continue` line skips a task

that has already been reached, and also the task the student holds, since the edge to that task is matched.

Every other task is marked as reached, with the current student as its predecessor. If the task is unused, the search has succeeded: `break` ends the inspection, and the `while` loop stops too, because `free_task` is no longer `None`. If the task is used, its owner is marked as reached and waits to be inspected; by the argument of Section 8.4, this owner has not been reached before, so `reached` never lists a student twice. Its predecessor need not be recorded separately: it is the task the owner holds, which `task_of` already records. Because `pop()` takes the most recently reached of the waiting students, this is not breadth-first search, but as Section 8.4 points out, the order of inspection does not matter.

The flip. If an unused task was found, the last loop flips M along the augmenting path, walking backward from the unused task to the root. At each step, `student` is the predecessor of the current task; this student gives up its old task, saved as `old_task`, and takes the current one. The walk continues from `old_task`, the task from which this student was reached, and in the next pass `owner` records the new holder of that task, its predecessor, so no out-of-date entry is left behind. At the root, `old_task` is `None`, because the root held no task, and the loop stops. The two dictionaries now describe the flipped matching, which by Lemma 8.5 has one more edge than M.

A failed search. The `while` loop stops only when an unused task has been found or `to_explore` is empty, that is, when every reached student has been inspected; so the search is complete in the sense of Section 8.4. Suppose that it ends without an unused task, and let A be the set of reached students. The three lines that build `needed` collect every task accepted by some student of A, so `needed` is $N(A)$. By Proposition 8.6, $N(A)$ has exactly one member fewer than A, so A is a shortage set. The function returns A and $N(A)$ at once, without trying the remaining students, because a shortage set already proves that no matching gives every student a task (Section 8.5).

The two test lines use the examples of Section 8.6, with the students a, b, c numbered $0, 1, 2$ and the tasks keeping their names $1, 2, 3$. The word in quotation marks at the start of each answer names its kind. The program displays

```
('assignment', {0: 2, 1: 1, 2: 3})
('shortage', [2, 0, 1], {1, 2})
```

In the first line, `{0: 2, 1: 1, 2: 3}` pairs student 0, that is a, with task 2, then b with task 1, and c with task 3: the assignment of Section 8.6, reached by a different route, because the program takes the roots in the order a, b, c rather than a, c, b. The search from a finds the unused task 1 at once, and a takes it. The search from b reaches task 1 from b, then its owner a, and then the unused task 2 from a, so it ends with `came_from` equal to `{1: 1, 2: 0}`. The flip gives task 2 to its predecessor a, whose old task is 1, and then task 1 to its predecessor b, whose old task is `None`, so the walk stops. Finally, the search from c reaches task 2, now held by a, and then the unused task 3, which c takes.

In the second test, task 3 has been removed from c's options. The searches from a and b run as before and give the matching $\{a\text{–}2,\ b\text{–}1\}$. The search from c reaches task 2 and its owner a, then a's other task 1 and its owner b, who accepts no further task, so the search fails. The list `[2, 0, 1]` contains the reached students c, a, b in the order of reaching, and `{1, 2}` is the set of tasks

they accept: three students, but only two tasks. This is the shortage set of Section 8.6, although the program reached it from a different matching.

Neither kind of answer has to be taken on trust, because each is a certificate in the sense of Section 8.5. To check an assignment, make sure that every student has a task that the student accepts and that no task appears twice. To check a shortage, count the tasks that the reported students accept: there must be fewer tasks than students.

The biased iteration with exact fractions

The last program checks the numerical experiment of Section 18.5. It computes the iterates of the biased update

$$y_{n+1} = \frac{3y_n + 1}{4} + \frac{1}{100}, \qquad y_0 = 0,$$

with exact fractions. For each n from 0 to 20, it checks the closed form $y_n = \frac{26}{25}\bigl(1 - (\frac{3}{4})^n\bigr)$ and the error certificate, the bound of part 2 of Theorem 18.1, which here reads

$$|y_n - 1| \le \left(\frac{3}{4}\right)^n e_0 + \eta\,\frac{1 - (3/4)^n}{1 - 3/4}, \qquad e_0 = |y_0 - 1| = 1, \quad \eta = \frac{1}{100}.$$

```
from fractions import Fraction

q = Fraction(3, 4)        # contraction factor of T(x) = (3x + 1)/4
eta = Fraction(1, 100)    # the bias added at every update
y = Fraction(0)           # the starting point y_0
e0 = abs(y - 1)           # initial error; the fixed point of T is 1
for n in range(21):
    closed = Fraction(26, 25) * (1 - q**n)
    assert y == closed                  # closed form, checked exactly
    error = abs(y - 1)
    bound = q**n * e0 + eta * (1 - q**n) / (1 - q)
    assert error <= bound               # bound, checked exactly
    print(n, round(float(error), 4), round(float(bound), 4))
    y = (3 * y + 1) / 4 + eta           # the biased update gives y_{n+1}
```

A line `assert` *condition* does nothing when the condition is true, and stops the program with an error message when it is false. So if the program displays all 21 lines, the closed form and the bound have both been confirmed, in exact arithmetic, for $n = 0, 1, \ldots, 20$. At $n = 0$ the inequality reads $1 \le 1$, which needs no theorem; part 2 of Theorem 18.1 covers every $n \ge 1$.

The function `float` converts a fraction to the nearest floating-point number, and `round(...,4)` rounds it to four decimal places. Both happen only in the `print` line, after the exact checks, so rounding affects only what is displayed. The displayed bounds are approximations, some of them rounded down; a bound used as a guarantee must be rounded up instead, as Section 12.5 explained. Here is part of the output:

```
0 1.0 1.0
1 0.74 0.76
2 0.545 0.58
...
10 0.0186 0.0941
11 0.0039 0.0805
12 0.0071 0.0704
...
20 0.0367 0.043
```

Python leaves out trailing zeros, so `0.58` stands for 0.5800 and `1.0` for 1.0000. The row for $n = 1$ shows the decimal forms of the exact values $37/50$ and $19/25$ found by hand in the solution to Exercise 18.6. As Section 18.5 describes, the true error in the middle column falls until $n = 11$ and then rises again toward $1/25 = 0.04$. The bound in the last column, by contrast, decreases at every step toward the same limit.

B.4 Checking the Lean examples

Install Lean, with the editor VS Code and its Lean extension, by following the official instructions [9] (Section 15.1). The examples are written for Lean version 4.19.0. Lean changes from one version to the next, so if an example that you copied exactly is rejected, check the version first: `lean --version` in a terminal shows it. The installation includes a small program called *elan*, which manages the versions of Lean and lets each folder choose its own. To use version 4.19.0, choose a folder for your Lean work and create in it a plain text file named `lean-toolchain` whose only line is

```
leanprover/lean4:v4.19.0
```

From then on, whenever Lean is started in that folder, elan uses version 4.19.0, downloading it first if necessary.

Then create, in the same folder, a file named `proofs.lean` with the shape of Section 15.1:

```
import Std

namespace MathCourse

-- the definitions and theorems go here

end MathCourse
```

Everything from `--` to the end of a line is a comment, which Lean ignores. Between the two lines that contain `MathCourse`, copy the Lean *declarations*, the displayed blocks of code that introduce a definition with `def` or a theorem with `theorem`, from Chapter 15, then Chapter 16, then the solutions for Chapters 15 and 16 in Appendix A, each in the order of the book. The order matters because Lean reads the file from top to bottom and accepts a name only after it has been defined; for example, the definition of `Inj` in Section 15.5 must come before `inj_comp` and every

later theorem that uses `Inj`.

Leave out the other displays: some show what Lean prints, such as a proof state or an error message, and a few repeat lines of a longer proof. The `#print axioms` commands of Section 16.6 are discussed below, and the one-line `theorem bad` of that section does not belong in the file, since it uses `sorry` on purpose.

Running the check

Open the file in VS Code. Lean checks it while you type, and when the cursor is inside a proof, the infoview beside the file shows the proof state there. To check the whole file at once from a terminal, move to its folder, as in Section B.1, and type

```
lean proofs.lean
```

If the command prints nothing, every declaration was accepted, with no error and no warning. An *error* means that the declaration containing it was rejected; a *warning*, that it was accepted but something in it deserves a second look. With all the declarations in the file, every one is accepted, but Lean prints two warnings, each followed by a note on how to switch such warnings off. The first looks like this:

```
proofs.lean:98:5: warning: unused variable `hf`
note: this linter can be disabled with `set_option linter.unusedVariables false`
```

Every message begins with the file name, the line and the column of the problem, and then says whether it is an error or a warning. Columns are counted from 0, so column 5 is the sixth character of the line, here the start of the name `hf`. Your line number may differ from 98, depending on the blank lines you leave between declarations. The second warning says the same about `hg`. Both concern the theorem `surj_comp_assumed`, and Section 16.5 explains why it never needs these two hypotheses.

Asking which axioms a proof uses

To list the axioms on which a theorem depends (Section 16.6), add a line such as

```
#print axioms MathCourse.surj_comp
```

after the line `end MathCourse`; since it stands outside the namespace, it uses the full name, with the prefix `MathCourse`. Lean answers

```
'MathCourse.surj_comp' does not depend on any axioms
```

and from then on `lean proofs.lean` prints this line as well. Some theorems, such as `not_surj_succ` and `succ_surj_int` of Section 16.7, list the standard axiom `propext` instead, through the library lemmas they use; as that section explains, this is no cause for concern.

The command `#check`, which Section 16.7 uses to display the exact statement of a library lemma, goes in the file the same way, on a line of its own outside every proof: the line `#check`

`Nat.succ_ne_zero`, for example, makes Lean print the statement of that lemma.

What an unfinished proof looks like

To see for yourself what Section 16.6 describes, temporarily replace the last line of the proof of `surj_comp`, `rw [ha, hb]`, by the single word `sorry`, keeping the same indentation. Lean still accepts the file, but it prints the warning `declaration uses 'sorry'` at the name of the theorem, and the `#print axioms` line above now reports

```
'MathCourse.surj_comp' depends on axioms: [sorryAx]
```

Put the original line back before you rely on the file again: a file that contains `sorry` has not proved what it states.

Where to go next

Once the statements, goals and proof states of Chapters 15 and 16 feel familiar, a natural next step is *Mathematics in Lean*, an interactive textbook with many exercises [15]. It is built on Mathlib, the large mathematical library that this book's examples do without, and its own instructions explain how to set it up.

APPENDIX C

A compact guide to notation and mathematical judgment

This appendix gathers two things that the book develops gradually: its notation (Section C.1), and the judgment that guides how its proofs are read and found (Sections C.2–C.5). It ends with a suggested rubric for the capstone investigation and a description of what the book aims to leave you able to do.

C.1 A notation map, with places to return to

The table below lists the main symbols of the book, roughly in order of appearance, with the place where each is explained. A one-line summary is only a reminder: to use a symbol with confidence again, reread that section and its examples.

A few symbols, and many letters, do more than one job; the surrounding sentence and the kind of object involved decide which.

- Vertical bars give the absolute value $|x|$ of a real number or the size $|S|$ of a finite set; double bars, as in $\|x\|_2$, give the length of a vector.
- $(1, 3)$ can be an interval or an ordered pair; $[1, 3]$ is an interval, while $[x]$ is an equivalence class.
- The arrow $\to$ appears in a function $f : A \to B$, a directed edge $u \to v$, and a convergent sequence $x_n \to x_*$; the barred arrow, as in $x \mapsto x^2$, names the rule of a function, and $\Rightarrow$ joins two statements.
- When f has an inverse, f^{-1} is that inverse function, so $f^{-1}(y)$ is a single input, not the reciprocal $1/f(y)$ (Section 3.2). The preimage $f^{-1}(T)$ of a set T of outputs is a set of inputs, defined for every function; when f has an inverse, it is the image of T under the inverse (Section 3.4).
- The letter q is a quotient in Chapter 5 and a contraction factor in Chapters 12 and 18. The capital A is a set in Chapter 3, a matrix in Chapter 10 and an event in Chapter 13, while A_s is an autocorrelation in Chapter 14. The letter I is an identity matrix in Chapter 10, but I_A is an indicator in Chapter 13; and e_j is a standard basis vector in Chapter 10, but e_n is the error at step n in Chapter 18.

Symbol	Meaning, and where it is explained
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	The natural numbers $0, 1, 2, \ldots$ (zero included), the integers, the rational numbers, and the real numbers (Section 1.8).
$\sum_{j=a}^{b} x_j$	The sum $x_a + x_{a+1} + \cdots + x_b$, with one term for each integer j from a to b (Section 1.8). When $b < a$ there are no terms, and this *empty sum* is 0 (Section 5.1). A sum such as $\sum_{v \in V} \deg(v)$ has one term for each member v of the finite set V, added in any order (Section 7.2).
$x < y, x \le y$	x is strictly smaller than y; x is smaller than or equal to y (Section 1.8).
$\sqrt{t}$	For a real number $t \ge 0$, the nonnegative number whose square is t; so $\sqrt{9} = 3$, not -3 (Section 1.8).
$P \Rightarrow Q$, $P \Leftrightarrow Q$	"If P, then Q"; "P if and only if Q," meaning both $P \Rightarrow Q$ and $Q \Rightarrow P$. Writing either states a claim, not that the claim has been proved (Section 2.1).
$\neg, \wedge, \vee$	Not; and; or. The "or" is inclusive: $P \vee Q$ is true when at least one of P and Q is true (Section 2.2).
$\forall, \exists$	For every; there exists (at least one). Quantifiers are read from left to right: a value chosen at an $\exists$ may depend on the values chosen before it, but not on those chosen after it, so changing the order can change the meaning (Section 2.3).
ε, δ	Traditional names for tolerances, positive amounts of error we are prepared to allow. They are names, not fixed numbers: "for every $\varepsilon > 0$" covers every positive value (Sections 2.3 and 11.1). In Chapter 18, ε_n is an update error (last row).
$\lvert x \rvert$	The absolute value of a real number x: its distance from 0 on the number line (Section 2.6).
$[a, b]$, (a, b), $[0, \infty)$	Intervals of real numbers. A square bracket includes its endpoint and a round bracket excludes it; $[0, \infty)$ is the set of all real x with $x \ge 0$ (Section 2.6).
$x \in A$, $x \notin A$	x is a member of the set A; x is not a member of A (Section 3.1).
$A \subseteq B$, $\varnothing$	Every member of A is also a member of B; the empty set, which has no members (Section 3.1).
$\{x \in A : P(x)\}$	The set of members of A that have the property P. Read the colon as "such that" (Section 3.1).
$A \cup B$, $A \cap B$	Union: the objects in at least one of A and B. Intersection: the objects in both (Section 3.1).
$A \setminus B$	The set of members of A that are not in B: members are removed, not numbers subtracted (Section 3.1).
$A \times B$	The set of ordered pairs (a, b) with $a \in A$ and $b \in B$ (Section 3.1).
$\lvert S \rvert$	The number of members of a finite set S, each counted once; $\lvert\varnothing\rvert = 0$ (Section 3.1).

Symbol	Meaning, and where it is explained
$f : A \to B, x \mapsto x^2$	A function with domain A and codomain B; the function that sends each input x to x^2 (Section 3.2).
$g \circ f$, id_A, f^{-1}	"g after f": apply f first, then g. The identity function on A, which sends every element to itself. The inverse of $f : A \to B$, when it exists: the function $f^{-1} : B \to A$ with $f^{-1}(f(x)) = x$ for every $x \in A$ and $f(f^{-1}(y)) = y$ for every $y \in B$ (Section 3.2).
$f(S)$, $f^{-1}(T)$	The image of a set S of inputs: all outputs $f(x)$ with $x \in S$. The preimage of a set T of outputs: all inputs x with $f(x) \in T$ (Section 3.4).
$d \mid a$, $\gcd(a, b)$	The positive integer d divides the integer a: $a = dk$ for some integer k. The greatest common divisor of nonnegative integers a and b, not both zero: the largest positive integer that divides both (Section 5.5).
$a \bmod m$	The remainder when the integer a is divided by the positive integer m; one of $0, 1, \ldots, m-1$ (Section 5.5).
$x \sim y$, $[x]$	x is related to y; the equivalence class of x, the set of all elements related to x (Section 6.1).
$a \equiv b \pmod m$	a is congruent to b modulo m: the positive integer m divides $a - b$ (Section 6.2).
$[a]_m$, $\mathbb{Z}/m\mathbb{Z}$	The residue class of a: the whole set of integers congruent to a modulo m, not just the one integer a. The set of all m residue classes (Section 6.2).
$x \preceq y$	x is below y in a partial order, a relation that is reflexive, antisymmetric and transitive (Section 6.6).
$\deg(v)$	The degree of a vertex v: the number of its neighbors (Section 7.1).
$u \to v$	A directed edge from u to v; v is then an out-neighbor of u (Section 7.5).
$N_1^+(v)$, $N_2^+(v)$	The first and second out-neighborhoods of v: the vertices at directed distance exactly one, and exactly two, from v (Section 7.7).
$N(s)$, $N(S)$	The tasks accepted by the student s; the tasks accepted by at least one student of the set S (Sections 8.1 and 8.5).
$S \subsetneq L$	S is a proper subset of L: $S \subseteq L$ and $S \neq L$ (Section 9.4).
$\mathbb{R}^n$	The set of vectors $(x_1, \ldots, x_n)$ with n real coordinates (Section 10.1).
A_{ij}, Ax	The entry of the matrix A in row i and column j; the vector obtained by applying A to x (Section 10.2).
$\ker T$	The kernel of a linear map T: the inputs that T sends to the zero vector (Section 10.3).
$u \cdot v$	The dot product: multiply corresponding coordinates and add. The result is a number, not a vector (Section 10.4).
$\|x\|_2$	The Euclidean length $\sqrt{x \cdot x}$ of a vector. The subscript 2 is part of the name, not a coordinate (Section 10.4).

Symbol	Meaning, and where it is explained
A^T, I	The transpose of A, obtained by exchanging rows and columns. The identity matrix, with ones on the diagonal and zeros elsewhere (Section 10.6).
(X, d), $d(x, y)$	A metric space: a set X with a chosen metric d, which gives every pair of points x, y a distance $d(x, y)$ and satisfies, for all points x, y, z: $d(x, y) \geq 0$, with $d(x, y) = 0$ exactly when $x = y$; $d(x, y) = d(y, x)$; and the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$. The choice of metric is part of the problem (Section 11.2).
d_2, d_∞	Two metrics on $\mathbb{R}^n$: the Euclidean distance $\|x - y\|_2$, and the largest of the coordinate differences $\|x_j - y_j\|$ (Section 11.2).
$x_n \to x_*$	The sequence converges to x_*: for every tolerance $\varepsilon > 0$, all terms from some index on are at distance less than ε from x_* (Section 11.3).
x_*	A name for a special point: a limit in Chapter 11, a fixed point in Chapters 12 and 18. The star is part of the name (Section 11.3).
$\sup A$, $\inf A$	The supremum (least upper bound) and infimum (greatest lower bound) of a set $A \subseteq \mathbb{R}$, when they exist. The supremum need not belong to A; if it does, it is the maximum (Section 11.8).
Tx	Short for $T(x)$ (Section 12.1).
Ω, $p(\omega)$	The sample space, a finite set of outcomes; the probability of the outcome ω. The numbers $p(\omega)$ are nonnegative and add up to 1 (Section 13.1).
$\mathbb{P}(A)$	The probability of an event $A \subseteq \Omega$: the sum of $p(\omega)$ over the outcomes $\omega \in A$ (Section 13.1).
$\mathbb{E}\,X$	The expectation of a random variable $X : \Omega \to \mathbb{R}$: the weighted average $\sum_{\omega \in \Omega} p(\omega) X(\omega)$ (Section 13.2).
I_A	The indicator of the event A: it equals 1 at outcomes in A and 0 at all other outcomes (Section 13.3).
A_s, C_s	For a sign sequence $(a_0, \ldots, a_{n-1})$, with every entry 1 or -1: the aperiodic autocorrelation $A_s = \sum_{j=0}^{n-1-s} a_j a_{j+s}$, for $1 \leq s \leq n-1$, with no wrapping around; and the periodic autocorrelation $C_s = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n}$, for $0 \leq s \leq n-1$, with positions taken modulo n (Sections 14.1 and 14.2).
`a : T`, `f x`, `:=`	In Lean: `a` has type `T` (for a proposition `P`, `h : P` says that `h` is a proof of `P`); the function `f` applied to `x`; "is defined as" or "is proved by" (Section 15.2).
`sorry`	In Lean, a placeholder for a missing proof. The command `#print axioms` shows whether a theorem depends on one: if it does, the list includes the axiom `sorryAx` (Section 16.6).
$B(c, r)$	The ball with center c and radius $r > 0$ in a metric space X: the points of X whose distance from c is strictly less than r (Section 17.2).

Symbol	Meaning, and where it is explained
y_n, ε_n, e_n	In Chapter 18: the points of an iteration whose updates may contain errors, so that y_{n+1} is only close to $T(y_n)$; the update error ε_n, with $d(y_{n+1}, T(y_n)) \le \varepsilon_n$; and the error $e_n = d(y_n, x_*)$, the distance from y_n to the fixed point (Sections 18.1 and 18.3).

C.2 Structures, what proofs use, and changes of description

Almost every object in this book is a set with something extra attached, which Section 6.4 calls a *structure*. It may be a *relation* (Section 6.1), which links or compares pairs of objects, as congruence modulo m and adjacency in a graph do, or an *operation*, such as addition, which combines two elements of a set into an element of the same set. Functions, also called maps (Section 3.2), let us carry a question from one description to another.

The extra structure decides which questions can even be asked.

- Addition and scaling let us form linear combinations of vectors (Chapter 10).
- Order lets us pick out a smallest or a largest object. Every nonempty set of natural numbers has a smallest member (the well-ordering property of Sections 4.2 and 5.2), and every nonempty finite one has a largest; this is how Chapter 7 chooses a shortest walk between two vertices joined by some walk, and a longest directed path in a nonempty finite directed graph.
- A distance lets us say that one point approximates another, and so speak of limits (Chapter 11). Without one, "x_n gets close to x_*" has no meaning.
- A probability on a finite set of outcomes lets us take weighted averages (Chapter 13).

Habit 3 (Section 3.4) asks which information a proof actually uses. For the main theorems of the book the answers are short.

- The proofs of Hall's theorem (Chapter 9) use only which students accept which tasks and the sizes of finite sets, never what the students and tasks are. So the theorem applies to every finite bipartite graph, whatever its vertices stand for.
- The proof of the contraction theorem (Theorem 12.2) uses, besides the facts that the space is nonempty and that T maps it into itself, only the contraction inequality, completeness, and the three requirements of a metric, above all the triangle inequality. It never adds points or multiplies them by numbers, so it works in metric spaces that have no addition at all.
- The proof of the Cauchy–Schwarz inequality (Theorem 10.2) uses only three properties of the dot product: linearity in each argument, symmetry, and positivity. So the same proof works for any other way of multiplying two vectors to get a number, provided it has these three properties and the length of u is defined as the square root of the product of u with itself. For example, on $\mathbb{R}^2$ the product $2u_1v_1 + u_2v_2$ is symmetric and linear in each argument, and positive, since $2u_1^2 + u_2^2$ is never negative and is zero only when $u = 0$. So $|2u_1v_1 + u_2v_2| \le \sqrt{2u_1^2 + u_2^2}\,\sqrt{2v_1^2 + v_2^2}$ for all $u, v \in \mathbb{R}^2$, with no new work.

- Chapter 14 turns the condition that a sign sequence has zero periodic correlation at every nonzero shift into the condition that different rows of its circulant matrix are orthogonal (Proposition 14.5). Adding up all the correlations shows that the order of the matrix, its number of rows, is a perfect square (Section 14.4), an argument that relies on the circulant pattern. Comparing three rows shows that an order greater than 2 is divisible by four (Lemma 14.6). That argument uses only that the entries are ± 1 and different rows are orthogonal, which is exactly what makes a square matrix a Hadamard matrix (Section 14.3), so it holds for every Hadamard matrix, circulant or not.

Noticing what a proof does *not* use tells you where its conclusion might reach further. This is a guide for guessing, not a proof: every proposed extension must still be checked against the argument, line by line.

Many steps in the book change the description of a problem without changing the problem, and Habit 6 (Section 6.6) asks what such a change preserves; a new description helps only when we know exactly what it keeps and what it throws away. Here are four from the book.

- *An inverse loses nothing.* If f has an inverse h, every input is recovered from its output as $x = h(f(x))$. In particular, $f(x) = f(y)$ gives $x = h(f(x)) = h(f(y)) = y$, so different inputs never share an output, and a question about inputs can be answered by looking at outputs (Section 3.3).
- *A quotient forgets on purpose.* Passing from an integer a to its residue class $[a]_m$ forgets everything except the remainder on division by m. This is safe only for rules that give the same answer for every representative of a class, which is why well-definedness has to be checked (Section 6.3).
- *A matrix turns a linear map into a table of numbers.* Column j of the matrix records where the map sends the standard basis vector e_j (Section 10.2), and composing linear maps corresponds to multiplying their matrices (Section 10.6).
- *An indicator turns an event into a quantity.* An indicator is 1 when its event occurs and 0 when it does not, so counting how many events occur becomes adding indicators, and linearity of expectation applies (Section 13.3).

Each of them gives a language in which the step we need is easier to state and to justify.

C.3 Choosing the next move

Mathematics can look like a long list of tricks to memorize, but it helps far more to recognize the kind of situation you are in. Four situations recur in this book, and each suggests a natural next move.

When a general claim looks doubtful

Look for a counterexample before you try to prove the claim. Start with small and boundary cases: zero, the empty set, the smallest allowed input, two objects that happen to be equal. A useful

counterexample is rarely found by trying cases at random; it is built to break the step that the claim seems to need.

- Proposition 3.3 shows that $g \circ f$ can be injective although g is not. There f maps $\{0, 1\}$ into $\{0, 1, 2\}$ and never reaches 2, while g sends 2 to the same output as 1. The composition applies g only to outputs of f, so it never meets this collision.
- In Section 18.2, the guess that small update errors still give convergence to the fixed point fails for a constant bias: the map $x \mapsto qx$ on $\mathbb{R}$, with $0 < q < 1$, has the fixed point 0, but adding the same $\eta > 0$ at every update moves the limit to $\eta/(1 - q)$, however small η is.

Each counterexample also showed what a correct statement must take into account: the outputs that f actually reaches, and errors that never die out.

When you need to show that something exists

When a direct construction is out of reach, ask what a failure would force. The book turns that question into a proof in three ways.

- *Search and repair.* The search of Chapter 8 either finds an augmenting path, which enlarges the matching, or fails and produces a group of students who together accept too few tasks (Proposition 8.6), so that no matching gives every student a task. Either outcome tells us something, which is the point of Habit 7 (Section 8.5).
- *An extremal choice.* Choose an object that is as large or as small as possible, and show that it already has the property you want. A longest directed path in a nonempty finite acyclic oriented graph ends at a sink, because an arrow leaving its last vertex would give either a longer path or a directed cycle (Theorem 7.8). Similarly, if a statement about natural numbers fails somewhere, choose the smallest number at which it fails: the statement holds at every smaller number, and this often leads to a contradiction (Section 5.2).
- *An average.* A random variable X on a finite probability space satisfies $X(\omega) \geq \mathbb{E}\, X$ at some outcome ω of positive probability, since outcomes of probability zero contribute nothing to $\mathbb{E}\, X$ and a weighted average cannot exceed all the values it averages (Proposition 13.5). Theorem 13.8 uses this: labeling the vertices of a graph with m edges at random gives a cut of expected size $m/2$, so some labeling gives a cut of size at least $m/2$, although the proof never says which. Habit 8 (Section 13.6) describes this approach.

To choose among them, ask: Can a partial solution be repaired one step at a time? Is there a natural quantity to make as large or as small as possible? Is there an average you can compute exactly?

When the full answer is out of reach

Ask what you really need to know. Often it is much less than the full answer.

- To know how close a point z is to the fixed point of a contraction, you need not find the fixed point: the residual estimate of Theorem 12.2 bounds the error by $d(z, Tz)/(1 - q)$, and Tz is

usually easy to compute. In Section 12.5, this certified that $x_2 = 10/27$ lies within $19/729$ of the fixed point of $T(x) = (x^2 + 1)/3$ on $[0, 1]$ before the fixed point had been found.

- To show that no circulant Hadamard matrix of order 12 exists, you do not need the complete answer claimed by the research of Section 14.7. The necessary condition of Proposition 14.7 is enough: an order greater than 1 must have the form $4u^2$ for a positive integer u, and 12 lies strictly between the first two such numbers, 4 and 16.

The risk in settling for less is answering an easier question than the one you were asked. If you replace the question by an easier one, say so, and compare the new version with the contract of the original, as Habit 2 (Section 2.6) asks. Proposition 14.7, for instance, settles order 12, but as an answer to "which orders are possible?" it is only partial: besides 1 and 4, which do occur, it still allows $16, 36, 64, \ldots$, and says nothing about whether any of them occurs.

When a bound appears

Ask whether the constant in the bound could be lowered and, if not, which example prevents any improvement. Such an example usually attains the bound, or comes as close to it as we like, so also ask when the bound is attained (Section 9.6).

- The Cauchy–Schwarz inequality $|u \cdot v| \le \|u\|_2 \, \|v\|_2$ holds with equality exactly when $v = 0$ or u is a multiple of v (Section 10.5). Because equality occurs for nonzero vectors, such as $u = v \neq 0$, the constant factor 1 on the right cannot be made smaller.
- In the constant-bias example of Section 18.4, the error bound of Theorem 18.1, part 2, holds with equality at every step: with bias $\eta > 0$, the errors are exactly $\eta(1 - q^n)/(1 - q)$, which approach $\eta/(1 - q)$ as n grows. So the factor $1/(1 - q)$ is sharp. Any smaller factor would give a bound below the true error once n is large enough, so no cleverer proof can lower it.

C.4 Turning a reading difficulty into a precise question

Suppose you understand what a theorem says but cannot follow one line of its proof. Asking an assistant, or a classmate, to explain the whole proof again rarely helps: the new explanation may skip the same step, or replace it by a different argument that you must then check from the beginning. It works better to turn the difficulty into a precise question, as Habit 4 (Section 4.5) suggests. Write down three things:

- the exact line that you do not follow;
- what is known at that point of the proof: the hypotheses, and the facts already established;
- the last step that you do understand.

Then ask which definition or earlier fact leads from what is known to the line. A good answer is usually short, one calculation written out in more detail or one small example, and rarely a new technique. Here are two such questions.

- In Case 1 of the inductive proof of Hall's theorem (Section 9.4), S is a nonempty proper

group of students with $|N(S)| = |S|$, and T is any group of the remaining students. A precise question is: "I know that the task sets $N(S)$ and $N(T)$ can overlap. Why is $|N(S) \cup N(T)| = |N(S)| + |N(T) \setminus N(S)|$?" The answer is a splitting: every task in the union lies in exactly one of the sets $N(S)$ and $N(T) \setminus N(S)$, and the sizes of two disjoint finite sets add (Section 3.1). Nothing more about graphs is needed. Exercise 9.3 asks you to write this out in full.

- From the proof of Theorem 18.1 (Section 18.3): "I see that each update error is later multiplied by q. Why is the weight of ε_j in the bound for e_n exactly q^{n-1-j}?" The answer is a count. The error ε_j is made in the update that produces y_{j+1}, and each of the $n - (j+1) = n - 1 - j$ later updates, which produce $y_{j+2}, \ldots, y_n$, contributes one factor q.

Once you have the answer, write the step in your own words. Then change one small feature and work out what changes.

In the Hall example, the proof continues as follows. The students of $S \cup T$ together accept exactly the tasks in $N(S) \cup N(T)$, so Hall's condition gives $|N(S) \cup N(T)| \geq |S \cup T|$, and $|S \cup T| = |S| + |T|$ because the two groups share no student. The identity from the question, with $|N(S)| = |S|$, turns the left side into $|S| + |N(T) \setminus N(S)|$, and subtracting $|S|$ leaves $|N(T) \setminus N(S)| \geq |T|$. Now suppose T may share students with S. The identity still holds, but $|S \cup T| = |S| + |T|$ does not, because a student in both groups is counted once on the left and twice on the right. The conclusion no longer follows, and it can even be false: for $T = S$ it would read $0 \geq |S|$, which fails because S is nonempty.

In the second example, if the update errors were numbered from 1, so that ε_j bounds the error in the update that produces y_j, then $n - j$ updates follow it, and its weight becomes q^{n-j}. Or suppose $q = 1$, so that the map never increases distances but need not shrink them. If it still has a fixed point x_*, the same estimate and count work, but every weight is 1, so the bound is $e_0 + \varepsilon_0 + \cdots + \varepsilon_{n-1}$. When every update error equals the same $\eta > 0$, this is $e_0 + n\eta$, which grows without limit, and the error really can grow this fast: every point is a fixed point of the identity map $T(x) = x$ on $\mathbb{R}$, so take $x_* = 0$, start at $y_0 = 0$, and set $y_{n+1} = y_n + \eta$; then $e_n = n\eta$, exactly the bound. If you can answer variations like these, the explanation has become your own: you can reason with it, not only repeat it.

The same approach works for a definition that you do not understand. Ask, in this order: What kind of object does it apply to? What exactly must such an object satisfy? What is one example, and what is a *near miss*, an object that fails only one of the requirements? For the definition of an odd integer (Definition 1.1), the objects are integers, and a is odd when $a = 2k + 1$ for some integer k. The number $7 = 2 \cdot 3 + 1$ is an example. The number 8 is a near miss: $8 = 2 \cdot 3.5 + 1$ has the right shape, but 3.5, the only k with $2k + 1 = 8$, is not an integer, so the one requirement that fails is "k is an integer". Finally, when the definition appears in a proof, decide which of the two directions of Section 1.4 is used: does the proof *unpack* the definition for an object it is given, or *recognize* that a new object satisfies it? The first proof of the book, Proposition 1.2, does both: it unpacks the fact that a and b are odd, and recognizes that $a + b$ is even.

C.5 What a finished reading record contains

A *reading record* (Section 17.8) is a written account of a proof you are studying: what you have read, what you have understood, and what remains open. A finished record contains at least:

- the exact statement of the result, copied from a named source, with the date or version number of what you read, because a preprint or a repository can change;
- the same statement in your own words;
- the objects that the statement is about, and the hypotheses on them;
- a map of the proof's dependencies: a table like the prerequisite map of Section 17.7, a diagram, or a paragraph;
- one delicate step of the proof, written out in full detail.

Some entries need a note beside them:

- for a result you use without having read its proof: its hypotheses, and why they hold in your situation;
- for a computer calculation: the range of cases it covered, and whether it used exact arithmetic or rounded decimals (Section B.2);
- for a result checked by Lean: exactly which statement Lean checked, which axioms its proof depends on (Section 16.6), and whether you ran the check yourself or are reporting what a source says (Section 17.8).

Give every entry the two labels of "Keeping track of evidence" in *To the reader*: how well you understand it (*understood*, *can follow*, *borrowed*, or *open*; see also Section 17.7), and what supports it (proved, cited from another source, checked by computer for finitely many cases, or not established). An entry that rests on one of the four starting facts listed there can say so, since those facts are recorded as assumptions.

The record ends with your own explanation of the proof and one variation you have worked out yourself, such as a hypothesis removed or a conclusion strengthened, with your reasons for what happens. A saved conversation with an assistant cannot replace this record: a conversation mixes guesses, corrections and results without marking which statements survived checking. The record should let another reader rebuild the final argument without reading the conversation.

C.6 A suggested capstone rubric

The table below suggests one way to assess the independent capstone of Section 18.7. The row "Understanding" assesses the short oral discussion that ends the investigation; the other rows follow that section's list of what the write-up should contain.

Quality	Weight	What a strong project shows
Claim and scope	20%	The exact objects, hypotheses and conclusion of each result. If the question changed during the project, both versions are stated and the difference is explained. Each claim is marked with its evidence: proved, cited from another source, checked by computer for finitely many cases, or not established.
Argument	35%	A complete proof or a counterexample, with a map of the background the argument needs. The delicate steps are written out in full, and each step names the results it depends on.
Understanding	20%	In the oral discussion, the student explains a delicate step without help and, when an assumption or an example is changed, correctly predicts what still holds and what fails.
Reproducibility	15%	Computer checks where relevant, with the program, the range of cases covered and the software used, so that anyone can repeat them. Otherwise, an explanation of how a reader can check the claims without a computer, for example by working small cases by hand.
Context	10%	Accurate credit to every source used; a modest statement of what is new compared with the published literature and what is new only to the student; an open account of how an AI assistant was used and of what remains unsettled.

Three faults cannot be outweighed by good presentation: a statement changed without saying so, a missing step on which the argument depends, and an unsupported claim of novelty. A project with any of them must be corrected before it can pass. The rubric rewards careful work on a small investigation of the student's own; nobody is expected to solve a famous problem.

C.7 What you can do at the end of the book

You do not need to remember every displayed formula in this book; what matters is what you can do. Faced with a statement, you can say exactly what it requires. Faced with a proof you do not yet follow, you can find where your understanding stops and ask for the one concept or step that would carry you past it. When an explanation arrives, from a book, a classmate or an assistant, you can test it on an example, see its role in the whole proof, and say clearly what has not yet been established.

The next unfamiliar theorem you meet will still contain mathematics you have not seen, but you now have a way in: identify the structures involved, lay out which steps depend on which, learn the missing pieces, and return to the argument. Working with an AI assistant is worthwhile when what it produces helps you understand and gives you something you can check, not when it merely fills a page with text that looks like mathematics.

References and further reading

[1] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. The textbook for MIT course 6.1200J, a free PDF from MIT OpenCourseWare; a good further source on logic, induction, graphs, matchings, and probability. https://ocw.mit.edu/courses/6-1200j-mathematics-for-computer-science-spring-2024/pages/readings/

[2] OpenAI, *math*: a public repository of mathematical preprints, with Lean formalizations and descriptions of what each formalization covers. https://github.com/openai/math

[3] OpenAI, "Seymour's second-neighborhood conjecture." Description of the scope of the Lean formalization, with a link to the accompanying preprint, *A proof of Seymour's second-neighborhood conjecture* (September 2026). https://raw.githubusercontent.com/openai/math/main/lean/docs/173.md

[4] OpenAI, "The circulant Hadamard and Barker-sequence conjectures." Description of the scope of the Lean formalization, with a link to the accompanying preprint, *The circulant Hadamard conjecture* (September 2026). https://raw.githubusercontent.com/openai/math/main/lean/docs/179.md

[5] OpenAI, `Main.lean`: the Lean file with the proof of the second-neighborhood theorem `exists_goodVertex`. https://raw.githubusercontent.com/openai/math/main/lean/OAI/Combinatorics/SecondNeighborhood/Main.lean

[6] OpenAI, `SeymourSecondNeighborhood.lean`: the Lean file with the official statement of the second-neighborhood theorem, against which the proof is checked. https://raw.githubusercontent.com/openai/math/main/lean/ComparatorChallenges/SeymourSecondNeighborhood.lean

[7] OpenAI, `Reduction.lean`: the Lean file with the reduction step of the second-neighborhood proof. https://raw.githubusercontent.com/openai/math/main/lean/OAI/Combinatorics/SecondNeighborhood/Reduction.lean

[8] OpenAI, configuration file for the formal check of the second-neighborhood theorem, naming the statement file, the proof file, the theorem, and the permitted axioms. https://raw.githubusercontent.com/openai/math/main/lean/ComparatorChallenges/SeymourSecondNeighborhood.json

[9] Lean, official installation instructions. https://lean-lang.org/install/

[10] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Propositions and Proofs." https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/

[11] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Quantifiers and Equality." https://lean-lang.org/theorem_proving_in_lean4/Quantifiers-and-Equality/

[12] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Tactics." https://lean-lang.org/theorem_proving_in_lean4/Tactics/

[13] *The Lean Language Reference*, section "Axioms." https://lean-lang.org/doc/reference/latest/Axioms/

[14] *The Lean Language Reference*, section "Validating a Lean Proof." https://lean-lang.org/doc/reference/latest/ValidatingProofs/

[15] J. Avigad and P. Massot, with contributors, *Mathematics in Lean*. An interactive textbook with exercises, built on the Mathlib library. https://leanprover-community.github.io/mathematics_in_lean/

[16] Python Software Foundation, *The Python Tutorial*, section "Data Structures." https://docs.python.org/3/tutorial/datastructures.html

[17] Python Software Foundation, *The Python Standard Library*, section "fractions—Rational numbers." https://docs.python.org/3/library/fractions.html

All web addresses were accessed in October 2026. The pages and repository files they point to may change after that date.